PoC

'25 년 C2J 변환 솔루션 PoC 용

PoC

PoC

별첨 #1

PoC

목 차

범례) 1

1. 프로그램별 사용 방법 1

1.1 계좌번호 채번(FUBcNumbering.getAcnoNumbering()) 1

1.2 관리번호 채번(FUBcNumbering.getMgtNoNumbering()) 3

1.3 비밀번호사용제한 체크(FUBcValidation.validationPassword()) 5

1.4 통화코드 조회(FUBcCode.getCurrencyCode()) 7

1.5 기준환율 조회(FUBcRate.getBaseExchangeRate()) 9

1.6 시장환율 조회(FUBcRate.getMarketExchangeRate()) 12

1.7 평균기준환율 조회(FUBcRate.getAvgExchangeRate()) 14

1.8 상품선물환율 조회(FUBcRate.getPrdctExchangeRate()) 16

1.9 어음교환결제시각 조회(N/A) 18

1.10 부점기본 조회(FUBcBranch.getBranchBasic()) 19

1.11 외국환부점변환 조회(FUBcBranch.getFXBrnChng()) 23

1.12 관할/대출실행팀 소속부점코드 조회(FUBcBranch.getJrsdLnExeTeamBrnCd()) 27

1.13 부점 폐쇄에 의한 통합부점 조회(FUBcBranch.getIntegrationBranch()) 29

1.14 통합부점코드에 속한 통폐합된 부점 전체조회(FUBcBranch.getMrabBrnWhol()) 33

1.15 인수합병 부점의 통합지로코드 조회(FUBcBranch.getIntgraGiroCd()) 35

1.16 부점운영지원내역 조회(FUBcBranch.getBranchOperSportHist()) 38

1.17 부점주소, 전화번호 조회 (실시간)(FUBcBranch.getBranchAddrTelno()) 39

1.18 업무마감통제내용 변경(N/A) 42

1.19 우편번호 3자리로 지역별 소속부점코드 조회(FUBcBranch.getAreaBranchCode()) .. 43

1.20 전행 인스턴스코드 내용 조회(FUBcCode.getEnbnIncdContent()) 45

1.21 전행 인스턴스코드 복수 조회(FUBcCode.getEnbnInstanceCode()) 47

1.22 에러메시지코드, 책임자승인사유코드, 조치메시지코드 조회 (FUBcMessage.getMessageCode()) 49

1.23 직원기본 조회(FUBcEmployee.getPafiarEmpInfo()) 51

1.24 IT 직원정보 조회(FUBcEmployee.getITEmpInfo()) 58

1.25 다양한 조건의 전체직원 조회(FUBcEmployee.getWholPafiarEmpInfo()) 61

1.26 다양한 조건의 전체 IT 직원 조회(FUBcEmployee.getWholITEmpInfo()) 63
1.27 KB 기준 정보 조회/ 변경(FUBcBase.prcssKBBaseInfo()) 65
1.28 일수/월수/영업일수/공휴일수 산출(FUBcDate.calculateNumberOfDays()) 67
1.29 입력월수 차감후의 일수산출(FUBcDate.calculateSubtractionNumberOfDays()) 69
1.30 만기일/역만기일 산출(FUBcDate.calculateExpirDd()) 71
1.31 양일자간의 차이를 년수/월수/일수산출(FUBcDate.calculateDteBtnDfrn()) 73
1.32 월별 주기값 산출(FUBcDate.calculatePerMnCyclVal()) 75
1.33 시분초 연산(FUBcDate.calculateHHMMSS()) 76
1.34 해외정보수신기관 코드별 일수/월수/영업일수/공휴일수 산출 (FUBcDate.calculateOvsesNoday()) 78
1.35 해외정보수신기관 코드별 만기일/역만기일 산출(FUBcDate.calculateOvsesExpirDd()) 80
1.36 가산 및 감산월을 이용한 월초영업일/ 월말영업일/ 월말일자 산출 (FUBcDate.calculateBaseYmdBzoprDd()) 82
1.37 연령계산 산출(FUBcDate.calculateAge()) 84
1.38 윤년 체크(FUBcDateCheck.checkLpyrChk()) 86
1.39 휴일 체크(FUBcDateCheck.checkHoldyChk()) 87
1.40 영업일 산출 I (익영업일/익익영업일/3 영업일/4 영업일/5 영업일/전영업일/전전영업일)(FUBcDate.calculateBzoprDd()) 89
1.41 일자 산출 I(익일/익익일/전일/전전일) (FUBcDate.calculateDate()) 91
1.42 월초영업일/월말영업일/월말일 산출(FUBcDate.calculateEnmnBzoprDay()) 93
1.43 줄리안 일자<->그레고리언 일자변환(FUBcDateChange.changeJulTOGreg()) 95
1.44 양력/음력일자 변환(FUBcDateChange.changeSlcl()) 97
1.45 일자 유효성 체크(FUBcDateCheck.checkDateValidity()) 99
1.46 요일산출(FUBcDate.calculateDwk()) 100
1.47 해외정보수신기관 코드별 휴일 체크(FUBcDateCheck.checkOvsesHoldyChk()) 101
1.48 해외정보수신기관 코드별 영업일 산출(FUBcDate.calculateOvsesBzoprDd()) 104
1.49 해외정보수신기관 코드별 월초영업일/ 월말영업일/ 월말일 산출 (FUBcDate.calculateOvsesEnmnBzoprDay()) 106
1.50 일자 변환(년월 몇주차 요일에 맞는 일자반환)(FUBcDateChange.changeDteChng()) 108
1.51 세율기본 조회(FUBcRate.getTaxRates()) 110
1.52 통합 구계좌 변환(N/A) 114

1.53 로직으로 계좌변환(요구불 계좌)(N/A) 117
1.54 합병은행 통합계좌간 변환(N/A) 118
1.55 문자열 길이 변환(FUBcChange.changeLettersLineLength()) 119
1.56 숫자 표기된 금액을 한글/ 영문으로 변환(FUBcChange.changeAmountHanglEng()) . 120
1.57 가변길이 데이터 변환(FUBcChange.changeChangeableData()) 122
1.58 개인정보 보호를 위한 변환(FUBcChange.changePrivatePersonInfo()) 124
1.59 계좌번호 체크디지트 산출 및 검증(FUBcValidation.validationAccountNumber()). 126
1.60 M/S 계좌번호 체크디지트 산출 및 검증
(FUBcValidation.validationMSAccountNumber()) 130
1.61 일반 체크디지트 검증(FUBcValidation.validationCheckDigit()) 132
1.62 관리번호 체크디지트 산출 및 검증(FUBcValidation.validationMgtNo()) 137
1.63 M/S 유효성 검증 (ISO 3 TYPE)(N/A) 140
1.64 국가목록 조회(FUBcCode.getNationalCatalogue()) 141
1.65 은행목록 조회(FUBcCode.getBankCatalogue()) 143
1.66 타행환은행공동코드 조회 (FUBcCode.getOtrbnkCatalogue()) 145
1.67 에러발생시 센터컷 중단 여부, 중요 에러메시지 여부 체크
(FUBcMessage.getMsgCdOfCmnArea()) 147
1.68 거래 파라미터 유효성검증(FUBcValidation.validationTranParm()) 148
1.69 사용자 권한 및 Sales/OP 정보 조회(FUBcEmployee.getEmpidAthInfo()) 149
1.70 고객지정계좌 채번 및 취소 (N/A) 152
1.71 공통출력 데이터 조립(FUBcChange.setSlipShetCommon()) 153
1.72 거래제어 정보 조회(FUBcTransaction.getTranCtrlInfo()) 155
1.73 양편넣기-일수/월수/영업일수/공휴일수 양편넣기 산출
(FUBcDate.calculateNumberOfBsdeDays()) 158
1.74 양편넣기-입력월수 차감후의 일수산출
(FUBcDate.calculateSubtractionNumberOfBsdeDays()) 160
1.75 양편넣기-만기일/역만기일 산출(FUBcDate.calculateBsdeExpirDd()) 162
1.76 이미지 인식 코드(ECC) 암호화 I (FUBcEcc.getEncodeStringCode1()) 164
1.77 이미지 인식 코드(ECC) 암호화 II (FUBcEcc.getEncodeStringCode2()) 171
1.78 이미지 인식 코드(ECC) 복호화 I (FUBcEcc.getDecodeStringCode1()) 178
1.79 이미지 인식 코드(ECC) 복호화 II (FUBcEcc.getDecodeStringCode2()) 180
1.80 책임자승인사유 판단(FUBcAuthorization.prcssSpvsrAthorPtrn()) 182

PoC

1.81 EBCDIC 한글코드(1364) 검증(N/A) 186

1.82 분리된 데이터(TEXT) 결합(N/A) 187

1.83 대외기관에러메시지코드별 표준에러/조치메시지코드 조회 PoC
(FUBcMessage.getTermlEcptnlChnlMsgCd()) 187

1.84 데이터 공백(Space) 제거(N/A) 189

1.85 전표번호 채번-유/무전표 자동판단(N/A) 189

1.86 KSC5601 한글코드 검증(N/A) 191

1.87 월말 년월일 산출 (FUBcDate.calculateEnmnYmd()) 192

1.88 공통영역 기준일자 SET(N/A) 194

1.89 영업일 산출 II (익영업일/익익영업일/전영업일/전전영업일)
(FUBcDate.calculateBzoprDd2()) 195

1.90 일자 산출 II(익일/익익일/전일/전전일) (FUBcDate.calculateDate2()) 197

1.91 거래소휴장일 만기일/역만기일 산출(FUBcDate.calculateExchHoldyExpirDd()).... 199

1.92 거래소휴장일 양편넣기-만기일/역만기일 산출
(FUBcDate.calculateBsdeExchHoldyExpirDd()) 201

1.93 일자 변환 2(요청 차수의 요일에 해당하는 일자 반환)(FUBcDateChange.
changeDteChng2()) 203

1.94 거래소휴장일 체크(FUBcDateCheck.checkExchHoldyChk()) 205

1.95 만기일 산출 II (N/A) 206

1.96 전행 인스턴스코드 존재여부 조회(FUBcCode.getEnbnIncdExstYn()) PoC 207

1.97 전행 인스턴스코드 내용 조회 II(입력일자 없음) (FUBcCode.getEnbnIncdContent2())
209

1.98 VIP 라운지 여부 조회 (N/A) 211

1.99 직원 존재여부 다건 조회(FUBcEmployee.getEmpExstYn()) 212

1.100 직원별 담당어플리케이션코드 및 처리권한 여부 확인
(FUBcEmployee.getEmpPerUapplCdPrcssAthYn()) 214

1.101 전행 인스턴스코드 내용 조회 III (복수건조회-입력일자 없음)
(FUBcCode.getEnbnIncdContent3()) 216

1.102 휴일 Check - 휴일테이블 사용안함 (N/A) 218

1.103 부점한글명 조회(FUBcBranch.getBrnHanglName()) 219

1.104 전행 인스턴스코드 내용 조회 IV(인스턴스내용출력안함)
(FUBcCode.getEnbnIncdContent4()) 221

1.105 신분증이미지 Key 조립(N/A) 223

1.106 실명증표 이미지키 저장 및 조회(N/A) 224
1.107 실명증표 이미지키 복원저장(N/A) 227
1.108 전자공용전표 판단규칙(N/A) 229
1.109 전표구분 조회-유/무전표 자동판단(N/A) 230
1.110 카드인터페이스용 전표채번(N/A) 231
1.111 고객지정계좌 계좌번호 체크디짓 산출 및 검증(N/A) 232
1.112 직원권한 및 종료고객조회(FUBcEmployee.getEmpAthEndCust()) 233
1.113 실버바판매환율정보조회(N/A) 235
1.114 대기업금융센터관리부점리스트조회(FUBcBranch.getCnglFinacCnterMgtt()) 236
1.115 태블릿방문접수일정 조회 (N/A) 238
1.116 대기업금융센터 및 관리부점조회(FUBcBranch.getCnglFinacCnterMgtbrn()) 239
1.117 고객유형체크(FUBcValidation.validationCustomerPattern()) 241
1.118 직원 KB-PIN 정보 조회(FUBcEmployee.getEmpKBPinInfo()) 243
1.119 KB-PIN 조회 및 생성(FUBcEmployee.getKBPin()) 245
1.120 기타대체번호조회 및 생성(FUBcEmployee.getEtcAltrNo()) 247
1.121 내맘대로계좌등록(N/A) 249
1.122 부점운영정보 조회(FUBcBranch.getBranchOperationTime()) 251
1.123 주민번호검색(N/A) 253
1.124 주민번호 존재여부 확인(N/A) 254
1.125 지주직원기본 조회(FUBcEmployee.getGroupEmpInfo()) 255
1.126 부점유형기본 조회(FUBcBranch.getBrnPtrnInfo()) 257
1.127 직원기타정보 조회(FUBcEmployee.getEmpEtcInfo()) 258
1.128 부점영업시간정보 조회(FUBcBranch.getBranchBizOperationTime()) 263
1.129 고유식별정보 검출(FUBcValidation.validationUniqIdnfiInfo()) 265
1.130 지역본부장,부점장,내부통제자 조회(FUBcBranch.getBrnAreahqBrnmgrCpsn()) 268
1.131 기준환율 조회 II (FUBcRate.getBaseExchangeRate2()) 271
1.132 부점점심시간정보 조회 (FUBcBranch.getLnchTtmCntzWrkBrn()) 274
1.133 고유식별정보 검출 II (FUBcValidation.validationUniqIdnfiInfo()) 276

범례)
● : 필수항목
◎ : 선택항목

1. 프로그램별 사용 방법

1.1계좌번호 채번(FUBcNumbering.getAcnoNumbering())

프로그램 ID	FUBcNumbering. getAcnoNumbering()		언어 구분	JAVA(○)	
입출력명	requestData/responData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	계좌번호채번 관리번호채번규칙	SQLMAP 명	NAME SPACE	fubcnumbering
	영문명	THKJICN01 THKJICN04		SQL ID	selectMgtNoNbringRule selectAcnoNbring updateAcnoNbring insertAcnoNbring

프로그램 설명	표준화에서 정의한 계좌성 번호를 채번하는 것으로 12, 13, 14 자리의 계좌를 채번한다.
프로그램 규칙 및 제약사항	1. 채번단위는 요청채번과목코드와 부점코드이다. 2. 폐쇄부점에서는 채번 할수 없고, 활동부점인 경우만 채번이 가능하다. 3. As-Is 에서 고객지정계좌번호로 기채번된 일련번호가 존재시 다음 일련번호로 채번한다. 4. 채번과목코드는 As-Is 에서 사용하는 코드를 계속 사용하며, 일련번호가 최대채번일련번호를 초과 되었할 경우 차기과목코드는 표준화에서 정의해준 채번과목코드로 채번한다. 이때, 기존 채번과목코드로 채번일련번호 초과시 차기과목코드로 채번과목코드가 출력된다. 5. 체크 디지트 산출(FUBcValidation. validationAccountNumber()) 프로그램을 호출하여 체크 디지트를 출력값으로 전달한다. 단,환(031)은 체크디지트를 사용하지 않는다. 6. 계좌번호의 자릿수에 따라 편집(FUBcChange. changeChangeableData()) 프로그램을 호출하여 '-' 포함한 출력계좌변호와 '-' 이 없는 숫자로만 구성된 계좌번호를 출력값으로 전달한다. 가) 12 자리 계좌번호 편집 - 가계당좌예금(06), 당좌예금(18) 부점코드(4 자리) + 채번과목코드(2 자리) + 일련번호(5 자리) + 체크 디지트(1 자리) - 차기과목코드로 변경시 표준화의 확인이 필요하다. 나) 13 자리 계좌번호 편집 - 정기예금(15) 부점코드(4 자리) + 채번과목코드(2 자리) + 일련번호(6 자리) + 체크 디지트(1 자리) - 차기과목코드로 변경시 표준화의 확인이 필요하다. 다) 14 자리 계좌번호 편집 - 가), 나)를 제외한 나머지 과목코드 부점코드(4 자리) + 채번과목코드(2 자리) + 일련번호(7 자리) + 체크 디지트(1 자리) - 계좌번호(700 번대 채번과목코드) 부점코드(4 자리) + 채번과목코드(3 자리) + 일련번호(6 자리) + 체크 디지트(1 자리) - 차기과목코드로 변경시 표준화의 확인이 필요하다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;;`
	
	`public class FUBcNumbering extends com.kbstar.sqc.base.FunctionUnit {`

	[입력항목]
	String inGroupCoCd
	String inDmndNSbjctCd
	String inBrncd
	
	[출력항목]
	String outAcno
	String outEdtAcno
	String outDefinsNSbjctCd
	String outSerno
	String outChkDigit
	}
		● 그룹회사코드
		● 요청채번과목코드
		● 부점코드
		
		// 계좌번호
		// 편집계좌번호
		// 확정채번과목코드
		// 일련번호
		// 체크디지트

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 계좌번호 채번 (com.kbstar.kji.enbncmn, getAcnoNumbering)
	
	IDataSet getAcnoNumberingReq = new DataSet();
	
	getAcnoNumberingReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getAcnoNumberingReq.put("inDmndNSbjctCd", requestData.get("inDmndNSbjctCd"));
	getAcnoNumberingReq.put("inBrncd", requestData.get("inBrncd"));
	
	// @@ 공유메소드 호출 (계좌번호 채번 getAcnoNumbering)
	IDataSet getAcnoNumberingRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcNumbering.getAcnoNumbering", getAcnoNumberingReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outacno = getAcnoNumberingRes.getString("outAcno");
	String outedtacno = getAcnoNumberingRes.getString("outEdtAcno");
	String outdefinsnsbjctcd = getAcnoNumberingRes.getString("outDefinsNSbjctCd");
	String outserno = getAcnoNumberingRes.getString("outSerno");
	String outchkdigit = getAcnoNumberingRes.getString("outChkDigit");	

비 고		

1.2 관리번호 채번(FUBcNumbering.getMgtNoNumbering())

프로그램 ID	FUBcNumbering. getMgtNoNumbering()		언어 구분	JAVA(○)	
입출력명	requestData/responData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	관리번호채번 관리번호채번규칙	SQLMAP 명	NAME SPACE	fubcnumbering
	영문명	THKJICN03, THKJICN04		SQL ID	selectMgtNoNbringRule selectMgtNoNbring updateMgtNoNbring insertMgtNoNbring

프로그램 설명	관리번호 채번을 위하여 채번규칙을 전행공통팀에 등록하고 부여받은 채번식별코드를 이용하여 관리번호를 채번한다.
프로그램 규칙 및 제약사항	1. 부점단위 채번시 폐쇄부점에서는 채번 할수 없다. 활동점인 경우만 채번이 가능하다. 2. 어플리케이션구분코드와 채번식별코드는 필수입력항목이다. 3. 부점단위 채번시 부점코드 또는 외국환은행코드+외국환점번(국제팀 요건)이 입력된다. 4. 관리번호채번규칙 TABLE 의 내용은 아래와 같다. 가) 부점단위채번여부 : 부점단위 채번요건을 반영한다. 나) 년월일처리기준구분코드- 다), 라), 마)의 처리기준년월일이 된다. - ‘1’ :Common Area 일자정보의 시스템년월일 기준으로 채번한다. - ‘2’ :Common Area 일자정보의 거래기준년월일 기준으로 채번한다. - ‘3’ :Common Area 일자정보의 회계기준년월일 기준으로 채번한다. - ‘4’ :Common Area 일자정보의 시스템년월일 기준으로 줄리안 일자로 변환하여 채번한다. - ‘5’ :Common Area 일자정보의 거래기준년월일 기준으로 줄리안 일자로 변환하여 채번한다. - ‘6’ :Common Area 일자정보의 회계기준년월일 기준으로 줄리안 일자로 변환하여 채번한다. 다) 년단위채번여부 : 년단위로 채번되는 요건을 반영한다. 라) 월단위채번여부 : 월단위로 채번되는 요건을 반영한다. 마) 일단위채번여부 : 일단위로 채번되는 요건을 반영한다. 바) 관리최대 채번일련번호 : 최대채번일련번호값을 저장한다. 사) 검증번호산출유형구분코드 : 체크디지트 생성시 사용할 입력 구분코드 - ‘001’: 신용장번호 체크디지트 생성 - ‘002’: 내국신용장 체크디지트 생성 - ‘003’: 기관약정번호 체크디지트 생성 - ‘004’ : 외화대출자금배정 일련번호 체크디지트 - ‘005’ : 통합트레이드번호 체크디지트 - ‘006’ : 부점통합트레이드번호 체크디지트 - ‘007’ : 용지보상채권번호 11 자리 유효자리 정보 - ‘008’ : 기타외화지급보증 / 선박선수보증 아) 관리번호구성내용 : 관리번호의 구성체계를 코드화하여 관리한다. - ‘+F’ : 관리번호구성 중 고정값을 표시한다. - ‘+B’ : 관리번호구성 중 부점코드 위치 값을 표시한다. - ‘+Y’ : 관리번호구성 중 년도의 위치 값을 표시한다. - ‘+M’ : 관리번호구성 중 월의 위치 값을 표시한다. - ‘+D’ : 관리번호구성 중 일자의 위치 값을 표시한다.

	- '+C' : 체크 디지트를 표시한다. - '+N' : 관리번호채번일련번호의 자릿수를 표시한다. - '+J' : Julian 일자의 위치 값을 표시한다. 예) 수출환어음매입추심 관리번호 : N-07Q6-0809-LS-12345 수출환어음매입추심 관리번호 구성내용 : +FN-+B01+B02+B03+B04+F-+Y03+Y04+M01+M02+F-LS-+N05* 자) 작업주기구분코드 : 사용하지 않은 관리번호채번 TABLE 을 삭제하기 위한 주기이다.

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcNumbering **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd ; ● 그룹회사코드
	String inUapplCd; ● 어플리케이션코드
	String inNbringIdnfiCd; ● 채번식별코드
	String inBrncd ; ◎ 부점코드
	
	[출력항목]
	String outMgtNo ; // 관리번호
	String outEdtMgtNo ; // 편집관리번호
	String outNbringSerno ; // 일련번호
	String outChkDigit ; // 체크디지트

RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - 관리번호 채번 (com.kbstar.kji.enbncmn, getMgtNoNumbering)**
	IDataSet getMgtNoNumberingReq = new DataSet();
	
	getMgtNoNumberingReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getMgtNoNumberingReq.put("inUapplCd", requestData.get("inUapplCd"));
	getMgtNoNumberingReq.put("inNbringIdnfiCd", requestData.get("inNbringIdnfiCd"));
	getMgtNoNumberingReq.put("inBrncd", requestData.get("inBrncd"));
	
	// @@ 공유메소드 호출 (관리번호 채번 getMgtNoNumbering)
	IDataSet getMgtNoNumberingRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcNumbering.getMgtNoNumbering", getMgtNoNumberingReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outmgtno = getMgtNoNumberingRes.getString("outMgtNo"); // 관리번호
	String outedtmgtno = getMgtNoNumberingRes.getString("outEdtMgtNo"); // 편집관리번호
	String outnbringserno = getMgtNoNumberingRes.getString("outNbringSerno"); // 일련번호
	String outchkdigit = getMgtNoNumberingRes.getString("outChkDigit"); // 체크디지트

비 고	

1.3 비밀번호사용제한 체크(FUBcValidation.validationPassword())

프로그램 ID	FUBcValidation. validationPassword()		언어 구분	JAVA(○)	
입출력 명	requestData/responData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	법인, 개인, 개인사업자 고객의 비밀번호 설정시 비밀번호로 사용이 가능한지 여부를 체크한다.
프로그램 규칙 및 제약사항	1. 고객고유번호는 고객이 법인인 경우는 사업자등록번호가 되고, 개인인 경우와 개인사업자인 경우는 주민등록번호가 된다. 2. 고객고유번호와 비밀번호는 필수입력항목이고 자택전화번호, 직장전화번호, 휴대폰번호는 선택입력항목이다. 3. 자택전화번호, 직장전화번호, 휴대폰번호는 고객정보 테이블의 내용을 기준으로 입력하여야 하며, 전화국번호(4 자리)와 전화개별번호(4 자리)를 연결하여 8 자리를 파라메터로 전달한다. ** 주의 : 전화번호는 반드는 지역번호를 "제외한" 최대 8 자리의 형대로 입력되어져야하며, SPACE 와 숫자 외의 값은 모두 정당하지 않은 입력값으로 간주함.* 4. 중복 CIF 일때의 비밀번호 제한 검사는 중복 CIF 만큼 호출하여 사용하도록 한다. 5. 비밀번호로 사용하지 못하는 경우는 아래와 같다. 1) 숫자가 아닌 문자가 포함된 4 자리, 6 자리, 7 자리,8 자리 (예시: 12A4, B430, 123B34,123A456, 345K3456 …) 2) 주민등록번호/사업자등록번호 중 4 자리, 6 자리, 7 자리,8 자리를 사용한 경우 (예시: 주민등록번호가 741231-1234567 인 경우 7412, 1234, 4567, 741231, 74123112 …) 3) 동일한 숫자 4 자리, 6 자리, 7 자리,8 자리를 사용한 경우 (예시: 1111, 7777, 222222, 77777777 …) 4) 순방향/역방향의 연속 4 자리, 6 자리, 7 자리, 8 자리 숫자를 사용한 경우 (예시: 1234, 9876, 123456, 12345678 …) 5) 비밀번호가 4 자리인경우 (비밀번호가 6,7,8 자리인(IC 카드)경우는 전화번호와의 체크는 생략한다) ① 자택전화번호 뒷자리 4 자리를 사용한 경우, ② 자택전화번호 국번호 3 자리 또는 4 자리를 사용한 경우 (국번호가 3 자리인 경우는 앞에 '0'을 포함하여 4 자리로 체크한다.) ③ 직장전화번호 뒷자리 4 자리를 사용한 경우 ④ 직장전화번호 국번호 3 자리 또는 4 자리를 사용한 경우 (국번호가 3 자리인 경우는 앞에 '0'을 포함하여 4 자리로 체크한다.) ⑤ 휴대폰번호 뒷자리 4 자리를 사용한 경우 ⑥ 휴대폰번호 국번호 3 자리 또는 4 자리를 사용한 경우 (국번호가 3 자리인 경우는 앞에 '0'을 포함하여 4 자리로 체크한다.) ** 비밀번호의 암.복호화는 보안에서 하며 비밀번호 사용제한 체크를 하기 위해 보안에서 복호화된 비밀번호로 전행공통 모듈을 호출 한다

입출력 설명	**package** com.kbstar.kji.enbncmn.biz; **public class** FUBcValidation **extends** com.kbstar.sqc.base.FunctionUnit **[입력항목]** String inPpsnCoprCuniqno; String inPwd; String inOwhusTelno; String inWplcTelno; String inCphnno; **[출력항목]**	● 개인/법인고객고유번호 ● 비밀번호 ◎ 자택전화번호 ◎ 직장전화번호 ◎ 휴대폰번호
RTNSTAT 설명	N/A	

사용 설명(예시)	**// 공유메소드 호출 - 비밀번호사용제한 체크 (com.kbstar.kji.enbncmn, validationPassword)**
	IDataSet validationPasswordReq = new DataSet();
	
	validationPasswordReq.put("inPpsnCoprCuniqno", requestData.get("inPpsnCoprCuniqno"));
	validationPasswordReq.put("inPwd", requestData.get("inPwd"));
	validationPasswordReq.put("inOwhusTelno", requestData.get("inOwhusTelno"));
	validationPasswordReq.put("inWplcTelno", requestData.get("inWplcTelno"));
	validationPasswordReq.put("inCphnno", requestData.get("inCphnno"));
	
	// @@ 공유메소드 호출 (비밀번호사용제한 체크 validationPassword)
	IDataSet validationPasswordRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcValidation.validationPassword", validationPasswordReq, onlineCtx);	

비 고		

1.4 통화코드 조회(FUBcCode.getCurrencyCode())

프로그램 ID	FUBc Code.getCurrencyCode()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	통화목록	SQLMAP 명	NAME SPACE	fubccode
	영문명	THKJIEX01		SQL ID	selectListCurrencyCode1 selectListCurrencyCode2 selectListCurrencyCode3 selectListCurrencyCode4

프로그램 설명	통화코드 정보를 조회한다.	
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 통화코드일련번호를 입력으로 하여 통화코드를 조회한다. 2) '2' : 통화코드를 입력으로 하여 통화순번을 조회한다. 3) '3' : 당행에서 지정한 46 개의 통화정보를 제공한다. 4) '4' : 전체 통화정보를 제공한다. 2. 구분코드별 입력항목을 아래와 같다. 1) 구분코드가 '1'인 경우는 통화코드일련번호를 필수입력하여야 한다. 2) 구분코드가 '2'인 경우는 통화코드를 필수입력하여야 한다. 3) 구분코드가 '3', '4'인 경우는 구분코드값만 입력한다. 3. 당행에서 지정한 46 개의 조회내용은 조회순서일련번호를 기준으로 오름차순정렬로 보여준다. 4. 전체 통화에 대한 조회내용은 통화코드을 기준으로 보여준다.	

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcCode **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd;
	String inDstcd;
	String inCncycdSerno;
	String inCncycd;
	String inCrncyISOcd;
	
	[출력항목]
	int outArayCnt;
	List LIST;
	
	< LIST 항목 >
	String outCncycd;
	String outUseYn;
	String outCrncyIsocd;
	String outIsoNatnlCd;
	String outNatnlHanglName;
	String outDesigRNatnlDstcd;
	String outAnounCrncyDstcd;
	String outCncycdSerno;
		● 그룹회사코드
		● 구분코드
		◎ 통화코드일련번호
		◎ 통화코드
		◎ 통화ISO코드
		
		// 배열개수
		// 통화코드List
		
		// 통화코드
		// 사용여부
		// 통화 ISO 코드
		// ISO국가코드
		// 국가한글명
		// 지정영수국가구분코드
		// 고시통화구분코드

	String outN100UnitCrncyYn;
	String outCncycdHanglName;
	String outCncycdEngName;
	String outEuroCrncyUseYn;
	int outDecptNocip;
	String outUpdatYmd;
	int outInquirySeqSerno;
		// 통화코드일련번호
		// N100 단위통화여부
		// 통화코드한글명
		// 통화코드영문명
		// 유로통화사용여부
		// 소수점자리수
		// 갱신년월일
		// 조회순서일련번호

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 통화코드 조회 (com.kbstar.kji.enbncmn, getCurrencyCode)
	
	IDataSet getCurrencyCodeReq = new DataSet();
	
	getCurrencyCodeReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getCurrencyCodeReq.put("inDstcd", requestData.get("inDstcd"));
	getCurrencyCodeReq.put("inCncycdSerno", requestData.get("inCncycdSerno"));
	getCurrencyCodeReq.put("inCncycd", requestData.get("inCncycd"));
	getCurrencyCodeReq.put("inCrncyISOcd", requestData.get("inCrncyISOcd"));
	
	// @@ 공유메소드 호출 (통화코드 조회 getCurrencyCode)
	IDataSet getCurrencyCodeRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getCurrencyCode", getCurrencyCodeReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outaraycnt = getCurrencyCodeRes.getString("outArayCnt");
	IRecordSet getCurrencyCodeResRsList = getCurrencyCodeRes.getRecordSet("LIST");	

비 고		

1.5

PoC

PoC

PoC

기준환율 조회(FUBcRate.getBaseExchangeRate())

프로그램 ID	FUBcRate.getBaseExchangeRate()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	기준환율	SQLMAP 명	NAME SPACE	fubcrate
	영문명	THKJIEX04		SQL ID	selectBaseMaxExreturnNth selectBaseExchangeRate1 selectBaseExchangeRate2 selectBaseExchangeRate3

프로그램 설명	자본시장업무(CMBS)에서 전송되는 기준환율정보를 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 지정통화에 대한 기준환율 정보를 조회한다. 통화코드를 필수항목으로 입력하여야 한다. 2) '2' : 전체통화에 대한 기준환율 정보를 조회한다.(유로관련 통화제외) 3) '3' : 전체통화에 대한 기준환율 정보를 조회한다.(유로관련 통화포함) 2. 출력구분코드는 아래와 같다. (전체통화에 대한 기준환율 정보 조회시에 필수입력한다.) 1) '1' : 통화코드일련번호순으로 오름차순 정렬하여 조회한다. 2) '2' : 조회순서일련번호순으로 오름차순 정렬하여 조회한다. 3. 대상일자는 기준환율 적용시작일을 기준으로 한다. 대상일자 미입력시 거래기준년월일을 기준으로 조회한다. 4. 입력고시회차 미입력시 최종회차로 조회한다. 1) MAX 회차가 0 인(미고시)경우 리턴상태를 '07'로 한다. 5. 1 회차 환율,고시여부를 체크 1) 1 회차 환율, 고시여부를 체크하려면 환율회차에 1 을 입력하여 조회하고, 조회 데이터가 없으면 리턴상태코드를 '07'로 한다. 6. 환율고시전에 기준년월일에 대한 1 회차 환율데이터가 없을 경우에, 전일자로 환율을 조회하지 않고 리턴상태코드를 '07'로 한다. 업무가 판단하여 환율고시전의 환율이 필요한 경우에 전일자로 조회하여 사용한다. 7. 100 건 초과하여 환율건수가 있을 경우 오류처리 한다. 8. 2009.09.08 현재 일본,인도네시아,이태리,이탈리아,스페인,베트남 6 개통화는 100 단로 관리되며 일단위 자료가 필요한 업무는 N100 단위통화여부(XCJIEX01-O-CNCYCD)에 '1' 출력 값을 확인 하여 6 개 통화에 대해서는 일단위로 변환하여 사용 한다. 9. 정상인경우 1 회차매매기준환율과 등록시각을 항상 제공한다.

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcRate **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; // ● 그룹회사코드
	String inDstcd; // ● 구분코드
	String inOutptDstcd; // ◎ 출력구분코드
	String inBaseYmd; // ◎ 기준년월일
	String inCncycd; // ◎ 통화코드
	int inExrtAplyNth; // ◎ 환율적용회차
	
	[출력항목]
	String rtnStat; // 리턴상태코드

	String outBaseYmd;
	int outExrtAplyNth;
	int outArayCnt;
	List outBaseExrtList;
	
	< outBaseExrtList 항목 >
	String outCncycd;
	BigDecimal outCrrtz;
	BigDecimal outDealBaseExrt;
	BigDecimal outUsdRt;
	BigDecimal outTelchkBuyExrt;
	BigDecimal outTelChkSelExrt;
	BigDecimal outCshBuyExrt;
	BigDecimal outCshSellExrt;
	BigDecimal outTcSellExret;
	String outCrncyIsocd;
	String outNatnlCd;
	String outNatnlHanglName;
	String outN100UnitCrncyYn;
	String outCncycdHanglName;
	String outCncycdEngName;
	String outRegiFnshHms;
	BigDecimal outN1tmTdBaseExrt;
	PoC
		// 기준년월일
		// 환율적용회차
		// 배열개수
		// 기준환율List
		
		// 통화코드
		// CROSSRATE 율
		// 매매기준환율
		// 대미환산율
		// 전신환매입환율
		// 전신환매도환율
		// 현찰매입환율
		// 현찰매도환율
		// TC 매도환율
		// 통화ISO코드
		// 국가코드
		// 국가한글명
		// N100단위통화여부
		// 통화코드한글명
		// 통화코드영문명
		// 등록완료시각
		// 1회차매매기준환율

RTNSTAT 설명	00 : 정상, 07 : NOTFOUND	

사용 설명(예시)	// 공유메소드 호출 - 기준환율 조회 (com.kbstar.kji.enbncmn, getBaseExchangeRate)
	PoC
	IDataSet getBaseExchangeRateReq = new DataSet();
	
	getBaseExchangeRateReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getBaseExchangeRateReq.put("inDstcd", requestData.get("inDstcd"));
	getBaseExchangeRateReq.put("inOutptDstcd", requestData.get("inOutptDstcd"));
	getBaseExchangeRateReq.put("inBaseYmd", requestData.get("inBaseYmd"));
	getBaseExchangeRateReq.put("inCncycd", requestData.get("inCncycd"));
	getBaseExchangeRateReq.put("inExrtAplyNth", requestData.get("inExrtAplyNth"));
	
	// @@ 공유메소드 호출 (기준환율 조회 getBaseExchangeRate)
	IDataSet getBaseExchangeRateRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcRate.getBaseExchangeRate", getBaseExchangeRateReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	PoC
	PoC
	String rtnstat = getBaseExchangeRateRes.getString("rtnStat");
	String outbaseymd = getBaseExchangeRateRes.getString("outBaseYmd");
	PoC
	int outexrtaplynth = getBaseExchangeRateRes.getInt("outExrtAplyNth");
	int outaraycnt = getBaseExchangeRateRes.getInt("outArayCnt");
	PoC
	IRecordSet getBaseExchangeRateResRsOutbaseexrtlist =
	getBaseExchangeRateRes.getRecordSet("outBaseExrtList");	

비 고		

PoC

1.6

PoC

PoC

PoC

PoC

PoC

시장환율 조회(FUBcRate.getMarketExchangeRate())

프로그램 ID	FUBcRate. getMarketExchangeRate()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	시장연동환율	SQLMAP 명	NAME SPACE	fubcrate
	영문명	THKJIEX05		SQL ID	selectMaktMaxExreturnNth selectMarketExchangeRate1 selectMarketExchangeRate2

PoC

프로그램 설명	외환시장에서 거래되고 있는 환율정보를 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 지정통화에 대한 시장환율 정보를 조회한다. 통화코드를 필수항목으로 입력하여야 한다. 2) '2' : 전체통화에 대한 시장환율 정보를 조회한다. 2. 대상일자는 시장환율 적용시작일을 기준으로 한다. 대상일자 미입력시 거래기준년월일을 기준으로 조회한다. 3. 입력고시회차 미입력시 최종회차로 조회한다. 1) MAX 회차가 0 인경우 리턴상태를 '07'로 한다. 4. 1 회차 환율,고시여부를 체크 1) 1 회차 환율, 고시여부를 체크하려면 환율회차에 1 을 입력하여 조회하고, 조회 데이터가 없으면 리턴상태코드를 '07'로 한다. 5. 구분코드 '2' 전체통화에 대한 시장환율 정보 조회는 통화일련번호순으로 오름차순 정렬하여 조회한다. 6. 환율고시전 기준년월일에 대한 환율조회는 전일자로 환율을 조회하지 않고 '07'로 한다. 환율고시전 환율조회가 필요한 경우 전일자로 환율조회하여 사용한다.

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcRate extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inDstcd; ● 구분코드
	String inBaseYmd; ◎ 기준년월일
	String inCncycd; ◎ 통화코드
	int inExrtAplyNth; ◎ 환율적용회차
	
	[출력항목]
	String rtnStat; // 리턴상태코드
	String outBaseYmd; // 기준년월일
	int outExrtAplyNth; // 환율적용회차
	int outArayCnt; // 배열건수
	List outMaktExrtList; // 시장환율List
	
	< outMaktExrtList 항목 >
	String outCncycd; // 통화코드
	BigDecimal outBuyPcstExrt; // 매입원가환율
	BigDecimal outSellPcstExrt; // 매도원가환율

PoC

	BigDecimal outBuyPosiExrt; // 매입포지션율
	BigDecimal outSellPosiExrt; // 매도포지션율
	PoC

PoC RTNSTAT 설명	00 : 정상, 07 : 1 회차 환율 고시 미완료,

사용 설명(예시)	**// 공유메소드 호출 - 시장환율 조회 (com.kbstar.kji.enbncmn, getMarketExchangeRate)**
	IDataSet getMarketExchangeRateReq = new DataSet();
	getMarketExchangeRateReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getMarketExchangeRateReq.put("inDstcd", requestData.get("inDstcd"));
	getMarketExchangeRateReq.put("inBaseYmd", requestData.get("inBaseYmd"));
	getMarketExchangeRateReq.put("inCncycd", requestData.get("inCncycd"));
	getMarketExchangeRateReq.put("inExrtAplyNth", requestData.get("inExrtAplyNth"));
	// @@ 공유메소드 호출 (시장환율 조회 getMarketExchangeRate)
	PoC
	IDataSet getMarketExchangeRateRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcRate.getMarketExchangeRate", getMarketExchangeRateReq, onlineCtx);
	// @@ 출력 데이터 셋 처리
	String rtnstat = getMarketExchangeRateRes.getString("rtnStat"); // 리턴상태코드
	String outbaseymd = getMarketExchangeRateRes.getString("outBaseYmd"); // 기준년월일
	int outexrtaplynth = getMarketExchangeRateRes.getInt("outExrtAplyNth"); // 환율적용회차
	int outaraycnt = getMarketExchangeRateRes.getInt("outArayCnt"); // 배열개수
	IRecordSet getMarketExchangeRateResRsOutmaktexrtlist =
	getMarketExchangeRateRes.getRecordSet("outMaktExrtList"); // 시장환율 List
	PoC

비 고	

1.7

PoC

PoC

평균기준환율 조회(FUBcRate.getAvgExchangeRate())

프로그램 ID	FUBcRate.getAvgExchangeRate()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	기준환율	SQLMAP 명	NAME SPACE	fubcrate
	영문명	THKJIEX04		SQL ID	selectDealBaseExrt

프로그램 설명	기준시작년월일부터 기준종료년월일까지 1 회차 매매기준율의 평균을 구한다.
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같다. 1) '1' - 특정통화의 1 회차 매매기준환율의 평균을 구한다. 2. 기준시작년월일과 기준종료년월일 사이에 등록된 기준환율 1 회차의 레코드 수만큼의 평균을 계산한다. 3. 기준시작년월일과 기준종료년월일의 영업일수와 평균대상건수가 같으면 리턴상태코드는 정상('00')이고, 같지 않으면 '07'을 전달한다. 4. 평균 대상건수가 '0'인 경우 입력된 기준시작년월일의 전영업일에 해당되는 1 회차 매매기준 환율을 제공한다

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcRate **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inDstcd; ● 구분코드
	String inCncycd; ● 통화코드
	String inBaseStartYmDd; ● 기준시작년월일
	String inBaseEndYmDd; ● 기준종료년월일
	
	[출력항목]
	String rtnStat // 리턴상태코드
	BigDecimal outDealBaseExrt; // 평균매매기준환율
	int outBzoprNoday; // 영업일수
	int outAvgTagetNoitm; // 평균대상건수
	BigDecimal outAmndAdBaseExrt; // 수정평균매매기준율
	String outN100UnitCrncyYn; // N100 단위통화여부

RTNSTAT 설명	00 : 정상, 07 : 영업일수와 평균대상건수 상이

사용 설명(예시)	**// 공유메소드 호출 평균기준환율 조회 (com.kbstar.kji.enbncmn, getAvgExchangeRate)**
	
	IDataSet getAvgExchangeRateReq = new DataSet();
	
	getAvgExchangeRateReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getAvgExchangeRateReq.put("inDstcd", requestData.get("inDstcd"));
	getAvgExchangeRateReq.put("inCncycd", requestData.get("inCncycd"));
	getAvgExchangeRateReq.put("inBaseStartYmDd", requestData.get("inBaseStartYmDd"));
	getAvgExchangeRateReq.put("inBaseEndYmDd", requestData.get("inBaseEndYmDd"));
	PoC

	// @@ 공유메소드 호출 (평균기준환율 조회 getAvgExchangeRate)
	IDataSet getAvgExchangeRateRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcRate.getAvgExchangeRate", getAvgExchangeRateReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getAvgExchangeRateRes.getString("rtnStat");
	BigDecimal outdealbaseexrt = getAvgExchangeRateRes.getBigDecimal("outDealBaseExrt");
	int outbzoprnoday = getAvgExchangeRateRes.getInt("outBzoprNoday");
	int outavgtagetnoitm = getAvgExchangeRateRes.getInt("outAvgTagetNoitm");
	BigDecimal outamndadbaseexrt =getAvgExchangeRateRes.getBigDecimal("outAmndAdBaseExrt");
	String outn100unitcrncyyn = getAvgExchangeRateRes.getString("outN100UnitCrncyYn");

비 고	PoC

1.8

PoC

PoC

PoC

PoC

PoC

This document is strictly limited to the 2025 C2J conversation PoC

상품선물환율 조회(FUBcRate.getPrdctExchangeRate())

프로그램 ID	FUBc Rate.getPrdctExchangeRate()		언어 구분	JAVA(○)	
입출력 명	PrdctExrtBDTO		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	상품선물환율	SQLMAP 명	NAME SPACE	fubcrate
	영문명	THKJIEX06		SQL ID	selectPrdctMaxAplyNth selectListPrdctExchangeRate1 selectListPrdctExchangeRate2

프로그램 설명	대고객 거래에 사용되는 상품선물환율정보를 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 지정 상품선물구분에 대한 상품선물환율 정보를 조회한다. 상품선물구분은 필수항목으로 입력하여야 한다. 2) '2' : 전체 상품선물구분에 대한 상품선물환율 정보를 조회한다. 2. 대상일자는 기준환율 적용시작일을 기준으로 한다. 대상일자 미입력시 거래기준년월일을 기준으로 조회한다. 3. 입력적용회차 미입력시 최종적용회차로 조회한다. 1) MAX 회차가 0 인경우 리턴상태를 '07'로 한다.
입출력 설명	

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcRate extends
                com.kbstar.sqc.base.FunctionUnit

[입력항목]
  String inGroupCoCd;                  ● 그룹회사코드
  String inDstcd;                      ● 구분코드
  String inPrdctFtrsDstcd;             ◎ 상품선물구분코드
  String inBaseYmd;                    ◎ 기준년월일
  int inAplyNth;                       ◎ 적용회차

[출력항목]
  String rtnStat;                      // 리턴상태코드
  String outBaseYmd;                   // 기준년월일
  int outAplyNth;                      // 적용회차
  int outRegiHms;                      // 등록시각
  BigDecimal outPrdctFdBaseExrt;       // 상품선물매매기준환율
  BigDecimal outPrdctFtrsBuyPrc;       // 상품선물매입가격
  BigDecimal outPrdctFtrsSellPrc;      // 상품선물매도가격
  BigDecimal outOuncePerUsdExrt;       // 온스당 USD 환율
  BigDecimal outPrdctFAplyExrt;        // 상품선물적용환율

< outPrdctExrtList 항목 >
  String outRegiHms;                   // 등록시각
  BigDecimal outPrdctFdBaseExrt;       // 상품선물매매기준환율
  BigDecimal outPrdctFtrsBuyPrc;       // 상품선물매입가격
  BigDecimal outPrdctFtrsSellPrc;      // 상품선물매도가격
  BigDecimal outOuncePerUsdExrt;       // 온스당 USD 환율
  BigDecimal outPrdctFAplyExrt;        // 상품선물적용환율
```

RTNSTAT 설명	00 : 정상, 07 : 1회차 환율 고시 미완료

사용 설명(예시)	// 공유메소드 호출 - 상품선물환율 조회 (com.kbstar.kji.enbncmn, getPrdctExchangeRate)
	IDataSet getPrdctExchangeRateReq = new DataSet();
	
	getPrdctExchangeRateReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getPrdctExchangeRateReq.put("inDstcd", requestData.get("inDstcd"));
	getPrdctExchangeRateReq.put("inPrdctFtrsDstcd", requestData.get("inPrdctFtrsDstcd"));
	getPrdctExchangeRateReq.put("inBaseYmd", requestData.get("inBaseYmd"));
	getPrdctExchangeRateReq.put("inAplyNth", requestData.get("inAplyNth"));
	
	// @@ 공유메소드 호출 (상품선물환율 조회 getPrdctExchangeRate)
	IDataSet getPrdctExchangeRateRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcRate.getPrdctExchangeRate", getPrdctExchangeRateReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getPrdctExchangeRateRes.getString("rtnStat"); // 리턴상태코드
	String outbaseymd = getPrdctExchangeRateRes.getString("outBaseYmd"); // 기준년월일
	int outaplynth = getPrdctExchangeRateRes.getInt("outAplyNth"); // 적용회차
	String outregihms = getPrdctExchangeRateRes.getString("outRegiHms"); // 등록시각
	BigDecimal outprdctfdbaseexrt =
	getPrdctExchangeRateRes.getBigDecimal("outPrdctFdBaseExrt"); // 상품선물매매기준환율
	BigDecimal outprdctftrsbuyprc =
	getPrdctExchangeRateRes.getBigDecimal("outPrdctFtrsBuyPrc"); // 상품선물매입가격
	BigDecimal outprdctftrssellprc =
	getPrdctExchangeRateRes.getBigDecimal("outPrdctFtrsSellPrc"); // 상품선물매도가격
	BigDecimal outounceperusdexrt =
	getPrdctExchangeRateRes.getBigDecimal("outOuncePerUsdExrt"); // 온스당USD환율
	BigDecimal outprdctfaplyexrt =
	getPrdctExchangeRateRes.getBigDecimal("outPrdctFAplyExrt"); // 상품선물적용환율

비 고	00 : 정상, 07 : NOTFOUND

This document is strictly limited to the 2025 C2J conversation PoC

1.9 어음교환결제시각 조회(N/A)

프로그램 ID	N/A		언어 구분	N/A	
DTO 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	타점권결제연장에 대한 정보(타점권결제시각, 미결제어음통보시각, 부도어음통보시각, 등록부점코드, 타점연장사유구분)를 조회한다
프로그램 규칙 및 제약사항	1. 어음교환결제시각 조회는 자행입금분과 타행입금분으로 구분 한다. 2. 구분코드는 아래와 같다. 1) '1' : 자행입금분에 대한 어음교환결제시각 조회 2) '2' : 타행입금분에 대한 어음교환결제시각 조회 3. 구분코드별로 필수 입력항목이 달라진다. 1) '1' : 구분코드, 자기앞수표기타어음구분코드, 부점코드. (입력된 은행코드 와 교환소코드는 무시 하며 은행코드는 '004'를 default로 사용 한다) 2) '2' : 구분코드, 자기앞수표기타어음구분코드, 은행코드, 교환소코드 (입력된 부점코드는 무시 한다). 4. 자기앞수표기타어음구분코드는 아래와 같다. 1) '1' : 자기앞수표 2) '2' : 기타어음 5. 자행입금분에 대한 어음교환결제시각 조회시 부점이 폐쇄점인경우 통폐합점부점코드를 조회하여 자행입금분에 대한 어음교환결제시각을 조회한다.
비 고	

1.10

PoC

부점기본 조회(FUBcBranch.getBranchBasic())

프로그램 ID	FUBcBranch.getBranchBasic()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본 PoC	SQLMAP 명	NAME SPACE	fubcbranch
PoC	영문명	THKJIBR01		SQL ID	selectBranchBasic selectClsrBranchBasic

프로그램 설명	조회 요청일자 시점의 부점기본 정보를 조회한다
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 부점코드는 숫자로 입력되어야 한다. - 부점코드를 '0000' 입력하면 오류를 발생한다. 2. 일자에 대한 부점 기본 정보 조회 1) 년월일 입력 : 입력된 일자 당시의 부점 정보 2) 년월일 미입력 : 현재시점의 부점정보 조회 - 현재시점 : Common Area 에 포함되어있는 KB 기준정보의 거래처리기준년월일 3) 적용시작일자 및 적용종료일자 기준으로 조회 된다 (단 개점년월일이 적용시작일자보다 클 수 있고, 폐쇄년월일이 적용종료일자보다 +1 이므로 반드시 부점기본 정보에 포함되어 있는 개점년월일과 폐쇄년월일을 확인 해야 한다.) 3. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일은 01 ~ 31 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다. 4. 입력된 정보를 기준으로 부점기본 테이블을 조회하여 찾지 못하면 폐쇄부점인지 확인한다. **5. 폐쇄부점일경우 리턴상태코드를 '07' 로 한다.** - 폐쇄부점의 최종 통합부점을 조회하려면 부점 폐쇄에 의한 통합점 조회 (FUBcBranch.getIntegrationBranch())를 한다. - 폐쇄부점도 부점기본 정보와 동일하게 출력한다.

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcBranch extends
            com.kbstar.sqc.base.FunctionUnit

[입력항목]
  String inGroupCoCd ;          ● 그룹회사코드
  String inBrncd     ;          ● 부점코드
  String inYmd       ;          ◎ 년월일

[출력항목]
  String rtnStat            ;   // 리턴상태코드
  String outBrncd           ;   // 부점코드
  String outAplyStartYmd    ;   // 적용시작년월일
  String outAplyEndYmd      ;   // 적용종료년월일
  String outOpnnYmd         ;   // 개점년월일
  String outClsrYmd         ;   // 폐쇄년월일
  String outBnkCd           ;   // 은행코드
  String outBrnHanglName    ;   // 부점한글명
  String outBrnHAbrvnName   ;   // 부점한글약어명
                                // 부점영문명
```

PoC

```
String outBrnEngName      ;   // 우편번호
String outZip             ;   // 한글주소
String outHanglAddr       ;   // 영문주소
String outEngAddr         ;   // 부점구분
String outBrnDstcd        ;   // 출장소모점부점코드
String outSubrnPabrnBrncd ;   // 부점전화번호
String outBrnTelno        ;   // ARS전화번호
String outArsTelno        ;   // 팩스번호
String outFaxNo           ;   // 팩스번호용도내용
String outFaxNoUsagCtnt   ;   // 팩스번호1
String outFaxNo1          ;   // 팩스번호용도내용1
String outFaxNoUsagCtnt1  ;   // 사업자번호
String outBzno            ;   // 관할세무서명
String outJrsdTxofcName   ;   // 회계발생구분
String outAcctgOccurDstcd ;   // 게정처리부점코드
String outAcitmPrcssBrncd ;   // 영업지원본부부점코드
String outBzoprSportHQBrncd;  // 사업본부부점코드
String outBizHqBrncd      ;   // 어음교환소모점코드
String outBlclrhPabrnCd   ;   // 어음교환소코드
String outBlclrhCd        ;   // 국고결제모점코드
String outNtresrPPabrnCd  ;   // 한국은행부점구분코드
String outBokBrnDstcd     ;   // 공금예금모점코드
String outPbmnyDPabrnCd ;     // 은행외환구분코드
String outBnkFxDstcd      ;   // 외환부점코드
String outFxzBrncd         ;  // 외환부점구분코드
String outFxBrnDstcd      ;   // 외환취급부점구분코드
String outFxHndlnBrnDstcd ;   // 외환개시년월일
String outFxOpenigYmd     ;   // 외환종료년월일
String outFxEndYmd        ;   // 은행외환모점코드
String outBnkFxPabrncd    ;   // 외환모점외환부점코드
String outFxPabrnFxBrncd  ;   // 통합부점코드
String outIntgraBrncd     ;   // 수표지급지명
String outChekPaymtNmopc  ;   // 대출싱행킴부점코드
String outLnExeTeamBrncd  ;   // 원본서류보관부점코드
String outOrcpyDKeepBrncd ;   // 프로세스센터부점코드
String outPrcesCnterBrncd ;   // 내부통제자직원번호
String outIntnlCPsnEmpid ;    // 내부통제자휴대폰번호
String outIntnlCPsnCphnNo ;   // 부점장직원번호
String outBrnmgrEmpid     ;   // 부점장휴대폰번호
String outBrnmgrCphnNo    ;   // 부점항목변경내용
String outBrnItemModfiCtnt;   // 부점이력내용
String outBrnPshistCtnt     ; // 점포유형구분
String outStorPtrnDstcd   ;   // 부점장CPNO변경년월일
String outBrnmgrCphnModfiYmd      ;   // 내통자CPNO변경년월일
String outIntnlCtrlPsnCphnModfiYmd;   // 부점지주회사구분
String outSthlCoDstcd             ;   // 주소표시구분
String outAddrRepsntDstcd          ;  // 확장부점한글명
```

PoC

PoC

PoC

PoC

PoC

PoC

	String outExpanBrnHanglName ;	// 이메일주소
	String outEmad ;	// 거점점포부점코드
	String outFthStorBrncd ;	
RTNSTAT 설명	00 : 정상, 07 : 입력:부점이 폐쇄점인 경우	

사용 설명 (예시)	**// 공유메소드 호출 - 부점기본 조회 (com.kbstar.kji.enbncmn, getBranchBasic)**
	
	IDataSet getBranchBasicReq = new DataSet();
	
	getBranchBasicReq.put("inGroupCoCd", requestData.get("inGroupCoCd")); // 그룹회사코드
	getBranchBasicReq.put("inBrncd", requestData.get("inBrncd")); // 부점코드
	getBranchBasicReq.put("inYmd", requestData.get("inYmd")); // 기준년월일
	
	// @@ 공유메소드 호출 (부점기본 조회 getBranchBasic)
	IDataSet getBranchBasicRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getBranchBasic", getBranchBasicReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getBranchBasicRes.getString("rtnStat"); // 처리상태
	String outbrncd = getBranchBasicRes.getString("outBrncd"); // 부점코드
	String outaplystartymd = getBranchBasicRes.getString("outAplyStartYmd"); // 적용시작년월일
	String outaplyendymd = getBranchBasicRes.getString("outAplyEndYmd"); // 적용종료년월일
	String outopnnymd = getBranchBasicRes.getString("outOpnnYmd"); // 개점년월일
	String outclsrymd = getBranchBasicRes.getString("outClsrYmd"); // 폐쇄년월일
	String outbnkcd = getBranchBasicRes.getString("outBnkCd"); // 은행코드
	String outbrnhanglname = getBranchBasicRes.getString("outBrnHanglName");//부점한글명
	String outbrnhabrvnname = getBranchBasicRes.getString("outBrnHAbrvnName");//부점한글약어명
	String outbrnengname = getBranchBasicRes.getString("outBrnEngName"); // 부점영문명
	String outzip = getBranchBasicRes.getString("outZip"); // 우편번호
	String outhangladdr = getBranchBasicRes.getString("outHanglAddr"); // 한글주소
	String outengaddr = getBranchBasicRes.getString("outEngAddr"); // 영문주소
	String outbrndstcd = getBranchBasicRes.getString("outBrnDstcd"); // 부점구분
	String outsubrnpabrnbrncd = getBranchBasicRes.getString("outSubrnPabrnBrncd");//출장소모점
	String outbrntelno = getBranchBasicRes.getString("outBrnTelno"); // 부점전화번호
	String outarstelno = getBranchBasicRes.getString("outArsTelno"); // ARS 전화번호
	String outfaxno = getBranchBasicRes.getString("outFaxNo"); // 팩스번호
	String outfaxnousagctnt = getBranchBasicRes.getString("outFaxNoUsagCtnt"); //팩스용도내용
	String outfaxno1 = getBranchBasicRes.getString("outFaxNo1"); // 팩스번호 1
	String outfaxnousagctnt1 = getBranchBasicRes.getString("outFaxNoUsagCtnt1");//팩스용도내용 1
	String outbzno = getBranchBasicRes.getString("outBzno"); // 사업자번호
	String outjrsdtxofcname = getBranchBasicRes.getString("outJrsdTxofcName"); // 관할세무서명
	String outacctgoccurdstcd = getBranchBasicRes.getString("outAcctgOccurDstcd");//회계발생구
	String outacitmprcssbrncd = getBranchBasicRes.getString("outAcitmPrcssBrncd"); // 계정처리 부점코드
	String outbzoprsporthqbrncd = getBranchBasicRes.getString("outBzoprSportHqBrncd"); // 영업 지원본부부점코드
	String outbizhqbrncd = getBranchBasicRes.getString("outBizHqBrncd"); // 사업본부부점코드
	String outblclrhpabrncd = getBranchBasicRes.getString("outBlclrhPabrnCd"); // 어음교환소모 점코드
	String outblclrhcd = getBranchBasicRes.getString("outBlclrhCd"); // 어음교환소코드
	String outntresrppabrncd = getBranchBasicRes.getString("outNtresrPPabrnCd"); // 국고결제모 점코드
	String outbokbrndstcd = getBranchBasicRes.getString("outBokBrnDstcd"); // 한은부점구분
	String outpbmnydpabrncd = getBranchBasicRes.getString("outPbmnyDPabrnCd"); // 공금예금모점 코드
	String outbnkfxdstcd = getBranchBasicRes.getString("outBnkFxDstcd"); // 은행외환구분
	String outfxzbrncd = getBranchBasicRes.getString("outFxzBrncd"); // 외환부점코드
	String outfxbrndstcd = getBranchBasicRes.getString("outFxBrnDstcd"); // 외환부점구분
	String outfxhndlnbrndstcd = getBranchBasicRes.getString("outFxHndlnBrnDstcd"); // 외환취급 부점구분
	String outfxopenigymd = getBranchBasicRes.getString("outFxOpenigYmd"); // 외환개시년월일	

	String outfxendymd = getBranchBasicRes.getString("outFxEndYmd"); // 외환종료년월일
	String outbnkfxpabrncd = getBranchBasicRes.getString("outBnkFxPabrncd");//은행외환모점코드
	String outfxpabrnfxbrncd = getBranchBasicRes.getString("outFxPabrnFxBrncd"); // 외환모점외환부점코드
	String outintgrabrncd = getBranchBasicRes.getString("outIntgraBrncd"); // 통합부점코드
	String outchekpaymtnmopc = getBranchBasicRes.getString("outChekPaymtNmopc");//수표지급지명
	String outlnexeteambrncd = getBranchBasicRes.getString("outLnExeTeamBrncd"); // 대출싱행팀부점코드
	String outorcpydkeepbrncd = getBranchBasicRes.getString("outOrcpyDKeepBrncd"); // 원본서류보관부점코드
	String outprcescnterbrncd = getBranchBasicRes.getString("outPrcesCnterBrncd"); // 프로세스센터부점코드
	String outintnlcpsnempid = getBranchBasicRes.getString("outIntnlCPsnEmpid"); // 내부통제자직원번호
	String outintnlcpsncphnno = getBranchBasicRes.getString("outIntnlCPsnCphnNo"); // 내부통제자휴대폰번호
	String outbrnmgrempid = getBranchBasicRes.getString("outBrnmgrEmpid"); // 부점장직원번호
	String outbrnmgrcphnno = getBranchBasicRes.getString("outBrnmgrCphnNo");//부점장휴대폰번호
	String outbrnitemmodfictnt = getBranchBasicRes.getString("outBrnItemModfiCtnt"); // 부점항목변경내용
	String outbrnpshistctnt = getBranchBasicRes.getString("outBrnPshistCtnt"); // 부점이력내용
	String outstorptrndstcd = getBranchBasicRes.getString("outStorPtrnDstcd"); // 점포유형구분
	String outbrnmgrcphnmodfiymd = getBranchBasicRes.getString("outBrnmgrCphnModfiYmd"); // 부점장 CPNO 변경년월일
	String outintnlctrlpsncphnmodfiymd = getBranchBasicRes.getString("outIntnlCtrlPsnCphnModfiYmd"); // 내통자 CPNO 변경년월일
	String outsthlcodstcd = getBranchBasicRes.getString("outSthlCoDstcd"); // 부점지주회사구분
	String outaddrrepsntdstcd = getBranchBasicRes.getString("outAddrRepsntDstcd"); // 주소표시구분
	String outexpanbrnhanglname = getBranchBasicRes.getString("outExpanBrnHanglName"); // 확장부점한글명
	String outemad = getBranchBasicRes.getString("outEmad"); // 이메일주소
	String outfthstorbrncd = getBranchBasicRes.getString("outFthStorBrncd");//거점점포부점코드

비 고	

1.11

PoC

PoC

PoC

외국환부점변환 조회(FUBcBranch.getFXBrnChng())

프로그램 ID	FUBcBranch.getFXBrnChng()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본	SQLMAP 명 PoC	NAME SPACE	fubcbranch
	영문명	THKJIBR01		SQL ID	selectFXBrnChng selectClsrFXBrnChng

프로그램 설명	현재 시점의 입력 외국환부점의 기본 정보를 조회한다. 폐쇄부점일 경우 해당 폐쇄부점 정보를 조회하고 리턴상태코드를 '07'로 출력한다
프로그램 규칙 및 제약사항	1. 입력 체크 - 입력:은행외환구분코드가 Space 로 입력되면 안됨. - 입력:외환부점코드가 Space 로 입력되면 안됨. 2. 일자에 대한 부점 기본 정보 조회 1) 년월일 입력 : 입력된 일자 당시의 부점 정보 2) 년월일 미입력 : 현재 시점의 부점정보 조회 - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준일 3. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일은 01 ~ 31 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다. 4).적용시작일자 및 적용종료일자 기준으로 조회 된다 (단 개점년월일이 적용시작일자보다 클 수 있고, 폐쇄년월일이 적용종료일자보다 +1 일이므로 반드시 부점기본 정보에 포함되어 있는 개점년월일과 폐쇄년월일을 확인해야 한다.) 4. 입력: 은행외환구분코드, 외환부점코드 - 은행외환구분코드 07:구국민(장은포함), 10:구주택(동남포함) 5. 입력 은행외환구분코드 + 외환부점코드가 폐쇄부점일 경우 - 해당 폐쇄부점 정보를 조회하고, 리턴상태코드를 '07'로 출력한다

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcBranch extends`
	`com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	String inGroupCoCd ; // ● 그룹회사코드
	String inBnkFxDstcd; // ● 은행외환구분코드
	String inFxBrncd ; // ● 외국환부점코드
	String inYmd ; // ● 년월일
	
	[출력항목]
	String rtnStat ; // 리턴상태코드
	String outBrncd ; // 부점코드
	String outAplyStartYmd ; // 적용시작년월일
	String outAplyEndYmd ; // 적용종료년월일
	String outOpnnYmd ; // 개점년월일
	String outClsrYmd ; // 폐쇄년월일
	String outBnkCd ; // 은행코드
	String outBrnHanglName ; // 부점한글명
	// 부점한글약어명

```
String outBrnHAbrvnName   ;                    // 부점영문명
String outBrnEngName      ;                    // 우편번호
String outZip             ;                    // 한글주소
String outHanglAddr       ;                    // 영문주소
String outEngAddr         ;                    // 부점구분
String outBrnDstcd        ;                    // 출장소모점부점코드
String outSubrnPabrnBrncd ;                    // 부점전화번호
String outBrnTelno        ;                    // ARS전화번호
String outArsTelno        ;                    // 팩스번호
String outFaxNo           ;                    // 팩스번호용도내용
String outFaxNoUsagCtnt   ;                    // 팩스번호1
String outFaxNo1          ;                    // 팩스번호용도내용1
String outFaxNoUsagCtnt1  ;                    // 사업자번호
String outBzno            ;                    // 관할세무서명
String outJrsdTxofcName   ;                    // 회계발생구분
String outAcctgOccurDstcd ;                    // 게정처리부점코드
String outAcitmPrcssBrncd ;                    // 영업지원본부부점코드
String outBzoprSportHQBrncd;                   // 사업본부부점코드
String outBizHqBrncd      ;                    // 어음교환소모점코드
String outBlclrhPabrnCd   ;                    // 어음교환소코드
String outBlclrhCd        ;                    // 국고결제모점코드
String outNtresrPPabrnCd  ;                    // 한국은행부점구분코드
String outBokBrnDstcd     ;                    // 공금예금모점코드
String outPbmnyDPabrnCd ;                      // 은행외환구분코드
String outBnkFxDstcd      ;                    // 외환부점코드
String outFxzBrncd         ;                   // 외환부점구분코드
String outFxBrnDstcd      ;                    // 외환취급부점구분코드
String outFxHndlnBrnDstcd ;                    // 외환개시년월일
String outFxOpenigYmd     ;                    // 외환종료년월일
String outFxEndYmd        ;                    // 은행외환모점코드
String outBnkFxPabrncd    ;                    // 외환모점외환부점코드
String outFxPabrnFxBrncd  ;                    // 통합부점코드
String outIntgraBrncd     ;                    // 수표지급지명
String outChekPaymtNmopc  ;                    // 대출싱행킴부점코드
String outLnExeTeamBrncd  ;                    // 원본서류보관부점코드
String outOrcpyDKeepBrncd ;                    // 프로세스센터부점코드
String outPrcesCnterBrncd ;                    // 내부통제자직원번호
String outIntnlCPsnEmpid ;                     // 내부통제자휴대폰번호
String outIntnlCPsnCphnNo ;                    // 부점장직원번호
String outBrnmgrEmpid     ;                    // 부점장휴대폰번호
String outBrnmgrCphnNo    ;                    // 부점항목변경내용
String outBrnItemModfiCtnt;                    // 부점이력내용
String outBrnPshistCtnt    ;                   // 점포유형구분
String outStorPtrnDstcd           ;            // 부점장CPNO변경년월일
String outBrnmgrCphnModfiYmd      ;            // 내통자CPNO변경년월일
String outIntnlCtrlPsnCphnModfiYmd ;           // 부점지주회사구분
String outSthlCoDstcd             ;            // 주소표시구분
```

PoC

PoC

PoC

PoC

PoC

	String outAddrRepsntDstcd ;
	String outExpanBrnHanglName ;
	String outEmad ;
	String outFthStorBrncd ;
		// 확장부점한글명
		// 이메일주소
		// 거점점포부점코드

RTNSTAT 설명	00 : 정상, 07 : 입력 부점이 폐쇄부점인 경우	

사용 설명(예시)	**// 공유메소드 호출 - 외국환부점변환 조회 (com.kbstar.kji.enbncmn, getFXBrnChng)**
	
	IDataSet getFXBrnChngReq = new DataSet();
	
	getFXBrnChngReq.put("inGroupCoCd", requestData.get("inGroupCoCd")); //그룹회사코드
	getFXBrnChngReq.put("inBnkFxDstcd", requestData.get("inBnkFxDstcd"));//은행외환구분코드
	getFXBrnChngReq.put("inFxBrncd", requestData.get("inFxBrncd")); // 외국환부점코드
	getFXBrnChngReq.put("inYmd", requestData.get("inYmd")); // 기준년월일
	
	// @@ 공유메소드 호출 (외국환부점변환 조회 getFXBrnChng)
	IDataSet getFXBrnChngRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getFXBrnChng", getFXBrnChngReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getFXBrnChngRes.getString("rtnStat"); // 처리상태
	String outbrncd = getFXBrnChngRes.getString("outBrncd"); // 부점코드
	String outaplystartymd = getFXBrnChngRes.getString("outAplyStartYmd"); // 적용시작년월일
	String outaplyendymd = getFXBrnChngRes.getString("outAplyEndYmd"); // 적용종료년월일
	String outopnnymd = getFXBrnChngRes.getString("outOpnnYmd"); // 개점년월일
	String outclsrymd = getFXBrnChngRes.getString("outClsrYmd"); // 폐쇄년월일
	String outbnkcd = getFXBrnChngRes.getString("outBnkCd"); // 은행코드
	String outbrnhanglname = getFXBrnChngRes.getString("outBrnHanglName"); // 부점한글명
	String outbrnhabrvnname = getFXBrnChngRes.getString("outBrnHAbrvnName"); // 부점한글약어명
	String outbrnengname = getFXBrnChngRes.getString("outBrnEngName"); // 부점영문명
	String outzip = getFXBrnChngRes.getString("outZip"); // 우편번호
	String outhangladdr = getFXBrnChngRes.getString("outHanglAddr"); // 한글주소
	String outengaddr = getFXBrnChngRes.getString("outEngAddr"); // 영문주소
	String outbrndstcd = getFXBrnChngRes.getString("outBrnDstcd"); // 부점구분
	String outsubrnpabrnbrncd = getFXBrnChngRes.getString("outSubrnPabrnBrncd"); // 출장소모점부점코드
	String outbrntelno = getFXBrnChngRes.getString("outBrnTelno"); // 부점전화번호
	String outarstelno = getFXBrnChngRes.getString("outArsTelno"); // ARS 전화번호
	String outfaxno = getFXBrnChngRes.getString("outFaxNo"); // 팩스번호
	String outfaxnousagctnt = getFXBrnChngRes.getString("outFaxNoUsagCtnt"); // 팩스번호용도내용
	String outfaxno1 = getFXBrnChngRes.getString("outFaxNo1"); // 팩스번호 1
	String outfaxnousagctnt1 = getFXBrnChngRes.getString("outFaxNoUsagCtnt1"); // 팩스번호용도내용 1
	String outbzno = getFXBrnChngRes.getString("outBzno"); // 사업자번호
	String outjrsdtxofcname = getFXBrnChngRes.getString("outJrsdTxofcName"); // 관할세무서명
	String outacctgoccurdstcd = getFXBrnChngRes.getString("outAcctgOccurDstcd"); // 회계발생구분
	String outacitmprcssbrncd = getFXBrnChngRes.getString("outAcitmPrcssBrncd"); // 계정처리부점코드
	String outbzoprsporthqbrncd = getFXBrnChngRes.getString("outBzoprSportHqBrncd"); // 영업지원본부부점코드
	String outbizhqbrncd = getFXBrnChngRes.getString("outBizHqBrncd"); // 사업본부부점코드
	String outblclrhpabrncd = getFXBrnChngRes.getString("outBlclrhPabrnCd"); // 어음교환소모점코드
	String outblclrhcd = getFXBrnChngRes.getString("outBlclrhCd"); // 어음교환소코드
	String outntresrppabrncd = getFXBrnChngRes.getString("outNtresrPPabrnCd"); // 국고결제모점코드
	String outbokbrndstcd = getFXBrnChngRes.getString("outBokBrnDstcd"); // 한은부점구분
	String outpbmnydpabrncd = getFXBrnChngRes.getString("outPbmnyDPabrnCd"); // 공금예금모점코드	

	String outbnkfxdstcd = getFXBrnChngRes.getString("outBnkFxDstcd"); // 은행외환구분
	String outfxzbrncd = getFXBrnChngRes.getString("outFxzBrncd"); // 외환부점코드
	String outfxbrndstcd = getFXBrnChngRes.getString("outFxBrnDstcd"); // 외환부점구분
	String outfxhndlnbrndstcd = getFXBrnChngRes.getString("outFxHndlnBrnDstcd"); // 외환취급부점구분
	String outfxopenigymd = getFXBrnChngRes.getString("outFxOpenigYmd"); // 외환개시년월일
	String outfxendymd = getFXBrnChngRes.getString("outFxEndYmd"); // 외환종료년월일
	String outbnkfxpabrncd = getFXBrnChngRes.getString("outBnkFxPabrncd"); // 은행외환모점코드
	String outfxpabrnfxbrncd = getFXBrnChngRes.getString("outFxPabrnFxBrncd"); // 외환모점외환부점코드
	String outintgrabrncd = getFXBrnChngRes.getString("outIntgraBrncd"); // 통합부점코드
	String outchekpaymtnmopc = getFXBrnChngRes.getString("outChekPaymtNmopc"); // 수표지급지명
	String outlnexeteambrncd = getFXBrnChngRes.getString("outLnExeTeamBrncd"); // 대출싱행팀부점코드
	String outorcpydkeepbrncd = getFXBrnChngRes.getString("outOrcpyDKeepBrncd"); // 원본서류보관부점코드
	String outprcescnterbrncd = getFXBrnChngRes.getString("outPrcesCnterBrncd"); // 프로세스센터부점코드
	String outintnlcpsnempid = getFXBrnChngRes.getString("outIntnlCPsnEmpid"); // 내부통제자직원번호
	String outintnlcpsncphnno = getFXBrnChngRes.getString("outIntnlCPsnCphnNo"); // 내부통제자휴대폰번호
	String outbrnmgrempid = getFXBrnChngRes.getString("outBrnmgrEmpid"); // 부점장직원번호
	String outbrnmgrcphnno = getFXBrnChngRes.getString("outBrnmgrCphnNo"); // 부점장휴대폰번호
	String outbrnitemmodfictnt = getFXBrnChngRes.getString("outBrnItemModfiCtnt"); // 부점항목변경내용
	String outbrnpshistctnt = getFXBrnChngRes.getString("outBrnPshistCtnt"); // 부점이력내용
	String outstorptrndstcd = getFXBrnChngRes.getString("outStorPtrnDstcd"); // 점포유형구분
	String outbrnmgrcphnmodfiymd = getFXBrnChngRes.getString("outBrnmgrCphnModfiYmd"); // 부점장 CPNO 변경년월일
	String outintnlctrlpsncphnmodfiymd = getFXBrnChngRes.getString("outIntnlCtrlPsnCphnModfiYmd"); // 내통자 CPNO 변경년월일
	String outsthlcodstcd = getFXBrnChngRes.getString("outSthlCoDstcd"); // 부점지주회사구분
	String outaddrrepsntdstcd = getFXBrnChngRes.getString("outAddrRepsntDstcd"); // 주소표시구분
	String outexpanbrnhanglname = getFXBrnChngRes.getString("outExpanBrnHanglName"); // 확장부점한글명
	String outemad = getFXBrnChngRes.getString("outEmad"); // 이메일주소
	String outfthstorbrncd = getFXBrnChngRes.getString("outFthStorBrncd"); // 거점점포부점코드

비 고	

1.12

관할/대출실행팀 소속부점코드 조회(FUBcBranch.getJrsdLnExeTeamBrnCd())

프로그램 ID	FUBcBranch. getJrsdLnExeTeamBrnCd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR01		SQL ID	selectJrsdLnExeTeamBrnCd

프로그램 설명	입력된 부점에 대해 구분코드별 소속 부점코드를 조회한다.
프로그램 규칙 및 제약사항	1. 입력 체크 1) 구분코드 체크 - '1','2','3','4','5','6','7' ,'8','9','A' 외 값 입력시 오류처리 2) 부점코드 체크 - 입력:부점코드는 숫자로 입력되어야 한다. - 입력:부점코드가 '0000'입력되면 안됨 2. 구분코드는 아래와 같다. 1) '1' : 소속지역본부 부점 코드 소속지역본부부점코드로 소속된 부점코드를 모두 조회한다. 2) '2' : 대출실행팀 부점코드 대출실행팀 부점코드로 소속된 부점코드를 모두 조회한다. 3) '3' : 원본서류보관 부점코드 원본서류보관 부점코드로 소속된 부점코드를 모두 조회한다. 4) '4' : 프로세스센터 부점코드 프로세스센터 부점코드로 소속된 부점코드를 모두 조회한다. 5) '5' : 사업본부 부점코드 사업본부 부점코드로 소속된 부점코드를 모두 조회한다. 6) '6' : 어음교환소 부점코드 어음교환소 부점코드로 소속된 부점코드를 모두 조회한다. 7) '7' : 출장소모점부점코드 출장소모점부점코드로 소속된 부점코드를 모두 조회한다. 8) '8' : 지역본부 부점코드(거점점포부점코드 조회) 지역본부에 소속된 거점점포부점코드를 모두 조회한다. 9) '9' : 거점점포부점코드(거점포함) 거점점포부점코드에 소속된 부점코드를 모두 조회한다.(거점포함) A) 'A' : 거점점포부점코드(거점 불포함) 거점점포부점코드에 소속된 부점코드를 모두 조회한다.(거점 불포함) 3. 일자에 대한 부점 기본 정보 조회 1) 년월일 입력 : 입력된 일자 당시의 부점 정보 2) 년월일 미입력 : 현재 시점의 부점정보 조회 - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준년월일 3). 적용시작일자 및 적용종료일자 기준으로 조회 된다 (단 개점년월일이 적용시작일자보다 클 수 있고, 폐쇄년월일이 적용종료일자보다 +1 일이므로 반드시 부점기본 정보에 포함되어 있는 개점년월일과 폐쇄년월일을 확인해야 한다.)

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcBranch extends`
	`com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;`
	`String inDstcd ;`
		● 그룹회사코드

	String inBrncd ; ● 구분코드
	String inYmd ; ● 부점코드
	◎ 기준년월일
	
	[출력항목]
	int outBrnNoitm; // 부점건수
	IRecordSet LIST1; // LIST1
	IrecordSet LIST2; // LIST2
	
	< LIST1 항목 >
	String outBrncd ; // 부점코드
	
	< LIST2 항목 >
	String outBrnHanglName; // 부점한글명

RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - 관할/대출실행팀 소속부점코드 조회 (com.kbstar.kji.enbncmn, getJrsdLnExeTeamBrnCd)**
	
	IDataSet getJrsdLnExeTeamBrnCdReq = new DataSet();
	
	getJrsdLnExeTeamBrnCdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getJrsdLnExeTeamBrnCdReq.put("inDstcd", requestData.get("inDstcd")); // 구분코드
	getJrsdLnExeTeamBrnCdReq.put("inBrncd", requestData.get("inBrncd")); // 부점코드
	getJrsdLnExeTeamBrnCdReq.put("inYmd", requestData.get("inYmd")); // 기준년월일
	
	// @@ 공유메소드 호출 (관할/대출실행팀 소속부점코드 조회 getJrsdLnExeTeamBrnCd)
	IDataSet getJrsdLnExeTeamBrnCdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcBranch.getJrsdLnExeTeamBrnCd", getJrsdLnExeTeamBrnCdReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outbrnnoitm = getJrsdLnExeTeamBrnCdRes.getString("outBrnNoitm");
	IRecordSet getJrsdLnExeTeamBrnCdResRsList1 =
	getJrsdLnExeTeamBrnCdRes.getRecordSet("LIST1");
	IRecordSet getJrsdLnExeTeamBrnCdResRsList2 =
	getJrsdLnExeTeamBrnCdRes.getRecordSet("LIST2");

비 고	

1.13 부점 폐쇄에 의한 통합부점 조회(FUBcBranch.getIntegrationBranch())

프로그램 ID	FUBcBranch. getIntegrationBranch()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR01		SQL ID	selectIntegrationBranch selectClosureBranch selectActivityBranch

PoC

프로그램 설명	폐쇄부점코드에 대한 최종 통합부점의 부점기본 내역을 조회한다..
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 부점코드는 숫자로 입력되어야 한다. - 부점코드가 '0000' 입력되면 안됨 2. 폐쇄부점코드에 대한 최종 통합부점의 부점기본 내역을 조회한다. 예) A -> B, B -> C 인 경우 A 에 대해 조회하면 C 부점의 정보가 조회된다. 3. 최종 통합된 부점코드를 사용하고 싶은 경우에는 XCJIBR04-O-BRNCD 항목을 사용하여야 한다. 예) inBrncd = 1100 -> outBrncd = 9876 4. 부점이 폐쇄부점이 아닌 경우 리턴상태코드는 '07' 로 하고 입력부점정보를 출력한다. 1) 적용시작일자 및 적용종료일자 기준으로 조회 된다 (단 개점년월일이 적용시작일자보다 클 수 있고, 폐쇄년월일이 적용종료일자보다 +1 일이므로 반드시 부점기본 정보에 포함되어 있는 개점년월일과 폐쇄년월일을 확인 해야 한다.)

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcBranch extends
                com.kbstar.sqc.base.FunctionUnit

[입력항목]
  String inGroupCoCd ;          ● 그룹회사코드
  String inBrncd     ;          ● 부점코드

[출력항목]
  String rtnStat            ;   // 리턴상태코드
  String outBrncd           ;   // 부점코드
  String outAplyStartYmd    ;   // 적용시작년월일
  String outAplyEndYmd      ;   // 적용종료년월일
  String outOpnnYmd         ;   // 개점년월일
  String outClsrYmd         ;   // 폐쇄년월일
  String outBnkCd           ;   // 은행코드
  String outBrnHanglName    ;   // 부점한글명
  String outBrnHAbrvnName   ;   // 부점한글약어명
  String outBrnEngName      ;   // 부점영문명
  String outZip             ;   // 우편번호
  String outHanglAddr       ;   // 한글주소
  String outEngAddr         ;   // 영문주소
  String outBrnDstcd        ;   // 부점구분
  String outSubrnPabrnBrncd ;   // 출장소모점부점코드
                                // 부점전화번호
```

	String outBrnTelno ;	// ARS전화번호
	String outArsTelno ;	// 팩스번호
	String outFaxNo ;	// 팩스번호용도내용
	String outFaxNoUsagCtnt ;	// 팩스번호1
	String outFaxNo1 ;	// 팩스번호용도내용1
	String outFaxNoUsagCtnt1 ;	// 사업자번호
	String outBzno ;	// 관할세무서명
	String outJrsdTxofcName ;	// 회계발생구분
	String outAcctgOccurDstcd ;	// 게정처리부점코드
	String outAcitmPrcssBrncd ;	// 영업지원본부부점코드
	String outBzoprSportHQBrncd;	// 사업본부부점코드
	String outBizHqBrncd ;	// 어음교환소모점코드
	String outBlclrhPabrnCd ;	// 어음교환소코드
	String outBlclrhCd ;	// 국고결제모점코드
	String outNtresrPPabrnCd ;	// 한국은행부점구분코드
	String outBokBrnDstcd ;	// 공금예금모점코드
	String outPbmnyDPabrnCd ;	// 은행외환구분코드
	String outBnkFxDstcd ;	// 외환부점코드
	String outFxzBrncd ;	// 외환부점구분코드
	String outFxBrnDstcd ;	// 외환취급부점구분코드
	String outFxHndlnBrnDstcd ;	// 외환개시년월일
	String outFxOpenigYmd ;	// 외환종료년월일
	String outFxEndYmd ;	// 은행외환모점코드
	String outBnkFxPabrncd ;	// 외환모점외환부점코드
	String outFxPabrnFxBrncd ;	// 통합부점코드
	String outIntgraBrncd ;	(사용중지)
		// 수표지급지명
	String outChekPaymtNmopc ;	// 대출싱행킴부점코드
	String outLnExeTeamBrncd ;	// 원본서류보관부점코드
	String outOrcpyDKeepBrncd ;	// 프로세스센터부점코드
	String outPrcesCnterBrncd ;	// 내부통제자직원번호
	String outIntnlCPsnEmpid ;	// 내부통제자휴대폰번호
	String outIntnlCPsnCphnNo ;	// 부점장직원번호
	String outBrnmgrEmpid ;	// 부점장휴대폰번호
	String outBrnmgrCphnNo ;	// 부점항목변경내용
	String outBrnItemModfiCtnt;	// 부점이력내용
	String outBrnPshistCtnt ;	// 점포유형구분
	String outStorPtrnDstcd	// 부점장CPNO변경년월일
	String outBrnmgrCphnModfiYmd	// 내통자CPNO변경년월일
	String outIntnlCtrlPsnCphnModfiYmd	// 부점지주회사구분
	String outSthlCoDstcd	// 주소표시구분
	String outAddrRepsntDstcd	// 확장부점한글명
	String outExpanBrnHanglName	// 이메일주소
	String outEmad	// 거점점포부점코드
	String outFthStorBrncd	
RTNSTAT 설명	00 : 정상, 07 : 입력 부점이 폐쇄부점이 아닌경우	

사용 설명(예시)	**// 공유메소드 호출 - 부점 폐쇄에 의한 통합부점 조회 (com.kbstar.kji.enbncmn, getIntegrationBranch)**
	
	IDataSet getIntegrationBranchReq = new DataSet();PoC
	
	getIntegrationBranchReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getIntegrationBranchReq.put("inBrncd", requestData.get("inBrncd"));
	
	// @@ 공유메소드 호출 (부점 폐쇄에 의한 통합부점 조회 getIntegrationBranch)
	IDataSet getIntegrationBranchRes = callSharedMethodByDirect ("com.kbstar.kji.enbncmn", "FUBcBranch.getIntegrationBranch", getIntegrationBranchReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	
	String rtnstat = getIntegrationBranchRes.getString("rtnStat"); // 처리상태
	String outbrncd = getIntegrationBranchRes.getString("outBrncd"); // 부점코드
	String outaplystartymd = getIntegrationBranchRes.getString("outAplyStartYmd"); // 적용시작년월일
	String outaplyendymd = getIntegrationBranchRes.getString("outAplyEndYmd"); // 적용종료년월일
	String outopnnymd = getIntegrationBranchRes.getString("outOpnnYmd"); // 개점년월일
	String outclsrymd = getIntegrationBranchRes.getString("outClsrYmd"); // 폐쇄년월일
	String outbnkcd = getIntegrationBranchRes.getString("outBnkCd"); // 은행코드
	PoC String outbrnhanglname = getIntegrationBranchRes.getString("outBrnHanglName"); // 부점한글명
	String outbrnhabrvnname = getIntegrationBranchRes.getString("outBrnHAbrvnName"); // 부점한글약어명
	String outbrnengname = getIntegrationBranchRes.getString("outBrnEngName"); // 부점영문명
	String outzip = getIntegrationBranchRes.getString("outZip"); // 우편번호
	String outhangladdr = getIntegrationBranchRes.getString("outHanglAddr"); // 한글주소
	String outengaddr = getIntegrationBranchRes.getString("outEngAddr"); // 영문주소 PoC
	String outbrndstcd = getIntegrationBranchRes.getString("outBrnDstcd"); // 부점구분
	String outsubrnpabrnbrncd = getIntegrationBranchRes.getString("outSubrnPabrnBrncd"); // 출장소모점부점코드
	String outbrntelno = getIntegrationBranchRes.getString("outBrnTelno"); // 부점전화번호
	String outarstelno = getIntegrationBranchRes.getString("outArsTelno"); // ARS 전화번호
	String outfaxno = getIntegrationBranchRes.getString("outFaxNo"); // 팩스번호
	String outfaxnousagctnt = getIntegrationBranchRes.getString("outFaxNoUsagCtnt"); // 팩스번호용도내용
	String outfaxno1 = getIntegrationBranchRes.getString("outFaxNo1"); // 팩스번호 1 PoC
	String outfaxnousagctnt1 = getIntegrationBranchRes.getString("outFaxNoUsagCtnt1"); // 팩스번호용도내용 1
	String outbzno = getIntegrationBranchRes.getString("outBzno"); // 사업자번호
	String outjrsdtxofcname = getIntegrationBranchRes.getString("outJrsdTxofcName"); // 관할세무서명
	String outacctgoccurdstcd = getIntegrationBranchRes.getString("outAcctgOccurDstcd"); // 회계발생구분
	String outacitmprcssbrncd = getIntegrationBranchRes.getString("outAcitmPrcssBrncd"); // 계정처리부점코드
	String outbzoprsporthqbrncd = getIntegrationBranchRes.getString("outBzoprSportHqBrncd"); // 영업지원본부부점코드 PoC
	String outbizhqbrncd = getIntegrationBranchRes.getString("outBizHqBrncd"); // 사업본부부점코드
	String outblclrhpabrncd = getIntegrationBranchRes.getString("outBlclrhPabrnCd"); // 어음교환소모점코드
	String outblclrhcd = getIntegrationBranchRes.getString("outBlclrhCd"); // 어음교환소코드
	String outntresrppabrncd = getIntegrationBranchRes.getString("outNtresrPPabrnCd"); // 국고결제모점코드 PoC
	String outbokbrndstcd = getIntegrationBranchRes.getString("outBokBrnDstcd"); // 한은부점구분
	String outpbmnydpabrncd = getIntegrationBranchRes.getString("outPbmnyDPabrnCd"); // 공금예금모점코드
	String outbnkfxdstcd = getIntegrationBranchRes.getString("outBnkFxDstcd");//은행외환구분

	String outfxzbrncd = getIntegrationBranchRes.getString("outFxzBrncd"); // 외환부점코드
	String outfxbrndstcd = getIntegrationBranchRes.getString("outFxBrnDstcd"); // 외환부점구분
	String outfxhndlnbrndstcd = getIntegrationBranchRes.getString("outFxHndlnBrnDstcd"); // 외환취급부점구분
	String outfxopenigymd = getIntegrationBranchRes.getString("outFxOpenigYmd"); // 외환개시년월일
	String outfxendymd = getIntegrationBranchRes.getString("outFxEndYmd"); // 외환종료년월일
	String outbnkfxpabrncd = getIntegrationBranchRes.getString("outBnkFxPabrncd"); // 은행외환모점코드
	String outfxpabrnfxbrncd = getIntegrationBranchRes.getString("outFxPabrnFxBrncd"); // 외환모점외환부점코드
	String outintgrabrncd = getIntegrationBranchRes.getString("outIntgraBrncd"); // 통합부점코드
	String outchekpaymtnmopc = getIntegrationBranchRes.getString("outChekPaymtNmopc"); // 수표지급지명
	String outlnexeteambrncd = getIntegrationBranchRes.getString("outLnExeTeamBrncd"); // 대출실행팀부점코드
	String outorcpydkeepbrncd = getIntegrationBranchRes.getString("outOrcpyDKeepBrncd"); // 원본서류보관부점코드
	String outprcescnterbrncd = getIntegrationBranchRes.getString("outPrcesCnterBrncd"); // 프로세스센터부점코드
	String outintnlcpsnempid = getIntegrationBranchRes.getString("outIntnlCPsnEmpid"); // 내부통제자직원번호
	String outintnlcpsncphnno = getIntegrationBranchRes.getString("outIntnlCPsnCphnNo"); // 내부통제자휴대폰번호
	String outbrnmgrempid = getIntegrationBranchRes.getString("outBrnmgrEmpid"); // 부점장직원번호
	String outbrnmgrcphnno = getIntegrationBranchRes.getString("outBrnmgrCphnNo"); // 부점장휴대폰번호
	String outbrnitemmodfictnt = getIntegrationBranchRes.getString("outBrnItemModfiCtnt"); // 부점항목변경내용
	String outbrnpshistctnt = getIntegrationBranchRes.getString("outBrnPshistCtnt"); // 부점이력내용
	String outstorptrndstcd = getIntegrationBranchRes.getString("outStorPtrnDstcd"); // 점포유형구분
	String outbrnmgrcphnmodfiymd = getIntegrationBranchRes.getString("outBrnmgrCphnModfiYmd"); // 부점장 CPNO 변경년월일
	String outintnlctrlpsncphnmodfiymd = getIntegrationBranchRes.getString("outIntnlCtrlPsnCphnModfiYmd"); // 내통자 CPNO 변경년월일
	String outsthlcodstcd = getIntegrationBranchRes.getString("outSthlCoDstcd"); // 부점지주회사구분
	String outaddrrepsntdstcd = getIntegrationBranchRes.getString("outAddrRepsntDstcd"); // 주소표시구분
	String outexpanbrnhanglname = getIntegrationBranchRes.getString("outExpanBrnHanglName"); // 확장부점한글명
	String outemad = getIntegrationBranchRes.getString("outEmad"); // 이메일주소
	String outfthstorbrncd = getIntegrationBranchRes.getString("outFthStorBrncd"); // 거점점포부점코드

비 고	

PoC

1.14 통합부점코드에 속한 통폐합된 부점 전체조회(FUBcBranch.getMrabBrnWhol())

프로그램 ID	FUBcBranch.getMrabBrnWhol()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR01		SQL ID	selectMrabBrnWhol

프로그램 설명	입력된 통합부점코드에 대해 통폐합된 부점코드 및 폐쇄년월일을 모두 조회한다.
프로그램 규칙 및 제약사항	1. 부점코드는 통합부점코드임. 1) 부점코드는 숫자로 입력되어야 한다. 2) 부점코드가 '0000' 입력되면 안됨 2. 통합부점코드로 통폐합된 부점의 부점코드, 폐쇄년월일, 부점한글명을 모두 조회한다. 만약 통합부점코드로 조회한 결과 한 건도 없을 경우 리턴상태코드를 '07'로 한다. 3. 조회되는 기준은 1차로 통폐합된 부점만 대상이 된다. 예) A <- B, B <- C, A <- D일 경우 A부점에 대해 1차로 통폐합된 B,D가 조회대상이 된다. 폐쇄년월일을 선적용할 수 있으므로 출력되는 폐쇄년월일은 미래일자가 될 수 있다.

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcBranch **extends**
	com.kbstar.sqc.base.FunctionUnit
	PoC
	
	[입력항목]
	String inGroupCoCd; // ● 그룹회사코드
	String inBrncd ; // ● 부점코드
	PoC
	
	[출력항목]
	String rtnStat ; // 리턴상태코드
	int outBrnNoitm ; // 부점건수
	List<LIST> outMrabBrnWholList; // 통폐합부점 List
	PoC
	
	< LIST 항목 >
	String outBrncd ; // 부점코드
	String outClsrYmd; // 폐쇄년월일
	String outBrnHanglName; // 부점한글명

RTNSTAT 설명	00 : 정상, 07 : 통폐합된 부점건수 없음 PoC PoC

사용 설명(예시)	**// 공유메소드 호출 - 통합부점코드에 속한 통폐합된 부점 전체조회 (com.kbstar.kji.enbncmn, getMrabBrnWhol)**
	
	IDataSet getMrabBrnWholReq = new DataSet();
	
	getMrabBrnWholReq.put("inGroupCoCd", requestData.get("inGroupCoCd")); // 그룹회사코드
	getMrabBrnWholReq.put("inBrncd", requestData.get("inBrncd")); // 부점코드

	// @@ 공유메소드 호출 (통합부점코드에 속한 통폐합된 부점 전체조회 getMrabBrnWhol)
	IDataSet getMrabBrnWholRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getMrabBrnWhol", getMrabBrnWholReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getMrabBrnWholRes.getString("rtnStat"); // 처리상태
	String outbrnnoitm = getMrabBrnWholRes.getString("outBrnNoitm"); // 부점건수
	IRecordSet getMrabBrnWholResRsList = getMrabBrnWholRes.getRecordSet("LIST"); // LIST

비 고	PoC

1.15

PoC

PoC

PoC

PoC

PoC

PoC

인수합병 부점의 통합지로코드 조회(FUBcBranch.getIntgraGiroCd())

프로그램 ID	FUBcBranch.getIntgraGiroCd() PoC		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	통합은행 부점코드 대응명세	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR02		SQL ID	selectIntgraGiroCd

PoC

프로그램 설명	M&A, P&A 된 은행의 인수합병부점의 통합 부점코드를 조회한다.
프로그램 규칙 및 제약사항	1. 인수합병은행부점코드 Chcek 1) 숫자로 입력되어야 한다. 2) '000000' 입력되면 안됨 2. 구분코드는 아래와 같이 입력받는다. 1) '01' : 주택점번을 국민부점코드로 조회 2) '02' : 국민부점코드(6 자리)를 국민부점코드(통합후부점코드)로 조회 3) '03' : 국민부점코드(변경후 부점코드 4 자리)를 주택점번(국민점포에 부여 주택체계 4 자리) 로 조회 4) '04' : 국민부점코드를 국민부점코드(6 자리) 5) '05' : 폐쇄부점코드로 통합부점코드를 조회 6) '06' : 대동부점코드로 국민지로코드(통합후부실점코드)를 조회 7) '07' : 장기부점코드로 국민지로코드(통합후부실점코드)를 조회 8) '08' : 구국민출장소(6 자리)로 지로코드(4 자리)를 조회 9) '09' : 동남부점코드로 주택부점코드를 조회 10) '10' : 통합후부점코드로 구주택부점명을 조회 11) '01', '02', '03', '04', '05', '06', '07', '08', '09','10' 외 값이면 오류

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcBranch extends com.kbstar.sqc.base.FunctionUnit`
	
	PoC PoC
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inDstcd ; ● 구분코드
	String inBrnPastCd; ● 부점과거코드
	
	[출력항목]
	String outIntgraAcBrnName; // 통합후코드부점명
	String outBrnIntgraAfCd; // 부점통합후코드
	String outPastCdBrnName; // 과거코드부점명

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 인수합병 부점의 통합지로코드 조회 (com.kbstar.kji.enbncmn, getIntgraGiroCd)
	
	IDataSet getIntgraGiroCdReq = new DataSet();
	
	getIntgraGiroCdReq.put("inGroupCoCd", requestData.get("inGroupCoCd")); // 그룹회사코드
	getIntgraGiroCdReq.put("inDstcd", requestData.get("inDstcd")); // 구분코드
	getIntgraGiroCdReq.put("inBrnPastCd", requestData.get("inBrnPastCd")); // 부점과거코드
	
	// @@ 공유메소드 호출 (인수합병 부점의 통합지로코드 조회 getIntgraGiroCd)
	IDataSet getIntgraGiroCdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcBranch.getIntgraGiroCd", getIntgraGiroCdReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outintgraacbrnname =
	getIntgraGiroCdRes.getString("outIntgraAcBrnName"); // 통합후코드부점명
	String outbrnintgraafcd =
	getIntgraGiroCdRes.getString("outBrnIntgraAfCd"); // 부점통합후코드
	String outpastcdbrnname =
	getIntgraGiroCdRes.getString("outPastCdBrnName"); // 과거코드부점명

비 고	

1.16

부점운영지원내역 조회(FUBcBranch.getBranchOperSportHist())

프로그램 ID	FUBcBranch.getBranchOperSport Hist()		언어 구분	JAVA(O)	
입출력 명	requestData/responseData		사용 구분	온라인(O), 배치(O)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점운영 지원내역	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR03		SQL ID	selectBranchOper SportHist

프로그램 설명	부점과 관계된 부수적인 정보를 관리하고 있는 부점운영지원내역 테이블의 조회 한다.
프로그램 규칙 및 제약사항	1. 입력 데이터 체크. 1) 그룹회사코드 체크 - SPACE 로 입력되면 안됨. 2) 부점코드 체크 - 숫자로 입력 되어야 한다. - ‘0000’이 입력되면 안됨. 2. 구분코드는 아래와 같다. 1) ‘1’ : 부점운영지원내역 조회

입출력 설명	PoC
	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcBranch extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; // 그룹회사코드
	String inDstcd; // 구분코드
	String inBrncd; // 부점코드
	
	[출력항목]
	String outBrncd; // 부점코드
	String outBrnClsrYmd; // 부점폐쇄년월일
	String outChbiClsngYmd; // 중요증서마감년월일
	String outChbiClsngYn; // 중요증서마감여부
	String outBzwkClsngCtrlYmd; // 업무마감통제년월일
	String outBzwkClsngCtrlCtnt; // 업무마감통제내용
	String outScrtBrnDstcd; // 유동화부점구분
	String outBokTaLnPabrnCd; // 한은총액대출모점코드
	String outCncrEcludYn; // 집중화제외여부
	String outSpexpRhAbilYn; // 특급송금취급가능여부
	String outChbiClsngHms; // 중요증서마감시각
	String outDeedClsngUserNo; // 증서마감사용자번호
	String outlnbzMgtCnterYno; // 여신관리센터여부
	String outcorpLCnterBrncd; // 기업여신센터부점코드
	String outhecoSCnterBrncd; // 가계담보센터부점코드
	String outhecoCCnterBrncd; // 가계신용센터부점코드
	String outcorpFCnterBrncd; // 기업기금센터부점코드
	String outhecoFCnterBrncd; // 가계기금센터부점코드
	String outAreaECnterBrncd; // 지역심사센터부점코드
	String outFsstPrcssBrnCd; // 수입인지처리부점코드
	String outPbmnyDAreaDstcd; // 공금예금지역구분코드
	String outBnkCprno; // 은행법인등록번호
	String outBnkChfFname; // 은행장성명
	
	PoC PoC PoC PoC PoC PoC

RTNSTAT 설명	N/A

사용 설명(예시)

```
// 공유메소드 호출 - 부점운영지원내역 조회 (com.kbstar.kji.enbncmn,
getBranchOperSportHist)

 IDataSet getBranchOperSportHistReq = new DataSet();

 getBranchOperSportHistReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
 getBranchOperSportHistReq.put("inDstcd", requestData.get("inDstcd")); // 구분코드
 getBranchOperSportHistReq.put("inBrncd", requestData.get("inBrncd")); // 부점코드

// @@ 공유메소드 호출 (부점운영지원내역 조회 getBranchOperSportHist)
  IDataSet getBranchOperSportHistRes = callSharedMethodByDirect
                    ("com.kbstar.kji.enbncmn", "FUBcBranch.getBranchOperSportHist",
                                        getBranchOperSportHistReq, onlineCtx);

// @@ 출력 데이터 셋 처리
String outbrncd = getBranchOperSportHistRes.getString("outBrncd"); // 부점코드
String outbrnclsrymd = getBranchOperSportHistRes.getString("outBrnClsrYmd"); // 부점폐쇄
년월일
String outchbiclsngymd = getBranchOperSportHistRes.getString("outChbiClsngYmd"); // 중요
증서마감년월일
String outchbiclsnghms = getBranchOperSportHistRes.getString("outChbiClsngHms"); // 중요
증서마감시각
String outchbiclsngyn = getBranchOperSportHistRes.getString("outChbiClsngYn"); // 중요증
서마감여부
String outdeedclsnguserno = getBranchOperSportHistRes.getString("outDeedClsngUserNo");
// 증서마감사용자번호
String outbzwkclsngctrlymd = getBranchOperSportHistRes.getString("outBzwkClsngCtrlYmd");
// 업무마감통제년월일
String outbzwkclsngctrlctnt =
getBranchOperSportHistRes.getString("outBzwkClsngCtrlCtnt"); // 업무마감통제내용
String outboktalnpabrncd = getBranchOperSportHistRes.getString("outBokTaLnPabrncd"); //
한은총액대출모점코드
String outcncrecludyn = getBranchOperSportHistRes.getString("outCncrEcludYn"); // 집중화
제외여부
String outspexprhabilyn = getBranchOperSportHistRes.getString("outSpexpRhAbilYn"); // 특
급송금취급가능여부
String outlnbzmgtcnteryno = getBranchOperSportHistRes.getString("outlnbzMgtCnterYno");
// 여신관리센터여부
String outcorplcnterbrncd = getBranchOperSportHistRes.getString("outcorpLCnterBrncd");
// 기업여신센터부점코드
String outhecoscnterbrncd = getBranchOperSportHistRes.getString("outhecoSCnterBrncd");
// 가계담보센터부점코드
String outhecoccnterbrncd = getBranchOperSportHistRes.getString("outhecoCCnterBrncd");
// 가계신용센터부점코드
String outcorpfcnterbrncd = getBranchOperSportHistRes.getString("outcorpFCnterBrncd");
// 기업기금센터부점코드
String outhecofcnterbrncd = getBranchOperSportHistRes.getString("outhecoFCnterBrncd");
// 가계기금센터부점코드
String outareaecnterbrncd = getBranchOperSportHistRes.getString("outAreaECnterBrncd");
// 지역심사센터부점코드
String outfsstprcssbrncd = getBranchOperSportHistRes.getString("outFsstPrcssBrnCd"); //
수입인지처리부점코드
String outpbmnydareadstcd = getBranchOperSportHistRes.getString("outPbmnyDAreaDstcd");
// 공금예금지역구분
String outbnkcprno = getBranchOperSportHistRes.getString("outBnkCprno"); // 은행법인등록
번호
String outbnkchffname = getBranchOperSportHistRes.getString("outBnkChfFname"); // 은행장
성명
String outlsgbvbrndstcd = getBranchOperSportHistRes.getString("outLsgbVBrnDstcd"); // 지
자체금고부점구분
String outwdarlsgbcd = getBranchOperSportHistRes.getString("outWdarLsgbCd"); // 광역지자
체코드
String outfndtlsgbcd = getBranchOperSportHistRes.getString("outFndtLsgbCd"); // 기초지자
체코드
```

	String outfxaaexetagetyn = getBranchOperSportHistRes.getString("outFxAaExeTagetYn"); // 외환후선실행대상여부
	String outcnglcnteryn = getBranchOperSportHistRes.getString("outCnglCnterYn"); // 대기업센터여부
	String outcnglcnterbrncd = getBranchOperSportHistRes.getString("outCnglCnterBrnCd"); // 대기업센터부점코드

비고	

1.17 부점주소, 전화번호 조회 (실시간)(FUBcBranch.getBranchAddrTelno())

프로그램 ID	FUBcBranch. getBranchAddrTelno()		언어 구분	JAVA(○)	
입출력 명	BrnAddrTelnoBDTO		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR01		SQL ID	selectBranchAddr Telno selectClsrBranch AddrTelno

프로그램 설명	부점 주소 및 전화번호 등 실시간으로 변경되는 부점정보의 조회	
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 입력:부점코드는 숫자로 입력되어야 한다. - 입력:부점코드가 '0000'입력되면 안됨 2. 일자에 대한 부점 기본 정보 조회 1) 기준년월일 입력 : 입력된 일자 당시의 부점 정보 2) 기준년월일 미입력 : 현재 시점의 부점정보 조회 - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준일 3. 기준년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일은 01 ~ 31 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다. 4). 적용시작일자 및 적용종료일자 기준으로 조회 된다 (단, 개점년월일이 적용시작일자보다 클수 있고, 폐쇄년월일이 적용종료일자보다 +1 일 이므로 반드시 부점기본 정보에 포함되어 있는 개점년월일과 폐쇄년월일을 확인해야 한다.) 4. 입력된 정보를 기준으로 부점기본 테이블을 조회하여 찾지 못하면 폐쇄부점인지 확인한다. 5. 폐쇄부점일경우 리턴상태코드를 '07'로 한다. - 폐쇄부점의 최종 통합부점을 조회하려면 부점 폐쇄에 의한 통합점 조회 (FUBcBranch.getIntegrationBranch())를 한다. - 폐쇄부점도 부점기본 정보와 동일하게 출력한다.	

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcBranch extends`
	`com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	String inGroupCoCd ;
	String inBrncd ;
	String inYmd ;
	
	[출력항목]
	String rtnStat ;
	String outBrncd ;
	String outBrnTelno ;
	String outClcntrTelno ;
	String outHanglDtailAddr ;
	String outZip ;
	String outBzno ;
		● 그룹회사코드
		● 부점코드
		◎ 기준년월일
		
		// 리턴상태코드
		// 부점코드
		// 부점전화번호
		// 콜센터전화번호
		// 한글상세주소
		// 우편번호
		// 사업자등록번호

	String outFaxNo ; // 팩스번호
	String outFaxNoCtnt ; // 팩스번호내용
	String outFaxNo1 ; // 팩스번호 1
	String outFaxNo1Ctnt ; // 팩스번호 1 내용
	String outNtresrPPabrnCd ; // 국고결제모점코드
	String outPbmnyDpoPabCd ; // 공금예금모점코드
	String outIntnlCtrlPsnEid; // 내부통제자직원번호
	String outIntnlCPsnCphnno; // 내부통제자휴대폰번호
	String outBrnmgrEmpid ; // 부점장직원번호
	String outBrnmgrCphnno ; // 부점장휴대폰번호

RTNSTAT 설명	00 : 정상, 07 : 입력:부점이 폐쇄점인 경우

사용 설명(예시)	**// 공유메소드 호출 - 부점주소,전화번호 조회 (com.kbstar.kji.enbncmn, getBranchAddrTelno)**
	IDataSet getBranchAddrTelnoReq = new DataSet();
	
	getBranchAddrTelnoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getBranchAddrTelnoReq.put("inBrncd", requestData.get("inBrncd"));
	getBranchAddrTelnoReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (부점주소,전화번호 조회 getBranchAddrTelno)
	IDataSet getBranchAddrTelnoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcBranch.getBranchAddrTelno", getBranchAddrTelnoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getBranchAddrTelnoRes.getString("rtnStat"); // 처리상태
	String outbrncd = getBranchAddrTelnoRes.getString("outBrncd"); // 부점코드
	String outbrntelno = getBranchAddrTelnoRes.getString("outBrnTelno"); // 부점전화번호
	String outhangladdr = getBranchAddrTelnoRes.getString("outHanglAddr"); // 한글주소
	String outzip = getBranchAddrTelnoRes.getString("outZip"); // 우편번호
	String outbzno = getBranchAddrTelnoRes.getString("outBzno"); // 사업자등록번호
	String outfaxno = getBranchAddrTelnoRes.getString("outFaxNo"); // 팩스번호
	String outfaxnoctnt = getBranchAddrTelnoRes.getString("outFaxNoCtnt"); // 팩스번호내용
	String outfaxno1 = getBranchAddrTelnoRes.getString("outFaxNo1"); // 팩스번호 1
	String outfaxno1ctnt = getBranchAddrTelnoRes.getString("outFaxNo1Ctnt");//팩스번호 1 내용
	String outntresrppabrncd = getBranchAddrTelnoRes.getString("outNtresrPPabrnCd"); // 국고결제모점코드
	String outpbmnydpopabcd = getBranchAddrTelnoRes.getString("outPbmnyDpoPabCd"); // 공금예금모점코드
	String outintnlctrlpsneid = getBranchAddrTelnoRes.getString("outIntnlCtrlPsnEid"); // 내부통제자직원번호
	String outintnlcpsncphnno = getBranchAddrTelnoRes.getString("outIntnlCPsnCphnno"); // 내부통제자휴대폰번호
	String outbrnmgrempid = getBranchAddrTelnoRes.getString("outBrnmgrEmpid"); // 부점장직원번호
	String outbrnmgrcphnno = getBranchAddrTelnoRes.getString("outBrnmgrCphnno"); // 부점장휴대폰번호
	String outaddrrepsntdstcd = getBranchAddrTelnoRes.getString("outAddrRepsntDstcd"); // 주소표시구분
	String outexpanbrnhanglname = getBranchAddrTelnoRes.getString("outExpanBrnHanglName"); // 확장부점한글명
	String outemad = getBranchAddrTelnoRes.getString("outEmad"); // 이메일주소

비 고	

PoC

1.18 업무마감통제내용 변경(N/A)

프로그램 ID	N/A		언어 구분	N/A	
DTO 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	부점 운영지원 업무마감 통제 내용 및 업무마감통제년월일 변경
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 부점코드는 숫자로 입력되어야 한다. - 부점코드가 '0000' 입력되면 안됨 2. 통제번호 체크 - 통제번호는 통제하고자 하는 업무에 고유한 위치값(절대위치값) - 통제번호는 숫자로 입력되어야 한다. - 통제번호는 001 ~ 200 범위값 이어야 한다. 3. 통제여부 체크 - '0' : 통제 해제 - '1' : 통제 4. Common Area 의 거래처리기준일과 부점운영지원 Table 의 업무마감통제년월일과 비교 - 거래처리기준일과 업무마감통제년월일이 일치 하지 않을경우 업무마감 통제내역을 '0' 으로 업무통제를 해제 한 다음 업무마감 통제년월일을 거래처리 기준일로 변경한다. - 거래처리기준일과 업무마감통제년월일이 일치 하는경우 해당 통제번호의 통제내역을 입력 통제여부로 변경한다. 5. 업무통제내역 변경내용. - 부점운영지원 테이블의 통제번호에 해당하는 통제여부가 '0' 이고 입력통제여부가 '1' 인경우 통제내역을 '1' 로 변경 - 부점운영지원 테이블의 통제번호에 해당하는 통제여부가 '1' 이고 입력통제여부가 '0' 인경우 통제내역을 '2' 로 변경 - 부점운영지원 테이블의 통제번호에 해당하는 통제여부가 '2' 이고 입력통제여부가 '1' 인경우 통제내역을 '1' 로 변경 - 그외 경우는 에러처리한다.
비 고	

PoC

1.19

PoC

우편번호 3자리로 지역별 소속부점코드 조회 (FUBcBranch.getAreaBranchCode())

프로그램 ID	FUBcBranch.getAreaBranchCode()		언어 구분	JAVA(○)	
입출력 명	AreaBrnCdBDTO		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR01		SQL ID	selectAreaBranchCode

PoC

프로그램 설명	우편번호 앞 3자리(지역)에 속한 부점코드를 모두 조회한다.(예:100-중구(서울))	
프로그램 규칙 및 제약사항	1. 해당 구의 우편번호 3자리로 부점기본 조회 - 우편번호로 소속된 부점코드를 모두 조회한다. 2. 적용시작일자 및 적용종료일자 기준으로 조회 된다 (단, 개점년월일이 적용시작일자보다 클 수 있고, 폐쇄년월일이 적용종료일자보다 +1 일 이므로 반드시 부점기본 정보에 포함되어 있는 개점년월일과 폐쇄년월일을 확인해야 한다.) 3. 본부부서가 아닌경우만 대상으로 한다.	

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcBranch extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd;
	String inZip ;
	
	[출력항목]
	int outBrnNoitm;
	List LIST1;
	List LIST2;
	
	< LIST1 항목 >
	List outBrncd ;
	< LIST2 항목 >
	List outBrnHanglName;
		● 그룹회사코드
		● 우편번호 3자리
		
		// 부점건수
		// List1
		// List2
		
		// 부점코드
		
		// 부점한글명

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 우편번호 3자리로 지역별 소속부점코드 조회 (com.kbstar.kji.enbncmn, getAreaBranchCode)
	
	IDataSet getAreaBranchCodeReq = new DataSet();
	
	getAreaBranchCodeReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));	

PoC

PoC
 	
PoC
	getAreaBranchCodeReq.put("inZip", requestData.get("inZip"));
	
	// @@ 공유메소드 호출 (우편번호 3자리로 지역별 소속부점코드 조회 getAreaBranchCode)
	PoC
	IDataSet getAreaBranchCodeRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getAreaBranchCode", getAreaBranchCodeReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outbrnnoitm = getAreaBranchCodeRes.getString("outBrnNoitm"); // 부점건수
	IRecordSet getAreaBranchCodeResRsList1 = getAreaBranchCodeRes.getRecordSet("LIST1");
	IRecordSet getAreaBranchCodeResRsList2 = getAreaBranchCodeRes.getRecordSet("LIST2");

비 고	

1.20

PoC

PoC

PoC

PoC

전행 인스턴스코드 내용 조회(FUBcCode.getEnbnIncdContent())

프로그램 ID	FUBcCode.getEnbnIncdContent()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	업무 인스턴스 코드	SQLMAP 명	NAME SPACE	fubccode
	영문명	THKJIUI02		SQL ID	selectEnbnIncdContent selectListEnbnIncdContent

프로그램 설명	인스턴스식별자, 인스턴스코드에 맞는 업무 인스턴스 내용을 조회한다.
프로그램 규칙 및 제약사항	**1. 구분코드 체크** 1) '1' : 업무인스턴스코드 단건조회 2) '2' : 업무인스턴스식별자에 의한 코드조회 **2. 구분코드별 처리** 1) '1' : 업무인스턴스식별자와 업무인스턴스코드를 입력받아 단건을 조회 2) '2' : 업무인스턴스식별자를 입력받아 식별자에 해당하는 인스턴스코드 전체를 조회 **3. 일자에 대한 처리** 1) 기준년월일 입력 : 입력된 일자 당시의 인스턴스코드 조회 2) 기준년월일 미입력 : 현재 시점의 인스턴스코드 조회 - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준일 4. 축약인스턴스내용(outCmpsInstncCtnt) 다시 사용할 수 있도록 개선됨.

입출력 설명	**[입력항목]**
	String inGroupCoCd; ● 그룹회사코드
	String inDstcd; ● 구분코드
	String inInstncIdnfr; ● 업무인스턴스 식별자
	String inInstncCd; ◎ 인스턴스 코드
	String inYmd; ◎ 년월일
	
	[출력항목]
	int outInquryNoitm ; // 조회건수
	recordSet LIST; // 전행인스턴스 List
	-field outInstncCd // -인스턴스 코드
	-field outInstncCtnt // -인스턴스 내용
	}

RTNSTAT 설명	N/A

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcCode extends
                com.kbstar.sqc.base.FunctionUnit {
```

PoC

사용 설명(예시)	//공유메소드 호출-전행 인스턴스코드 내용 조회
	(com.kbstar.kji.enbncmn, getEnbnIncdContent)
	
	IDataSet getReq = new DataSet();
	PoC
	getEnbnIncdReq.put("inGroupCoCd" , requestData.get("inGroupCoCd"));
	getEnbnIncdReq.put("inDstcd" , requestData.get("inDstcd"));
	getEnbnIncdReq.put("inInstncIdnfr", requestData.get("inInstncIdnfr"));
	getEnbnIncdReq.put("inInstncCd" , requestData.get("inInstncCd"));
	getEnbnIncdReq.put("inYmd" , requestData.get("inYmd"));
	PoC
	
	// 공유메소드 호출(전행 인스턴스코드 내용 조회 getEnbnIncdContent)
	PoC
	IDataSet getEnbnIncdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getEnbnIncdContent", getEnbnIncdContentReq, onlineCtx);
	
	PoC
	// 출력 데이터 셋 처리
	String outinqurynoitm = getEnbnIncdRes.getString("outInquryNoitm");
	IRecordSet getEnbnIncdResRsList = getEnbnIncdRes.getRecordSet("LIST");
	for(IRecord getEnbnIncdResRsListRec : getEnbnIncdResRsList){
	String outinstnccd = getEnbnIncdResRsListRec.getString("outInstncCd");
	String outinstncctnt = getEnbnIncdResRsListRec.getString("outInstncCtnt");
	}

비 고	

1.21

PoC

PoC

전행 인스턴스코드 복수 조회(FUBcCode.getEnbnInstanceCode())

프로그램 ID	FUBcCode.getEnbnInstanceCode()		언어 구분	JAVA(○) PoC	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	업무 인스턴스 코드	SQLMAP 명	NAME SPACE	fubccode
	영문명	THKJIUI02		SQL ID	selectEnbnInstanceCode

프로그램 설명	인스턴스식별자, 인스턴스코드를 복수개로 받아 사용가능여부를 확인한다. 주로 입력 PC 프로그램에서 입력항목의 정합성 검증을 할 때 사용한다. PoC
프로그램 규칙 및 제약사항	1. 조회요청 건수는 입력항목의 인스턴스 개수와 같다. 2. 일자에 대한 처리 1) 기준년월일 입력 : 입력된 일자 당시의 인스턴스코드 조회 2) 기준년월일 미입력 : 현재 시점의 인스턴스코드 조회 - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준일 3. 입력항목의 인스턴스식별자, 인스턴스코드에 대한 개별 조회를 하여 찾지 못하면 (조회결과 데이터가 없는 경우) 출력항목별로 사용가능여부에 '1'(가능), '0'(불가)을 전달한다. 4. 한 건이라도 불가능건수가 있으면 리턴상태코드는 '08'(사용불가건수 존재)를 전달한다.

DTO 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcCode extends
                com.kbstar.sqc.base.FunctionUnit {

 [입력항목]
  String inGroupCoCd                    ● 그룹회사코드
  int inInquryDmnoNoitm                 ● 조회요청건수
  recordSet List1
        String inInstncIdnfr            ● 인스턴스 식별자
  recordSet List2
        String inInstncCd               ● 인스턴스 코드
  String inYmd                          ◎ 년월일

 [출력항목]
  String rtnStat                        // 리턴상태
  int outNomalNoitm                     // 정상건수
  int outErrNoitm                       // 오류건수
  recordSet List                        // (List)
        -field outInstncIdnfr           //  인스턴스식별자
        -field outInstncCd              //  인스턴스코드
        -field outValdStartYmd          //  유효시작년월일
        -field outValdEndYmd            //  유효종료년월일
        -field outInstncCtnt            //  인스턴스내용
```

RTNSTAT 설명	'00' : 정상의 경우, '08' : 사용불가 건수 존재(한건이라도 에러가 있으면)

사용 설명(예시)	//공유메소드 호출 - 전행 인스턴스코드 복수 조회
	(com.kbstar.kji.enbncmn, getEnbnInstanceCode)
	
	IDataSet getEnbnInstanceCodeReq = new DataSet();
	
	getEnbnInstanceCodeReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEnbnInstanceCodeReq.put("inInquryDmnoNoitm", requestData.get("inInquryDmnoNoitm"));
	getEnbnInstanceCodeReq.put("LIST1", requestData.getRecordSet("LIST1"));
	getEnbnInstanceCodeReq.put("LIST2", requestData.getRecordSet("LIST2"));
	getEnbnInstanceCodeReq.put("inYmd", requestData.get("inYmd"));
	
	// 공유메소드 호출 (전행 인스턴스코드 복수 조회 getEnbnInstanceCode)
	IDataSet getEnbnInstanceCodeRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getEnbnInstanceCode", getEnbnInstanceCodeReq, onlineCtx);
	
	// 출력 데이터 셋 처리
	String rtnstat = getEnbnInstanceCodeRes.getString("rtnStat");
	String outnomalnoitm = getEnbnInstanceCodeRes.getString("outNomalNoitm");
	String outerrnoitm = getEnbnInstanceCodeRes.getString("outErrNoitm");
	IRecordSet getEnbnInstanceCodeResRsList = getEnbnInstanceCodeRes.getRecordSet("LIST");
	For (IRecord getEnbnInstanceCodeResRsListRec : getEnbnInstanceCodeResRsList){
	String outinstncidnfr = getEnbnInstanceCodeResRsListRec.getString("outInstncIdnfr");
	String outinstncd = getEnbnInstanceCodeResRsListRec.getString("outInstncCd");
	String outvaldstartymd = getEnbnInstanceCodeResRsListRec.getString("outValdStartYmd");
	String outvaldendymd = getEnbnInstanceCodeResRsListRec.getString("outValdEndYmd");
	String outinstncctnt = getEnbnInstanceCodeResRsListRec.getString("outInstncCtnt");

비 고	

1.22 에러메시지코드, 책임자승인사유코드, 조치메시지코드 조회 (FUBcMessage.getMessageCode())

프로그램 ID	FUBcMessage.getMessageCode()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	에러메시지목록	SQLMAP 명	NAME SPACE	fubcmessage
	영문명	THKJIER01		SQL ID	selectMessageCode

PoC

프로그램 설명	에러메시지목록의 에러정보를 읽어서 에러메시지코드, 책임자승인사유코드, 조치메시지코드를 조회한다.
프로그램 규칙 및 제약사항	1. 에러정보가 존재하지 않을 경우 오류를 반환한다. 2. 책임자승인사유코드 조회시는 에러메시지코드(책임자승인사유코드), 출력내용의 내용을 참조하고 나머지 출력항목값은 SPACE 가 된다. 3. 메시지 종류와 거래채널구분에 따라 유틸리티에서 내부적으로 아래와 같이 조회한다. - 조치메시지이고 채널구분이 02(자동화기기), 04(콜센터)이면 입력된 채널구분에 해당하는 조치메시지를 제공한다. - 에러메시지이고 채널구분이 대내외서버(11 ~ 99)이면 입력된 채널구분에 해당하는 에러메시지를 제공한다. 따라서 채널구분이 대외망인경우에는 XCJIER01-I-CHNL-DSTCD 항목을 반드시 SET 하여 모듈을 사용하도록 한다. - 기타의 경우는 01(단말)에 해당하는 에러/조치메시지를 제공한다. * 채널구분 : 업무인스턴스 117409000 참조 01:단말, 02:자동화기기, 04:콜센터, 11:연계업무, 12:대량이체, 13:CMS 공동망, 14:타행환공동망, 15:전자금융공동망, 16:PG 공동망,17:CD 공동망,18:ARS 공동망 (2009.7.9 기준) 4. 해당 채널에 대한 메시지코드가 없는 경우 아래와 같이 처리된다. - 에러메시지 : Default 에러메시지코드(B8000001)의 해당 채널용 메시지정보가 제공된다. - 조치메시지 : Default 조치메시지코드(UKJI9999)의 해당 채널용 메시지정보가 제공된다. - 해당 채널용 Default 메시지코드가 미정의된 경우 오류가 발생하며 사용자 맞춤메시지로 해당 메시지코드와 채널구분코드가 제공된다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcMessage extends`
	
	`com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inLangDstcd;` ● 언어구분
	`String inErrMsgCd ;` ● 에러메시지코드
	`String inChnlDstcd;` ● 채널구분코드
	
	[출력항목]
	`String outLangDstcd ;` // 언어구분코드
	`String outErrMsgCd ;` // 에러메시지코드
	`String outChnlDstcd ;` // 채널구분코드
	`String outRegiYmd ;` // 등록년월일
	`String outChnlErrMsgCd ;` // 채널에러메시지코드
	`String outErrStusDstcd ;` // 에러상태구분코드
	`String outCncutDscnDstcd ;` // 센터컷중단구분코드

PoC

	String outImpntEmAplcblYn;
	String outSpvsrRGrdDstcd;
	String outAplcnUapplCd;
	String outTreatRpMsgCtnt ;
	String outOutptCtnt;
		// 중요에러해당여부
		// 책임자사유등급구분코드
		// 신청어플리케이션코드
		// 조치담당자메시지내용
		// 출력내용

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 메시지코드(에러/책임자/조치)조회 (com.kbstar.kji.enbncmn, getMessageCode)
	
	IDataSet getMessageCodeReq = new DataSet();
	
	getMessageCodeReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getMessageCodeReq.put("inLangDstcd", requestData.get("inLangDstcd"));
	getMessageCodeReq.put("inErrMsgCd", requestData.get("inErrMsgCd"));
	getMessageCodeReq.put("inChnlDstcd", requestData.get("inChnlDstcd"));
	
	// @@ 공유메소드 호출 (메시지코드(에러/책임자/조치)조회 getMessageCode)
	IDataSet getMessageCodeRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcMessage.getMessageCode", getMessageCodeReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outlangdstcd = getMessageCodeRes.getString("outLangDstcd");
	String outerrmsgcd = getMessageCodeRes.getString("outErrMsgCd");
	String outchnldstcd = getMessageCodeRes.getString("outChnlDstcd");
	String outregiymd = getMessageCodeRes.getString("outRegiYmd");
	String outchnlerrmsgcd = getMessageCodeRes.getString("outChnlErrMsgCd");
	String outerrstusdstcd = getMessageCodeRes.getString("outErrStusDstcd");
	String outcncutdscndstcd = getMessageCodeRes.getString("outCncutDscnDstcd");
	String outimpntemaplcblyn = getMessageCodeRes.getString("outImpntEmAplcblYn");
	String outspvsrathordstcd = getMessageCodeRes.getString("outSpvsrAthorDstcd");
	String outspvsrrgrddstcd = getMessageCodeRes.getString("outSpvsrRGrdDstcd");
	String outaplcnuapplcd = getMessageCodeRes.getString("outAplcnUapplCd");
	String outtreatrpmsgctnt = getMessageCodeRes.getString("outTreatRpMsgCtnt");
	String outoutptctnt = getMessageCodeRes.getString("outOutptCtnt");	

비 고		

PoC

1.23 직원기본 조회(FUBcEmployee.getPafiarEmpInfo())

프로그램 ID	FUBcEmployee. getPafiarEmpInfo()		언어 구분	PoC JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	직원기본	PoC SQLMAP 명	NAME SPACE	fubcemployee
	영문명	THKJIHR01		SQL ID	selectPafiarEmpInfo

프로그램 설명	직원번호에 대한 인사직원정보를 조회한다.
프로그램 규칙 및 제약사항	1. 보안팀에서 수신한 인사직원정보를 기반으로 한다. 2. 직원의 부서이동 이력정보는 조회할 수 없다. 3. 직원번호 또는 직원주민번호(직원식별자)를 반드시 입력하여야 한다. 단, 직원번호와 주민번호(식별자)를 동시에 입력하여 조회는 불가능하다. 4. 직원정보는 직원번호, 직원명, 소속부점코드, 퇴직년월일, 부점장여부가 해당된다. PoC 예를 들어 현재 존재하는 직원정보를 조회하고자 할 경우 직원명, 소속부점코드, 퇴직년월일로 업무가 판단한다. 5. 사용자정보는 사용자소속부점코드, 사용자유형구분코드, 사용자인증여부, SOD 사용자유형구분코드, SOD 영업점유형구분코드, 입출금가능여부, 무자원대체가능여부, 근무상태구분코드, 사용자인증년월일이 해당된다. 6. 직원고객식별자(10) 제공 추가 직원고객식별자가 없으면 공백으로 제공되고 직원기본은 정상으로 조회 되며, 퇴사후 재 입행하여 직원번호가 여러개 인 경우 마지막 직원번호 정보가 제공 된다. ** 주의 : 직원주민등록번호는 2016.11 월 15 부터 제공되지 않음(KB-PIN 변환적용관련) 직원고객식별자 추가제공되며, 직원주민등록번호 꼭 필요한업무는 대체모듈인 BCEmployee.getEmpKBPinInfo()를 사용

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcEmployee extends com.kbstar.sqc.base.FunctionUnit` PoC
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inEmpid ; ◎ 직원번호
	String inEmpSsno; ◎ 주민번호
	
	[출력항목]
	String outEmpid; // 직원번호
	String outEmpHanglFname; // 직원한글성명
	String outEmpHanjaName; // 직원한자명
	String outEmpEngName; // 직원영문명
	String outPafiarBrnDstcd; // 인사부점구분
	String outBelngBrncd; // 소속부점코드
	String outEmpExusTelno; // 직원전용전화번호
	String outEmad; // 이메일주소
	String outDptchStartYmd; // 파견시작년월일
	String outDptchEndYmd; // 파견종료년월일
	String outDptchYn; // 파견여부
	String outDptchPBrnName; // 파견인사부점명
	String outDptchPBrnDstcd; // 파견인사부점구분
	String outOfodrYmd; // 발령년월일
	String outTmrstYmd; // 휴직년월일
	PoC
	PoC

	String outRtireYmd;	// 퇴직년월일
	String outMainDtyDstcd;	// 주직무구분
	String outMainDtyDsticName;	// 주직무구분명
	String outSubDtyDstcd1;	// 부직무구분코드 1
	String outSubDtyDsticName1;	// 부직무구분명 1
	String outSubDtyDstcd2;	// 부직무구분코드 2
	String outSubDtyDsticName2;	// 부직무구분명 2
	String outSubDtyDstcd3;	// 부직무구분코드 3
	String outSubDtyDsticName3;	// 부직무구분명 3
	String outSubDtyDstcd4;	// 부직무구분코드 4
	String outSubDtyDsticName4;	// 부직무구분명 4
	String outSubDtyDstcd5;	// 부직무구분코드 5
	String outSubDtyDsticName5;	// 부직무구분명 5
	String outSubDtyDstcd6;	// 부직무구분코드 6
	String outSubDtyDsticName6;	// 부직무구분명 6
	String outSubDtyDstcd7;	// 부직무구분코드 7
	String outSubDtyDsticName7;	// 부직무구분명 7
	String outSubDtyDstcd8;	// 부직무구분코드 8
	String outSubDtyDsticName8;	// 부직무구분명 8
	String outSubDtyDstcd9;	// 부직무구분코드 9
	String outSubDtyDsticName9;	// 부직무구분명 9
	String outSubDtyDstcd10;	// 부직무구분코드 10
	String outSubDtyDsticName10;	// 부직무구분명 10
	String outBrnmgrYn;	// 부점장여부
	String outJobtyDstcd;	// 직군구분코드
	String outJobtyDsticName;	// 직군구분명
	String outJobclDstcd;	// 직급구분
	String outJobclDsticName;	// 직급구분명
	String outPafiarJobtlDstcd;	// 인사직위구분
	String outPafiarJdhName;	// 인사직위구분한글명
	String outPafiarJdEngName;	// 인사직위구분영문명
	String outCnpstnPBrnDstcd1;	// 겸직인사부점구분코드 1
	String outCnpstnAcctgBrncd1;	// 겸직회계부점코드 1
	String outCnpstnPjDstcd1;	// 겸직인사직위구분코드 1
	String outCnpstnPjdName1;	// 겸직인사직위구분명 1
	String outCnpstnStartYmd1;	// 겸직시작년월일 1
	String outCnpstnEndYmd1;	// 겸직종료년월일 1
	String outCnpstnPBrnDstcd2;	// 겸직인사부점구분코드 2
	String outCnpstnAcctgBrncd2;	// 겸직회계부점코드 2
	String outCnpstnPjDstcd2;	// 겸직인사직위구분코드 2
	String outCnpstnPjdName2;	// 겸직인사직위구분명 2
	String outCnpstnStartYmd2;	// 겸직시작년월일 2
	String outCnpstnEndYmd2;	// 겸직종료년월일 2
	String outCnpstnPBrnDstcd3;	// 겸직인사부점구분코드 3
	String outCnpstnAcctgBrncd3;	// 겸직회계부점코드 3
	String outCnpstnPjDstcd3;	// 겸직인사직위구분코드 3
	String outCnpstnPjdName3;	// 겸직인사직위구분명 3
	String outCnpstnStartYmd3;	// 겸직시작년월일 3
	String outCnpstnEndYmd3;	// 겸직종료년월일 3
	String outCnpstnPBrnDstcd4;	// 겸직인사부점구분코드 4
	String outCnpstnAcctgBrncd4;	// 겸직회계부점코드 4
	String outCnpstnPjDstcd4;	// 겸직인사직위구분코드 4
	String outCnpstnPjdName4;	// 겸직인사직위구분명 4
	String outCnpstnStartYmd4;	// 겸직시작년월일 4
	String outCnpstnEndYmd4;	// 겸직종료년월일 4
	String outCnpstnPBrnDstcd5;	// 겸직인사부점구분코드 5
	String outCnpstnAcctgBrncd5;	// 겸직회계부점코드 5
	String outCnpstnPjDstcd5;	// 겸직인사직위구분코드 5
	String outCnpstnPjdName5;	// 겸직인사직위구분명 5
	String outCnpstnStartYmd5;	// 겸직시작년월일 5
	String outCnpstnEndYmd5;	// 겸직종료년월일 5
	String outUserBzwkBrncd;	// 사용자업무부점코드
	String outUserRoleCtnt;	// 사용자역할내용
	String outEmpDOrignDstcd;	// 직원데이터원천구분
	String outUserPtrnDstcd;	// 사용자유형구분

	String outSodUserPtrnDstcd;
	String outSodBbrnPtrnDstcd;
	String outRcdrwAbilYn;
	String outNbngRtAbilYn;
	String outWrkStusDstcd;
	String outExrtMsgRecvYn;
	String outAtmRspblPsnYn;
	String outFxRRspblPsnYn;
	String outMfUserCertnYmd;
	String outMfUserCertnHms;
	String outEmpSsno;
	String outCtrcESalryDstcd;
	String outOvrtAplcnYmd1;
	String outOvrtAStartHms1;
	String outOvrtAplcnEndHms1;
	String outOvrtAplcnYmd2;
	String outOvrtAStartHms2;
	String outOvrtAplcnEndHms2;
	String outBancaRspblPsnYn;
	String outLsdtTranYn;
	String outTeamDstcdName;
	String outCnslIntgreBrncd;
	String outEmpCphnNo;
	String outSthlCoDstcd;
	String outAddrRepsntDstcd;
	String outEmpCustIdnfr;
		// SOD 사용자유형구분
		// SOD 영업점유형구분
		// 입출금가능여부
		// 무자원대체가능여부
		// 근무상태구분
		// 환율메시지수신여부
		// 자동화기기담당자여부
		// 외환출납담당자여부
		// MF 사용자인증년월일
		// MF 사용자인증시각
		// 직원주민번호
		// 계약직원급여구분코드
		// 야근신청년월일 1
		// 야근신청시작시각 1
		// 야근신청종료시각 1
		// 야근신청년월일 2
		// 야근신청시작시각 2
		// 야근신청종료시각 2
		// 방카슈랑스담당자여부
		// 전일자거래여부
		// 팀 구분명
		// 연결통합부점코드
		// 직원휴대폰번호
		// 지주회사구분
		// 주소표시구분
		// 직원고객식별자

RTNSTAT 설명	N/A	

사용 설명(예시)	**// 공유메소드 호출 - 직원기본조회 (com.kbstar.kji.enbncmn, getPafiarEmpInfo)**
	
	IDataSet getPafiarEmpInfoReq = **new** DataSet();
	
	getPafiarEmpInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getPafiarEmpInfoReq.put("inEmpid", requestData.get("inEmpid"));
	getPafiarEmpInfoReq.put("inEmpSsno", requestData.get("inEmpSsno"));
	getPafiarEmpInfoReq.put("inEmpCustIdnfr", requestData.get("inEmpCustIdnfr"));
	
	// @@ 공유메소드 호출 (직원기본조회 getPafiarEmpInfo)
	IDataSet getPafiarEmpInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEmployee.getPafiarEmpInfo", getPafiarEmpInfoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outempid = getPafiarEmpInfoRes.getString("outEmpid"); // 직원번호
	String outemphanglfname = getPafiarEmpInfoRes.getString("outEmpHanglFname"); // 직원한글성명
	String outemphanjaname = getPafiarEmpInfoRes.getString("outEmpHanjaName"); // 직원한자명
	String outempengname = getPafiarEmpInfoRes.getString("outEmpEngName"); // 직원영문명
	String outempdorigndstcd = getPafiarEmpInfoRes.getString("outEmpDOrignDstcd"); // 직원데이터원천구분
	String outempssno = getPafiarEmpInfoRes.getString("outEmpSsno"); // 직원주민번호
	String outpafiarbrndstcd = getPafiarEmpInfoRes.getString("outPafiarBrnDstcd"); // 인사부점구분
	String outbelngbrncd = getPafiarEmpInfoRes.getString("outBelngBrncd"); // 소속부점코드
	String outempexustelno = getPafiarEmpInfoRes.getString("outEmpExusTelno"); //	

PoC

```
직원전용전화번호
String outemad = getPafiarEmpInfoRes.getString("outEmad"); // 이메일주소
String outdptchstartymd = getPafiarEmpInfoRes.getString("outDptchStartYmd"); // 파견시작년월일
String outdptchendymd = getPafiarEmpInfoRes.getString("outDptchEndYmd"); // 파견종료년월일
String outdptchyn = getPafiarEmpInfoRes.getString("outDptchYn"); // 파견여부
String outdptchpbrnname = getPafiarEmpInfoRes.getString("outDptchPBrnName"); //
파견인사부점명
String outdptchpbrndstcd = getPafiarEmpInfoRes.getString("outDptchPBrnDstcd"); //
파견인사부점구분
String outofodrymd = getPafiarEmpInfoRes.getString("outOfodrYmd"); // 발령년월일
String outtmrstymd = getPafiarEmpInfoRes.getString("outTmrstYmd"); // 휴직년월일
String outrtireymd = getPafiarEmpInfoRes.getString("outRtireYmd"); // 퇴직년월일
String outmaindtydstcd = getPafiarEmpInfoRes.getString("outMainDtyDstcd"); // 주직무구분
String outmaindtydsticname = getPafiarEmpInfoRes.getString("outMainDtyDsticName"); //
주직무구분명
String outsubdtydstcd1 = getPafiarEmpInfoRes.getString("outSubDtyDstcd1"); // 부직무구분1
String outsubdtydsticname1 = getPafiarEmpInfoRes.getString("outSubDtyDsticName1"); //
부직무구분명1
String outsubdtydstcd2 = getPafiarEmpInfoRes.getString("outSubDtyDstcd2"); // 부직무구분2
String outsubdtydsticname2 = getPafiarEmpInfoRes.getString("outSubDtyDsticName2"); //
부직무구분명2
String outsubdtydstcd3 = getPafiarEmpInfoRes.getString("outSubDtyDstcd3"); // 부직무구분3
String outsubdtydsticname3 = getPafiarEmpInfoRes.getString("outSubDtyDsticName3"); //
부직무구분명3
String outsubdtydstcd4 = getPafiarEmpInfoRes.getString("outSubDtyDstcd4"); // 부직무구분4
String outsubdtydsticname4 = getPafiarEmpInfoRes.getString("outSubDtyDsticName4"); //
부직무구분명4
String outsubdtydstcd5 = getPafiarEmpInfoRes.getString("outSubDtyDstcd5"); // 부직무구분5
String outsubdtydsticname5 = getPafiarEmpInfoRes.getString("outSubDtyDsticName5"); //
부직무구분명5
String outsubdtydstcd6 = getPafiarEmpInfoRes.getString("outSubDtyDstcd6"); // 부직무구분6
String outsubdtydsticname6 = getPafiarEmpInfoRes.getString("outSubDtyDsticName6"); //
부직무구분명6
String outsubdtydstcd7 = getPafiarEmpInfoRes.getString("outSubDtyDstcd7"); // 부직무구분7
String outsubdtydsticname7 = getPafiarEmpInfoRes.getString("outSubDtyDsticName7"); //
부직무구분명7
String outsubdtydstcd8 = getPafiarEmpInfoRes.getString("outSubDtyDstcd8"); // 부직무구분8
String outsubdtydsticname8 = getPafiarEmpInfoRes.getString("outSubDtyDsticName8"); //
부직무구분명8
String outsubdtydstcd9 = getPafiarEmpInfoRes.getString("outSubDtyDstcd9"); // 부직무구분9
String outsubdtydsticname9 = getPafiarEmpInfoRes.getString("outSubDtyDsticName9"); //
부직무구분명9
String outsubdtydstcd10 = getPafiarEmpInfoRes.getString("outSubDtyDstcd10"); // 부직무구분10
String outsubdtydsticname10 = getPafiarEmpInfoRes.getString("outSubDtyDsticName10"); //
부직무구분명10
String outbrnmgryn = getPafiarEmpInfoRes.getString("outBrnmgrYn"); // 부점장여부
```

```
String outjobtydstcd = getPafiarEmpInfoRes.getString("outJobtyDstcd"); // 직군구분
String outjobtydsticname = getPafiarEmpInfoRes.getString("outJobtyDsticName"); // 직군구분명
String outjobcldstcd = getPafiarEmpInfoRes.getString("outJobclDstcd"); // 직급구분
String outjobcldsticname = getPafiarEmpInfoRes.getString("outJobclDsticName"); // 직급구분명
String outpafiarjobtldstcd = getPafiarEmpInfoRes.getString("outPafiarJobtlDstcd"); // 인사직위구분
String outpafiarjdhname = getPafiarEmpInfoRes.getString("outPafiarJdhName"); //
인사직위구분한글명
String outpafiarjdengname = getPafiarEmpInfoRes.getString("outPafiarJdEngName"); //
인사직위구분영문명
String outctrcesalrydstcd = getPafiarEmpInfoRes.getString("outCtrcESalryDstcd"); //
계약직원급여구분
String outovrtaplcnymd1 = getPafiarEmpInfoRes.getString("outOvrtAplcnYmd1"); //
야근신청년월일1
String outovrtastarthms1 = getPafiarEmpInfoRes.getString("outOvrtAStartHms1"); //
야근신청시작시각1
String outovrtaplcnendhms1 = getPafiarEmpInfoRes.getString("outOvrtAplcnEndHms1"); //
야근신청종료시각1
String outovrtaplcnymd2 = getPafiarEmpInfoRes.getString("outOvrtAplcnYmd2"); //
야근신청년월일2
String outovrtastarthms2 = getPafiarEmpInfoRes.getString("outOvrtAStartHms2"); //
야근신청시작시각2
String outovrtaplcnendhms2 = getPafiarEmpInfoRes.getString("outOvrtAplcnEndHms2"); //
야근신청종료시각2
String outcnpstnpbrndstcd1 = getPafiarEmpInfoRes.getString("outCnpstnPBrnDstcd1"); //
겸직인사부점구분1
String outcnpstnacctgbrncd1 = getPafiarEmpInfoRes.getString("outCnpstnAcctgBrncd1"); //
겸직회계부점코드1
String outcnpstnpjdstcd1 = getPafiarEmpInfoRes.getString("outCnpstnPjDstcd1"); //
겸직인사직위구분1
String outcnpstnpjdname1 = getPafiarEmpInfoRes.getString("outCnpstnPjdName1"); //
겸직인사직위구분명1
String outcnpstnstartymd1 = getPafiarEmpInfoRes.getString("outCnpstnStartYmd1"); //
겸직시작년월일1
String outcnpstnendymd1 = getPafiarEmpInfoRes.getString("outCnpstnEndYmd1"); //
겸직종료년월일1
String outcnpstnpbrndstcd2 = getPafiarEmpInfoRes.getString("outCnpstnPBrnDstcd2"); //
겸직인사부점구분2
String outcnpstnacctgbrncd2 = getPafiarEmpInfoRes.getString("outCnpstnAcctgBrncd2"); //
겸직회계부점코드2
String outcnpstnpjdstcd2 = getPafiarEmpInfoRes.getString("outCnpstnPjDstcd2"); //
겸직인사직위구분2
String outcnpstnpjdname2 = getPafiarEmpInfoRes.getString("outCnpstnPjdName2"); //
겸직인사직위구분명2
String outcnpstnstartymd2 = getPafiarEmpInfoRes.getString("outCnpstnStartYmd2"); //
겸직시작년월일2
String outcnpstnendymd2 = getPafiarEmpInfoRes.getString("outCnpstnEndYmd2"); //
```

```
겸직종료년월일2
String outcnpstnpbrndstcd3 = getPafiarEmpInfoRes.getString("outCnpstnPBrnDstcd3"); //
겸직인사부점구분3
String outcnpstnacctgbrncd3 = getPafiarEmpInfoRes.getString("outCnpstnAcctgBrncd3"); //
겸직회계부점코드3
String outcnpstnpjdstcd3 = getPafiarEmpInfoRes.getString("outCnpstnPjDstcd3"); //
겸직인사직위구분3
String outcnpstnpjdname3 = getPafiarEmpInfoRes.getString("outCnpstnPjdName3"); //
겸직인사직위구분명3
String outcnpstnstartymd3 = getPafiarEmpInfoRes.getString("outCnpstnStartYmd3"); //
겸직시작년월일3
String outcnpstnendymd3 = getPafiarEmpInfoRes.getString("outCnpstnEndYmd3"); //
겸직종료년월일3
String outcnpstnpbrndstcd4 = getPafiarEmpInfoRes.getString("outCnpstnPBrnDstcd4"); //
겸직인사부점구분4
String outcnpstnacctgbrncd4 = getPafiarEmpInfoRes.getString("outCnpstnAcctgBrncd4"); //
겸직회계부점코드4
String outcnpstnpjdstcd4 = getPafiarEmpInfoRes.getString("outCnpstnPjDstcd4"); //
겸직인사직위구분4
String outcnpstnpjdname4 = getPafiarEmpInfoRes.getString("outCnpstnPjdName4"); //
겸직인사직위구분명4
String outcnpstnstartymd4 = getPafiarEmpInfoRes.getString("outCnpstnStartYmd4"); //
겸직시작년월일4
String outcnpstnendymd4 = getPafiarEmpInfoRes.getString("outCnpstnEndYmd4"); //
겸직종료년월일4
String outcnpstnpbrndstcd5 = getPafiarEmpInfoRes.getString("outCnpstnPBrnDstcd5"); //
겸직인사부점구분5
String outcnpstnacctgbrncd5 = getPafiarEmpInfoRes.getString("outCnpstnAcctgBrncd5"); //
겸직회계부점코드5
String outcnpstnpjdstcd5 = getPafiarEmpInfoRes.getString("outCnpstnPjDstcd5"); //
겸직인사직위구분5
String outcnpstnpjdname5 = getPafiarEmpInfoRes.getString("outCnpstnPjdName5"); //
겸직인사직위구분명5
String outcnpstnstartymd5 = getPafiarEmpInfoRes.getString("outCnpstnStartYmd5"); //
겸직시작년월일5
String outcnpstnendymd5 = getPafiarEmpInfoRes.getString("outCnpstnEndYmd5"); //
겸직종료년월일5
String outuserbzwkbrncd = getPafiarEmpInfoRes.getString("outUserBzwkBrncd"); //
사용자업무부점코드
String outuserptrndstcd = getPafiarEmpInfoRes.getString("outUserPtrnDstcd"); // 사용자유형구분
String outsoduserptrndstcd = getPafiarEmpInfoRes.getString("outSodUserPtrnDstcd"); //
SOD사용자유형구분
String outsodbbrnptrndstcd = getPafiarEmpInfoRes.getString("outSodBbrnPtrnDstcd"); //
SOD영업점유형구분
String outrcdrwabilyn = getPafiarEmpInfoRes.getString("outRcdrwAbilYn"); // 입출금가능여부
String outnbngraltrabilyn = getPafiarEmpInfoRes.getString("outNbngRAltrAbilYn"); //
```

	무자원대체가능여부
	String outwrkstusdstcd = getPafiarEmpInfoRes.getString("outWrkStusDstcd"); // 근무상태구분
	String outexrtmsgrecvyn = getPafiarEmpInfoRes.getString("outExrtMsgRecvYn"); // 환율메시지수신여부
	String outatmrspblpsnyn = getPafiarEmpInfoRes.getString("outAtmRspblPsnYn"); // 자동화기기담당자여부
	String outfxrrspblpsnyn = getPafiarEmpInfoRes.getString("outFxRRspblPsnYn"); // 외환출납담당자여부
	String outbancarspblpsnyn = getPafiarEmpInfoRes.getString("outBancaRspblPsnYn"); // 방카슈랑스담당자여부
	String outuserrolectnt = getPafiarEmpInfoRes.getString("outUserRoleCtnt"); // 사용자역할내용
	String outmfusercertnymd = getPafiarEmpInfoRes.getString("outMfUserCertnYmd"); // MF사용자인증년월일
	String outmfusercertnhms = getPafiarEmpInfoRes.getString("outMfUserCertnHms"); // MF사용자인증시각
	String outlsdttranyn = getPafiarEmpInfoRes.getString("outLsdtTranYn"); // 전일자거래여부
	String outsyslastprcssyms = getPafiarEmpInfoRes.getString("outSysLastPrcssYms"); // 시스템최종처리일시
	String outsyslastuno = getPafiarEmpInfoRes.getString("outSysLastUno"); // 시스템최종사용자번호
	String outteamdstcdname = getPafiarEmpInfoRes.getString("outTeamDstcdName"); // 팀구분명
	String outcnslintgrebrncd = getPafiarEmpInfoRes.getString("outCnslIntgreBrncd"); // 연결통합부점코드
	String outempcphnno = getPafiarEmpInfoRes.getString("outEmpCphnNo"); // 직원휴대폰번호
	String outbrncd1 = getPafiarEmpInfoRes.getString("outBrncd1"); // 부점코드1
	String outbrncd2 = getPafiarEmpInfoRes.getString("outBrncd2"); // 부점코드2
	String outbrncd3 = getPafiarEmpInfoRes.getString("outBrncd3"); // 부점코드3
	String outempdsticabrvnname1 = getPafiarEmpInfoRes.getString("outEmpDsticAbrvnName1"); // 직원구분약어명1
	String outempdsticabrvnname2 = getPafiarEmpInfoRes.getString("outEmpDsticAbrvnName2"); // 직원구분약어명2
	String outempdsticabrvnname3 = getPafiarEmpInfoRes.getString("outEmpDsticAbrvnName3"); // 직원구분약어명3
	String outsthlcodstcd = getPafiarEmpInfoRes.getString("outSthlCoDstcd"); // 지주회사구분
	String outaddrrepsntdstcd = getPafiarEmpInfoRes.getString("outAddrRepsntDstcd"); // 주소표시구분
	String outempcustidnfr = getPafiarEmpInfoRes.getString("outEmpCustIdnfr"); // 직원고객식별자

비 고	

PoC

1.24 IT 직원정보 조회(FUBcEmployee.getITEmpInfo())

프로그램 ID	FUBcEmployee.getITEmpInfo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	IT직원기본	SQLMAP 명	NAME SPACE	fubcemployee
	영문명	THKJIHR03		SQL ID	selectITEmpInfo

프로그램 설명	직원번호에 대한 IT 직원정보를 조회한다.
프로그램 규칙 및 제약사항	1. 보안팀에서 수신한 IT 직원정보를 기반으로 한다. 2. 조회시점에 근무 중인 IT 직원정보만 조회한다. 단, 직원번호에 해당하는 모든 사무분담업무를 확인하려면 다양한 조건의 전체 IT 직원정보 조회(FUBcEmployee.getWholITEmpInfo())를 이용해야 한다. 3. 소속업무구분코드는 해당직원의 어플리케이션을 의미하며, 단위업무구분코드는 사무분담구분을 의미한다.

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcEmployee **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inEmpid ; ● 직원번호
	String inBelngUapplCd; ● 소속어플리케이션코드
	String inUnitBzwkDstcd; ● 단위업무구분
	String inYmd; ◎ 년월일
	
	[출력항목]
	String outEmpid; // 직원번호
	String outBelngUapplCd; // 소속어플리케이션코드
	String outUnitBzwkDstcd; // 단위업무구분
	String outAplyStartYmd; // 적용시작년월일
	String outAplyEndYmd; // 적용종료년월일
	String outEmpHanglFname; // 직원한글성명
	String outBelngDvsnDstcd; // 소속부서구분
	String outBelngDvsnName; // 소속부서명
	String outBelngTeamDstcd; // 소속팀구분
	String outBelngTeamName; // 소속팀명
	String outAcctgBrncd; // 회계부점코드
	String outDvsnChfzEmpid; // 부서장직원번호
	String outDvsnCHanglFname; // 부서장한글성명
	String outTmldEmpid; // 팀장직원번호
	String outTmldHanglFname; // 팀장한글성명
	String outEmpExusTelno; // 직원전용전화번호
	String outEmpCphnNo; // 직원휴대폰번호

	String outEmad; // 이메일주소
	String outOwhusZip; // 자택우편번호
	String outEmpOwhusAddr; // 직원자택주소
	String outOutsrcSTeamName; // 외주지원팀명
	String outOutsrcBCoName; // 외주소속회사명
	String outOutsrcStartYmd; // 외주시작년월일
	String outOutsrcEndYmd; // 외주종료년월일
	String outItHqWrkCtnt; // IT 본부근무내용
	String outUnitBzwkName; // 단위업무명
	String outDsticName; // 구분명
	String outInptStartYmd; // 투입시작년월일
	String outInptEndYmd; // 투입종료년월일
	String outEmpDsticCtnt; // 직원구분내용
	String outItTDstic; // IT 재직구분
	String outItTDsticCtnt; // IT 재직구분내용
	String outItMovinYmd; // IT 전입년월일
	String outItMovoutYmd; // IT 전출년월일

RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - IT직원정보 조회 (com.kbstar.kji.enbncmn, getITEmpInfo)**
	
	IDataSet getITEmpInfoReq = new DataSet();
	
	getITEmpInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getITEmpInfoReq.put("inEmpid", requestData.get("inEmpid"));
	getITEmpInfoReq.put("inBelngUapplCd", requestData.get("inBelngUapplCd"));
	getITEmpInfoReq.put("inUnitBzwkDstcd", requestData.get("inUnitBzwkDstcd"));
	getITEmpInfoReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (IT직원정보 조회 getITEmpInfo)
	IDataSet getITEmpInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEmployee.getITEmpInfo", getITEmpInfoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outempid = getITEmpInfoRes.getString("outEmpid"); // 직원번호
	String outbelnguapplcd = getITEmpInfoRes.getString("outBelngUapplCd");// 소속어플리케이션코드
	String outunitbzwkdstcd = getITEmpInfoRes.getString("outUnitBzwkDstcd");// 단위업무구분
	String outaplystartymd = getITEmpInfoRes.getString("outAplyStartYmd");// 적용시작년월일
	String outaplyendymd = getITEmpInfoRes.getString("outAplyEndYmd");//적용종료년월일
	String outemphanglfname = getITEmpInfoRes.getString("outEmpHanglFname");// 직원한글성명
	String outbelngdvsndstcd = getITEmpInfoRes.getString("outBelngDvsnDstcd");//소속부서구분
	String outbelngdvsnname = getITEmpInfoRes.getString("outBelngDvsnName"); // 소속부서명
	String outbelngteamdstcd = getITEmpInfoRes.getString("outBelngTeamDstcd"); // 소속팀구분
	String outbelngteamname = getITEmpInfoRes.getString("outBelngTeamName"); // 소속팀명
	String outacctgbrncd = getITEmpInfoRes.getString("outAcctgBrncd"); // 회계부점코드

	String outdvsnchfzempid = getITEmpInfoRes.getString("outDvsnChfzEmpid"); // 부서장직원번호
	String outdvsnchanglfname = getITEmpInfoRes.getString("outDvsnCHanglFname"); // 부서장한글성명
	String outtmldempid = getITEmpInfoRes.getString("outTmldEmpid"); // 팀장직원번호
	String outtmldhanglfname = getITEmpInfoRes.getString("outTmldHanglFname"); // 팀장한글성명
	String outempexustelno = getITEmpInfoRes.getString("outEmpExusTelno"); // 직원전용전화번호
	String outempcphnno = getITEmpInfoRes.getString("outEmpCphnNo"); // 직원휴대폰번호
	String outemad = getITEmpInfoRes.getString("outEmad"); // 이메일주소
	String outowhuszip = getITEmpInfoRes.getString("outOwhusZip"); // 자택우편번호
	String outempowhusaddr = getITEmpInfoRes.getString("outEmpOwhusAddr"); // 직원자택주소
	String outoutsrcsteamname = getITEmpInfoRes.getString("outOutsrcSTeamName");// 외주지원팀명
	String outoutsrcbconame = getITEmpInfoRes.getString("outOutsrcBCoName");// 외주소속회사명
	String outoutsrcstartymd = getITEmpInfoRes.getString("outOutsrcStartYmd");// 외주시작년월일
	String outoutsrcendymd = getITEmpInfoRes.getString("outOutsrcEndYmd"); // 외주종료년월일
	String outithqwrkctnt = getITEmpInfoRes.getString("outItHqWrkCtnt"); // IT본부근무내용
	String outunitbzwkname = getITEmpInfoRes.getString("outUnitBzwkName"); // 단위업무명
	String outdsticname = getITEmpInfoRes.getString("outDsticName"); // 구분명
	String outinptstartymd = getITEmpInfoRes.getString("outInptStartYmd");// 투입시작년월일
	String outinptendymd = getITEmpInfoRes.getString("outInptEndYmd");// 투입종료년월일
	String outempdsticctnt = getITEmpInfoRes.getString("outEmpDsticCtnt");// 직원구분내용
	String outittdstic = getITEmpInfoRes.getString("outItTDstic");// IT재직구분
	String outittdsticctnt = getITEmpInfoRes.getString("outItTDsticCtnt");// IT재직구분내용
	String outitmovinymd = getITEmpInfoRes.getString("outItMovinYmd");// IT전입년월일
	String outitmovoutymd = getITEmpInfoRes.getString("outItMovoutYmd");// IT전출년월일
	String outaddrrepsntdstcd = getITEmpInfoRes.getString("outAddrRepsntDstcd"); // 주소표시구분
	String outpafiarjobtldstcd = getITEmpInfoRes.getString("outPafiarJobtlDstcd"); // 인사직위구분
	String outspotprxyempid = getITEmpInfoRes.getString("outSpotPrxyEmpid"); // 현장대리인직원번호
	String outspotprxyhanglfname = getITEmpInfoRes.getString("outSpotPrxyHanglFname"); // 현장대리인한글성명

비 고	

PoC

1.25 다양한 조건의 전체직원 조회(FUBcEmployee.getWholPafiarEmpInfo())

프로그램 ID	FUBcEmployee. getWholPafiarEmpInfo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	직원기본	SQLMAP 명	NAME SPACE	fubcemployee
	영문명	THKJIHR01		SQL ID	selectWholPafiar EmpInfo1 selectWholPafiar EmpInfo2

프로그램 설명	다양한 조회 조건에 의한 전체 직원번호를 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 1) '1' : 소속부점코드의 소속직원 전체를 조회(퇴직직원포함) 2) '2' : 사용자 업무부점코드의 소속직원 전체를 조회(퇴직직원포함) 3) '3' : 소속부점코드의 소속 재직직원 전체를 조회(2021.08 추가) 4) '4' : 사용자 업무부점코드의 소속 재직직원 전체를 조회(2021.08 추가) 2. 해당 구분코드에 대한 직원번호 및 직원한글성명을 조회한다. 3. 직원정보는 최대 600 명 까지 조회가능하며 초과시 에러(UKJI1001)처리 한다. ** 600 명을 초과, 에러발생시 해당업무는 직원기본(THKJIHR01)테이블을 직접 참조하여 조회한다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcEmployee extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd ;` ● 그룹회사코드
	`String inDstcd ;` ● 구분코드
	`String inBelngBrncd;` ● 소속부점코드
	
	[출력항목]
	`int outArayCnt ;` // 배열개수
	`List LIST;` // 직원 List
	
	< LIST 항목 >
	`String outEmpid ;` // 직원번호
	`String outEmpHanglFname;` // 직원한글성명

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 다양한 조건의 전체직원 조회 (com.kbstar.kji.enbncmn, getWholPafiarEmpInfo)
	
	IDataSet getWholPafiarEmpInfoReq = new DataSet();
	
	getWholPafiarEmpInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getWholPafiarEmpInfoReq.put("inDstcd", requestData.get("inDstcd"));
	getWholPafiarEmpInfoReq.put("inBelngBrncd", requestData.get("inBelngBrncd"));
	
	// @@ 공유메소드 호출 (다양한 조건의 전체직원 조회 getWholPafiarEmpInfo)
	IDataSet getWholPafiarEmpInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcEmployee.getWholPafiarEmpInfo", getWholPafiarEmpInfoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outaraycnt = getWholPafiarEmpInfoRes.getString("outArayCnt"); // 배열개수
	IRecordSet getWholPafiarEmpInfoResRsList = getWholPafiarEmpInfoRes.getRecordSet("LIST"); // recordSet1

비 고	

PoC

PoC

PoC

PoC

1.26

다양한 조건의 전체 IT 직원 조회(FUBcEmployee.getWholITEmpInfo()))

프로그램 ID	FUBcEmployee. getWholITEmpInfo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	IT직원기본	SQLMAP 명	NAME SPACE	fubcemployee
	영문명	THKJIHR03		SQL ID	selectWholITEmpInfo1 selectWholITEmpInfo2

프로그램 설명	다양한 조회 조건에 대한 전체 IT 직원번호를 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 '1' : 소속팀구분코드의 전체 IT 직원 조회 '2' : IT 직원번호에 대한 복수업무 정보를 조회한다. 2. 해당 구분코드에 대한 IT 직원기본의 직원번호, 직원한글성명, 소속업무구분, 단위업무구분 코드를 조회한다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcEmployee extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inDstcd; ● 구분코드
	String inBelngTeamDstcd; ◎ 소속팀구분코드
	String inEmpid; ◎ 직원번호
	String inYmd; ◎ 년월일
	
	[출력항목]
	int outArayCnt ; // 배열개수
	List LIST ; // 직원 List
	
	< LIST 항목 >
	String outEmpId; //직원번호
	String outEmpHanglFname; //직원한글성명
	String outBelngUapplCd; //소속어플리케이션코드
	String outUnitBzwkDstcd; //단위업무구분

RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - 다양한 조건의 전체IT직원 조회 (com.kbstar.kji.enbncmn, getWholITEmpInfo)**
	
	`IDataSet getWholITEmpInfoReq = new DataSet();`
	
	`getWholITEmpInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));`
	`getWholITEmpInfoReq.put("inDstcd", requestData.get("inDstcd"));`

PoC

	getWholITEmpInfoReq.put("inBelngTeamDstcd", requestData.get("inBelngTeamDstcd"));
	getWholITEmpInfoReq.put("inEmpid", requestData.get("inEmpid"));
	getWholITEmpInfoReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (다양한 조건의 전체IT직원 조회 getWholITEmpInfo)
	IDataSet getWholITEmpInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEmployee.getWholITEmpInfo", getWholITEmpInfoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	IRecordSet getWholITEmpInfoResRsList = getWholITEmpInfoRes.getRecordSet("LIST");

비 고	

PoC

1.27

PoC

PoC

PoC

PoC

KB 기준 정보 조회/ 변경(FUBcBase.prcssKBBaseInfo())

프로그램 ID PoC	FUBcBase.prcssKBBaseInfo()		언어 구분	PoC JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	KB기준정보	SQLMAP 명	NAME SPACE	fubcbase
	영문명	THKJIKB01		SQL ID	selectKBBaseInfo updateCncutFnshYn updateAcldntRrppFnshYn

프로그램 설명	KB 기준 정보를 조회하고 센터컷완료여부, 사고신고접수분처리완료여부에 대한 변경을 한다.
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같이 입력받는다. 1) '1' : KB 기준 정보 조회 2) '2' : 센터컷완료여부 변경 3) '3' : 사고접수분완료여부 변경 4) '4' : IT 통제여부 변경 5) '5' : 업무통제여부 변경 2. 구분코드는 필수입력항목이다. 3. 구분코드 '2', '3', '4', '5' 인 경우는 완료(통제)여부('0', '1')가 필수입력항목이다. 4. 센터컷완료여부, 사고접수분완료여부, IT 통제여부, 업무통제여부 변경시에는 출력항목은 SPACE 이고 정상적으로 변경시 리턴상태코드가 '00'(정상)으로 반환된다.

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcBase **extends** com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd;; ● 그룹회사코드
	String inDstcd; PoC ● 구분코드
	String inFnshYn; ◎ 완료여부
	
	[출력항목]
	String outTranBaseYmd; // 거래기준년월일
	String outBfBfBzoprYmd; // 전전영업년월일
	String outBfBzoprYmd; PoC // 전영업년월일
	String outNowBzoprYmd; PoC // 금영업년월일
	String outNxbizYmd; // 익영업년월일
	String outN2BzoprYmd; // N2 영업년월일
	String outN3BzoprYmd; PoC // N3 영업년월일
	String outN4BzoprYmd; // N4 영업년월일
	String outN5BzoprYmd; // N5 영업년월일
	String outLunrTranBaseYmd; // 음력거래기준년월일
	String outLunrBzoprYmd; // 음력영업년월일
	String outCncutFnshYn; // 센터컷완료여부
	String outAcidntRpFnshYn; // 사고접수분완료여부
	String outTranCtrlYn; // 거래통제여부
	String outTranCtrlModfiYmd;

	String outTranCtrlModfiHms; // 거래통제변경년월일
	// 거래통제변경시각
	PoC

RTNSTAT 설명	N/A PoC

사용 설명(예시)	// 공유메소드 호출 - KB기준정보 조회/변경 (com.kbstar.kji.enbncmn, prcssKBBaseInfo)
	
	IDataSet prcssKBBaseInfoReq = new DataSet();
	
	prcssKBBaseInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	prcssKBBaseInfoReq.put("inDstcd", requestData.get("inDstcd"));
	prcssKBBaseInfoReq.put("inFnshYn", requestData.get("inFnshYn"));
	
	// @@ 공유메소드 호출 (KB기준정보 조회/변경 prcssKBBaseInfo)
	IDataSet prcssKBBaseInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBase.prcssKBBaseInfo", prcssKBBaseInfoReq, onlineCtx);
	PoC
	
	// @@ 출력 데이터 셋 처리
	String outtranbaseymd = prcssKBBaseInfoRes.getString("outTranBaseYmd");
	String outbfbfbzoprymd = prcssKBBaseInfoRes.getString("outBfBfBzoprYmd");
	String outbfbzoprymd = prcssKBBaseInfoRes.getString("outBfBzoprYmd");
	String outnowbzoprymd = prcssKBBaseInfoRes.getString("outNowBzoprYmd");
	String outnxbizymd = prcssKBBaseInfoRes.getString("outNxbizYmd");
	String outn2bzoprymd = prcssKBBaseInfoRes.getString("outN2BzoprYmd");
	String outn3bzoprymd = prcssKBBaseInfoRes.getString("outN3BzoprYmd");
	String outn4bzoprymd = prcssKBBaseInfoRes.getString("outN4BzoprYmd");
	String outn5bzoprymd = prcssKBBaseInfoRes.getString("outN5BzoprYmd");
	String outlunrtranbaseymd = prcssKBBaseInfoRes.getString("outLunrTranBaseYmd");
	String outlunrbzoprymd = prcssKBBaseInfoRes.getString("outLunrBzoprYmd");
	String outcncutfnshyn = prcssKBBaseInfoRes.getString("outCncutFnshYn");
	String outacidntrpfnshyn = prcssKBBaseInfoRes.getString("outAcidntRpFnshYn");
	String outtranctrlyn = prcssKBBaseInfoRes.getString("outTranCtrlYn");
	String outtranctrlmodfiymd = prcssKBBaseInfoRes.getString("outTranCtrlModfiYmd");
	String outtranctrlmodfihms = prcssKBBaseInfoRes.getString("outTranCtrlModfiHms");
	PoC PoC PoC

비 고	

1.28 일수/월수/영업일수/공휴일수 산출(FUBcDate.calculateNumberOfDays())

PoC

프로그램 ID	FUBcDate.calculateNumberOfDays()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	휴일목록	SQLMAP 명	NAME SPACE	fubcdate
	영문명	THKJIH001		SQL ID	selectHoldyCnt

프로그램 설명	시작년월일과 종료년월일에 대한 일수, 월수, 영업일수, 공휴일수, 윤년일수를 산출한다. PoC
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같이 입력받는다. 1) '1' : 일수/월수/윤년일수 산출 2) '2' : 일수/월수/영업일수/공휴일수/윤년일수 산출 *2. 구분코드가 '2' 인경우는 1976 년 부터 2200 년 까지의 일자만 산출이 가능하다.* 3. 일수, 영업일수, 공휴일수는 한편넣기(시작년월일 불포함,종료년월일 포함)를 한다. 4. 일수산출시 윤년을 포함한 일수를 산출함으로 상품 중 목돈마련상여금저축, 목돈마련월급여저축은 비즈니스 요건에 맞추어 일수에서 윤년일수를 차감하여 사용한다. 5. 월수는 아래와 같이 계산한다. 1) 월수 = (종료년-시작년) * 12 + (종료월 - 시작월) 2) 종료년월일의(예:20080115) 일자가 시작년월일의(예:20080930) 일자보다 작은 경우는 (월수 - 1)을 월수로 한다. 즉, 종료년월일의 일자가 15 이고 시작년월일의 일자가 30 일로 두 일자간의 차이가 한달이 되지 않으므로 월수에서 1 을 차감하여 월수를 산출한다 PoC PoC 3) 시작년월일과(예:20080131) 종료년월일이(예:20080930) 말일자에 해당하고 종료년월일의 일자(30)가 시작년월일의 일자(31)보다 작은 경우 리턴상태코드는 '07' 값을 리턴한다. 예시] 1) 시작년월일: 2007-10-31, 종료년월일: 2008-03-31 -> 일수: 152, 월수: 5, 영업일수: 102, 공휴일수: 50, 윤년일수: 1, 리턴상태코드: 00 2) 시작년월일: 2007-10-31, 종료년월일: 2008-03-27 -> 일수: 148, 월수: 4, 영업일수: 100 공휴일수: 48, 윤년일수: 1, 리턴상태코드: 00 3) 시작년월일: 2007-10-31, 종료년월일: 2008-02-29 PoC -> 일수: 121, 월수: 4, 영업일수: 81, 공휴일수: 40, 윤년일수: 1, 리턴상태코드: 07
입출력 설명 PoC	

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDate extends
                com.kbstar.sqc.base.FunctionUnit

[입력항목]
  String inGroupCoCd;          ● 그룹회사코드
  String inDstcd;              ● 구분코드
  String inStartYmd;           ● 시작년월일
  String inEndYmd;             ● 종료년월일

[출력항목]
  String rtnStat;              // 리턴상태코드
  int outNoday;                // 일수
  int outNomn;                 // 월수
  int outBzoprNoday;           // 영업일수
  int outPbhdayNoday;          // 공휴일수
  int outLpyrNoday;            // 윤년일수
```

RTNSTAT 설명	00 : 정상, 07 : 종료일의 말일이 시작일의 말일보다 작은 경우

사용 설명(예시)	// 공유메소드 호출 - 일수/월수/영업일수/공휴일수 산출 (com.kbstar.kji.enbncmn, calculateNumberOfDays)
	
	IDataSet calculateNumberOfDaysReq = new DataSet();
	
	calculateNumberOfDaysReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateNumberOfDaysReq.put("inDstcd", requestData.get("inDstcd"));
	calculateNumberOfDaysReq.put("inStartYmd", requestData.get("inStartYmd
	calculateNumberOfDaysReq.put("inEndYmd", requestData.get("inEndYmd"));
	
	// @@ 공유메소드 호출 (일수/월수/영업일수/공휴일수 산출 calculateNumberOfDays)
	PoC
	IDataSet calculateNumberOfDaysRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateNumberOfDays", calculateNumberOfDaysReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outnoday = calculateNumberOfDaysRes.getString("outNoday");
	String outnomn = calculateNumberOfDaysRes.getString("outNomn");
	String outbzoprnoday = calculateNumberOfDaysRes.getString("outBzoprNoday
	String outpbhdaynoday = calculateNumberOfDaysRes.getString("outPbhdayNoday");
	String outlpyrnoday = calculateNumberOfDaysRes.getString("outLpyrNoday");
	String rtnstat = calculateNumberOfDaysRes.getString("rtnStat");

비 고	

1.29

PoC

PoC

PoC

PoC

PoC

입력월수 차감후의 일수산출(FUBcDate.calculateSubtractionNumberOfDays())

프로그램 ID	FUBcDate. calculateSubtractionNumberOfDays()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	휴일목록	SQLMAP 명	NAME SPACE	fubcdate
	영문명	THKJIH001		SQL ID	selectHoldyCnt

프로그램 설명	시작년월일과 종료년월일에 대한 입력월수가 주어질 때, 입력월수 차감 후의 일수를 산출한다.
프로그램 규칙 및 제약사항	1. 구분코드 처리 '1' : 입력 월수만큼 차감후 산출한다. 예) 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 0 출력일수 : 90 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 1 출력일수 : 61 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 2 출력일수 : 30 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 3 출력일수 : 0 '2' : 입력 월수에 -1 을 한 월수만큼 산출한다. 예) 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 0 출력일수 : 오류 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 1 출력일수 : 90 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 2 출력일수 : 61 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 3 출력일수 : 30 2. *산출한 일수가 1 일 이상이고 7 일 미만일 경우, 입력 월수를 차감하여 산출된 시작일을 휴일체크한다. 산출된 시작일이 영업일이 아니면 이후 도래하는 최초의 영업일을 산출(FUBcDate.calculateBzoprDd())하여 입력된 종료일과 비교한 후 산출된 영업일이 종료일과 같거나 큰 경우 출력일수는 '0'으로 처리한다. 단, 산출된 영업일이 종료일보다 작은 경우에는 입력월수를 차감하여 산출된 시작일과 입력된 종료일 사이의 일수를 RETURN 한다.* 예) 시작년월일 : '20081002', 종료년월일 : '20081006', 월수 : 0 출력일수 : 4 시작년월일 : '20081002', 종료년월일 : '20081006', 월수 : 1 출력일수 : 0 시작년월일 : '20081003', 종료년월일 : '20081006', 월수 : 0 출력일수 : 0 시작년월일 : '20081003', 종료년월일 : '20081007', 월수 : 0 출력일수 : 4 *3. 산출 시작년월일이 입력 종료일보다 큰 경우는 에러출력하지않고 일수를 '0'으로 처리한다. 처리 상태코드는 '00' 정상으로 출력하다.* *4. 시작/종료년월일은 1976 년~2200 년 사이의 범위에서만 입력 가능하다.*

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcDate extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inDstcd;` ● 구분코드
	`String inStartYmd;` ● 시작년월일
	`String inEndYmd;` ● 종료년월일
	`int inNomn;` ● 월수
	
	[출력항목]
	`int outNoday;` // 일수

PoC RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 입력월수 차감후의 일수산출 (com.kbstar.kji.enbncmn, calculateSubtractionNumberOfDays)
	
	IDataSet calculateSubtractionNumberOfDaysReq = new DataSet();
	
	calculateSubtractionNumberOfDaysReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateSubtractionNumberOfDaysReq.put("inDstcd", requestData.get("inDstcd
	calculateSubtractionNumberOfDaysReq.put("inStartYmd", requestData.get("inStartYmd"));
	calculateSubtractionNumberOfDaysReq.put("inEndYmd", requestData.get("inEndYmd"));
	calculateSubtractionNumberOfDaysReq.put("inNomn", requestData.get("inNomn"));
	
	// @@ 공유메소드 호출 (입력월수 차감후의 일수산출 calculateSubtractionNumberOfDays)
	IDataSet calculateSubtractionNumberOfDaysRes = callSharedMethodByDirect ("com.kbstar.kji.enbncmn", "FUBcDate.calculateSubtractionNumberOfDays", calculateSubtractionNumberOfDaysReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outnoday = calculateSubtractionNumberOfDaysRes.getString("outNoday"); // 일수

비 고	

PoC

1.30

PoC

PoC

PoC

PoC

This document is strictly limited to the 2025 C2J conversation PoC

만기일/역만기일 산출(FUBcDate.calculateExpirDd())

프로그램 ID	FUBcDate.calculateExpirDd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	휴일목록	SQLMAP 명	NAME SPACE	fubcdate
	영문명	THKJIH001		SQL ID	getHoldyCnt

프로그램 설명	시작년월일과 일수/월수를 입력받아서 만기일과 역만기일을 산출한다. 단, 구분코드 1, 3, 5, 6 은 한편넣기(시작일불산입,종료일산입)를 적용한 만기일과 역만기일을 산출 한다.
프로그램 규칙 및 제약사항	*1. 영업일과 관련됨 구분값(5,6)은 1976 년과 2200 의 사이의 일자만 산출이 가능하다* 2. 구분코드는 아래와 같다. 1) '1' : 시작년월일과 일수를 입력하여 만기일을 산출한다. - 만기일이 영업일이 아닌경우도 산출한다. 예시] 시작년월일 : 2008-01-28, 일수 : 90 => 만기일 : 2008-04-27 2) '2' : 시작년월일과 월수를 입력하여 만기일을 산출한다. 만기일이 영업일이 아닌경우도 산출한다. 예시] 시작년월일 : 2008-01-28, 월수 : 1 => 만기일 : 2008-02-28 3) '3' : 시작년월일과 일수를 입력하여 역만기일을 산출한다. 역만기일이 영업일이 아닌경우도 산출한다. 예시] 시작년월일 : 2008-01-28, 일수 : 90 => 역만기일 : 2007-10-30 4) '4' : 시작년월일과 월수를 입력하여 역만기일을 산출한다. 역만기일이 영업일이 아닌경우도 산출한다. 예시] 시작년월일 : 2008-01-28, 월수 : 1 => 역만기일 : 2007-12-28 5) '5' : 시작년월일과 영업일수를 입력하여 만기일을 산출한다. 예시] 시작년월일 : 2008-01-28, 일수 : 90 => 만기일 : 2008-06-12 6) '6' : 시작년월일과 영업일수를 입력하여 역만기일을 산출한다. 예시] 시작년월일 : 2008-06-12, 일수 : 90 => 역만기일 : 2008-01-28 7) '7' : 시작년월일과 월수를 입력하여 만기일을 산출한다. - 시작년월일이 오류인 날짜를 입력하여도 정당성 체크를 하지 않으며, 입력된 월수만큼 가산 해준다. - 산출된 만기일이 월말일을 초과하는 경우 해당월의 말일자를 출력한다. - 시작년월일과 월수("0")를 입력하면 시작년월일의 만기일이 산출되며 시작년월일이 오류인 날짜를 입력하여도 정당성 체크를 하지 않는다. 예시] 시작년월일 : 2008-02-31, 월수 : 1 => 만기일 : 2008-03-31 시작년월일 : 2008-02-31, 월수 : 2 => 만기일 : 2008-04-30 시작년월일 : 2008-02-31, 월수 : 0 => 만기일 : 2008-02-29 8) '8' : 시작년월일과 월수를 입력하여 역만기일을 산출한다. - 시작년월일이 오류인 날짜를 입력하여도 정당성 체크를 하지 않으며, 입력된 월수만큼 감산 해준다. - 산출된 만기일이 월말일을 초과하는 경우 해당월의 말일자를 출력한다. - 시작년월일과 월수("0")를 입력하면 시작년월일의 만기일이 산출되며 시작년월일이 오류인 날짜를 입력하여도 정당성 체크를 하지 않는다. 예시] 시작년월일 : 2008-02-31, 월수 : 1 => 역만기일 : 2008-01-31 시작년월일 : 2008-02-31, 월수 : 0 => 역만기일 : 2008-02-29 3. 윤년이 포함된 경우 윤년일수를 합산하여 만기일과 역만기일을 산출하였다. 4. '5', '6'에 대한 만기일/역만기일은 영업일수만을 계산하여 산출한 일자이다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcDate extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inDstcd;` ● 구분코드

PoC	String inYmd; int inNodayNomn; [출력항목] String outYmd;	● 년월일 ● 일수/월수 // 출력년월일
RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 만기일/역만기일 산출 (com.kbstar.kji.enbncmn, calculateExpirDd)
	
	IDataSet calculateExpirDdReq = new DataSet();
	
	calculateExpirDdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateExpirDdReq.put("inDstcd", requestData.get("inDstcd"));
	calculateExpirDdReq.put("inYmd", requestData.get("inYmd"));
	calculateExpirDdReq.put("inNodayNomn", requestData.get("inNodayNomn"));
	
	// @@ 공유메소드 호출 (만기일/역만기일 산출 calculateExpirDd)
	IDataSet calculateExpirDdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateExpirDd", calculateExpirDdReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outymd = calculateExpirDdRes.getString("outYmd"); // 출력년월일	

비 고		

1.31

PoC

양일자간의 차이를 년수/월수/일수산출(FUBcDate.calculateDteBtnDfrn())

프로그램 ID	FUBcDate.calculateDteBtnDfrn()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	시작년월일과 종료년월일의 차이를 년수, 월수, 일수 단순차이를 산출한다.
프로그램 규칙 및 제약사항	1. 종료년월일과 시작년월일을 종료년도-시작년도, 종료월-시작월, 종료일-시작일을 각각 계산하여 년, 월, 일을 구한다. 2. 입력종료년월일은 입력시작년월일보다 큰 값이어야 한다. 3. 일수산출은 종료년월일의 일자에서 시작년월일의 일자를 차감한 값이 된다. 단, 종료년월일이 월말이고 종료년월일의 일자가 시작년월일의 일자보다 작은경우는 일수가 0 이 된다. PoC 예시] 1) 윤년이 있는 경우: 시작년월일: 2007-10-01, 종료년월일: 2008-02-29 -> 출력년수 : 0, 출력월수 : 4, 출력일수 : 28 2) 시작일과 종료일이 월말이고 종료년월일의 일자가 작은경우 : PoC 시작년월일: 2007-08-31, 종료년월일: 2008-04-30 -> 출력년수 : 0, 출력월수 : 8, 출력일수 : 0

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcDate **extends**
	com.kbstar.sqc.base.FunctionUnit
	PoC
	
	[입력항목]
	String inStartYmd; ● 시작년월일
	String inEndYmd ; ● 종료년월일
	
	[출력항목]
	String outNoy ; // 년수
	String outNomn ; // 월수
	String outNoday; // 일수
	PoC

RTNSTAT 설명	PoC N/A

사용 설명(예시)	**// 공유메소드 호출 - 양일자간의 차이를 년수/월수/일수산출 (com.kbstar.kji.enbncmn, calculateDteBtnDfrn)**
	
	IDataSet calculateDteBtnDfrnReq = **new** DataSet();
	
	calculateDteBtnDfrnReq.put("inStartYmd", requestData.get("inStartYmd"));
	calculateDteBtnDfrnReq.put("inEndYmd", requestData.get("inEndYmd"));
	
	// @@ 공유메소드 호출 (양일자간의 차이를 년수/월수/일수산출 calculateDteBtnDfrn)
	IDataSet calculateDteBtnDfrnRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",

	"FUBcDate.calculateDteBtnDfrn", calculateDteBtnDfrnReq, onlineCtx);
	// @@ 출력 데이터 셋 처리
	String outnoy = calculateDteBtnDfrnRes.getString("outNoy"); // 년수
	String outnomn = calculateDteBtnDfrnRes.getString("outNomn"); // 월수
	String outnoday = calculateDteBtnDfrnRes.getString("outNoday"); // 일수
	PoC

비 고	

1.32

PoC

PoC

PoC

PoC

PoC

월별 주기값 산출(FUBcDate.calculatePerMnCyclVal())

프로그램 ID	FUBcDate.calculatePerMnCyclVal()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력년월일이 해당월의 몇 번째 주가 되는지 조회한다.
프로그램 규칙 및 제약사항	1. 월에 해당하는 주기는 토요일을 기준으로 한다. 즉, 해당월의 첫번째 토요일인 해당월의 첫번째주가 되고, 해당월일 두번째 토요일은 해당월의 두번째주가 된다. 예시] 입력년월일 : 2008-01-05 => 출력년월 : 2008-01, 출력주기 : 1 (첫번째주) 입력년월일 : 2008-01-16 => 출력년월 : 2008-01, 출력주기 : 3 (세번째주) 2. 마지막 주기값의 일자보다 입력년월일이 큰 경우는 주기값을 익월의 첫번째 주로 한다. 예시] 입력년월일 : 2008-01-31 => 출력년월 : 2008-02, 출력주기 : 1 (첫번째주)

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcDate extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inYmd; ● 년월일
	
	[출력항목]
	String outYm ; // 년월
	String outCycl; // 주기

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 월별 주기값 산출 (com.kbstar.kji.enbncmn, calculatePerMnCyclVal)
	
	IDataSet calculatePerMnCyclValReq = new DataSet();
	
	calculatePerMnCyclValReq.put("inYmd", requestData.get("inYmd")); // 년월일
	
	// @@ 공유메소드 호출 (월별 주기값 산출 calculatePerMnCyclVal)
	IDataSet calculatePerMnCyclValRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculatePerMnCyclVal", calculatePerMnCyclValReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outym = calculatePerMnCyclValRes.getString("outYm"); // 년월
	String outcycl = calculatePerMnCyclValRes.getString("outCycl"); // 주기

비 고	

1.33 시분초 연산(FUBcDate.calculateHHMMSS())

프로그램 ID	FUBcDate.calculateHHMMSS()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	변환전시각과 연산하고자 하는 시, 분, 초를 입력받아서 각각 합산하여 산출한다.
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같이 입력받는다. 1) '1' : 기준 시분초에 입력 시/분/초 가산 하여 시분초 출력 2) '2' : 기준 시분초에 입력 시/분/초 감산 하여 시분초 출력 2. 가감산 연산시 제한 사항. 1) 가산 연산시 연산 결과가 99 시를 초과 하면 에러 처리 함. 2) 감산 연산시 연산 결과가 0 시 미만 발생 하면 에러 처리 함 2. 변환전시각은 필수입력항목이다. 3. 연산하고자하는 시, 분, 초 중 하나는 입력하여야 한다. 4. 출력항목 변환후시각 1 의 시는 24 시를 기준, 분은 60 분을 기준, 초는 60 초를 기준으로 출력하고, 출력항목 변환후시각 2 의 시는 24 시를 기준으로 연산하지 않고, 연산한 값을 그대로 출력한다. 예시] 구분코드 '1'인 경우, 1) 변환전시각: 112033, 시:00, 분: 30, 초: 00 → 변환후시각 1: 115033, 변환후시각 2: 115033 2) 변환전시각: 231234, 시: 02, 분: 00, 초: 00 → 변환후시각 1: 01:12:34, 변환후시각 2: 25:12:34

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcDate **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inDstcd; ● 구분코드
	String inChngBfHms; ● 변환전시각
	int inHh; ● 시
	int inMm; ● 분
	int inSs; ● 초
	
	[출력항목]
	String outChngAfHms1; // 변환후시각 1
	String outChngAfHms2; // 변환후시각 2

RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - 시분초 연산 (com.kbstar.kji.enbncmn, calculateHHMMSS)**
	
	IDataSet calculateHHMMSSReq = **new** DataSet();
	
	calculateHHMMSSReq.put("inDstcd", requestData.get("inDstcd"));
	
	calculateHHMMSSReq.put("inChngBfHms", requestData.get("inChngBfHms"));
	calculateHHMMSSReq.put("inHh", requestData.get("inHh"));
	calculateHHMMSSReq.put("inMm", requestData.get("inMm"));
	calculateHHMMSSReq.put("inSs", requestData.get("inSs"));
	
	// 공유메소드 호출 - 시분초 연산 (com.kbstar.kji.enbncmn, calculateHHMMSS)
	
	IDataSet calculateHHMMSSRes = **new** DataSet();
	
	calculateHHMMSSRes = **this**.getFunctionUnit("enbncmn", "calculateHHMMSS", calculateHHMMSSReq);
	
	// 출력 데이터 처리
	if (calculateHHMMSSRes.isSuccess()) {
	// 출력 데이터 가져오기
	boolean hasNext = calculateHHMMSSRes.next();
	while (hasNext) {
	// 현재 레코드 처리
	boolean isCurrentRecordValid = **true**;
	if (!isCurrentRecordValid) {
	continue;
	}

	calculateHHMMSSReq.put("inChngBfHms", requestData.get("inChngBfHms"));
	calculateHHMMSSReq.put("inHh", requestData.get("inHh"));
	calculateHHMMSSReq.put("inMm", requestData.get("inMm"));
	calculateHHMMSSReq.put("inSs", requestData.get("inSs"));
	PoC
	// @@ 공유메소드 호출 (시분초 연산 calculateHHMMSS)
	IDataSet calculateHHMMSSRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateHHMMSS", calculateHHMMSSReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outchngafhms1 = calculateHHMMSSRes.getString("outChngAfHms1");
	String outchngafhms2 = calculateHHMMSSRes.getString("outChngAfHms2");

비 고	

1.34

PoC

PoC

PoC

PoC

PoC

해외정보수신기관 코드별 일수/월수/영업일수/공휴일수 산출 (FUBcDate.calculateOvsesNoday())

프로그램 ID	FUBcDate.calculateOvsesNoday()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	해외기관휴일목록	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIH002		SQL ID	selectInHoldyIrInstiCd

프로그램 설명	해외정보수신기관 코드의 시작년월일과 종료년월일에 대한 일수, 월수, 영업일수, 공휴일수, 윤년일수를 산출한다.
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같이 입력받는다. 1) '1' : 일수/월수/윤년일수 산출 2) '2' : 일수/월수/영업일수/공휴일수/윤년일수 산출 2. 구분코드가 2 인 경우에는 2009 년부터 2200 사이의 일자만 산출이 가능하다. 3. 일수, 영업일수, 공휴일수는 한편넣가(시작년월일 불포함,종료년월일 포함)를 한다. 4. 동 기능을 사용가능한 헤외정보수신기관코드는 해외기관 휴일목록에서 관리하는 휴일정보수신기관코드에 한한다. 사용가능한 해외정보수신기관코드 정보는 1.48 해외정보수신기관 코드별 휴일 체크 (FUBcDateCheck.checkOvsesHoldyChk())를 참조한다. 5. 월수는 아래와 같이 계산한다. 1) 월수 = (종료년-시작년) * 12 + (종료월-시작월) 2) 종료년월일의(예:20080115) 일자가 시작년월일의(예:20080930) 일자보다 작은 경우는 (월수 - 1)을 월수로 한다. 즉, 종료년월일의 일자가 15이고 시작년월일의 일자가 30일로 두 일자간의 차이가 한달이 되지 않으므로 월수에서 1을 차감하여 월수를 산출한다. 3) 시작년월일과(예:20090131) 종료년월일이(예:20090930) 말일자에 해당하고 종료년월일의 일자가 시작년월일의 일자보다 작은 경우 리턴상태코드는 '07' 값을 리턴한다.

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcDate **extends** com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inDstcd; ● 구분코드
	String inStartYmd; ● 시작년월일
	String inEndYmd; ● 종료년월일
	String inHoldyIrInstiCd; ● 휴일정보수신기관코드
	
	[출력항목]
	String rtnStat; // 일수
	int outNoday; // 월수
	int outNomn; // 영업일수
	int outBzoprNoday; // 공휴일수
	int outPbhdayNoday;; // 윤년일수
	int outLpyrNoday;

RTNSTAT 설명	00 : 정상, 07 : 종료일의 말일이 시작일의 말일보다 작은 경우
사용 설명(예시)	**// 공유메소드 호출 - 해외정보수신기관 코드별 일수/월수/영업일수/공휴일수 산출** **(com.kbstar.kji.enbncmn, calculateOvsesNoday)**

```
IDataSet calculateOvsesNodayReq = new DataSet();

calculateOvsesNodayReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
calculateOvsesNodayReq.put("inDstcd", requestData.get("inDstcd"));
calculateOvsesNodayReq.put("inStartYmd", requestData.get("inStartYmd"));
calculateOvsesNodayReq.put("inEndYmd", requestData.get("inEndYmd"));
calculateOvsesNodayReq.put("inHoldyIrInstiCd", requestData.get("inHoldyIrInstiCd"));

// @@ 공유메소드 호출 (해외정보수신기관 코드별 일수/월수/영업일수/공휴일수 산출
                                                        calculateOvsesNoday)
IDataSet calculateOvsesNodayRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
                "FUBcDate.calculateOvsesNoday", calculateOvsesNodayReq, onlineCtx);

// @@ 출력 데이터 셋 처리
String outnoday = calculateOvsesNodayRes.getString("outNoday"); // 일수
String outnomn = calculateOvsesNodayRes.getString("outNomn"); // 월수
String outbzoprnoday = calculateOvsesNodayRes.getString("outBzoprNoday"); // 영업일수
String outpbhdaynoday = calculateOvsesNodayRes.getString("outPbhdayNoday");
String outlpyrnoday = calculateOvsesNodayRes.getString("outLpyrNoday"); // 윤년일수
String rtnstat = calculateOvsesNodayRes.getString("rtnStat"); // 리턴상태
```

비 고	

1.35

PoC

PoC

PoC

해외정보수신기관 코드별 만기일/역만기일 산출 (FUBcDate.calculateOvsesExpirDd())

PoC

프로그램 ID	FUBcDate.calculateOvsesExpirDd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	해외기관휴일목록	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIH002		SQL ID	selectInHoldyIrInstiCd

프로그램 설명	시작년월일과 영업일수를 입력받아서 해외정보수신기관 코드별 만기일과 역만기일을 산출한다.
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같다. 1) '1' : 시작년월일과 영업일수를 입력하여 해외 만기일을 산출한다. 예시] 시작년월일 : 2008-06-27, 일수 : 90, 휴일정보수신기관코드: BeB => 만기일 : 2008-04-26 2) '2' : 시작년월일과 영업일수를 입력하여 해외 역만기일을 산출한다. 예시] 시작년월일 : 2008-06-27, 일수 : 90, 휴일정보수신기관코드: BeB ==> 역만기일 : 2008-03-29 2. 시작년월일 및 산출된 만기일/역만기일의 년도가 2009 년부터 2200 년 사이값이어야 한다. 3. 윤년이 포함된 경우 윤년일수를 합산하여 만기일과 역만기일을 산출하였다. 4. '1', '2'에 대한 만기일/역만기일은 영업일수만을 계산하여 산출한 일자이다. 5. 해외정보수신기관 코드별 영업일 산출 가능 해외정보수신기관코드는 해외기관 휴일목록에서 관리하는 해외정보수신기관코드에 한한다.(대소문자 구분있음) 사용가능한 해외정보수신기관코드 정보는 1.48 해외정보 수신기관 코드별 휴일 체크 (FUBcDateCheck.checkOvsesHoldyChk())를 참조한다. 6. 시작년월일과 일수/월수로 만기일/역만기일 산출은 만기일/역만기일 산출(FUBcDate. calculateExpirDd()) 프로그램을 사용한다.

입출력 설명	PoC
	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcDate extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; // ● 그룹회사코드
	String inDstcd; // ● 구분코드
	String inYmd; // ● 년월일
	int inBzoprNoday; // ● 영업일수
	String inHoldyIrInstiCd; // ● 휴일정보수신기관코드
	
	[출력항목]
	String outYmd; // 출력년월일
	
	PoC PoC PoC PoC

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 해외정보수신기관 코드별 만기일/역만기일 산출
	(com.kbstar.kji.enbncmn, calculateOvsesExpirDd)
	
	IDataSet calculateOvsesExpirDdReq = new DataSet();
	calculateOvsesExpirDdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));

	calculateOvsesExpirDdReq.put("inDstcd", requestData.get("inDstcd"));
	calculateOvsesExpirDdReq.put("inYmd", requestData.get("inYmd"));
	calculateOvsesExpirDdReq.put("inBzoprNoday", requestData.get("inBzoprNoday"));
	calculateOvsesExpirDdReq.put("inHoldyIrInstiCd", requestData.get("inHoldyIrInstiCd"));
	PoC
	
	// @@ 공유메소드 호출 (해외정보수신기관 코드별 만기일/역만기일 산출 calculateOvsesExpirDd)
	IDataSet calculateOvsesExpirDdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDate.calculateOvsesExpirDd", calculateOvsesExpirDdReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outymd = calculateOvsesExpirDdRes.getString("outYmd"); // 출력년월일

비 고	

1.36

PoC

PoC

PoC

PoC

PoC

가산 및 감산월을 이용한 월초영업일/ 월말영업일/ 월말일자 산출 (FUBcDate.calculateBaseYmdBzoprDd())

PoC

프로그램 ID	FUBcDate. calculateBaseYmdBzoprDd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	휴일목록	SQLMAP 명	NAME SPACE	fubcdate
	영문명	THKJIH001		SQL ID	selectHoldyCnt

PoC

프로그램 설명	기준일자에 대한 가산월/감산월을 입력하여 가산월/감산월에 해당하는 대상일자를 산출하고, 대상월에 대한 월초영업일/월말영업일/월말일자를 산출한다.
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같다. 1) '1' : 기준년월일과 가산월을 입력하여 기준년월일에 가산월말큼 가산하여 대상년월일를 구하고 대상년월일에 대한 월초영업일, 월말영업일, 월말일자를 산출한다. 예시] 시작년월일: 2008-04-25, 가산월: 7 => 월초영업일: 20081103, 월말영업일:20081128, 월말일자:20081130 2) '2' : 시작년월일과 감산월을 입력하여 기준년월일에 감산월말큼 감산하여 대상년월일를을 구하고 대상년월일에 대한 월초영업일, 월말영업일, 월말일자를 산출한다. 예시] 시작년월일 : 2008-08-31, 감산월 : 3 => 월초영업일: 20080502, 월말영업일: 20080530, 월말일자: 20080531 *2. 기준년월일에 가산/감산한 대상년월일은 1976~2200 년사이의 값이어야 한다.*

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcDate **extends** com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inDstcd; ● 구분코드
	String inYmd; ● 년월일
	int inPlsSbtrMn; ● 가산월/감산월
	
	[출력항목]
	String outMnThbgBzoprYmd; // 월초영업년월일
	String outEnmnBzoprYmd; // 월말영업년월일
	String outEnmnYmd; // 월말년월일

RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - 가산 및 감산월을 이용한 월초영업일/월말영업일/월말일자 산출 (com.kbstar.kji.enbncmn, calculateBaseYmdBzoprDd)**
	
	IDataSet calculateBaseYmdBzoprDdReq = **new** DataSet();
	
	calculateBaseYmdBzoprDdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateBaseYmdBzoprDdReq.put("inDstcd", requestData.get("inDstcd"));
	calculateBaseYmdBzoprDdReq.put("inYmd", requestData.get("inYmd"));
	calculateBaseYmdBzoprDdReq.put("inPlsSbtrMn", requestData.get("inPlsSbtrMn"));
	
	PoC

PoC

	// @@ 공유메소드 호출 (가산 및 감산월을 이용한 월초영업일/월말영업일/월말일자 산출 calculateBaseYmdBzoprDd) IDataSet calculateBaseYmdBzoprDdRes = callSharedMethodByDirect ("com.kbstar.kji.enbncmn", "FUBcDate.calculateBaseYmdBzoprDd", calculateBaseYmdBzoprDdReq, onlineCtx); // @@ 출력 데이터 셋 처리 String outmnthbgbzoprymd = calculateBaseYmdBzoprDdRes.getString("outMnThbgBzoprYmd"); String outenmnbzoprymd = calculateBaseYmdBzoprDdRes.getString("outEnmnBzoprYmd"); String outenmnymd = calculateBaseYmdBzoprDdRes.getString("outEnmnYmd");
비 고	

PoC

1.37

PoC

PoC

PoC

PoC

연령계산 산출(FUBcDate.calculateAge())

프로그램 ID	FUBcDate.calculateAge()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	생년월일을 입력받아서 만 나이, 일반 나이, 보험회사용 나이를 계산한다.
프로그램 규칙 및 제약사항	1. 생년월일과 비교할 년월일이 입력되지 않으면 거래기준년월일을 기준으로 계산한다. 2. 만연령 계산은 아래와 같다. 1) 생일이 지났을 경우 : 년월일의 년도 - 생일년도 2) 생일이 지나지 않았을 경우 : 년월일의 년도 - 생일년도 - 1 3. 일반연령 계산은 아래와 같다. 1) 년월일의 년도 - 생일년도 + 1 4. 보험연령 계산은 아래와 같다. 1) 생일이 6 개월 이상 지난경우 : 만 나이 + 1 2) 생일이 6 개월 미만 지난경우 : 만 나이 5. 구분코드 : '1' 인 경우 주민등록번호(외국인등록번호)로 연령계산 ('1' 인 경우는 주민등록번호나, 거소신고증에 의해 만들어진 외국인 등록번호가 필수 입력 사항이며, 입력된 생년월일이 있어도 무시하고 주민등록번호나,외국인등록번호를 참조 하여 연령계산) *거소신고증 없이 만들어진 외국인 등록번호인 경우 본 모듈 절대 사용불가(연령 계산이 잘못 될 수 있음) 주민등록번호 , 외국인등록번호 7 번째자리체크 (아래 해당값이 아닌 경우 오류처리) - 0,9 : 1800 년도 내국인 - 1,2 : 1900 년도 내국인 - 3,4 : 2000 년도 내국인 - 5,6 : 1900 년도 외국인 - 7,8 : 2000 년도 외국인 으로 간주하여 계산 구분코드가 '1' 이 아닌 모든경우는 입력된 생년월일을 가지고 연령계산 (생년월일 필수 입력)

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcDate extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inDstcd;` ◎ 구분코드
	`String inSsno;` ◎ 주민번호
	`String inBrdt;` ● 생년월일
	`String inYmd;` ◎ 년월일
	
	[출력항목]
	`int outFiAge;` // 만연령

PoC PoC PoC PoC PoC

	int outGnralAge; int outIsurncAge;	// 일반연령 // 보험연령
RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 연령계산 산출 (com.kbstar.kji.enbncmn, calculateAge)
	
	IDataSet calculateAgeReq = new DataSet();
	
	calculateAgeReq.put("inDstcd", requestData.get("inDstcd")); // 구분코드
	calculateAgeReq.put("inSsno", requestData.get("inSsno")); // 주민번호
	calculateAgeReq.put("inBrdt", requestData.get("inBrdt")); // 생년월일
	calculateAgeReq.put("inYmd", requestData.get("inYmd")); // 년월일
	
	// @@ 공유메소드 호출 (연령계산 산출 calculateAge)
	IDataSet calculateAgeRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateAge", calculateAgeReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outfiage = calculateAgeRes.getString("outFiAge"); // 만연령
	String outgnralage = calculateAgeRes.getString("outGnralAge"); // 일반연령
	String outisurncage = calculateAgeRes.getString("outIsurncAge"); // 보험연령	

비 고		

1.38

윤년 체크(FUBcDateCheck.checkLpyrChk())

PoC

프로그램 ID	FUBcDateCheck.checkLpyrChk()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형 PoC	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력년에 대해 윤년인지 여부를 제공한다
PoC 프로그램 규칙 및 제약사항	1. 입력년은 아래의 조건을 만족해야한다. 1) 숫자로만 구성되어야 한다. 2) '0000'는 오류처리 한다. 2. 윤년여부는 아래와 같다. 1) '1' : 윤년 2) '0' : 평년 3. 윤년여부 체크 방법. 1) 4로 나눈 나머지가 0이고 100으로나눈 나머지가 0이 아닌경우 윤년 2) 400으로 나누어 나머지가 0인 경우 윤년 3) 그외 평년 예시] 가) 입력년 : 2007 => 출력윤년여부 : 0 나) 입력년 : 2008 => 출력윤년여부 : 1
입출력 설명	package com.kbstar.kji.enbncmn.biz; public class FUBcDateCheck extends com.kbstar.sqc.base.FunctionUnit [입력항목] String inYr; ● 년 [출력항목] String outLpyrYn; // 윤년여부
RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 윤년체크 (com.kbstar.kji.enbncmn, checkLpyrChk)
	
	IDataSet checkLpyrChkReq = new DataSet(); PoC
	
	checkLpyrChkReq.put("inYr", requestData.get("inYr")); // 년도
	
	// @@ 공유메소드 호출 (윤년체크 checkLpyrChk)
	IDataSet checkLpyrChkRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDateCheck.checkLpyrChk", checkLpyrChkReq, onlineCtx);
	// @@ 출력 데이터 셋 처리
	String outlpyryn = checkLpyrChkRes.getString("outLpyrYn"); // 윤년여부 PoC

비 고	PoC

1.39 휴일 체크(FUBcDateCheck.checkHoldyChk())

프로그램 ID	FUBcDateCheck. checkHoldyChk()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	공휴일목록	SQLMAP 명	NAME SPACE	fubcdatecheck
	영문명	THKJI001		SQL ID	selectCheckHoldyChk

PoC

프로그램 설명	입력일자에 대해 휴일구분을 제공 한다. (리턴상태코드 00 : 평일, 07 : 휴일)
프로그램 규칙 및 제약사항	*1. 구분코드 아래와 같다.* *1) '1' : 입력된 일자의 휴일을 체크 한다.* *2) '2' : 입력된 일자의 휴일을 체크 및 영업일을 산출 한다.* *- 입력일자가 영업일인경우 입력일자를 영업년월일에 SET 하여 출력한다.* *- 입력일자가 휴일인경우 익영업일을 영업년월일에 SET 하여 출력한다.* 2. 입력년월일은 아래의 조건을 만족해야 한다. *1) 입력년은 1976 년과 2200 년 사이 값이어야 한다.* 2) 입력월은 01 ~ 12 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 29 일보다 크지 않아야 한다. - 입력년의 윤년여부를 판단하여 윤년이 아닌경우 2 월 28 일보다 크지 않아야 한다 3. 휴일구분코드는 아래와 같다. 1) '0' : 평일(영업일) 2) '1' : 법정공휴일(2014 년부터 대체휴일 포함) 3) '2' : 임시공휴일 4) '3' : 근무토요일(영업일)(1976 ~ 2002 년 06 월까지 토요일) 5) '4' : 일요일 6) '5' : 토요휴무일(2002 년 07 월 ~ 토요일) 7) '6' : 근로자의날 4. 휴일 체크 1) 공휴일목록 테이블을 입력된 년월일로 조회한다. 공휴일 테이블에는 '1'(법정공휴일), '2'(임시공휴일), '4'(일요일), '5'(토요일휴무), '6'(근로자의날)만 존재한다. 2) 공휴일목록 테이블에 존재 하지 않을 때에는 요일산출 프로그램 (FUBcDate.calculateDwk())을 호출하여 요일을 확인하고, 일요일은 '4'(일요일)를 출력하고 토요일인 경우는 입력 년월이 2002 년 06 월까지는 '3'(근무토요일)으로 그 이후는 '5'(토요일휴무)로 출력한다. 3) 1), 2)의 요건이 아닌 경우는 '0'(평일-영업일)으로 출력한다. 5. 리턴상태코드 '00'(정상)은 휴일구분코드가 '0'(평일-영업일), '3'(근무토요일-영업일)인 경우이고, '07'(휴일)은 휴일구분코드가 '1'(법정공휴일), '2'(임시공휴일), '4'(일요일), '5'(토요휴무일), '6'(근로자의날)인 경우이다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcDateCheck extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inDstcd;` ● 구분코드
	`String inYmd;` ● 년월일
	
	[출력항목]
	`String rtnStat;` // 리턴상태코드
	`String outHoldyDstic;` // 휴일구분코드
	`String outHoldyHanglName;` // 휴일한글명
	`String outBzoprYmd;` // 영업년월일

RTNSTAT 설명	00 : 정상(평일인경우), 07 : 정상(휴일인경우)

PoC PoC PoC PoC PoC

사용 설명(예시)	// 공유메소드 호출 - 휴일체크 (com.kbstar.kji.enbncmn, checkHoldyChk)
	
	IDataSet checkHoldyChkReq = new DataSet();
	
	checkHoldyChkReq.put("inGroupCoCd", requestData.get("inGroupCoCd
	checkHoldyChkReq.put("inDstcd", requestData.get("inDstcd"));
	checkHoldyChkReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (휴일체크 checkHoldyChk)
	IDataSet checkHoldyChkRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDateCheck.checkHoldyChk", checkHoldyChkReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outfoldydstic = checkHoldyChkRes.getString("outFoldyDstic");
	String outholdyhanglname = checkHoldyChkRes.getString("outHoldyHanglName");
	String outbzoprymd = checkHoldyChkRes.getString("outBzoprYmd");
	String rtnstat = checkHoldyChkRes.getString("rtnStat");

비 고	

PoC PoC PoC

1.40

PoC

PoC

영업일 산출 I (익영업일/익익영업일/3 영업일/4 영업일/5 영업일/전영업일/전전영업일)(FUBcDate.calculateBzoprDd())

프로그램 ID	FUBcDate.calculateBzoprDd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명

입력일자에 대해 익영업년월일, 익익영업년월일, 3 영업년월일, 4 영업년월일, 5 영업년월일, 전영업년월일, 전전영업년월일 제공 한다

프로그램 규칙 및 제약사항

주의) 일부 영업일이 필요한 경우는 영업일 산출 II(FUBcDate.calculateBzoprDd2())를 사용 하면
성능면에서 효율적이다

영업일 산출 II (FUBcDate.calculateBzoprDd2()) 기능

구분 1 : 전영업일 제공
구분 2 : 익영업일 제공
구분 3 : 전영업일/ 익영업일 제공
구분 4 : 전영업일/ 전전영업일 제공
구분 5 : 익영업일/ 익익영업일 제공

PoC

PoC

1. 입력년월일은 아래의 조건을 만족해야 한다.
   1) 입력년은 1976 ~ 2200 년 사이 값이어야 한다.
   2) 입력월은 01 ~ 12 사이 값이어야 한다.
   3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다.
      - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 일로 한다.
2. 익영업년월일, 익익영업년월일, 3 영업년월일, 4 영업년월일, 5 영업년월일, 전영업년월일, 전전영업년월일의 산출
   1) 휴일을 체크하여 평일 및 근무토요일인 경우의 일자를 구한다
   예시] 입력일 2008 년 4 월 5 일

월	화	수	목	금	토	일
3 월 31 일	1	2	3 전전영업년월일	4 전영업년월일	5 입력년월일	6
7 익영업년월일	8 익익영업년월일	9 3 영업년월일	10 4 영업년월일	11 5 영업년월일	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

PoC

PoC

PoC

PoC

가) 입력년월일: 2008-04-05
=> 출력익영업일: 2008-04-07, 출력익익영업일: 2008-04-08,
출력 3 영업일: 2008-04-09, 출력 4 영업일: 2008-04-10, 출력 5 영업일: 2008-04-11,
출력전영업일: 2008-04-04, 출력전전영업일: 2008-04-03
나) 입력년월일: 2008-04-21
=> 출력익영업일: 2008-04-22, 출력익익영업일: 2008-04-23,
출력 3 영업일: 2008-04-24, 출력 4 영업일: 2008-04-25, 출력 5 영업일: 2008-04-28,
출력전영업일: 2008-04-18, 출력전전영업일: 2008-04-17

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDate extends
                    com.kbstar.sqc.base.FunctionUnit
```

[입력항목]

	String inGroupCoCd;
	String inYmd;
	
	[출력항목]
	String outBfBfBzoprYmd
	String outBfBzoprYmd;;
	String outNxbizYmd;
	String outTdatYmd;
	String out3bzoprYmd;
	String out4bzoprYmd;
	String out5bzoprYmd;
		● 그룹회사코
		● 년월일
		
		// 전전영업년월일
		// 전영업년월일
		// 익영업년월일
		// 익익영업년월일
		// 3 영업년월일
		// 4 영업년월일
		// 5 영업년월일

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 영업일산출 I (익/익익/3/4/5영업일 전/전전영업일)
	(com.kbstar.kji.enbncmn, calculateBzoprDd)
	
	IDataSet calculateBzoprDdReq = new DataSet();
	
	calculateBzoprDdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateBzoprDdReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출(영업일산출 I(익/익익/3/4/5영업일 전/전전영업일) calculateBzoprDd)
	IDataSet calculateBzoprDdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateBzoprDd", calculateBzoprDdReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outbfbfbzoprymd = calculateBzoprDdRes.getString("outBfBfBzoprYmd");
	String outbfbzoprymd = calculateBzoprDdRes.getString("outBfBzoprYmd");
	String outnxbizymd = calculateBzoprDdRes.getString("outNxbizYmd");
	String outtdatymd = calculateBzoprDdRes.getString("outTdatYmd");
	String out3bzoprymd = calculateBzoprDdRes.getString("out3bzoprYmd");
	String out4bzoprymd = calculateBzoprDdRes.getString("out4bzoprYmd");
	String out5bzoprymd = calculateBzoprDdRes.getString("out5bzoprYmd");	

비 고		

1.41

일자 산출 I(익일/익익일/전일/전전일) (FUBcDate.calculateDate())

프로그램 ID	FUBcDate.calculateDate()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력일자에 대해 전전년월일, 전년월일 ,익년월일, 익익년월일을 제공한다
프로그램 규칙 및 제약사항	주의) 일부 일이 필요한 경우는 일자 산출 II(FUBcDate.calculateDate2())를 사용 하면 성능면에서 효율적이다 일자 산출 II (FUBcDate.calculateDate2()) 기능 구분 1 : 전일 제공 구분 2 : 익일 제공 구분 3 : 전일/ 익일 제공 구분 4 : 전일/ 전전일 제공 구분 5 : 익일/ 익익업일 제공 1. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우에 2 월의 마지막일자는 29 일로 한다. 예시] 입력일 2008 년 4 월 5 일

월	화	수	목	금	토	일
	1	2	3 전전일	4 전일	5 입력일	6 익일
7 익익일	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

가) 입력년월일 : 2008-04-05
=> 출력익일: 2008-04-06, 출력익익일: 2008-04-07,
출력전일: 2008-04-04, 출력전전영업일: 2008-04-03
나) 입력년월일: 2008-04-21
=> 출력익일: 2008-04-22, 출력익익일: 2008-04-23,
출력전일: 2008-04-20, 출력전전일: 2008-04-19

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDate extends
                    com.kbstar.sqc.base.FunctionUnit

[입력항목]
    String inYmd;                          ● 년월일

[출력항목]
    String outNextYmd;                     // 익년월일
    String outNextNextYmd;                 // 익익년월일
    String outBfYmd;                       // 전년월일
    String outBfBfYmd;                     // 전전년월일
```

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 일자산출 I (익일/익익일/전일/전전일) (com.kbstar.kji.enbncmn, calculateDate)
	
	IDataSet calculateDateReq = new DataSet();
	
	calculateDateReq.put("inYmd", requestData.get("inYmd")); // 년월일
	
	// @@ 공유메소드 호출 (일자산출 I (익일/익익일/전일/전전일) calculateDate)
	IDataSet calculateDateRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateDate", calculateDateReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outnextymd = calculateDateRes.getString("outNextYmd"); // 익년월일
	String outnextnextymd = calculateDateRes.getString("outNextNextYmd"); // 익익년월일
	String outbfymd = calculateDateRes.getString("outBfYmd"); // 전년월일
	String outbfbfymd = calculateDateRes.getString("outBfBfYmd"); // 전전년월일

비 고	

PoC

PoC

PoC

PoC

1.42

PoC

PoC

월초영업일/월말영업일/월말일 산출(FUBcDate.calculateEnmnBzoprDay())

프로그램 ID	FUBcDate.calculateEnmnBzoprDay()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명

입력년월일에 대해 월초영업년월일/월말영업년월일/월말년월일을 제공한다

프로그램 규칙 및 제약사항

1. 입력년월일은 아래의 조건을 만족해야 한다.
   1) 입력년은 1976 ~ 2200 년 사이 값이어야 한다.
   2) 입력월은 01 ~ 12 사이 값이어야 한다.
   3) 입력일이 입력월의 말일자보다 크지않은 값이어야 한다.
      - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 일로 한다.
2. 월초영업년월일/월말영업년월일/월말년월일 산출
   1) 월초영업년월일은 1 일에서부터 휴일여부를 확인하고 휴일인 경우는 1 일자부터 +1 일씩 더한다
   2) 월말년월일은 입력월의 말일자로 산출한다.
   3) 월말영업년월일은 2)에서 구해진 말일자로 휴일여부를 확인하고, 휴일인 경우는 말일자에서 -1 일씩 차감한다.
   4) 입력년월일과 월말영업년월일 비교하여 아래와 같이 리턴상태코드를 다르게 준다
      - '00' : 입력년월일이 월말영업년월일보다 작은 경우
      - '06' : 입력년월일이 월말영업년월일보다 크거나 같은 경우
      - '07' : 입력년월일이 월말영업년월일보다 크거나 같으면서 입력월이 12 월인 경우

PoC

예시] 2008 년 4 월

월	화	수	목	금	토	일
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

가) 입력년월일 : 2008-04-05
=> 출력월초영업년월일: 2008-04-01, 출력월말년월일: 2008-04-30,
⇨ 출력월말영업일: 2008-04-30, 리턴상태코드: 00

입출력 설명

PoC

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDate extends
                        com.kbstar.sqc.base.FunctionUnit

[입력항목]
    String inGroupCoCd                    ● 그룹회사코드
    String inYmd;                         ● 년월일

[출력항목]
    String rtnStat;                       // 리턴상태코드
    String outMnThbgBzoprYmd;             // 월초영업년월일
    String outEnmnYmd;                    // 월말년월일
    String outEnmnBzoprYmd;               // 월말영업년월일
```

PoC

RTNSTAT 설명

00 : 정상(입력일자가 월중), 06 : 입력일자가 월말, 07 : 입력일자가 년말, 09 : ERROR

사용 설명(예시)

PoC	// 공유메소드 호출 - 월초영업일/월말영업일/월말일 산출 (com.kbstar.kji.enbncmn, calculateEnmnBzoprDay)
	
	IDataSet calculateEnmnBzoprDayReq = new DataSet(); PoC
	
	calculateEnmnBzoprDayReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateEnmnBzoprDayReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (월초영업일/월말영업일/월말일 산출 calculateEnmnBzoprDay)
	IDataSet calculateEnmnBzoprDayRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateEnmnBzoprDay", calculateEnmnBzoprDayReq, onlineCtx); PoC
	
	// @@ 출력 데이터 셋 처리 PoC
	String outmnthbgbzoprymd = calculateEnmnBzoprDayRes.getString("outMnThbgBzoprYmd");
	String outenmnymd = calculateEnmnBzoprDayRes.getString("outEnmnYmd");
	String outenmnbzoprymd = calculateEnmnBzoprDayRes.getString("outEnmnBzoprYmd");
	String rtnstat = calculateEnmnBzoprDayRes.getString("rtnStat");

비 고	

PoC

1.43

PoC

줄리안 일자<->그레고리언 일자변환(FUBcDateChange.changeJulTOGreg())

프로그램 ID	FUBcDateChange. changeJulTOGreg()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력일자에 대해 줄리안/그레고리안 일자을 제공한다
프로그램 규칙 및 제약사항	**1. 구분코드** 1) '1' : 그레고리안 일자 -> 줄리안 일자 - 입력형태 : YYYYMMDD 2) '2' : 줄리안 일자 -> 그레고리안 일자 - 입력형태 : YYYYDDDD **2. 구분코드 '1' 인경우 입력년월일은 아래의 조건을 만족해야 한다.** 1) 숫자로 입력되어야 한다 2) 입력월은 01 ~ 12 사이 값이어야 한다. 3) 입력일이 입력월의 말일자보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 일로 한다. **3. 구분코드 '2' 인경우 입력줄리안일은 아래의 조건을 만족해야 한다.** 1) 숫자로 입력되어야 한다 2) 윤년인 경우 366 보다 크지 않은 값이어야 한다. 3) 윤년인 아닌경우 365 보다 크지 않은 값이어야 한다. 4) 0 보다 큰 값이어야 한다. **4. 구분코드 '1' 인경우(그레고리안 -> 줄리안)** 1) 입력년월일: 20080119 => 출력년도: 2008, 출력일수: 19 2) 입력년월일: 20080303 (윤년) => 출력년도: 2008, 출력일수: 63 **5. 구분코드 '2' 인경우(줄리안 -> 그레고리안)** 1) 입력년월일 : 20080019 => 출력년도: 2008, 출력일수: 2008-01-19 2) 입력 년월일을은 반드시 년도 4 자리 일수 4 자리로 입력한다. 2008100 (X) -> 20080100(0), 20081(X) -> 20080001(O)
입출력 설명	**package** com.kbstar.kji.enbncmn.biz; **public class** FUBcDateChange **extends** com.kbstar.sqc.base.FunctionUnit **[입력항목]** String inDstcd; ● 구분코드 String inYmd; ● 년월일 **[출력항목]** String outNoday; // 일수/그레고리안 일자
RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - 줄리안일자<->그레고리언 일자변환 (com.kbstar.kji.enbncmn, changeJulTOGreg)**
	
	IDataSet changeJulTOGregReq = new DataSet();
	
	changeJulTOGregReq.put("inDstcd", requestData.get("inDstcd"));
	changeJulTOGregReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (줄리안일자<->그레고리언 일자변환 changeJulTOGreg)

PoC

	IDataSet changeJulTOGregRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", *"FUBcDateChange.changeJulTOGreg", changeJulTOGregReq, onlineCtx);* **// @@ 출력 데이터 셋 처리** String outnoday = changeJulTOGregRes.getString("outNoday"); // 일수/그레고리언일자
비 고	

PoC

PoC

1.44

PoC

PoC

PoC

양력/음력일자 변환(FUBcDateChange.changeSlcl())

프로그램 ID	FUBcDateChange.changeSlcl()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력 년월일과 변환구분을 기준으로 음력년월일 또는 양력년월일로 변환한다
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같다. 1) '1' : 양력년월일을 음력년월일로 변환한다. 2) '2' : 음력년월일을 양력년월일로 변환한다. 윤달인 경우 변환년월일 1 에는 첫번째 양력년월일을, 변환년월일 2 에는 두번째 양력년월일을 출력한다. 단, 윤달의 말일이 첫번째달 29 일-두번째달 30, 또는 첫번째달 30 일-두번째달 29 일 처럼 틀린 경우는 변환년월일이 30 일인경우 29 일에 대해서는 변환이 되지않음 2. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력년은 1901 ~ 2200 사이 값이어야 한다. 2) 입력월은 01 ~ 12 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다. 3. 입력 년월일이 1901-02-18 이전이면 양력 -> 음력변환 로직으로 처리가 안되므로 별도의 매핑정보를 이용한다. 예] 가) 구분코드 : 1, 입력년월일 : 2008-02-11 => 출력변환년월일 : 2008-01-05 나) 구분코드 : 2, 입력년월일 : 2008-01-23 => 출력변환년월일 : 2008-02-29

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	`public class FUBcDateChange extends com.kbstar.sqc.base.FunctionUnit`
	[입력항목]
	`String inDstcd;` ● 구분코드
	`String inYmd;` ● 년월일
	[출력항목]
	`int outChngNoitm;` // 변환건수
	`String outChngYmd1` // 변환년월일 1
	`String outChngYmd2;` // 변환년월일 2

RTNSTAT 설명	N/A

사용 설명(예시)	`// 공유메소드 호출 - 양력/음력일자 변환 (com.kbstar.kji.enbncmn, changeSlcl)`
	`IDataSet changeSlclReq = new DataSet();`
	`changeSlclReq.put("inDstcd", requestData.get("inDstcd")); // 구분코드`
	`changeSlclReq.put("inYmd", requestData.get("inYmd")); // 년월일`
	`// @@ 공유메소드 호출 (양력/음력일자 변환 changeSlcl)`
	`IDataSet changeSlclRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDateChange.changeSlcl", changeSlclReq, onlineCtx);`
	`// @@ 출력 데이터 셋 처리`

	String outchngnoitm = changeSlclRes.getString("outChngNoitm"); // 변환건수
	String outchngymd1 = changeSlclRes.getString("outChngYmd1"); // 변환년월일1
	String outchngymd2 = changeSlclRes.getString("outChngYmd2"); // 변환년월일 2

비 고	

1.45

일자 유효성 체크(FUBcDateCheck.checkDateValidity())

프로그램 ID	FUBcDateCheck. checkDateValidity()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력된 년월일이 일자 Type, Range 가 맞는지 체크하고, 입력된 년도가 윤년인지 여부를 확인한다.
프로그램 규칙 및 제약사항	1. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력년월일은 숫자. 2) 입력년월일은 00000000 아닌경우. 2. 입력월은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 3. 입력일은 아래의 조건을 만족해야 한다. 1) 입력월이 1, 3, 5, 7, 8, 10, 12 월이면 일자는 1 보다 크고 31 보다 작다. 2) 입력월이 4, 6, 9, 11 월이면 일자는 1 보다 크고 30 보다 작다. 3) 입력년월이 윤년 2 월이면 일자는 1 보다 크고 29 보다 작다. 4) 입력년월이 평년 2 월이면 일자는 1 보다 크고 28 보다 작다.
입출력 설명	package com.kbstar.kji.enbncmn.biz; public class FUBcDateCheck extends com.kbstar.sqc.base.FunctionUnit [입력항목] String inYmd; ● 기준년월일 [출력항목] String outLpyrYn; // 윤년여부
RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 일자유효성체크 (com.kbstar.kji.enbncmn, checkDateValidity)
	
	IDataSet checkDateValidityReq = new DataSet();
	
	checkDateValidityReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (일자유효성체크 checkDateValidity)
	IDataSet checkDateValidityRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDateCheck.checkDateValidity", checkDateValidityReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outlpyryn = checkDateValidityRes.getString("outLpyrYn");

비 고	

1.46 요일산출(FUBcDate.calculateDwk())

프로그램 ID	FUBcDate.calculateDwk()		언어 구분	JAVA(○)	
PoC 입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력년월일에 대해 요일을 제공한다
프로그램 규칙 및 제약사항	1. 입력년월일은 아래의 조건을 만족해야 한다. 1) 숫자로 입력되어야 한다 2) 입력월은 01 ~ 12 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2월의 마지막일자는 29일로 한다. 2. 요일구분은 아래와 같다. 1) '1' : 월요일 2) '2' : 화요일 3) '3' : 수요일 4) '4' : 목요일 5) '5' : 금요일 6) '6' : 토요일 7) '7' : 일요일
입출력 설명 PoC	package com.kbstar.kji.enbncmn.biz; public class FUBcDate extends com.kbstar.sqc.base.FunctionUnit PoC [입력항목] String inYmd; ● 년월일 [출력항목] String outDwkDstic; // 요일구분코드 PoC
RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 요일산출 (com.kbstar.kji.enbncmn, calculateDwk)
	
	PoC IDataSet calculateDwkReq = new DataSet();
	
	calculateDwkReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (요일산출 calculateDwk)
	IDataSet calculateDwkRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDate.calculateDwk", calculateDwkReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리 PoC
	String outdwkdstic = calculateDwkRes.getString("outDwkDstic");

비 고	

1.47 해외정보수신기관 코드별 휴일 체크(FUBcDateCheck.checkOvsesHoldyChk())

프로그램 ID	FUBcDateCheck. checkOvsesHoldyChk()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	해외기관휴일목록	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIH002		SQL ID	selectInHoldyIrInstiCd selectOvsesHoldy

프로그램 설명	입력년월일에 대해 해외휴일정보수신기관 코드별 휴일구분코드를 제공한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 입력된 일자의 휴일을 체크 한다. 2) '2' : 입력된 일자의 휴일을 체크 및 영업일을 산출 한다. - 입력일자가 영업일인경우 입력일자를 영업년월일에 SET 하여 출력한다. - 입력일자가 휴일인경우 익영업일을 영업년월일에 SET 하여 출력한다. 2. 휴일정보수신기관코드표

	휴일정보수신기관코드	통화코드	ISO 국가코드	국가명	휴일정보수신기관명	휴일수신기관구분명
1	AmB	NLG	NL	Netherlands	Amsterdam bank holidays	Bank
2	AuB	NZD	NZ	New Zealand	Auckland bank holidays	Bank
3	BeB	CNY	CN	China	Beijing bank holidays	Bank
4	BmB	INR	IN	India	Mumbai bank holidays	Bank
5	BrB	BEF	BE	Belgium	Brussels bank holidays	Bank
6	Ecl	BEF	BE	Belgium	Euroclear holidays	Other
7	FrB	DEM	DE	Germany	Frankfurt bank holidays	Bank
8	HKB	HKD	HK	Hong Kong	Hong Kong bank holidays	Bank
9	JaB	IDR	ID	Indonesia	Jakarta bank holidays	Bank
10	KLB	MYR	MY	Malaysia	Kuala Lumpur bank holidays	Bank
11	LnB	GBP	GB	England	London bank holidays	Bank
12	LxB	LUF	LU	Luxembourg	Luxembourg bank holidays	Bank
13	MdB	ESP	ES	Spain	Madrid bank holidays	Bank
14	MeB	AUD	AU	Australia	Melbourne bank holidays	Bank
15	MiB	ITL	IT	Italy	Milan bank holidays	Bank
16	MnB	PHP	PH	Philippines	Manila bank holidays	Bank
17	NYB	USD	US	United States	New York bank holidays	Bank
18	PaB	FRF	FR	France	Paris bank holidays	Bank
19	SeB	KRW	KR	South Korea	Seoul bank holidays	Bank
20	SiB	SGD	SG	Singapore	Singapore bank holidays	Bank
21	TkB	JPY	JP	Japan	Tokyo bank holidays	Bank
22	TpB	TWD	TW	Taiwan	Taipei bank holidays	Bank
23	TrB	CAD	CA	Canada	Toronto bank holidays	Bank
24	ZuB	CHF	CH	Switzerland	Zurich bank holidays	Bank

3. 휴일정보수신기관코드 체크
1) 휴일정보수신기관코드 체크: 해외기관휴일목록 테이블에 존재유무 확인, 미존재시 에러처리 한다.
4. 입력년월일은 아래의 조건을 만족해야 한다.
1) *입력년은 2009 년과 2200 년 사이* 값이어야 한다.
2) 입력월은 01 ~ 12 사이 값이어야 한다.

3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다.
- 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 29 일 보다 크지 않아야 한다.
- 입력년의 윤년여부를 판단하여 윤년이 아닌경우 2 월 28 일 보다 크지 않아야 한다

5. 휴일구분코드는 아래와 같다.
1) '0' : 평일
2) '1' : 법정공휴일
3) '2' : 임시공휴일
4) '3' : 근무토요일
5) '4' : 일요일
6) '5' : 토요휴무일
7) '6' : 근로자의날

6. 휴일 체크
1) 해외기관휴일목록 테이블을 입력된 년월일로 조회한다. 해외기관휴일 테이블에는 '1'(법정공휴일), '2'(임시공휴일), '6'(근로자의날)만 존재한다.
2) 해외기관휴일목록 테이블에 존재 하지 않을 때에는 '0'(평일-영업일)으로 출력한다.

7. 리턴상태코드 '00'(정상)은 휴일구분코드가 '0'(평일-영업일) 경우이고, '07'(휴일)은 휴일구분코드가 '1'(법정공휴일), '2'(임시공휴일), '4'(일요일), '5'(토요휴무일), '6'(근로자의날)인 경우이다.

*** 해외 휴일 데이터 정보**
- 해외휴일 데이터는 CMBS 에서 SWAPS Monitors(휴일자료) 업체로부터 제공 받는다
- 원천 데이터 변경요건이 있으면 CMBS 에서 변경하여 전행공통에 전달 되어지며(D-1) 해외 휴일을 사용하는 업무팀은 이를 인지하고 사용하여야 한다(CMBS 의 업무 요건에 따라 변경가능)
- 따라서 CMBS 에 의존 적이지 않고 업무 특성에 따라 해외휴일 데이터가 필요한 경우는 해당 업무팀에서 해외휴일 데이터를 별도로 구성 관리하여야 한다

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDateCheck extends
                    com.kbstar.sqc.base.FunctionUnit
```

[입력항목]

```
     String inGroupCoCd;
     String inDstcd;
     String inHoldyIrInstiCd;
     String inYmd;
```

- 그룹회사코드
- 구분코드
- 휴일정보수신기관코드
- 년월일

PoC

[출력항목]

```
     String rtnStat;
     String outHoldyDstcd;
     String outIsoNatnlCd;
     String outNatnlName;
     String outCncycd;
     String outHoldyRiDsticName;
     String outHoldyRrInstiName;
     String outOvsesHEngName;
     String outBzoprYmd;
```

```
// 휴일구분코드
// ISO 국가코드
// 국가명
// 통화코드
// 휴일수신기관구분명
// 휴일정보수신기관명
// 해외휴일영문명
// 영업년월일
```

PoC PoC PoC PoC

RTNSTAT 설명

00 : 정상(평일인경우), 07 : 정상(휴일인경우),

PoC PoC

사용 설명(예시)

// 공유메소드 호출 - 해외정보수신기관 코드별 휴일체크 (com.kbstar.kji.enbncmn, checkOvsesHoldyChk)

```
IDataSet checkOvsesHoldyChkReq = new DataSet();

checkOvsesHoldyChkReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
checkOvsesHoldyChkReq.put("inDstcd", requestData.get("inDstcd"));
checkOvsesHoldyChkReq.put("inHoldyIrInstiCd", requestData.get("inHoldyIrInstiCd
checkOvsesHoldyChkReq.put("inYmd", requestData.get("inYmd"));
```

	// @@ 공유메소드 호출 (해외정보수신기관 코드별 휴일체크 checkOvsesHoldyChk) *IDataSet checkOvsesHoldyChkRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",* *"FUBcDateCheck.checkOvsesHoldyChk", checkOvsesHoldyChkReq, onlineCtx);* // @@ 출력 데이터 셋 처리 String outholdydstcd = checkOvsesHoldyChkRes.getString("outHoldyDstcd"); String outisonatnlcd = checkOvsesHoldyChkRes.getString("outIsoNatnlCd"); String outnatnlname = checkOvsesHoldyChkRes.getString("outNatnlName"); String outcncycd = checkOvsesHoldyChkRes.getString("outCncycd"); String outholdyridsticname = checkOvsesHoldyChkRes.getString("outHoldyRiDsticName"); String outholdyrrinstiname = checkOvsesHoldyChkRes.getString("outHoldyRrInstiName"); String outovseshengname = checkOvsesHoldyChkRes.getString("outOvsesHEngName"); String outbzoprymd = checkOvsesHoldyChkRes.getString("outBzoprYmd"); String rtnstat = checkOvsesHoldyChkRes.getString("rtnStat");
비 고	

PoC
1.48

PoC

PoC

PoC

PoC

해외정보수신기관 코드별 영업일 산출(FUBcDate.calculateOvsesBzoprDd())

프로그램 ID	FUBcDate.calculateOvsesBzoprDd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	해외기관휴일목록	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIH002		SQL ID	selectInHoldyIrInstiCd

프로그램 설명	입력년월일에 대해 해외정보수신기관 코드별 익영업년월일, 익익영업년월일, 3 영업년월일, 4 영업년월일, 5 영업년월일, 전영업년월일, 전전영업년월일 제공한다
프로그램 규칙 및 제약사항	1. 휴일정보수신기관코드 입력 체크 1) 휴일정보수신기관코드 체크: 해외기관 휴일목록 테이블에 존재유무 확인, 미존재시 에러처리 한다. 사용가능한 휴일정보수신기관코드 정보는 1.48 해외정보수신기관 코드별 휴일 체크 (FUBcDateCheck.checkOvsesHoldyChk())를 참조한다. 2 입력년월일은 아래의 조건을 만족해야 한다. *1) 입력년은 2009 ~ 2200 년 사이 값이어야 한다.* 2) 입력월은 01 ~ 12 사이 값이어야 한다. 3) 입력일이 입력월의 말일자보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 일로 한다. 3. 익영업년월일, 익익영업년월일, 3 영업년월일, 4 영업년월일, 5 영업년월일, 전영업년월일, 전전영업년월일의 산출 1) 해외정보수신기관 코드별 휴일 체크 프로그램(FUBcDateCheck.checkOvsesHoldyChk())을 호출하여 휴일구분코드가 '0'(평일)과 '3'(근무토요일)인 경우의 일자를 구한다 예시] 입력일 2008 년 4 월 5 일

월	화	수	목	금	토	일
3 월 31 일	1	2	3 전전영업년월일	4 전영업년월일	5 입력년월일	6
7 익영업년월일	8 익익영업년월일	9 3 영업년월일	10 4 영업년월일	11 5 영업년월일	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

가) 입력년월일: 2008-04-05
=> 익영업년월일: 2008-04-07, 익익영업년월일: 2008-04-08, 3 영업년월일: 2008-04-09, 4 영업년월일: 2008-04-10, 영업년월일: 2008-04-11, 전영업년월일: 2008-04-04, 전전영업년월일: 2008-04-0

나) 입력년월일: 2008-04-21
=> 익영업년월일: 2008-04-22, 익익영업년월일: 2008-04-23, 3 영업년월일: 2008-04-24, 4 영업년월일: 2008-04-25, 5 영업년월일: 2008-04-28, 전영업년월일: 2008-04-18, 전전영업년월일: 2008-04-17

*** 해외 휴일 데이터 정보**
- 해외휴일 데이터는 CMBS 에서 SWAPS Monitors(휴일자료) 업체로부터 제공 받는다
- 원천 데이터 변경요건이 있으면 CMBS 에서 변경하여 전행공통에 전달 되어지며(D-1) 해외 휴일을 사용하는 업무팀은 이를 인지하고 사용하여야 한다(CMBS 의 업무 요건에 따라 변경가능)
- 따라서 CMBS 에 의존 적이지 않고 업무 특성에 따라 해외휴일 데이터가 필요한 경우는 해당 업무팀에서 해외휴일 데이터를 별도로 구성 관리하여야 한다

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDate extends
                    com.kbstar.sqc.base.FunctionUnit
```

	[입력항목]
	String inGroupCoCd;
	String inHoldyIrInstiCd;
	String inYmd;
	
	[출력항목]
	String outBfBfBzoprYmd;
	String outBfBzoprYmd;
	String outNowBzoprYmd;
	String outNxbizYmd;
	String outTdatYmd;
	String out3BzoprYmd;
	String out4BzoprYmd;
	String out5BzoprYmd;
		● 그룹회사코드
		● 휴일정보수신기관코드
		● 년월일
		
		// 전전영업년월일
		// 전영업년월일
		// 금영업년월일
		// 익영업년월일
		// 익익영업년월일
		// 3 영업년월일
		// 4 영업년월일
		// 5 영업년월일

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 해외정보수신기관 코드별 영업일 산출 (com.kbstar.kji.enbncmn, calculateOvsesBzoprDd)
	
	IDataSet calculateOvsesBzoprDdReq = new DataSet();
	
	calculateOvsesBzoprDdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateOvsesBzoprDdReq.put("inHoldyIrInstiCd", requestData.get("inHoldyIrInstiCd"));
	calculateOvsesBzoprDdReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (해외정보수신기관 코드별 영업일 산출 calculateOvsesBzoprDd)
	IDataSet calculateOvsesBzoprDdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDate.calculateOvsesBzoprDd", calculateOvsesBzoprDdReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outbfbfbzoprymd = calculateOvsesBzoprDdRes.getString("outBfBfBzoprYmd");
	String outbfbzoprymd = calculateOvsesBzoprDdRes.getString("outBfBzoprYmd");
	String outnxbizymd = calculateOvsesBzoprDdRes.getString("outNxbizYmd");
	String outtdatymd = calculateOvsesBzoprDdRes.getString("outTdatYmd");
	String out3bzoprymd = calculateOvsesBzoprDdRes.getString("out3BzoprYmd");
	String out4bzoprymd = calculateOvsesBzoprDdRes.getString("out4BzoprYmd");
	String out5bzoprymd = calculateOvsesBzoprDdRes.getString("out5BzoprYmd");	

비 고		

1.49 해외정보수신기관 코드별 월초영업일/ 월말영업일/ 월말일 산출 (FUBcDate.calculateOvsesEnmnBzoprDay())

프로그램 ID	FUBcDate. calculateOvsesEnmnBzoprDay()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	해외기관휴일목록	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIH002		SQL ID	selectInHoldyIrInstiCd

프로그램 설명	입력년월일에 대해 해외정보수신기관 코드별 월초영업년월일/월말영업년월일/월말년월일을 제공한다
프로그램 규칙 및 제약사항	1. 해외정보수신기관코드 입력체크 1) 해외정보수신기관코드 체크: 해외기관 휴일목록 테이블에 존재유무 확인, 미존재시 에러처리 한다. 사용가능한 헤외정보수신기관코드 정보는 1.48 해외정보수신기관 코드별 휴일 체크 (FUBcDateCheck.checkOvsesHoldyChk())를 참조한다. 2. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력년은 2009 ~ 2200 년 사이 값이어야 한다. 2) 입력월은 01 ~ 12 사이 값이어야 한다. 3) 입력일이 입력월의 말일자보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 일로 한다. 3. 월초영업년월일/월말영업년월일/월말년월일 산출 1) 월초영업년월일은 1 일에서부터 해외정보수신기관 코드별 휴일 체크 프로그램 (FUBcDateCheck.checkOvsesHoldyChk())으로 휴일여부를 확인하고 휴일인 경우는 1 일자부터 +1 일씩 더한다 2) 월말년월일은 입력월의 말일자로 산출한다. 3) 월말영업년월일은 2)에서 구해진 말일자로 해외 휴일체크 프로그램 (FUBcDateCheck.checkOvsesHoldyChk())으로 휴일여부를 확인하고, 휴일인 경우는 말일자에서 -1 일씩 차감한다. 4) 입력년월일과 월말영업년월일 비교하여 아래와 같이 리턴상태코드를 다르게 준다 - '00' : 입력년월일이 월말영업년월일보다 작은 경우 - '06' : 입력년월일이 월말영업년월일보다 크거나 같은 경우 - '07' : 입력년월일이 월말영업년월일보다 크거나 같으면서 입력월이 12 월인 경우 예시] 2008 년 4 월

월	화	수	목	금	토	일
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

가) 입력년월일: 2008-04-05
=> 월말년월일: 2008-04-30, 월말영업년월일: 2008-04-30, 리턴상태코드: 00

*** 해외 휴일 데이터 정보**
- 해외휴일 데이터는 CMBS 에서 SWAPS Monitors(휴일자료) 업체로부터 제공 받는다
- 원천 데이터 변경요건이 있으면 CMBS 에서 변경하여 전행공통에 전달 되어지며(D-1) 해외 휴일을 사용하는 업무팀은 이를 인지하고 사용하여야 한다(CMBS 의 업무 요건에 따라 변경가능)
- 따라서 CMBS 에 의존 적이지 않고 업무 특성에 따라 해외휴일 데이터가 필요한 경우는 해당 업무팀에서 해외휴일 데이터를 별도로 구성 관리하여야 한다

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDate extends
                    com.kbstar.sqc.base.FunctionUnit
```

	[입력항목]
	String inGroupCoCd; // ● 그룹회사코드
	String inHoldyIrInstiCd; // ● 해외정보수신기관코드
	String inYmd; // ● 년월일
	
	[출력항목]
	String rtnStat; // 리턴상태코드
	String outMnThbgbzoprYmd; // 월초영업년월일
	String outEnmnYmd; // 월말년월일
	String outEnmnBzoprYmd; // 월말영업년월일

RTNSTAT 설명	00 : 정상(입력일자가 월중), 06 : 입력일자가 월말, 07 : 입력일자가 년말, 09 : ERROR

사용 설명(예시)	**// 공유메소드 호출 - 해외정보수신기관 코드별 월초영업일/월말영업일/월말일 산출**
	(com.kbstar.kji.enbncmn, calculateOvsesEnmnBzoprDay)
	
	IDataSet calculateOvsesEnmnBzoprDayReq = new DataSet();
	
	calculateOvsesEnmnBzoprDayReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateOvsesEnmnBzoprDayReq.put("inHoldyIrInstiCd", requestData.get("inHoldyIrInstiCd"));
	calculateOvsesEnmnBzoprDayReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (해외정보수신기관 코드별 월초영업일/월말영업일/월말일 산출
	calculateOvsesEnmnBzoprDay)
	IDataSet calculateOvsesEnmnBzoprDayRes = callSharedMethodByDirect ("com.kbstar.kji.enbncmn", "FUBcDate.calculateOvsesEnmnBzoprDay", calculateOvsesEnmnBzoprDayReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = calculateOvsesEnmnBzoprDayRes.getString("rtnStat");
	String outmnthbgbzoprymd = calculateOvsesEnmnBzoprDayRes.getString("outMnThbgBzoprYmd");
	String outenmnymd = calculateOvsesEnmnBzoprDayRes.getString("outEnmnYmd");
	String outenmnbzoprymd = calculateOvsesEnmnBzoprDayRes.getString("outEnmnBzoprYmd");

비 고	

1.50

일자 변환(년월 몇주차 요일에 맞는 일자반환)(FUBcDateChange.changeDteChng())

프로그램 ID	FUBcDateChange. changeDteChng()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력년월과 요청 주차와 요일을 입력받아 일자(년월일)를 반환한다.
프로그램 규칙 및 제약사항	**1. 월에 해당하는 주기는 토요일을 기준으로 한다. 따라서, 월의 마지막 일자가 토요일이 아니면 다음월의 첫번째 토요일에 귀속되어 다음월의 첫번째 주가 된다.** 예시] 입력년월: 200810, 요청주차: 2, 요일구분: 1 출력년월일: 20081006 입력년월: 200811, 요청주차: 1, 요일구분: 1 출력년월일: 20081027 **2. 요일구분 값** 1) '1' - 월요일 2) '2' - 화요일 3) '3' - 수요일 4) '4' - 목요일 5) '5' - 금요일 6) '6' - 토요일 7) '7' - 일요일 **3. 5 주차까지 요청할 수 있으며, 년월의 5 주차가 없으면 리턴상태코드에 '07'을 전달한다. 5 주차의 존재 유무를 판단하는 기준은 토요일이며, 따라서 5 주차에 해당하는 토요일이 없으면 다른 요일도 없다고 간주한다.** 예)2009 년 4 월 입력값-> 년월: 200904 요청주차: 5 요일구분: 2 -> ***리턴상태코드: 07***

일	월	화	수	목	금	*토: 기준*
			1	2	3	*4* 1 주기
5	6	7	8	9	10	*11* 2 주기
12	13	14	15	16	17	*18* 3 주기
19	20	21	22	23	24	*25* 4 주기
26	*27*	28	29	30		*5 주기 미존재*

입출력 설명

```
package com.kbstar.kjienbncmn.biz;

public class FUBcDateChange extends
                    com.kbstar.sqc.base.FunctionUnit

[입력항목]
    String inYm;                    ● 년월
    String inDmndWeekTh;            ● 요청주차
    String inDwkDstic;              ● 요일구분

[출력항목]
    String rtnStat;                 // 리턴상태코드
    String outYmd ;                 // 년월일
```

RTNSTAT 설명	00 : 정상, 07 : 입력년월의 5 주차가 없는경우

사용 설명(예시)	// 공유메소드 호출 - 년월 몇주차 요일에 맞는 일자변환
	(com.kbstar.kji.enbncmn, changeDteChng)
	IDataSet changeDteChngReq = new DataSet();
	
	changeDteChngReq.put("inYm", requestData.get("inYm"));
	changeDteChngReq.put("inDmndWeekTh", requestData.get("inDmndWeekTh"));
	changeDteChngReq.put("inDwkDstic", requestData.get("inDwkDstic"));
	
	// @@ 공유메소드 호출 (년월 몇주차 요일에 맞는 일자변환 changeDteChng)
	~~IDataSet changeDteChngRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",~~
	~~"FUBcDateChange.changeDteChng", changeDteChngReq, onlineCtx);~~
	
	// @@ 출력 데이터 셋 처리
	String outymd = changeDteChngRes.getString("outYmd");
	String rtnstat = changeDteChngRes.getString("rtnStat");

비 고	

1.51

PoC

PoC

PoC

PoC

PoC

세율기본 조회(FUBcRate.getTaxRates())

프로그램 ID	FUBcRate.getTaxRates()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	이자소득원천징수세율	SQLMAP 명	NAME SPACE	bcrate
	영문명	THKJITX03		SQL ID	selectTxCdCnt selectNatnlCdCnt selectTaxRates

프로그램 설명	세율코드,국가코드, 시작년월일, 종료년월일로 세율기본 테이블을 조회하여 세율구간별로 세율을 제공한다.
프로그램 규칙 및 제약사항	1. 세율대분류구분코드, 세율중분류구분코드, 세율소분류구분코드에 따른 세율코드는 아래와 같다. 1) 세율대분류구분 1:이자소득, 2:배당소득, 3:연금소득, 4:과징금, 9:기타 2) 세율중분류구분 1:개인세율, 2:법인세율, 3:협약세율개인, 4:협약세율법인 3) 세율소분류구분 001:비과세, 002:과세, 003:분리과세, 004:세금우대, 005:추징, 006:근로자장기저축, 007:개인연금, 008:장기주택마련저축, 009:연금신탁, 010:장기주식, 011:채권,012:근로자우대,013:근로자주식투자신탁 4) 세율코드(세율대분류+세율중분류+세율소분류+일련번호: 7 자리)

세율코드	내용	세율코드	내용
1100101	개인비과세	1100102	증권대체 비과세
1100103	생계형 비과세	1100201	개인과세
1100301	분리과세	1100401	세금우대
1100501	추징세율(일반)	1100502	추징세율(비과세및세금우대)
1100601	근로자장기저축(2%)	1100701	개인연금추징세율(4%)
1100801	장기주택마련저축추징세율(4%)	1100802	장기주택마련저축추징세율(8%)
1101101	채권개인과세(20%)	1101102	채권개인과세(22%)
1101103	채권분리과세(20%)	1101104	채권분리과세(22%)
1101105	채권세금우대(20%)	1101106	채권세금우대(22%)
1200101	법인비과세	1200201	법인과세
1201101	채권법인과세(20%)	1201102	채권법인과세(22%)
1300201	협약세율(개인)	1400201	협약세율(법인)
2100101	배당소득개인비과세	2100201	배당소득개인과세
2100301	배당소득분리과세	2100302	배당소득분리과세저율
2100401	배당소득개인우대	2101001	배당소득개인장기주식
2200101	배당소득법인비과세	2200201	배당소득법인과세
2300201	배당소득협약세율(개인)	2400201	배당소득협약세율(법인)
2400202	배당소득주식소유지분율별배당협약세율		
3100901	연금신탁(연금소득)(5%)	3100902	연금신탁해지가산세율(2%)
4100201	과징세율	9100201	기타소득(20%)

2. 국가코드별 관리

- 협약세율에 따른 세율코드는 이자소득협약세율개인, 이자소득협약세율법인, 배당소득협약세율개인, 배당소득협약세율법인으로 관리한다.
- 아래 국가코드만 관리하며 그외 국가는 에러처리한다.
- 협약세율과 관련없는 세율기본 조회시 국가코드는 '100'(대한민국)으로 한다.

국가코드	국가한글명	국가코드	국가한글명
102	일본	226	오스트리아
112	필리핀	227	포르투갈
122	말레이시아	229	핀란드
123	싱가폴	231	아일랜드
124	인도네시아	232	스페인
131	타이(태국)	233	아이슬랜드
132	미얀마	237	몰타
134	네팔	272	아제르바이젠(구소련)
136	라오스	273	벨라루스(구소련)
138	카자흐스탄(구소련)	276, 980	우크라이나
141	파키스탄	277	알바니아
142	인도	280	크로아티아
143	스리랑카	282	리투아니아
145	방글라데시	285	슬로베니아
148	우즈베키스탄(구소련)	286	슬로바키아
153	쿠웨이트	292	폴렌드
155	사우디아라비아	293	러시아
157	요르단	294	체코슬라바키아
161	이스라엘	295	헝가리
164	아랍에미리트연합국	296	루마니아
174	오만	297	불가리아
191	베트남	301	캐나다
192	몽고	303	미국
193	중국	401	멕시코
201	프랑스	403	칠레
202	이태리	407	브라질
203	독일	411	베네주엘라
204	네덜란드	621	이집트
205	벨기에	623	튜니시아
206	룩셈부르그	624	알제리아
208	그리스	625	모로코
209	터어키	661	남아공화국
221	노르웨이	701	호주
222	스웨덴	702	뉴우질랜드
223	덴마크	707	파푸아뉴기니아
224	영국	744	피지
225	스위스	850,851	말레이지아라부안

3. 추징세율/과징세율 관리
   1) 추징세율 : 소득세율 및 주민세율로 나뉘어 적용한다. 농특세율, 교육세율, 방위세율은 0 %로 적용한다.
   2) 과징세율 : 소득세율로 적용한다. 주민세율, 농특세율,교육세율,방위세율은 0%로 적용한다.

4. 세율코드 변경요건의 처리
   1) 하나의 상품이 세율코드가 변경되었을 경우 각각의 세율코드와 기간별로 처리되어야 함
   예시]세금우대상품에서 일반상품으로 변경시 각각의 세율코드로 기간을 나누어 요청해야 한다.
5. 이율변경 구간의 이자금액
   1) 이율변경 구간이 2 개 이상 존재시 각 구간별로 요청해야 한다.
6. 최대 요청 가능한 과세코드는 10 개이다.
7. 최대 출력 가능한 과세코드는 100 개이다.
8. 출력항목의 입력건수는 입력항목의 입력건수와 동일한 값을 출력한다. 즉 입력항목의 첫번째 항목의 출력이면 출력항목의 입력건수는 '1'이고, 두번째 입력항목의 출력이면 출력항목의 입력건수는 '2'가 된다.
9. 복수건의 세율조회시 데이터를 찾지 못하면 출력건수의 마지막 배열에 정보를 전달한다. 즉, 10 건까지는 정상적으로 출력항목을 조립하고 에러가 발생하면 11 번째 출력항목의 입력건수는 에러가 발생한 입력항목의 입력건수를 표시하여 몇 번째에서 에러가 발생했는지 정보를 주고 리턴상태코드는 '08'로 한다.

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcRate extends
                    com.kbstar.sqc.base.FunctionUnit

[입력항목]
     String inGroupCoCd;                    ● 그룹회사코드
     int inInptNoitm;                       ● 입력건수
     List  inInputIdx;                      ● 입력 List

[출력항목]
     String rtnStat;                        // 리턴상태코드
     int outOutptNoitm;                     // 출력건수
     List outInputIdx;                      // 출력 List

< inInputIdx 항목 >
    String inTxrtCd;                        ● 세율코드
    String inNatnlCd;                       ◎ 국가코드
    String inStartYmd;                      ● 시작년월일
    String inEndYmd;                        ● 종료년월일

< outInputIdx 항목 >
    int outInptNoitm;                       // 입력건수
    String outAplyStartDt;                  // 적용시작일자
    String outAplyEndDt;                    // 적용종료일자
    BigDecimal outIncmtxRt;                 // 소득세율(법인세율)
    BigDecimal outRsitaxRt;                 // 주민세율
    BigDecimal outSprtaxRt;                 // 농특세율
    BigDecimal outEduTxrt;                  // 교육세율
    BigDecimal outDfnsTxrt;                 // 방위세율
    String outTxrtCdHanglName;              // 세율코드한글명
    String outNatnlHanglName;               // 국가한글명
    int outAplySOwnPortnRt;                 // 적용시작소유지분율
    int outAplyEOwnPortnRt;                 // 적용종료소유지분율
    int outOwnPortnRtMgtNo;                 // 소유지분율관리번호
```

RTNSTAT 설명	00 : 정상, 08 : 복수개 입력시 ERROR 발생

사용 설명(예시)

```
// 공유메소드 호출 - 세율기본조회 (com.kbstar.kji.enbncmn, getTaxRates)

IDataSet getTaxRatesReq = new DataSet();

getTaxRatesReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
getTaxRatesReq.put("inInptNoitm", requestData.get("inInptNoitm"));
getTaxRatesReq.put("inInputIdx", requestData.getRecordSet("inInputIdx"));
```

	// @@ 공유메소드 호출 (세율기본 조회 getTaxRates) IDataSet getTaxRatesRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcRate.getTaxRates", getTaxRatesReq, onlineCtx); // @@ 출력 데이터 셋 처리 String rtnstat = getTaxRatesRes.getString("rtnStat"); int outoutptnoitm = getTaxRatesRes.getInt("outOutptNoitm"); IRecordSet getTaxRatesResRsOutinputidx = getTaxRatesRes.getRecordSet("outInputIdx"); PoC
비 고	

PoC

PoC

PoC

PoC

PoC

1.52 통합 구계좌 변환(FUBcAccountChange.changeIntgraAcnChng())

프로그램 ID	FUBcAccountChange. changeIntgraAcnChng()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	PoC 한글명	계좌변경내역 고객지정계좌채번	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIAC01 THKJICN05		SQL ID	N/A

프로그램 설명 PoC	구계좌에 대해 표준계좌로 변환(단, 구국민 요구불계좌는 로직으로 변환)하며, 반대로 표준계좌를 구계좌로 변환(단, 구국민 요구불계좌는 DB 로 변환)한다. 참고] - 통합국민계좌 : 국민은행과 주택은행 통합후의 계좌 - 구)국민계좌 : 통합국민은행 이전의 국민은행 계좌 - 표준계좌 : 표준화 원칙에 맞는 계좌 - 구계좌 : 구)국민계좌, 국민카드론 계좌, 구)국민의 외화예금계좌 (13 자리계좌) 보증기금, 특수채권과 같은 15 자리 계좌는 To-Be 에서 모두 14 자리로 전환된다. PoC ※ 고객지정계좌의 변환은 미제공
프로그램 규칙 및 제약사항	1. 구계좌를 입력받으면 계좌종류를 판단하여 계좌번호로 조회한다. PoC 2. 현재 사용되고 있는 계좌만 변환 대상이 된다. (과거의 은행 통폐합으로 변환되었던 계좌 중 대동은행계좌, 장기은행계좌, 부산금고계좌, 전남금고계좌, 대구금고계좌는 사용하지 않고 있다.) PoC 3. 대동은행계좌, 장기은행계좌, 부산금고계좌, 전남금고계좌, 대구금고계좌에 대한 변환정보 조회시는 합병은행 통합계좌간 변환(FUBcAccount.changeIntgraAccountBtn())모듈을 사용한다. 4. 계좌변환 대상과 판단기준은 다음과 같다. 1) 구)국민계좌: 계좌자리수가 12 이고 계좌번호의 과목코드가 '06'(가계당좌예금), '18'(당좌예금)이 아닌 계좌 2) 국민카드론계좌: 계좌자리수가 11 이고 여신코드 첫번째 자리가 '9'가 아닌 계좌 3) 구)국민외화예금 2: 계좌자리수가 13 이고 계좌번호의 과목코드가 '42','43'인 계좌 4) 고객지정계좌: 계좌자리수가 11 이고 첫번째 자리가 '9'로 시작하는 계좌 5) 보증기금 및 특수채권계좌: 계좌자리수가 15 인 계좌 6) 정기예금계좌 : 계좌자리수 13 자리이면서 뒤에 취급번호 2 자리를 가진 과목코드가 '15'인 계좌 예) xxxx15-xx-xxxxx-xx(취급번호 2 자리) 5. "정기예금계좌변환" 시에는 반드시 일련번호를 포함한 15 자리 계좌번호를 입력해야 구) 국민계좌로 변환된 값을 반환받을 수 있음. 따라서, 13 자리 통합 정기예금계좌번호를 입력하고 모듈을 호출하는 경우에는 관련된 구)국민계좌가 존재하더라도 변환계좌번호에 입력된 13 자리계좌번호를 그대로 반환함. 예시 1) 정기예금계좌 15 자리 입력 . 입력계좌번호 : 524915017645601 . 변환계좌번호 : 5242230176455 예시 2) 정기예금계좌 13 자리 입력 . 입력계좌번호 : 5249150176456 . 변환계좌번호 : 52491501764 56 6. 구)국민 정기예금계좌(12 자리)를 입력값으로 하여 계좌변환 요청을 한 경우, 변환계좌번호에 취급일련번호를 포함한 15 자리 계좌번호를 반환한다 예시) . 입력계좌번호 : 524230176455 . 변환계좌번호 : 524915017645601 7. 편집용 계좌번호를 사용하는 업무는 CJIOT03 기능을 이용하여 편집 됩니다. 또는 차단말에 출력하기 위하여 편집하는 경우는 단

This document is strictly limited to the 2025 C2J conversation PoC

PoC

말의 멀티패턴 처리방안
을 이용하시면 됩니다(단말에서 편집)

8. 계좌구분코드는 아래와 같다.
1) '1' : 표준계좌
2) '2' : 구)국민은행계좌
3) '3' : 국민카드론계좌
4) '4' : 구)국민외화예금 2
5) '5' : 보증기금, 특수채권
6) '6' : 고객지정계좌
7) '7' : 보증기금, 특수채권
(15 자리 계좌, 과목 '05','09','16','44,'33'- 통합국민계좌)
8) '8' : KB 내맘대로계좌
- '0'으로 시작하는 10~11 자리 계좌

9. 계좌번호 체계
1) 11 자리 : 국민카드론계좌, 여신코드(2)+년도(2)+일련번호(6)+검증번호(1)
고객지정계좌, 고객지정계좌구분(1)+일련번호(2)+고객지정번호(8)
2) 12 자리 : 당좌(가계)예금계좌, 점번(4)+과목(2)+일련번호(5)+검증번호(1)
구)국민계좌, 점번(3)+과목(2)+일련번호(6)+검증번호(1)
3) 13 자리 : 정기예금, 무역금융, 점번(4)+과목(2)+일련번호(6)+검증번호(1)
구)국민외화예금 2, 점번(3)+과목(2)+일련번호(6)+검증번호(1)+구분(1)
4) 14 자리 : 과목 2 자리계좌, 점번(4)+과목(2)+일련번호(7)+검증번호(1)
과목 3 자리계좌, 점번(4)+과목(3)+일련번호(6)+검증번호(1)
5) 15 자리 : 보증기금계좌, 점번(4)+과목(2)+일련번호(8)+검증번호(1)
특수채권계좌, 점번(4)+과목(2)+일련번호(7)+검증번호(1)+계정구분(1)
(가수가지급 계좌는 변환을 하지 않는다)

10. 구분코드 :
- '1' 인 경우
표준계좌 입력시 변환테이블을 참조 하지 않고 입력계좌를 변환계좌로 출력
표준계좌가 아닌경우 변환 테이블을 이용 변환 후 변환 계좌로 출력
- 미입력한 경우
표준계좌인 경우 변환테이블을 참조하여 계좌가 있으면 구 계좌를 변환계좌로
출력 테이블에 없으면 입력계좌를 변환계좌로 출력
표준계좌가 아닌경우 변환 테이블을 이용 변환 후 변환 계좌로 출력

~~※ 고객지정계좌, KB 내맘대로계좌의 변환은 미제공~~

11. KB 내맘대로계좌 처리
- 입력
> 구분코드 --- '1' OR 미입력
> 계좌번호 --- KB 내맘대로계좌
- 출력
> 계좌번호 ---------- KB 내맘대로계좌
> 계좌구분코드 -------- 8
> 변환계좌번호 -------- 통합계좌(표준계좌)
> 변환계좌구분코드 ---- 1
> 변환계좌번호 2 ------- 구국민계좌(통합계좌에 따른 구국민 계좌 존재시)
> 변환계좌구분코드 2 --- 2 (통합계좌에 따른 구국민 계좌 존재시)

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcAccountChange extends
                    com.kbstar.sqc.base.FunctionUnit {
```

[입력항목]

```
String inGroupCoCd;
String inDstcd;
String inAcno;
```

<입력항목>
- 그룹회사코드
- 구분코드
- 계좌번호

[출력항목]

```
String outAcno
```

<출력항목>
계좌번호

	String outDstcd String outChngAcno String outChngAcnDstcd String outChngAcno2 String outChngAcnDstcd2	계좌구분코드 변환계좌번호 변환계좌구분코드 변환계좌번호 2 변환계좌구분코드 2
RTNSTAT 설명	N/A	
사용 설명(예시)		
비 고		

1.53

PoC

PoC

PoC

PoC

PoC

PoC

로직으로 계좌변환(요구불 계좌)(FUBcAccountChange.changeAcnChngByLogic())

프로그램 ID	FUBcAccountChange.changeAcnChngByLogic()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	구)국민계좌 중 요구불 과목에 대해 로직으로 계좌변환을 한다.
프로그램 규칙 및 제약사항	1. 통합 구계좌 변환(FUBcAccount.changeIntgraOldAccount()) 프로그램에서 구계좌에서 표준계좌로 변환할 때, 구)국민의 요구불 계좌는 성능을 위하여 로직으로 변환을 한다. 1) 구)국민계좌의 요구불계좌: 계좌과목코드가 '01'(보통예금), '21'(저축예금), '24'(자유저축예금), '25'(기업자유예금), '04'(당좌예금), '05'(가계당좌예금) 인 계좌 2) 구)국민계좌의 과목코드는 통합계좌의 과목코드로 변경하여 변환해준다. 가) '01'(보통예금) -> '01'(보통예금) 나) '21'(저축예금) -> '02'(저축예금) 다) '24'(자유저축예금) -> '25'(자유저축예금) 라) '25'(기업자유예금) -> '37'(기업자유예금) 마) '04'(당좌예금) -> '18'(당좌예금) 바) '05'(가계당좌예금) -> '06'(가계당좌예금) 사) '26' (연계업무) -> '90'(연계업무) 3) '01'(보통예금), '21'(저축예금), '24'(자유저축예금), '25'(기업자유예금), '26' (연계업무)인 경우는 14 자리로 계좌 변환을 하고 '04'(당좌예금), '05'(가계당좌예금) 계좌는 12 자리로 계좌 변환을 한다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	`public class FUBcAccountChange extends com.kbstar.sqc.base.FunctionUnit {`
	[입력항목]
	`String inOlAcno;`
	[출력항목]
	`String outNewAcno`
	
	<입력항목>
	● 구계좌번호
	
	<출력항목>
	신계좌번호

RTNSTAT 설명	N/A
사용 설명(예시)	
비 고	

1.54

합병은행 통합계좌간 변환(N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	합병은행 통합계좌간 변환프로그램으로 구계좌와 차세대계좌간 변환 프로그램 (FUBcAccount.changeIntegraOldAccount())으로 찾을 수 없는 경우 사용한다. 동남은행의 현금카드 거래는 동 프로그램을 이용하여 계좌변환을 해야한다.
프로그램 규칙 및 제약사항	1. 변환할 계좌종류에 따라 계좌유형구분코드와 변환요청계좌번호를 입력하여 입력 변환요청계좌번호에 맵핑되는 변환계좌번호를 조회한다. 2. 계좌유형구분코드는 다음과 같다. 1) '01' : 통합계좌 -> 구)국민계좌 2) '02' : 구)국민계좌 -> 통합계좌 3) '03' : 대동은행계좌 -> 구)국민계좌 4) '04' : 장기은행계좌 -> 구)국민계좌 5) '05' : 부산금고계좌 -> 구)국민계좌 6) '06' : 전남금고계좌 -> 구)국민계좌 7) '07' : 대구금고계좌 -> 구)국민계좌 8) '08' : 구)국민계좌 -> 대동은행계좌 9) '09' : 구)국민계좌 -> 장기은행계좌 10) '10' : 구)국민계좌 -> 부산금고계좌 11) '11' : 구)국민계좌 -> 전남금고계좌 12) '12' : 구)국민계좌 -> 대구금고계좌 13) '13' : 국민카드 -> 통합계좌 14) '14' : 통합계좌 -> 국민카드 15) '15' : 통합계좌 -> 동남계좌 16) '16' : 동남계좌 -> 통합계좌 17) '17' : 여신한도거래번호(14 자리) -> 한도계좌번호(14 자리) 18) '18' : 한도계좌번호(14 자리) -> 여신한도거래번호(14 자리) 3. 대동은행/장기은행/부산금고/전남금고/대구금고/국민카드론 계좌가 구)국민계좌로 변환시 3개의 계좌로 변환된다.
입출력 설명	
RTNSTAT 설명	N/A
사용 설명(예시)	
비 고	

1.55 문자열 길이 변환(FUBcChange.changeLettersLineLength())

프로그램 ID	FUBcChange. changeLettersLineLength()		PoC 언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력데이터를 받아 요청하는 길이만큼 절사하거나 공백을 채워서 반환한다.
프로그램 규칙 및 제약사항	1. 한글데이터의 길이 변경시 한글이 잘리는 경우에 마지막 1 BYTE 를 SI 로 변환한다. 예시] 입력데이터: 세계로향하는국민은행, 출력요구길이: 12 => 출력데이터: ‘SO’세계로향하’SI’ 2. 한글 EBCDIC 의 처리시 SO/SI 가 맞지 않으면 출력요구길이를 1 BYTE 더하여 SI 를 덧붙인다. 3. 혼합문자의 길이도 변경이 가능하다. 예시] 1) 한글 + 숫자 : 차세대 123 2) 한글 + 영문 : 차세대 ABC 4. MAINFRAME 은 EBCDIC 코드이므로 출력데이터 앞뒤에 SO/SI 를 붙여서 변경한다. PoC
입출력 설명	PoC package com.kbstar.kji.enbncmn.biz; public class FUBcChange extends com.kbstar.sqc.base.FunctionUnit [입력항목] String inInptData; ● 입력 데이터 int inOutptRqstLen; ● 출력요구 길이 [출력항목] PoC PoC String outOutptData; // 출력 데이터
RTNSTAT 설명	N/A PoC

사용 설명(예시)	// 공유메소드 호출 - 문자열 길이 변환 (com.kbstar.kji.enbncmn, changeLettersLineLength)
	
	IDataSet changeLettersLineLengthReq = new DataSet();
	
	changeLettersLineLengthReq.put("inInptData", requestData.get("inInptData"));
	changeLettersLineLengthReq.put("inOutptRqstLen", requestData.get("inOutptRqstLen"));
	
	// @@ 공유메소드 호출 (문자열 길이 변환 changeLettersLineLength)
	IDataSet changeLettersLineLengthRes = callSharedMethodByDirect
	("com.kbstar.kji.enbncmn", "FUBcChange.changeLettersLineLength",
	changeLettersLineLengthReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outoutptdata = changeLettersLineLengthRes.getString("outOutptData");

비 고	

PoC

1.56 숫자 표기된 금액을 한글/ 영문으로 변환 (FUBcChange.changeAmountHanglEng())

프로그램 ID	FUBcChange. changeAmountHanglEng()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	숫자 금액을 한글금액과 영문금액으로 변환한다. 한글금액 결과는 2 개를 출력해 주는데 하나는 '금'~'원정'을 붙여준다.
프로그램 규칙 및 제약사항	1. 입력금액의 자릿수는 (18,3)이며 한글금액의 경우에 소수점 이하는 절사한다. 1. 구분코드는 아래와 같다. 1) '1' : 한글금액으로 변환 (정수 15 자리만 변환) 2) '2' : 영문금액으로 변환 (정수 15 자리의 변환 및 소수점 이하 3 자리까지 변환) 예시] 입력숫자: 123456789012345 => 출력금액 1 : 금 일천이백삼십사억오천육백칠십팔만구천일십이원정 출력금액 2 : 일천이백삼십사억오천육백칠십팔만구천일십이 예시] 입력숫자 : 123456789012345.120 => 출력금액 1 : ONE HUNDRED TWENTY THREE TILLION FOUR HUNDRED FIFTY SIX BILLION SEVEN HUNDRED EIGHTY NINE MILLION SEVEN HUNDRED TWELVE THOUSAND THREE HUNDRED FORTY FIVE AND 120 2. 한글금액과 영문금액으로 변환시 변환된 길이를 함께 출력 해준다.

PoC PoC

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcChange extends
                        com.kbstar.sqc.base.FunctionUnit

[입력항목]
     String inDstcd;                          ● 구분코드
     BigDecimal inInptFigAmt;                 ◎ 입력숫자(금액)

[출력항목]
     String outOutptAmt1;                     // 출력금액 1
     int outChngLen1;                         // 변환길이 1
     String outOutptAmt2;                     // 출력금액 2
     int outChngLen2;                         // 변환길이 2
```

PoC PoC PoC

RTNSTAT 설명	N/A

사용 설명(예시)

```
// 공유메소드 호출 - 숫자 표기된 금액을 한글/영문으로 변환
                                (com.kbstar.kji.enbncmn, changeAmountHanglEng)

  IDataSet changeAmountHanglEngReq = new DataSet();

  changeAmountHanglEngReq.put("inDstcd", requestData.get("inDstcd"));
  changeAmountHanglEngReq.put("inInptFigAmt", requestData.get("inInptFigAmt"));

// @@ 공유메소드 호출 (숫자 표기된 금액을 한글/영문으로 변환 changeAmountHanglEng)
 IDataSet changeAmountHanglEngRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
                "FUBcChange.changeAmountHanglEng", changeAmountHanglEngReq, onlineCtx);
```

	// @@ 출력 데이터 셋 처리 String outoutptamt1 = changeAmountHanglEngRes.getString("outOutptAmt1"); String outchnglen1 = changeAmountHanglEngRes.getString("outChngLen1"); String outoutptamt2 = changeAmountHanglEngRes.getString("outOutptAmt2"); String outchnglen2 = changeAmountHanglEngRes.getString("outChngLen2");
비 고	

1.57

가변길이 데이터 변환(FUBcChange.changeChangeableData())

프로그램 ID	FUBcChange. changeChangeableData()		언어 구분	JAVA(○)	
PoC 입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	PoC SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	가변길이의 데이터(일반/이동 전화번호 및 계좌번호(고객지정계좌포함))를 일정한 형식으로 편집한다.
프로그램 규칙 및 제약사항	1. 입력되는 국가전화코드, 전화번호, 계좌번호에서 숫자만 추출하여 편집한다. 2. 구분코드 1) '1' : 입력전화번호에 구분자를 추가하여 출력한다. 2) '2' : 입력계좌번호에 구분자를 추가하여 출력한다. 3. 국가전화코드는 선택항목이고, 국가전화코드가 입력되면 출력되는 형태는 달라진다. PoC 예시] 국가전화코드: 82, 전화번호: 0220739999 => 출력 편집전화번호: 82-2-2073-9999 예시] 국가전화코드: 미입력, 전화번호: 0220739999 => 출력 편집전화번호: 02)2073-9999 4. 핸드폰번호의 경우는 편집형태를 달리한다. 1) 입력된 전화번호의 지역번호자리가 010, 011, 016 , 017, 018, 019 로 시작되는 경우에는 입력된 전화번호가 핸드폰번호인 것으로 간주함 예시] 전화번호: 01022223333 => 출력 편집전화번호: 010-2222-3333 5. 각 계좌번호 편집변환. 1) 계좌번호내 계정과목이 7 로 시작되는 계좌번호 14 자리. 예시] 123456-12-123456 PoC 2) 계좌번호내 계정과목이 06 인 가계당좌예금계좌번호와 18 인 당좌예금계좌번호 12 자리. 예시] 123456-12-1234 3) 계좌번호내 계정과목이 15 인 정기예금계좌번호, 계정과목이 66 인 무역금융/내국신용장 계좌번호 13 자리. 예시] 123456-12-1234567,123456-12-12345 4) 계좌번호내 계정과목이 42, 43 인 구)국민외화예금계좌번호 13 자리. 예시] 12356-12-12345 5) 계좌번호내계정과목이 33 인 신용보증계좌번호, 계정과목이 94 인 가수/가지급계좌번호, 05 인 특수채권 계좌번호 15 자리. 예시] 123456-12-1234567 6) 구)국민계좌번호 12 자리. 예시] 123-12-1234-123 PoC 7) 고객지정계좌번호 11 자리 예시] 9-12345678-12 PoC 8) KB 내맘대로계좌 0 으로 시작하는 10~11 자리 예시] 010-123-1234 , 010-1234-1234 9) 상기 이외의 계좌번호 14 자리. 예시] 123456-12-123456

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcChange extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inDstcd;
	String inNatnlTelCd;
	String inTelno;
	String inAcno;
	
	[출력항목]
	String outEdtTelno;
	String outEdtAcno;
		● 구분코드
		◎ 국가전화코드
		◎ 전화번호
		◎ 계좌번호
		
		// 출력 편집전화번호
		// 출력 편집계좌번호

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 가변길이 데이터 변환 (com.kbstar.kji.enbncmn, changeChangeableData)
	
	IDataSet changeChangeableDataReq = new DataSet();
	
	changeChangeableDataReq.put("inDstcd", requestData.get("inDstcd"));
	changeChangeableDataReq.put("inNatnlTelCd", requestData.get("inNatnlTelCd"));
	changeChangeableDataReq.put("inTelno", requestData.get("inTelno"));
	changeChangeableDataReq.put("inAcno", requestData.get("inAcno"));
	
	// @@ 공유메소드 호출 (가변길이 데이터 변환 changeChangeableData)
	IDataSet changeChangeableDataRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcChange.changeChangeableData", changeChangeableDataReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outedttelno = changeChangeableDataRes.getString("outEdtTelno");
	String outedtacno = changeChangeableDataRes.getString("outEdtAcno");	

비 고		

1.58

PoC

PoC

개인정보 보호를 위한 변환(FUBcChange.changePrivatePersonInfo())

프로그램 ID	FUBcChange. changePrivatePersonInfo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	보안을 위해 고객번호, 계좌번호, 입력데이타을 '*'로 아래와 같이 편집처리 하고. 계발계인 경우에 입력성명에 대해 100 인의 성명중 랜덤으로 선택하여 출력한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 고객번호. 계좌번호, 성명을 입력하는 경우이며, SYSTEM 구분에 따라 출력이 다르다. 가) 'P'(운영계) : 고객번호, 계좌번호, 성명을 변환없이 입력한대로 출력한다. 나) 'D'(개발계) : 입력값의 길이에 따라서 다르게 편집하여 출력 한다. - 고객번호, 계좌번호 처리 : 길이가 6 이상인 경우 오른쪽 4 자리를 '*'처리하고, 5 이하인 경우 왼쪽 4 자리를 '*'처리한다. - 성명 : 입력된 성명대신 100 인의 성명중 랜덤으로 선택하여 출력한다. 2) '2' : 입력데이타, 편집시작자리, 편집종료자리를 입력하는 경우, 입력데이터에 대해 원하는 자릿수만큼 '*'처리를 하는데, 특수문자 '-' 값이 있는 경우는 그 값을 제외하고 나머지만 '*'처리한다. 2. 구분코드가 '2'인경우 편집종료자릿수는 편집시작자릿수보다 큰 값이어야 한다. 입력값 중 공백이 있으면 공백이전 자릿수가 편집종료 자릿수가 된다. ** 공백제거 필요시 업무에서 공백제거 후 호출한다. > 오류처리 예시) inInptData {AB CDEF} inEdtStartFgrs {04} inEdtEndFgrs {05}

PoC

PoC

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcChange extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inDstcd;`
	`String inCno;`
	`String inAcno;`
	`String inCustnm;`
	`String inInptData;`
	`int inEdtStartFgrs;`
	`int inEdtEndFgrs;`
	
	[출력항목]
	`String outCno;`
	`String outAcno;`
	`String outCustnm;`
	`String outOutptData;`
		● 구분코드
		◎ 고객번호
		◎ 계좌번호
		◎ 고객명
		◎ 입력데이터
		◎ 편집시작자리
		◎ 편집종료자리
		
		// 출력고객번호
		// 출력계좌번호
		// 출력고객명
		// 출력데이터

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 개인정보 보호를 위한 변환
	(com.kbstar.kji.enbncmn, changePrivatePersonInfo)
	`IDataSet changePrivatePersonInfoReq = new DataSet();`
	
	`changePrivatePersonInfoReq.put("inDstcd", requestData.get("inDstcd"));`	

PoC

```
changePrivatePersonInfoReq.put("inCno", requestData.get("inCno"));
changePrivatePersonInfoReq.put("inAcno", requestData.get("inAcno"));
changePrivatePersonInfoReq.put("inCustnm", requestData.get("inCustnm"));
changePrivatePersonInfoReq.put("inInptData", requestData.get("inInptData"));
changePrivatePersonInfoReq.put("inEdtStartFgrs", requestData.get("inEdtStartFgrs"));
changePrivatePersonInfoReq.put("inEdtEndFgrs", requestData.get("inEdtEndFgrs"));

// @@ 공유메소드 호출 (개인정보 보호를 위한 변환 changePrivatePersonInfo)
 IDataSet changePrivatePersonInfoRes = callSharedMethodByDirect
                    ("com.kbstar.kji.enbncmn", "FUBcChange.changePrivatePersonInfo",
                                                changePrivatePersonInfoReq, onlineCtx);
// @@ 출력 데이터 셋 처리
  String outcno = changePrivatePersonInfoRes.getString("outCno");
  String outacno = changePrivatePersonInfoRes.getString("outAcno");
  String outcustnm = changePrivatePersonInfoRes.getString("outCustnm");
  String outoutptdata = changePrivatePersonInfoRes.getString("outOutptData");
```

비 고	

1.59

PoC

PoC

PoC

계좌번호 체크디지트 산출 및 검증(FUBcValidation.validationAccountNumber())

PoC

프로그램 ID	FUBcValidation. validationAccountNumber()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	1. 각 계좌번호별 체크디지트를 산출 및 검증 한다. 1) 계정과목이 7 로 시작되는 계좌번호 14 자리. 2) 계정과목이 06 인 가계당좌예금계좌번호와 18 인 당좌예금계좌번호 12 자리. 3) 계정과목이 15 인 정기예금계좌번호, 계정과목이 66 인 무역금융/내국신용장 계좌번호 13 자리. 4) 계정과목이 42, 43 인 구)국민외화예금계좌번호 13 자리. 5) 계정과목이 33 인 신용보증계좌번호, 계정과목이 94 인 가수/가지급계좌번호, 05 인 특수채권계좌번호 15 자리. 6) 구)국민계좌번호 12 자리. 7) 상기 이외의 계좌번호 14 자리 8) KB 내맘대로계좌번호는 '2' : 체크디지트 검증시 검증하지 않고 정상처리함 예) 01012341234,0171231234 2. 1), 2), 3), 7)는 체크디지트 산출 및 검증이 있으며, 4), 5), 6)은 체크디지트 검증만 있다. 입력 계좌번호는 숫자이외(공백,문자,특수문자 등)값은 제외하고 길이를 산출한다.

PoC

PoC
PoC
PoC
PoC

| 프로그램 규칙 및 제약사항 | 3. 구분코드
1) ‘1’ : 체크디지트 산출
2) ‘2’ : 체크디지트 검증
4. 계좌과목구분코드가 ‘7’로 시작하는 14 자리계좌는 아래와 같이 체크 디지트를 산출한다.
1) 체크 디지트산출식
입력계좌번호 1 번째자리 * 3 +
입력계좌번호 2 번째자리 * 1 +
입력계좌번호 3 번째자리 * 7 +
입력계좌번호 5 번째자리 * 3 +
입력계좌번호 6 번째자리 * 1 +
입력계좌번호 7 번째자리 * 7 +
입력계좌번호 8 번째자리 * 3 +
입력계좌번호 9 번째자리 * 1 +
입력계좌번호 10 번째자리 * 7 +
입력계좌번호 11 번째자리 * 3 +
입력계좌번호 12 번째자리 * 1 +
입력계좌번호 13 번째자리 * 7 + 2
2) 체크디지트 = 10 - 체크디지트 산출값
3) 입력계좌번호 14 번째 자리값과 체크 디지트가 일치하는지 여부 확인한다.
5. 가계당좌예금 12 자리계좌인경우 아래와 같이 체크 디지트를 산출한다.
1) 체크 디지트산출식
입력계좌번호 1 번째자리 * 3 +
입력계좌번호 2 번째자리 * 1 +
입력계좌번호 3 번째자리 * 7 +
입력계좌번호 5 번째자리 * 3 +
입력계좌번호 6 번째자리 * 1 +
입력계좌번호 7 번째자리 * 7 +
입력계좌번호 8 번째자리 * 3 +
입력계좌번호 9 번째자리 * 1 +
입력계좌번호 10 번째자리 * 7 +
입력계좌번호 11 번째자리 * 3
2) 체크 디지트 = 10 - 체크 디지트산출값
3) 입력계좌번호 12 번째 자리값과 체크 디지트가 일치하는지 여부 확인한다.
6. 정기예금, 무역금융/내국신용장 13 자리 계좌인경우 아래와 같이 체크 디지트를 산출한다.
1) 체크 디지트산출식
입력계좌번호 1 번째자리 * 3 +
입력계좌번호 2 번째자리 * 1 +
입력계좌번호 3 번째자리 * 7 +
입력계좌번호 5 번째자리 * 3 +
입력계좌번호 6 번째자리 * 1 +
입력계좌번호 7 번째자리 * 7 +
입력계좌번호 8 번째자리 * 3 +
입력계좌번호 9 번째자리 * 1 +
입력계좌번호 10 번째자리 * 7 +
입력계좌번호 11 번째자리 * 3 +
입력계좌번호 12 번째자리 * 1 + 2
2) 체크 디지트 = 10 - 체크 디지트산출값
3) 입력계좌번호 13 번째 자리값과 체크 디지트가 일치하는지 여부 확인한다 |

PoC

PoC

PoC

PoC

PoC

7. 당좌예금 12 자리계좌인경우 아래와 같이체크 디지트를 산출한다.
  1) 체크 디지트산출식
    입력계좌번호 1 번째자리 * 3 +
    입력계좌번호 2 번째자리 * 1 +
    입력계좌번호 3 번째자리 * 7 +
    입력계좌번호 5 번째자리 * 3 +
    입력계좌번호 6 번째자리 * 1 +
    입력계좌번호 7 번째자리 * 7 +
    입력계좌번호 8 번째자리 * 3 +
    입력계좌번호 9 번째자리 * 1 +
    입력계좌번호 10 번째자리 * 7 +
    입력계좌번호 11 번째자리 * 3 + 2
  2) 체크 디지트 = 10 - 체크 디지트산출값
  3) 입력계좌번호 12 번째 자리값과 체크 디지트가 일치하는지 여부 확인한다
8. 신용보증, 가수,가지급 15 자리계좌인경우 아래와 같이 체크 디지트를 검증만 한다.
  1) 체크 디지트산출식
    입력계좌번호 1 번째자리 * 3 +
    입력계좌번호 2 번째자리 * 1 +
    입력계좌번호 3 번째자리 * 7 +
    입력계좌번호 5 번째자리 * 3 +
    입력계좌번호 6 번째자리 * 1 +
    입력계좌번호 7 번째자리 * 7 +
    입력계좌번호 8 번째자리 * 3 +
    입력계좌번호 9 번째자리 * 1 +
    입력계좌번호 10 번째자리 * 7 +
    입력계좌번호 11 번째자리 * 3 +
    입력계좌번호 12 번째자리 * 1 +
    입력계좌번호 13 번째자리 * 7
    입력계좌번호 14 번째자리 * 3 + 2
  2) 체크 디지트 = 10 - 체크 디지트산출값
  3) 입력계좌번호 15 번째 자리값과 체크 디지트가 일치하는지 여부 확인한다.
9. 그 외 12 자리인 계좌일 경우는 구)국민계좌번호이므로 아래와 같이 체크 디지트를 검증만 한다.
  1) 체크 디지트 산출식 :
    입력구국민은행계좌번호 1번째자리 *6 +
    입력구국민은행계좌번호 2번째자리 *4 +
    입력구국민은행계좌번호 3번째자리 *3 +
    입력구국민은행계좌번호 4번째자리 *2 +
    입력구국민은행계좌번호 5번째자리 *9 +
    입력구국민은행계좌번호 6번째자리 *8 +
    입력구국민은행계좌번호 7번째자리 *7 +
    입력구국민은행계좌번호 8번째자리 *6 +
    입력구국민은행계좌번호 9번째자리 *4 +
    입력구국민은행계좌번호 10번째자리 *3 +
    입력구국민은행계좌번호 11 번째자리 *2
  2) 체크 디지트 : 연산합을 11로 나눈 나머지값을 구하여
    가) 나머지가 0,10이면 1
    나) 나머지가 1이면 0
    다) 그외는 11-나머지값
  3) 입력계좌번호 12번째 자리값과 체크 디지트가 일치하는지 여부 확인한다.
10. 고객지정계좌 11 자리계좌인경우 아래와 같이 체크 디지트를 산출한다.(계좌체계 미정)
  1) 체크 디지트 산출합 :
    입력고객지정계좌 2번째자리 *9 +
    입력고객지정계좌 4번째자리 *8 +
    입력고객지정계좌 5번째자리 *7 +
    입력고객지정계좌 6번째자리 *6 +
    입력고객지정계좌 7번째자리 *5 +
    입력고객지정계좌 8번째자리 *4 +
    입력고객지정계좌 9번째자리 *3 +
    입력고객지정계좌 10번째자리 *2 +

PoC

	입력고객지정계좌 11번째자리 *9 2) 체크 디지트 : 체크 디지트 산출합을 10 으로 나눈 나머지를 10 에서 뺀값 3) 입력계좌번호 3 번째 자리값과 체크 디지트가 일치하는지 여부 확인한다. 11. 그 외 14 자리 계좌인 경우 아래와 같이 체크 디지트를 산출한다. 1) 체크 디지트산출식 입력계좌번호 1 번째자리 * 3 + 입력계좌번호 2 번째자리 * 1 + 입력계좌번호 3 번째자리 * 7 + 입력계좌번호 5 번째자리 * 3 + 입력계좌번호 6 번째자리 * 1 + 입력계좌번호 7 번째자리 * 7 + 입력계좌번호 8 번째자리 * 3 + 입력계좌번호 9 번째자리 * 1 + 입력계좌번호 10 번째자리 * 7 + 입력계좌번호 11 번째자리 * 3 + 입력계좌번호 12 번째자리 * 1 + 입력계좌번호 13 번째자리 * 7 + 2 2) 체크디지트 = 10 - 체크디지트 산출값 3) 입력계좌번호 14 번째 자리값과 체크 디지트가 일치하는지 여부 확인한다.

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcValidation extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inDstcd; ● 구분코드
	String inAcno; ● 계좌번호
	
	[출력항목]
	String rtnStat; // 리턴상태코드
	String outChkDigit; // 체크디지트
	String outAcno; // 계좌번호

RTNSTAT 설명	00 : 정상, 07 : Check Digit ERROR

사용 설명(예시)	// 공유메소드 호출 - 계좌번호 체크디지트 산출 및 검증
	(com.kbstar.kji.enbncmn, validationAccountNumber)
	IDataSet validationAccountNumberReq = new DataSet();
	
	validationAccountNumberReq.put("inDstcd", requestData.get("inDstcd"));
	validationAccountNumberReq.put("inSbjctDstcd", requestData.get("inSbjctDstcd"));
	validationAccountNumberReq.put("inAcno", requestData.get("inAcno"));
	
	// @@ 공유메소드 호출 (계좌번호 체크디지트 산출 및 검증 validationAccountNumber)
	IDataSet validationAccountNumberRes = callSharedMethodByDirect
	("com.kbstar.kji.enbncmn", "FUBcValidation.validationAccountNumber",
	validationAccountNumberReq, onlineCtx);
	// @@ 출력 데이터 셋 처리
	String rtnstat = validationAccountNumberRes.getString("rtnStat");
	String outchkdigit = validationAccountNumberRes.getString("outChkDigit");
	String outacno = validationAccountNumberRes.getString("outAcno");

비 고	

1.60 M/S 계좌번호 체크디지트 산출 및 검증(FUBcValidation.validationMSAccountNumber())

프로그램 ID	FUBcValidation. validationMSAccountNumber()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	M/S 계좌번호 체크 디지트를 산출 및 검증한다.
프로그램 규칙 및 제약사항	1. 아래의 경우 사용된다. 1) 자동화기기에서 통장으로 업무처리시(MS 계좌정보 체크 디지트검증) 2) 통장정리시 MS 계좌정보 체크 디지트검증 2. 입력 MS 계좌번호는 아래의 조건을 만족해야 한다. 1) 숫자값이어야 한다. 2) 11, 12, 13, 14 자리의 계좌번호만 유효하다. 3) 자릿수만큼의 0 값이 입력된 경우 오류를 발생한다. 3. 시스템운영환경구분코드는 아래와 같다. (COMMON AREA : BICOM-SYS-OPER-EVIRN-DSTCD) 1) BICOM-SYS-OPER-EVIRN-DSTCD 가 운영 System(P)이면 시스템구분은 '0' 2) BICOM-SYS-OPER-EVIRN-DSTCD 가 개발 System(D)이면 시스템구분은 ';' 4. 체크 디지트 산출은 아래와 같다.(검증값) 1) 연산합 = 입력 MS 계좌번호 1 번째자리 * 1 + 입력 MS 계좌번호 2 번째자리 * 2 + 입력 MS 계좌번호 3 번째자리 * 3 + 입력 MS 계좌번호 4 번째자리 * 4 + 입력 MS 계좌번호 5 번째자리 * 5 + 입력 MS 계좌번호 6 번째자리 * 6 + 입력 MS 계좌번호 7 번째자리 * 7 + 입력 MS 계좌번호 8 번째자리 * 8 + 입력 MS 계좌번호 9 번째자리 * 9 + 입력 MS 계좌번호 10 번째자리 * 10 + 입력 MS 계좌번호 11 번째자리 * 11 + 입력 MS 계좌번호 12 번째자리 * 12 + 입력 MS 계좌번호 13 번째자리 * 13 + 입력 MS 계좌번호 14 번째자리 * 14 2) 체크 디지트(검증값) = 100 - 연산합 2 번째자리에서부터 2 자리값 5. 검증값 구분 산출은 아래와 같다. 1) 검증값구분 '=' 로 SETTINGS

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcValidation **extends** com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inMsAcno; ● MS 계좌번호
	
	[출력항목]
	String outValdnValDstic; // 검증값구분
	String outChkDigit; // 체크디지트
	String outSysDstic; // 시스템구분

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - M/S계좌번호 체크디지트 산출 및 검증
	(com.kbstar.kji.enbncmn, validationMSAccountNumber)
	
	IDataSet validationMSAccountNumberReq = new DataSet();
	
	validationMSAccountNumberReq.put("inMsAcno", requestData.get("inMsAcno"));
	
	// @@ 공유메소드 호출 (M/S계좌번호 체크디지트 산출 및 검증 validationMSAccountNumber)
	IDataSet validationMSAccountNumberRes = callSharedMethodByDirect
	("com.kbstar.kji.enbncmn", "FUBcValidation.validationMSAccountNumber",
	validationMSAccountNumberReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outvaldnvaldstic = validationMSAccountNumberRes.getString("outValdnValDstic");
	String outchkdigit = validationMSAccountNumberRes.getString("outChkDigit");
	String outsysdstic = validationMSAccountNumberRes.getString("outSysDstic");

비 고	

1.61

일반 체크디지트 검증(FUBcValidation.validationCheckDigit())

프로그램 ID	FUBcValidation. validationCheckDigit()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력 데이터의 체크디지트 산출 및 검증 결과 제공 한다. 산출 : 구)주택 3 자리 부점코드에서 변환된 4 자리 부점코드 산출, 지로코드 산출과 부점코드를 지로코드로 변환 산출 검증 : 구)주택 3 자리 부점코드에서 변환된 4 자리 부점코드 검증, 지로코드, 직원번호, 주민등록번호, 사업자등록번호, 법인등록번호, 외국인등록번호 검증과 부점코드를 지로코드로 변환 검증 ※ 참고 : 주민등록번호 체계 변경, 20201005 시행 > 관련내용 - 시행문 : 법무실 972, 2020-05-26 ,「 주민등록법 시행규칙 」 일부개정령(안) 입법예고 안내 주민등록번호를 신규 부여 · 변경하는 경우, 생년월일 · 성별 외에 지역표시번호를 폐지하고 임의번호를 부여하도록 함 - 알림방 : 전행공통 주민등록번호 관련 유틸 변경 안내 , 2020-09-16 > 해당 유틸은 주민등록번호 체계 변경전 번호에 대한 검증 기능입니다
프로그램 규칙 및 제약사항	1.입력 데이터 검증 1) 구분코드 : '01', '02', '03', '04', '05', '06', '07', '08', '09', '10' 외 값이 입력되면 오류 2) 구분코드에 따른 입력 데이터 검증 가) '01' : 구)주택 3 자리 부점코드에서 변환된 4 자리 부점코드 체크디지트 산출 디지트대상번호는 3 자리이며 '000' 또는 3 자리숫자인 값이 입력되어야 한다. 나) '02' : 구)주택 3 자리 부점코드에서 변환된 4 자리 부점코드 체크디지트 검증 디지트대상번호는 4 자리이며 '0000' 또는 4 자리숫자인 값이 입력되어야 한다. 다) '03' : 지로코드 체크디지트 산출 디지트대상번호는 6 자리이며 '000000' 또는 6 자리숫자인 값이 입력되어야 한다. 라) '04' : 지로코드 체크디지트 검증 디지트대상번호는 7 자리이며 '0000000' 또는 7 자리숫자인 값이 입력되어야 한다. 마) '05' : 부점코드를 지로코드로 변환 체크디지트 검증 디지트대상번호는 4 자리이며 '0000' 또는 4 자리숫자인 값이 입력되어야 한다. 바) '06' : 숫자로만된 직원번호 체크디지트 검증 디지트대상번호는 7 자리이며 '0000000' 또는 7 자리숫자인 값이 입력되어야 한다. 사) '07' : 주민등록번호 체크디지트 검증 디지트대상번호는 13 자리이며 '0000000000000' 또는 13 자리숫자인 값이 입력되어야 한다. 아) '08' : 사업자등록번호 체크디지트 검증 디지트대상번호는 10 자리이며 '0000000000' 또는 10 자리숫자인 값이 입력되어야 한다. 자) '09' : 법인등록번호번호 체크디지트 검증 디지트대상번호는 13 자리이며 '0000000000000' 또는 13 자리숫자인 값이 입력되어야 한다. 차) '10' : 외국인등록번호 체크디지트 검증 디지트대상번호는 13 자리이며 '0000000000000' 또는 13 자리숫자인 값이 입력되어야 한다. 2. 구분코드별 체크디지트 산출 및 검증 규칙 => 검증시 입력된 체크디지트와 산출된 체크디지트와 불일치하면 리턴상태코드를 '07'로 출력한다 1) 구분코드 '01' : 부점코드 체크디지트 산출은 아래와 같고 산출연산합은 숫자 1 자리이다. 가) 산출연산합 : 입력부점코드 1 번째자리 * 1 + 입력부점코드 2 번째자리 * 3 +

입력부점코드 3 번째자리 * 7 + 2
나) 산출체크디지트 : 10 - 산출연산합
다) 산출체크디지트를 출력한다.
2) 구분코드 '02' : 부점코드 체크디지트 검증은 아래와 같고 산출연산합은 숫자 1 자리이다.
가) 산출연산합 : 입력부점코드 1 번째자리 * 1 +
입력부점코드 2 번째자리 * 3 +
입력부점코드 3 번째자리 * 7 + 2
나) 산출체크디지트 : 10 - 산출연산합
다) 산출체크디지트를 출력한다.
라) 입력된 부점코드의 4 번째 자리값이 산출된 체크디지트와 일치하면 출력체크디지트가 되고, 입력된 부점코드가 출력부점코드가 된다.
3) 구분코드 '03' : 지로코드 체크디지트 산출은 아래와 같고 산출연산합은 숫자 1 자리이다.
가) 산출연산합 : 입력지로코드 1 번째자리 * 1 +
입력지로코드 2 번째자리 * 3 +
입력지로코드 3 번째자리 * 7 +
입력지로코드 4 번째자리 * 1 +
입력지로코드 5 번째자리 * 3 +
입력지로코드 6 번째자리 * 7
나) 산출체크디지트 : 10 - 체크디지트 산출값
다) 산출체크디지트를 출력한다.
4) 구분코드 '04' : 지로코드 체크디지트 검증은 아래와 같고 산출연산합은 숫자 1 자리이다.
가) 산출연산합 : 입력지로코드 1 번째자리 * 1 +
입력지로코드 2 번째자리 * 3 +
입력지로코드 3 번째자리 * 7 +
입력지로코드 4 번째자리 * 1 +
입력지로코드 5 번째자리 * 3 +
입력지로코드 6 번째자리 * 7
나) 산출체크디지트 : 10 - 체크디지트 산출값
다) 산출체크디지트를 출력한다.
라) 입력된 지로코드의 7 번째 자리값이 산출된 체크디지트와 일치하면 출력체크디지트가 되고, 입력된 지로코드가 출력지로코드가 된다.
5) 구분코드 '05' : 부점코드를 지로코드로 변환하고 체크디지트 검증
가) 부점코드와 조합가능한 은행코드는 아래와 같고 산출연산합은 숫자 1 자리이다.
- '004' : 국민은행
- '006' : 국민은행(구:주택은행)
- '019' : 국민은행
- '029' : 국민은행(구:동남은행)
나) 체크디지트 산출은 아래와 같다.
다) 산출지로코드 = 은행코드('004', '006', '019', '029') + 부점코드
라) 산출연산합 = 산출지로코드 1 번째자리 * 1 +
산출지로코드 2 번째자리 * 3 +
산출지로코드 3 번째자리 * 7 +
산출지로코드 4 번째자리 * 1 +
산출지로코드 5 번째자리 * 3
산출지로코드 6 번째자리 * 7
마) 산출체크디지트 = 10 - 연산합
바) 산출체크디지트를 출력한다.
사) 산출지로코드의 7 번째자리값과 체크디지트가 일치하면 산출지로코드가 출력지로코드가 되고, 일치하지 않으면 에러가 발생한다.
6) 구분코드 '06' : 직원번호 체크디지트 검증을 위한 체크디지트산출은 아래와 같고 산출연산합은 숫자 3 자리이고 산출연산값 숫자 2 자리, 산출체크디지트는 숫자 1 자리이다.
가) 산출연산합 = 입력직원번호 1 번째자리 * 2 +
입력직원번호 2 번째자리 * 3 +
입력직원번호 3 번째자리 * 4 +
입력직원번호 4 번째자리 * 5 +
입력직원번호 5 번째자리 * 6 +
입력직원번호 6 번째자리 * 7
나) 산출 연산값 : 산출연산합을 11로 나눈 나머지값을 구하여
- 나머지가 0, 10 이면 '1' -> 산출 체크디지트
- 나머지가 1이면 '0' -> 산출 체크디지트
- 그 외는 11-나머지값 나머지값 -> 산출 체크디지트
다) 입력된 직원번호의 7 번째 자리값이 체크디지트와 일치하면 출력체크디지트가 되고,

입력된 직원번호가 출력직원번호가 된다.
라) 新인사시스템 구축이후 체크디지트 사용하지 않음(HR부 확인)
- 2021.08.13 이후 입행직원은 체크디지트 사용하지 않음(상태코드 : ‘07’)

7) 구분코드 ‘07’ : 주민등록번호에 대한 검증을 위한 체크디지트산출은 아래와 같고 산출연산합은 숫자 3 자리이고 산출체크디지트는 숫자 1 자리이다.
가) 산출연산합 = 입력주민등록번호 1 번째자리 * 2 +
입력주민등록번호 2 번째자리 * 3 +
입력주민등록번호 3 번째자리 * 4 +
입력주민등록번호 4 번째자리 * 5 +
입력주민등록번호 5 번째자리 * 6 +
입력주민등록번호 6 번째자리 * 7 +
입력주민등록번호 7 번째자리 * 8 +
입력주민등록번호 8 번째자리 * 9 +
입력주민등록번호 9 번째자리 * 2 +
입력주민등록번호 10 번째자리 * 3 +
입력주민등록번호 11 번째자리 * 4 +
입력주민등록번호 12 번째자리 * 5
나) 산출체크디지트 = 11 - (연산합을 11 로 나눈 나머지)
다) 산출한 체크디지트와 입력주민등록번호의 13 번째 자리값과의 일치여부를 확인한다.

8) 구분코드 ‘08’ : 사업자등록번호에 대한 검증을 위한 체크디지트산출은 아래와 같고 산출연산합은 숫자 3 자리이고 산출체크디지트는 숫자 1 자리이다.
가) 체크디지트 산출값 = 입력사업자등록번호 9 번째자리값 * 5 값을 구한 후 구한값의 2 번째 자리값과 2 번째부터 2 자리값을 더한값
나) 산출연산합 = 입력사업자등록번호 1 번째자리 + 0 +
입력사업자등록번호 2 번째자리 * 3 +
입력사업자등록번호 3 번째자리 * 7 +
입력사업자등록번호 4 번째자리 + 0 +
입력사업자등록번호 5 번째자리 * 3 +
입력사업자등록번호 6 번째자리 * 7 +
입력사업자등록번호 8 번째자리 * 3 +
체크디지트 산출값 +
입력사업자등록번호 10 번째자리 + 0
다) 산출연산합의 3 번째값이 0 인지 확인한다.

9) 구분코드 ‘09’ : 법인번호에 대한 검증을 위한 체크디지트 산출은 아래와 같고 산출연산합은 숫자 3 자리이고 산출체크디지트는 숫자 1 자리이다.
가) 산출연산합 = 입력법인번호 1 번째자리 * 1 +
입력법인번호 2 번째자리 * 2 +
입력법인번호 3 번째자리 * 1 +
입력법인번호 4 번째자리 * 2 +
입력법인번호 5 번째자리 * 1 +
입력법인번호 6 번째자리 * 2 +
입력법인번호 7 번째자리 * 1 +
입력법인번호 8 번째자리 * 2 +
입력법인번호 9 번째자리 * 1 +
입력법인번호 10 번째자리 * 2 +
입력법인번호 11 번째자리 * 1 +
입력법인번호 12 번째자리 * 2
나) 산출체크디지트 = 10 - (산출연산합을 10 으로 나눈 나머지)
다) 산출체크디지트를 출력한다.
라) 입력법인번호 13 번째자리값과 체크디지트가 일치여부를 확인한다.

10) 구분코드 ‘10’ : 외국인 등록번호에 대한 검증을 위한 산출은 아래와 같고 산출연산합은 숫자 3 자리이고 산출체크디지트는 숫자 1 자리이다.
가) 산출연산합 = 입력외국인등록번호 1 번째자리 *2 +
입력외국인등록번호 2 번째자리 *3 +
입력외국인등록번호 3 번째자리 *4 +
입력외국인등록번호 4 번째자리 *5 +
입력외국인등록번호 5 번째자리 *6 +
입력외국인등록번호 6 번째자리 *7 +
입력외국인등록번호 7 번째자리 *8 +
입력외국인등록번호 8 번째자리 *9 +
입력외국인등록번호 9 번째자리 *2 +
입력외국인등록번호 10 번째자리 *3 +
입력외국인등록번호 11 번째자리 *4 +
입력외국인등록번호 12 번째자리 *5
나) 산출체크디지트 = 11 - (연산합을 11 로 나눈 나머지)

다) 산출체크디지트 = 산출체크디지트 + 2
라) 산출체크디지트를 출력한다.
마) 산출한 체크디지트와 입력외국인등록번호의 13 번째 자리값과의 일치여부를 확인한다.

3. 구분코드별 출력 규칙
1) '01' : 구)주택 3 자리 부점에서 변환된 4 자리 부점코드의 체크디지트를 산출
=> 체크디지트와 변환된 4 자리의 부점코드를 제공한다.
PoC
2) '02' : 구)주택 3 자리 부점에서 변환된 4 자리 부점코드의 체크디지트를 검증
=> 체크디지트와 변환된 4 자리의 부점코드를 결과로 제공하나 리턴상태코드로 검증결과를 확인한다.
3) '03' : 지로코드 체크디지트 산출
=> 체크디지트와 7 자리의 지로코드를 제공한다.
4) '04' : 지로코드 체크디지트 검증
=> 체크디지트와 7 자리의 지로코드를 결과로 제공하나 리턴상태코드로 검증결과를 확인한다.
5) '05' : 부점코드를 지로코드로 변환하고 체크디지트 검증
=> 체크디지트와 7 자리의 지로코드를 결과로 제공하나 리턴상태코드로 검증결과를 확인한다.
PoC
6) '06' : 직원번호 체크디지트 검증
=> 체크디지트와 7 자리의 직원번호를 결과로 제공하나 리턴상태코드로 검증결과를 확인한다.
7) '07' : 주민등록번호 체크디지트 검증
=> 체크디지트와 13 자리의 주민등록번호를 결과로 제공하나 리턴상태코드로 검증결과를 확인한다.
PoC
8) '08' : 사업자등록번호 체크디지트 검증
=> 체크디지트와 10 자리의 사업자등록번호를 결과로 제공하나 리턴상태코드로 검증결과를 확인한다.
9) '09' : 법인번호 체크디지트 검증
=> 체크디지트와 13 자리의 법인번호를 결과로 제공하나 리턴상태코드로 검증결과를 확인한다.
10) '10' : 외국인 등록번호 체크디지트 검증
=> 체크디지트와 13 자리의 외국인등록번호를 결과로 제공하나 리턴상태코드로 검증결과를 확인한다.

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcValidation extends
                com.kbstar.sqc.base.FunctionUnit

[입력항목]
    String inDstcd;                    ● 구분코드
    String inDigitTargetNo;            ● 디지트대상번호

[출력항목]
    String rtnStat;                    // 리턴상태코드
    String outChkDigit;                // 체크디지트
    String outDigitTargetNo;           // 디지트대상번호
```

PoC
PoC

RTNSTAT 설명

00 : 정상, 07 : Check Digit ERROR

사용 설명(예시)

PoC

```
// 공유메소드 호출 - 일반 체크디지트 검증(com.kbstar.kji.enbncmn, validationCheckDigit)

  IDataSet validationCheckDigitReq = new DataSet();

  validationCheckDigitReq.put("inDstcd", requestData.get("inDstcd"));
  validationCheckDigitReq.put("inDigitTargetNo", requestData.get("inDigitTargetNo"));

// @@ 공유메소드 호출 (일반 체크디지트 검증 validationCheckDigit)
  IDataSet validationCheckDigitRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
        "FUBcValidation.validationCheckDigit", validationCheckDigitReq, onlineCtx);
```

	// @@ 출력 데이터 셋 처리 String rtnstat = validationCheckDigitRes.getString("rtnStat"); String outchkdigit = validationCheckDigitRes.getString("outChkDigit"); String outdigittargetno = validationCheckDigitRes.getString("outDigitTargetNo");
비 고 PoC	PoC

1.62

PoC

PoC

PoC

PoC

관리번호 체크디지트 산출 및 검증(FUBcValidation.validationMgtNo())

프로그램 ID	FUBcValidation.validationMgtNo ()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	관리번호 체크디지트 산출 및 검증 결과를 제공 한다.
프로그램 규칙 및 제약사항 PoC	1.입력 데이터 검증 1) 구분코드 가) '1' : 체크디지트 산출 나) '2' : 체크디지트 검증 2) 산출유형구분코드 가) '001' : 신용장 일련번호 체크디지트 나) '002' : 내국신용장 일련번호 체크디지트 다) '003' : 기관약정 일련번호 체크디지트 라) '004' : 외화대출자금배정 일련번호 체크디지트 마) '005' : 통합트레이드번호 체크디지트 바) '006' : 부점통합트레이드번호 체크디지트 2. 구분코드별 체크디지트 산출 및 검증 규칙 => 검증시 입력데이터의 체크디지트와 산출체크디지트와 불일치시 리턴상태코드를 '07'로 출력한다 1) 산출유형구분코드 '001' : 신용장일련번호 체크디지트 산출은 아래와 같다. 가) 산출연산합 : 신용장 일련번호 1 번째자리 * 6 + 신용장 일련번호 2 번째자리 * 5 + 신용장 일련번호 3 번째자리 * 4 + 신용장 일련번호 4 번째자리 * 3 나) 나머지 = 산출연산합 / 11 로 나눈 나머지 다) 나머지가 0 이면 산출연산합 > 51 체크디지트 = 1 산출연산합 <= 51 체크디지트 = 0 라) 나머지가 0 이 아니면, 산출체크디지트 : 11 - 나머지 마) 신용장일련번호 5 번째 자리와 산출체크디지트 비교 일치여부를 확인한다. 2) 산출유형구분코드 '002 ': 내국신용장일련번호 체크디지트 산출은 아래와 같다. 가) 산출연산합 : 내국신용장 일련번호 1 번째자리 * 7 + 내국신용장 일련번호 2 번째자리 * 6 + 내국신용장 일련번호 3 번째자리 * 5 + 내국신용장 일련번호 4 번째자리 * 4 + 내국신용장 일련번호 5 번째자리 * 3 나) 나머지 = 산출연산합 / 11 로 나눈 나머지 다) 나머지가 0 이면 산출연산합 > 51 체크디지트 = 1 산출연산합 <= 51 체크디지트 = 0 라) 나머지가 0 이 아니면 산출체크디지트 : 11 - 나머지 마) 내국신용장일련번호 6 번째 자리와 산출:체크디지트 비교 일치여부를 확인한다. 3) 산출유형구분코드 '003 ': 기관약정번호일련번호 체크디지트 산출은 아래와 같다. 가) 산출연산합 : 기관약정 일련번호 1 번째자리 * 2 + 기관약정 일련번호 2 번째자리 * 3 + 기관약정 일련번호 3 번째자리 * 4 + 기관약정 일련번호 4 번째자리 * 5 + 기관약정 일련번호 5 번째자리 * 6 + 기관약정 일련번호 6 번째자리 * 7 나) 나머지 = 산출연산합 / 11 로 나눈 나머지 다) 나머지가 0 또는 10 이면 산출체크디지트 = 1 라) 나머지가 1 이면 산출체크디지트 = 0 마) 다), 라)가 아니면 산출체크디지트 : 11 - 나머지 바) 기관약정번호일련번호 7 번째 자리와 산출체크디지트 비교 일치여부를 확인한다.

PoC PoC PoC PoC

PoC

4) 산출유형구분코드 ‘ 004 ’ : 외화대출자금배정 일련번호 체크디지트 산출은 아래와 같다.
가) 산출연산합 : 외화대출자금배정 일련번호 1 번째자리 +
외화대출자금배정 일련번호 2 번째자리 +
외화대출자금배정 일련번호 3 번째자리 +
외화대출자금배정 일련번호 4 번째자리 +
외화대출자금배정 일련번호 5 번째자리 +
나) 산출체크디지트 = 산출연산합의 끝(일단위)자리 1 Byte
다) 외화대출자금배정 일련번호 6 번째 자리와 산출체크디지트 비교 일치여부를 확인한다.
5) 산출유형구분코드 ‘ 005 ’ : 통합트레이드번호 체크디지트 산출은 아래와 같다.
가) 산출연산합 : 통합트레이드번호 4 번째자리 * 7 +
통합트레이드번호 5 번째자리 * 6 +
통합트레이드번호 6 번째자리 * 5 +
통합트레이드번호 7 번째자리 * 4 +
통합트레이드번호 8 번째자리 * 3 +
통합트레이드번호 9 번째자리 * 7 +
통합트레이드번호 10 번째자리 * 6 +
통합트레이드번호 11 번째자리 * 5 +
통합트레이드번호 12 번째자리 * 4 +
통합트레이드번호 13 번째자리 * 3
나) 나머지 = 산출연산합 / 11 로 나눈 나머지
다) 나머지가 0 이면
산출연산합 > 51 체크디지트 = 1
산출연산합 <= 51 체크디지트 = 0
라) 나머지가 0 이 아니면,
산출체크디지트 : 11 - 나머지
마) 통합트레이드번호 14 번째 자리와 산출체크디지트 비교 일치여부를 확인한다.
6) 산출유형구분코드 ‘ 006 ’ : 부점통합트레이드번호 체크디지트 산출은 아래와 같다.
가) 산출연산합 : 부점통합트레이드번호 4 번째자리 * 7 +
부점통합트레이드번호 5 번째자리 * 6 +
부점통합트레이드번호 6 번째자리 * 5 +
부점통합트레이드번호 7 번째자리 * 4 +
부점통합트레이드번호 8 번째자리 * 3 +
부점통합트레이드번호 9 번째자리 * 7 +
부점통합트레이드번호 10 번째자리 * 6 +
부점통합트레이드번호 11 번째자리 * 5 +
부점통합트레이드번호 12 번째자리 * 4 +
부점통합트레이드번호 13 번째자리 * 3
나) 나머지 = 산출연산합 / 11 로 나눈 나머지
다) 나머지가 0 이면
산출연산합 > 51 체크디지트 = 1
산출연산합 <= 51 체크디지트 = 0
라) 나머지가 0 이 아니면,
산출체크디지트 : 11 - 나머지
마) 부점통합트레이드번호 14 번째 자리와 산출체크디지트 비교 일치여부를 확인한다.

입출력 설명

PoC

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcValidation extends
                com.kbstar.sqc.base.FunctionUnit

[입력항목]
    String inDstcd;                    // ● 구분코드
    String inCmptnPtrnDstcd;           // ● 산출유형구분코드
    String inDigitTargetNo;            // ● 디지트대상번호

[출력항목]
    String rtnStat;                    // 리턴상태코드
    String outCheckDigit;              // 체크디지트
    String outDigitTargetNo;           // 디지트대상번호
```

PoC

PoC

PoC

RTNSTAT 설명

00 : 정상, 07 : Check Digit ERROR

사용 설명(예시)

```
// 공유메소드 호출 - 관리번호 체크디지트 산출 및 검증
                                (com.kbstar.kji.enbncmn, validationMgtNo)

 IDataSet validationMgtNoReq = new DataSet();
```

PoC	validationMgtNoReq.put("inDstcd", requestData.get("inDstcd"));
	validationMgtNoReq.put("inCmptnPtrnDstcd", requestData.get("inCmptnPtrnDstcd"));
	validationMgtNoReq.put("inDigitTargetNo", requestData.get("inDigitTargetNo"));
	
	// @@ 공유메소드 호출 (관리번호 체크디지트 산출 및 검증 validationMgtNo)
	IDataSet validationMgtNoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcValidation.validationMgtNo", validationMgtNoReq, onlineCtx);
	// @@ 출력 데이터 셋 처리 PoC
	String rtnstat = validationMgtNoRes.getString("rtnStat");
	String outchkdigit = validationMgtNoRes.getString("outChkDigit");
	String outdigittargetno = validationMgtNoRes.getString("outDigitTargetNo");

비 고	

1.63

PoC

PoC

PoC

PoC

M/S 유효성 검증 (ISO 3 TYPE)(N/A)

프로그램 ID	N/A		언어 구분	N/A	
DTO 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	M/S 유효성 검증 (ISO 3 TYPE) 결과를 제공한다.
프로그램 규칙 및 제약사항	1. 입력값으로 들어오는 은행코드는 다음과 같다. 1) '004' : 구 국민 2) '006' : 구 주택 3) '029' : 구 동남 2. 유효기간은 필수 입력값 으로 한다. 3. 은행코드가 '004' (구 국민) 인 경우: 1) 보충데이터는 필수 입력값으로 한다. 2) 유효기간이 '0000' 값이 입력되면 오류처리 한다. 3) 유효기간이 '9999' 이고 보충데이터의 앞 5 자리가 '00000' 이면 오류처리한다. 4) 유효기간이 '9912' 이고 보충데이터의 앞 5 자리가 '09465' 또는 '09565' 또는 '09665' 이면 오류처리 한다. 5) 유효기간이 '9912' 이고 보충데이터의 앞 5 자리가 '58982' 또는 '90000' 또는 보충데이터 앞 7 자리와 주계좌번호의 5 번째자리부터의 7 자리데이터가 같으면 유효기간을 '9999' (유효기간 없음) 로 SET 하고 RETURN 값을 '07' 로 SET 한다. 단, 위의 조건을 만족하지 않는 경우에는 입력받은 유효기간을 유효기간으로 SET 하며 RETURN 코드를 '00' 정상으로 SET 한다. 4. 은행코드가 '006' (구 주택), '029' (구동남) 인 경우: 1) 카드 구분 영역은 필수 입력값으로 한다. 2) 카드 구분 값이 '1', '4', '5', '6', '9' 로 들어오면 유효기간을 입력받은 유효기간으로 SET 하고 RETURN 코드를 '00' 정상으로 SET 한다. 3) 카드구분에 위의 값과 다른 입력값이 들어오면 유효시간을 '9999' (유효기간 없음)로 SET 하고 RETURN 코드를 '07' 로 SET 한다.
비 고	

1.64

국가목록 조회(FUBcCode.getNationalCatalogue())

프로그램 ID	FUBcCode.getNationalCatalogue()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	국가목록 PoC	SQLMAP 명 PoC	NAME SPACE	fubccode
	영문명	THKJINA01		SQL ID	selectNationalCatalogue

프로그램 설명	국가목록를 국가코드, ISO 국가코드, ISO 국가번호코드, ISO 영문코드 등으로 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. PoC 1) '1' : 국가코드를 입력으로 하여 국가정보를 조회한다. 2) '2' : ISO 국가코드를 입력으로 하여 국가정보를 조회한다. 3) '3' : ISO 국가번호코드를 입력으로 하여 국가정보를 조회한다. 4) '4' : ISO 영문코드를 입력으로 하여 국가정보를 조회 한다. 5) '5' : 전체국가정보를 조회한다. 2. 구분코드별 입력항목을 아래와 같다. PoC 1) 구분코드가 '1'인 경우는 국가코드를 필수입력하여야 한다. 2) 구분코드가 '2'인 경우는 ISO 국가코드를 필수입력하여야 한다. 3) 구분코드가 '3'인 경우는 ISO 국가번호코드를 필수입력하여야 한다. 4) 구분코드가 '4'인 경우는 ISO 영문코드를 필수입력하여야 한다. 5) 구분코드가 '5'인 경우는 구분코드값만 입력한다. PoC

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcCode extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inDstcd;` ● 구분코드
	`String inNatnlCd;` ◎ 국가코드
	`String inIsoNatnlCd;` ◎ ISO 국가코드
	`String inIsoNatnlNoCd;` ◎ ISO 국가번호코드
	`String inIsoEngCd;` ◎ ISO 영문코드
	
	[출력항목] PoC
	`int outArayCnt;` // 배열개수
	`List LIST;` // 국가 List
	
	< LIST 항목 >
	`String outNatnlCd;` // 국가코드
	`String outIsoNatnlCd;` // ISO 국가코드
	`String outIsoNatnlNoCd;` // ISO 국가번호코드
	`String outIsoEngCd;` // ISO 영문코드
	`String outNatnlHanglName;` // 국가한글명
	`String outNatnlEngName;` // 국가영문명
	`String outCntntDstcd;` // 대륙구분
	`String outPrimNtinYn;` // 주요국여부
	`String outPrimUseCncycd;` // 주요사용통화코드

RTNSTAT 설명	N/A

PoC

사용 설명(예시)	// 공유메소드 호출 - 국가목록 조회 (com.kbstar.kji.enbncmn, getNationalCatalogue)
	
	IDataSet getNationalCatalogueReq = new DataSet();
	
	getNationalCatalogueReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getNationalCatalogueReq.put("inDstcd", requestData.get("inDstcd"));
	getNationalCatalogueReq.put("inNatnlCd", requestData.get("inNatnlCd"));
	getNationalCatalogueReq.put("inIsoNatnlCd", requestData.get("inIsoNatnlCd"));
	getNationalCatalogueReq.put("inIsoNatnlNoCd", requestData.get("inIsoNatnlNoCd"));
	getNationalCatalogueReq.put("inIsoEngCd", requestData.get("inIsoEngCd"));
	
	// @@ 공유메소드 호출 (국가목록 조회 getNationalCatalogue)
	IDataSet getNationalCatalogueRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getNationalCatalogue", getNationalCatalogueReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outaraycnt = getNationalCatalogueRes.getString("outArayCnt");
	IRecordSet getNationalCatalogueResRsList =
	getNationalCatalogueRes.getRecordSet("LIST");

비 고	

PoC
PoC
PoC
PoC
PoC

1.65

은행목록 조회(FUBcCode.getBankCatalogue())

프로그램 ID	FUBcCode.getBankCatalogue()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	은행목록	SQLMAP 명	NAME SPACE	fubccode
	영문명	THKJIBK01		SQL ID	selectBankCatalogue

PoC

프로그램 설명	은행목록테이블에 있는 은행명, 대표은행코드등을 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 특정 은행목록을 조회한다. 2) '2' : 전체 은행목록을 조회한다. 2. 구분코드별 입력항목을 아래와 같다. 1) 구분코드가 '1'인 경우는 그룹회사코드, 은행코드를 필수입력하여야 한다. 2) 구분코드가 '2'인 경우는 그룹회사코드를 필수입력하여야 한다

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcCode extends
                    com.kbstar.sqc.base.FunctionUnit

[입력항목]
     String inDstcd;                    ● 구분코드
     String inGroupCoCd;                ● 그룹회사코드
     String inBnkCd;                    ◎ 은행코드

[출력항목]
     int outArayCnt;                    // 배열개수
     List LIST;                         // 은행 List

    < LIST 항목 >                       // 은행코드
     String outBnkCd;                   // 은행명
     String outBnkName;                 // 은행약어명
     String outBnkAbrvnName;            // 대표은행코드
     String outRpsntBnkCd;              // 대표은행명
     String outRpsntBnkName;            // 포함은행내용
     String outInclsnBnkCtnt;           // 어음교환대상
     String outBlclrTagetBnkYn;                 은행여부
```

PoC

PoC

PoC

RTNSTAT 설명	N/A

사용 설명(예시)

```
// 공유메소드 호출 - 은행목록 조회 (com.kbstar.kji.enbncmn, getBankCatalogue)

  IDataSet getBankCatalogueReq = new DataSet();

  getBankCatalogueReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));//그룹회사코드
  getBankCatalogueReq.put("inDstcd", requestData.get("inDstcd")); // 구분코드
  getBankCatalogueReq.put("inBnkCd", requestData.get("inBnkCd")); // 은행코드

// @@ 공유메소드 호출 (은행목록 조회 getBankCatalogue)
  IDataSet getBankCatalogueRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
                        "FUBcCode.getBankCatalogue", getBankCatalogueReq, onlineCtx);
// @@ 출력 데이터 셋 처리
```

PoC

PoC

	String outaraycnt = getBankCatalogueRes.getString("outArayCnt"); // 조회건수
	IRecordSet getBankCatalogueResRsList = getBankCatalogueRes.getRecordSet("LIST");

비 고	

1.66

PoC

PoC

PoC

PoC

PoC

PoC

타행환은행공동코드 조회 (FUBcCode.getOtrbnkCatalogue())

프로그램 ID	FUBcCode.getOtrbnkCatalogue()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	은행공동코드목록	SQLMAP 명	NAME SPACE	fubccode
PoC	영문명	THKJIBK02	PoC	SQL ID	selectOtrbnkCatalogue

PoC

프로그램 설명	타행환은행공동코드목록 테이블에 있는 은행명, 부점명, 어음교환소코드 등을 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 특정 은행부점정보를 조회한다. 2. 구분코드별 입력항목을 아래와 같다. 1) 구분코드가 '1'인 경우는 그룹회사코드, 은행코드, 부점코드를 필수입력하여야 한다.

입출력 설명	**[입력항목]**
	String inDstcd; // ● 구분코드
	String inGroupCoCd; // ● 그룹회사코드
	String inBnkCd; // ● 은행코드
	String inBnncd; // ● 부점코드
	
	[출력항목]
	String outBnkCd; // 은행코드
	String outBrncd; // 부점코드
	String outStorClsrYn; // 점포폐쇄여부
	String outZip; // 우편번호
	String outBrnDtailAddr; // 부점상세주소
	String outBlclrhCd; // 어음교환소코드
	String outBnkName; // 은행명
	String outBrnName; // 부점명
	String outOlineEDpostNo; // 온라인실시 예금번호부
	String outBnkJntAreaDstcd; // 은행공동지역구분
	String outAplyYmd; // 적용년월일
	String outTbrnCd. // 자점코드
	String outTelno; // 전화번호
	String outFaxNo; // 팩스번호

RTNSTAT 설명	N/A
사용 설명(예시)	// 공유메소드 호출 - 타행은행공동코드 조회 (com.kbstar.kji.enbncmn, getOtrbnkCatalogue)

PoC

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcCode extends
                    com.kbstar.sqc.base.FunctionUnit
```

PoC

```
IDataSet getOtrbnkCatalogueReq = new DataSet();

getOtrbnkCatalogueReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
getOtrbnkCatalogueReq.put("inDstcd", requestData.get("inDstcd"));
getOtrbnkCatalogueReq.put("inBnkCd", requestData.get("inBnkCd"));
getOtrbnkCatalogueReq.put("inBrncd", requestData.get("inBrncd"));
```

	// @@ 공유메소드 호출 (타행은행공동코드 조회 getOtrbnkCatalogue)
	
	IDataSet getOtrbnkCatalogueRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getOtrbnkCatalogue", getOtrbnkCatalogueReq, onlineCtx);
	PoC
	
	// @@ 출력 데이터 셋 처리
	String outbnkcd = getOtrbnkCatalogueRes.getString("outBnkCd"); // 은행코드
	String outbrncd = getOtrbnkCatalogueRes.getString("outBrncd"); // 부점코드
	String outstorclsryn = getOtrbnkCatalogueRes.getString("outStorClsrYn");// 점포폐쇄여부
	String outzip = getOtrbnkCatalogueRes.getString("outZip"); // 우편번호
	String outbrndtailaddr = getOtrbnkCatalogueRes.getString("outBrnDtailAddr");// 부점주소
	String outblclrhcd = getOtrbnkCatalogueRes.getString("outBlclrhCd"); // 어음교환소코드
	String outbnkname = getOtrbnkCatalogueRes.getString("outBnkName"); // 은행명
	String outbrnname = getOtrbnkCatalogueRes.getString("outBrnName"); // 부점명
	PoC
	String outolineedpostno = getOtrbnkCatalogueRes.getString("outOlineEDpostNo"); // 온라인실시예금번호
	PoC
	String outbnkjntareadstcd = getOtrbnkCatalogueRes.getString("outBnkJntAreaDstcd"); // 은행공동지역구분
	String outaplyymd = getOtrbnkCatalogueRes.getString("outAplyYmd"); // 적용년월일
	String outtbrncd = getOtrbnkCatalogueRes.getString("outTbrnCd"); // 지점코드
	String outtelno = getOtrbnkCatalogueRes.getString("outTelno"); // 전화번호
	String outfaxno = getOtrbnkCatalogueRes.getString("outFaxNo"); // 팩스번호
	String outfxbrnmgtcd = getOtrbnkCatalogueRes.getString("outFXBrnMgtCd"); // 외환부점관리코드
	String outclsrbrnmgtcd = getOtrbnkCatalogueRes.getString("outClsrBrnMgtCd"); // 폐쇄부점관리코드
	String outemad = getOtrbnkCatalogueRes.getString("outEmad"); // 이메일주소
	String outaddrrepsntdstcd = getOtrbnkCatalogueRes.getString("outAddrRepsntDstcd"); // 주소표시구분

비 고	PoC

1.67

PoC PoC

에러발생시 센터컷 중단 여부, 중요 에러메시지 여부 체크 (FUBcMessage.getMsgCdOfCmnArea())

프로그램 ID	FUBcMessage. getMsgCdOfCmnArea()		언어 구분	JAVA(○)	
입출력 명	N/A		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	에러메시지목록	SQLMAP 명	NAME SPACE	fubcmessage
	영문명	THKJIER01		SQL ID	selectMsgCdOfCmnArea

프로그램 설명	COMMON AREA 의 에러메시지코드를 읽어서 센터컷중단 여부와 중요 에러메시지 여부를 표시한다.
프로그램 규칙 및 제약사항	1. COMMON AREA 의 에러메시지코드에 해당하는 코드를 에러메시지목록 테이블에서 읽어서 센터컷중단요구여부의 값을 표시한다. 테이블에는 센터컷중단구분코드 값이 아래와 같다. 1) '01' - Tran cancel 처리, 2) '02' - 에러발생시 즉시 ABEND 처리 및 후속조치 발행 3) '03' - TASK 별 일정 횟수 RETRY 처리 후 ABEND 처리 발행 -> 10 개의 에러메시지코드 중 하나라도 '02'가 있으면 '1'(중단요구)을 표시하고, 그렇지 않으면 '0'을 표시한다. 2. COMMON AREA 의 에러메시지코드에 해당하는 코드를 에러메시지목록 테이블에서 읽어서 중요 에러메시지 해당여부의 값을 COMMON AREA 의 에러중요메시지여부 속성에 표시한다. 3. 에러메시지코드 및 조치메시지 코드를 에러메시지목록 테이블에 있는지 체크하여 없을 경우에는 대표에러메시지코드('B8099999'-등록된 메시지 내용이 아닙니다)와 대표조치메세지코드('UKJI9999'-업무상담센터(IT 업무상담 Tel:4119)로 연락주세요)로 변경된다.
입출력 설명	N/A
RTNSTAT 설명	N/A
사용 설명(예시)	// 공유메소드 호출 - 에러발생시 센터컷 중단 여부, 중요 에러메시지 여부 체크 (com.kbstar.kji.enbncmn, getMsgCdOfCmnArea) IDataSet getMsgCdOfCmnAreaReq = new DataSet(); // @@ 공유메소드 호출 (센터컷중단여부, 중요에러메시지여부체크 getMsgCdOfCmnArea) *IDataSet getMsgCdOfCmnAreaRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcMessage.getMsgCdOfCmnArea", getMsgCdOfCmnAreaReq, onlineCtx);*
비 고	

1.68 거래 파라미터 유효성검증(FUBcValidation.validationTranParm())

프로그램 ID	FUBcValidation.validationTranParm()		언어 구분	JAVA(○)	
입출력 명	N/A		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	거래 발생시 Common Area 의 거래제어 파라미터를 검증하고 유효값이 아닌 경우 에러 처리를 하며, Common Area 무자원 대체입금 허용여부를 설정한다. (거래제어 방안 진행 중으로 추후 변경 가능)
프로그램 규칙 및 제약사항	1. 거래 통제여부 1) 거래 통제여부가 '1'(통제)인 경우에 해당 거래코드를 에러처리 한다. 2. 사고신고 미완료시 거래제한 여부 1) 사고신고 미완료시 거래제한 여부가 '1'(제한)인 경우 사고신고 완료여부를 체크하여 완료가 아닌 경우 에러처리 한다. 3. 무자원 대체입금 허용 구분코드 1) 무자원 대체입금 허용인 경우 Common Area 무자원 대체입금 허용여부를 설정한다. 4. 영업일 구분(0: 영업일, 1: 휴일)에 따라 각 부점의 구분코드별 거래가능여부를 체크하여 '0'(불가능)인 경우 에러처리 한다. 1) 영얼일 구분 가) 영업일 거래가능부점 내용 A) 본부부서 거래가능여부 '0'(불가능)인 경우에 영업일 거래가능 특정부점 참조여부가 '0'(불참조)이면 에러처리하고, 그렇지 않으면 거래가능 특정부점 테이블과 비교하여 있으면 거래를 허용한다. B) 지역본부, 지점, 출장소, 해외사무소, 해외영업점, 가족회사, 지원센터, PB 센터, 기업금융지점, 대출실행센터, 여신관리센터, 콜센터, 지역심사센터, 국민가크사본부부서, 업무지원센터, 카드지원반, 유동화점의 경우 가)-A)와 동일 기준으로 처리한다 나) 휴일 거래가능부점 내용 A) 본부부서 거래가능여부 '0'(불가능)인 경우에 휴일 거래가능 특정부점 참조여부가 '0'(불참조)이면 에러처리하고, 그렇지 않으면 휴일 거래가능 특정부점 테이블과 비교하여 있으면 거래를 허용한다. B) 지역본부, 지점, 출장소, 해외사무소, 해외영업점, 가족회사, 지원센터, PB 센터, 기업금융지점, 대출실행센터, 여신관리센터, 콜센터, 지역심사센터, 국민가크사본부부서, 업무지원센터, 카드지원반, 유동화점의 경우 나)-A)와 동일 기준으로 처리한다
입출력 설명	N/A
RTNSTAT 설명	N/A
사용 설명(예시)	// 공유메소드 호출 - 거래 파라미터 유효성검증 (com.kbstar.kji.enbncmn, validationTranParm) IDataSet validationTranParmReq = new DataSet(); // @@ 공유메소드 호출 (거래 파라미터 유효성검증 validationTranParm) IDataSet validationTranParmRes = callSharedMethodByDirect ("com.kbstar.kji.enbncmn", "FUBcValidation.validationTranParm", validationTranParmReq, onlineCtx);
비 고	

PoC

1.69 사용자 권한 및 Sales/OP 정보 조회(FUBcEmployee.getEmpidAthInfo())

프로그램 ID	FUBcEmployee.getEmpidAthInfo( )		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	직원기본	SQLMAP 명	NAME SPACE	fubcemployee
	영문명	THKJIHR01		SQL ID	selectEmpidAthInfo

프로그램 설명	입력된 사용자번호(직원번호)에 따라 SOD 영업점유형구분, SOD 사용자유형구분, Sales/OP 구분, 입력제한구분을 조회한다.
프로그램 규칙 및 제약사항	1. 출력항목 처리결과 가) ‘00’ : 정상적인 직원번호, 또는 직원번호 첫째 자리부터 5 자리가 ‘00000’인경우 나) ’07’ : 퇴직한 직원, 또는 유효하지 않은 직원 번호 인경우 2. 직원번호에 대한 권한정보 1) SOD 사용자유형구분코드

01	SOD 분리 온라인텔러 - OP	02	SOD 분리 BOM - OP	03	SOD 분리 BOO - OP
04	SOD 분리 제신고텔러 - OP	05	SOD 분리 제신고겸임 온라인텔러 - OP	06	SOD 분리 상판 OP 겸임 온라인텔러 - OP
07	SOD 분리 상판(수신)팀원 - SALES	08	SOD 분리 상판(수신+개인여신)팀원 - SALES	09	SOD 분리 상판 OP
10	SOD 분리 상판팀장 - SALES	11	SOD 분리 소호여신팀원 - SALES	12	SOD 분리 소호여신팀장 - SALES
13	SOD 분리 소호여신 OP	14	SOD 분리 VM - SALES	15	VIP OP(VIP TELLER)
16	영업관리자 - OP	17	지점장 - SALES	18	마스터권한 - SALES
19	PB (JR. PB) - SALES	20	PB PO - OP	21	PB 온라인 - OP
22	상판 제신고 팀원 - SALES	23	VM 대체 상판 팀원 - SALES	24	온라인텔러 (PT) - OP
25	제신고겸임 온라인텔러 (PT) - OP	26	JR. VM - SALES	27	VIP TELLER - OP
28	VIP TELLER(PT) - OP	29	VIP 창구 VM - SALES	30	환전텔러 - OP
31	환전자출납 - OP	32	기업금융팀장	33	기업금융팀원

2) SOD 영업점유형구분코드
가) 211 : 표준업무분리영업점(702 개). SOD 적용
나) 311 : 온라인창구 인원수(제신고텔러포함)가 3 명 이하인 영업점(217 개). 온라인에 제신고입출금허용
다) 411 : 지점장 및 청경을 제외한 총인원수가 8 명 이하인 지점 또는 온라인텔러가 2 명이하 또는 상판팀원의 합이 1 명 이하인 지점(102 개). SOD 제외
라) 611 : 독립형 PB 센터(12 개)
마) 711 : 전환형 PB 센터(3 개)
바) 811 : SOD 제외 PB 센터(3 개)
사) 999 : 긴급 SOD 미적용
3) SALES/OP 구분
가) 00 : SALES
나) 01 : OPERATOR
4) 입력제한구분
가) 0 : 입력가능

	나) 9 : 입력제한 5) 부점구분 : 부점기본 테이블의 부점구분코드로서 27 개의 부점구분이 있음. 3. 입력제한구분 처리 요건 1) 입력을 제한하는 경우 가) SOD 사용자유형구분(01,02,03,04,05,06,09,16,20,21,24,25,30,31)이면서 SOD 영업점유형구분(211,311,611,711,기타영업점유형구분)인 경우 나) SOD 사용자유형구분(07,08,10,11,12,13,14,15,17,18,19,22,23,26,29,32,33)이면서 SOD 영업점유형구분(기타영업점유형구분)인 경우 다) SOD 사용자유형구분(27,28)이면서 SOD 영업점유형구분(211,311,411,611,711,811,기타영업점유형구분)인경우 2) 입력을 허용하는 경우 가) SOD 사용자유형구분(01,02,03,04,05,06,09,16,20,21,24,25,30,31)이면서 SOD 영업점유형구분(411,811)인 경우 나) SOD 사용자유형구분(07,08,10,11,12,13,14,15,17,18,19,22,23,26,29,32,33)이면서 SOD 영업점유형구분(211,311,411,611,711,811)인 경우 4. 3.의 처리 요건에서 아래의 경우를 일시 허용한다. (2011.02.24) - 영업점유형이 (311,711)이고, 조작자권한타입이 (02,03,06,09,16,20,21,31)의 경우, 입력제한(9)에서 허용(0)으로 변경

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcEmployee extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd ; ● 그룹회사코드
	List inList ● 입력 List
	
	[출력항목]
	List LIST; // 출력 List
	
	< inList 항목 >
	String inEmpId; // 직원번호
	
	< LIST 항목 >
	String outPrcssRsult; // 처리결과
	String outSodBbrnPtrnDstcd; // SOD 영업점유형구분
	String outSodUserPtrnDstcd; // SOD 사용자유형구분
	String outEmpid; // 직원번호
	String outSlsOprtrDdstic; // SALES/OP 구분
	String outInptRsrctDstic; // 입력제한구분

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 사용자권한 및 Sales/OP 정보 조회
	(com.kbstar.kji.enbncmn, getEmpidAthInfo)
	
	IDataSet getEmpidAthInfoReq = new DataSet();
	
	getEmpidAthInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEmpidAthInfoReq.put("inList", requestData.getRecordSet("inList"));
	
	// @@ 공유메소드 호출 (사용자 권한 및 Sales/OP 정보 조회 getEmpidAthInfo)
	IDataSet getEmpidAthInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",

	"FUBcEmployee.getEmpidAthInfo", getEmpidAthInfoReq, onlineCtx); // @@ 출력 데이터 셋 처리 IRecordSet getEmpidAthInfoResRsList = getEmpidAthInfoRes.getRecordSet("LIST");
비 고	

PoC
PoC
PoC
PoC
PoC

1.70 고객지정계좌 채번 및 취소 (N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	고객지정계좌의 채번 및 취소에 따라 고객지정계좌 채번 테이블에 연결계좌번호, 사용여부 정보를 반영한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 고객지정계좌를 채번한다. 2) '2' : 기채번된 고객지정계좌를 취소한다. 단, 취소요청은 신규거래의 오조작 취소만을 허용한다. 2. 구분코드별 입력항목을 아래와 같다. 1) 구분코드가 '1'인 경우 그룹회사코드, 고객지정계좌, 연결계좌번호가 필수입력항목이다. 2) 구분코드가 '2'인 경우 그룹회사코드, 고객지정계좌가 필수입력항목이다. 3. 고객지정계좌가 관리계좌인 경우에 취급점과 테이블의 계좌신규부점코드와 비교하여 같은 경우에만 거래를 허용한다. 4. 연결계좌번호는 보통예금(01), 저축예금(02), 가계당좌(06), 당좌예금(18), 기업자유예금(37)만 허용한다.
비 고	

1.71 공통출력 데이터 조립(FUBcChange.setSlipShetCommon())

프로그램 ID	FUBcChange.setSlipShetCommon( )		언어 구분	JAVA(O)	
입출력 명	requestData/responseData		사용 구분	온라인(O), 배 치(O)	
			공통 유형	공통Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명

전표 및 장표의 공통출력을 위하여 공통출력항목을 인수받아 전행공통의 Common Area 에 Set 한다. 동 정보는 연동편집이 끝난 후 z-KESA 및 u-KESA 가 전행공통을 호출하면 출력편집이 수행된다.

프로그램 규칙 및 제약사항

1. 공통출력 데이터 조립항목
   1) 정보보안문구구분코드

보안 등급	보안 문구	분류 기준
A	" 본 자료는 고객의 실명정보가 표시된 금융자료이니 실명법에 의거 처리바람 "	고객고유 번호 등 고객정보 출력(계좌,고객정보 포함)
B	" 본 자료는 고객의 비밀보장 대상 자료이므로 본인 외 자료제공을 금함 "	고객번호(정보) 등을 제외한 계좌관련 정보 출력
C	" 본 자료는 당 행의 기밀사항이 포함된 자료이므로 무단으로 외부유출 및 제공을 금함 "	은행회계 정보 출력(계좌,고객정보 제외)
D	" 해당 사항 없음 "	자료 제공 기준을 해당업무 기준에 준함

   2) 거래종류한글명 : 업무별 해당거래 종류
      예) 신규, 상환, 상계, 전출, 해약, 강제집행, 재예치, 연동대출이자상환, 타행환 연동, 이자수납, 잡수익환급, 일반대출, 신탁대출, 부도, 보증료, 당첨금 등
   3) 업무고유인자한글명 : 업무별 추가인자 필요 항목-생략 가능하며 업무팀 자유 선택
      예) 현금, 대체, 무통, 유통, 현금/대체 등
2. 거래종류한글명 및 업무고유인자한글명은 각각 20 Bytes 를 초과할 수 없다.

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcChange extends
                com.kbstar.sqc.base.FunctionUnit

[입력항목]
    String inInfoSWrdsDstcd;
    String inTranKndHanglName;
    String inBzwkUtHanglName;
```

- ● 정보보안문구구분코드
- ● 거래종류한글명
- ◎ 업무고유인자한글명

RTNSTAT 설명

N/A

사용 설명(예시)

```
// 공유메소드 호출 - 공통출력 데이터 조립 (com.kbstar.kji.enbncmn, setSlipShetCommon)

IDataSet setSlipShetCommonReq = new DataSet();

setSlipShetCommonReq.put("inInfoSWrdsDstcd", requestData.get("inInfoSWrdsDstcd"));
setSlipShetCommonReq.put("inTranKndHanglName", requestData.get("inTranKndHanglName"));
setSlipShetCommonReq.put("inBzwkUtHanglName", requestData.get("inBzwkUtHanglName"));
```

	// @@ 공유메소드 호출 (공통출력 데이터 조립 setSlipShetCommon) *IDataSet setSlipShetCommonRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",* *"FUBcChange.setSlipShetCommon", setSlipShetCommonReq, onlineCtx);*
비 고	

PoC
PoC
PoC
PoC
PoC
PoC

1.72 거래제어 정보 조회(FUBcTransaction.getTranCtrlInfo())

프로그램 ID	FUBcTransaction.getTranCtrlInfo()		언어 구분	JAVA(O)	
입출력 명	requestData/responseData		사용 구분	온라인(O), 배 치(O)	
			공통 유형	공통Utility	
사용 TABLE 명	한글명	거래제어	SQLMAP 명	NAME SPACE	fubctransaction
	영문명	THKJITR01		SQL ID	selectTranCtrlInfo

프로그램 설명	거래제어코드 번호에 대한 거래제어정보를 조회한다.
프로그램 규칙 및 제약사항	1. 일자에 대한 거래제어 정보 조회 1) 년월일 입력 : 입력된 일자 당시의 거래제어 정보 2) 년월일 미입력 : 현재시점의 거래제어 정보 조회 - 현재시점 : Common Area 에 포함되어있는 KB 기준정보의 거래처리기준년월일 3) 적용시작일자 및 적용종료일자 기준으로 조회 된다 2. 거래제어코드번호는 z-KESA 와 u-KESA 환경에 따라 다른 기준으로 생성된다. 1) z-KESA 거래제어코드번호 : 거래코드 10 자리 2) u-KESA 거래제어코드번호 : 거래코드 10 자리 또는 PC Class 명(100 bytes 이하) 3. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일은 01 ~ 31 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다.

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcTransaction **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inTranCtrlCd; ● 거래제어코드
	String inTranCtrlDtalsNo; ● 거래제어세부번호
	String inYmd; ◎ 년월일
	
	[출력항목]
	String outAplyStartYmd; //적용시작년월일
	String outAplyEndYmd; //적용종료년월일
	String outtranCtrlCdName; //거래제어코드명
	String outCnclAbilYn; //취소가능여부
	String outCorrcAbilYn; //정정가능여부
	String outNonOpbrnAbilYn; //비개설부점가능여부
	String outLsdtAbilYn; //전일자가능여부
	String outIdyrekAbilYn; //기산일가능여부
	String outClsngAfAbilYn; //마감후가능여부
	String outGnldrOTranDstcd; //계리발생거래구분
	String outNbngRtAbilYn; //무자원대체가능여부
	String outAcidntNfshAbilYn; //사고미완료가능여부
	String outPlanSTranAbilYn; //계획정지거래가능여부
	String outHqDvsnAbilDstcd; //본부부서가능구분
	String outBzoprShAbilDstcd; //영업지원본부가능구분
	String outBrnAbilDstcd; //부점가능구분
	String outSubrnAbilDstcd; //출장소가능구분
	String outOvsesOAbilDstcd; //해외사무소가능구분
	String outOvsesBAbilDstcd; //해외영업점가능구분
	String outOvsesLcAbilDstcd; //해외현지법인가능구분
	String outSportCAbilDstcd; //지원센터가능구분
	String outPbCnterAbilDstcd; //PB 센터가능구분
	String outPbSubrnAbilDstcd; //PB 출장소가능구분
	String outLnEcAbilDstcd; //대출실행센터가능구분

```
String outLnbzMcAbilDstcd;          //여신관리센터가능구분
String outClcntrAbilDstcd;          //콜센터가능구분
String outAreaEcAbilDstcd;          //지역심사센터가능구분
String outCardHdAbilDstcd;          //카드본부부서가능구분
String outCardBAbilDstcd;           //카드영업점가능구분
String outBzwkScAbilDstcd;          //업무지원센터가능구분
String outCardSbAbilDstcd;          //카드지원반가능구분
String outScrtBrnAbilDstcd;         //유동화부점가능구분
String outCorpBrnAbilDstcd;         //기업부점가능구분
String outCorpSAbilDstcd;           //기업출장소가능구분
String outCnglBrnAbilDstcd;         //대기업부점가능구분
String outCnglSAbilDstcd;           //대기업출장소가능구분
String outPpsnCbAbilDstcd;          //개인복합부점가능구분
String outCorpCbAbilDstcd;          //기업복합부점가능구분
String outPbCBrnAbilDstcd;          //PB 복합부점가능구분
String outFmlyCoAbilDstcd;          //가족회사가능구분
String outSpltiB1AbilDstcd;         //특수부점 1 가능구분
String outSpltiB2AbilDstcd;         //특수부점 2 가능구분
String outSpltiB3AbilDstcd;         //특수부점 3 가능구분
String outSpltiB4AbilDstcd;         //특수부점 4 가능구분
String outSpltiB5AbilDstcd;         //특수부점 5 가능구분
String outaplyPlfmDstcd;            // 적용플랫폼구분
String outRspblPsnEmpid;            //담당자직원번호
String outRsempTelno;               //담당직원전화번호
```

RTNSTAT 설명

N/A

사용 설명(예시)

```
// 공유메소드 호출 - 거래제어정보 조회 (com.kbstar.kji.enbncmn, getTranCtrlInfo)

IDataSet getTranCtrlInfoReq = new DataSet();

getTranCtrlInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
getTranCtrlInfoReq.put("inTranCtrlCd", requestData.get("inTranCtrlCd"));
getTranCtrlInfoReq.put("inTranCtrlDtalsNo", requestData.get("inTranCtrlDtalsNo"));
getTranCtrlInfoReq.put("inYmd", requestData.get("inYmd"));

// @@ 공유메소드 호출 (거래제어정보 조회 getTranCtrlInfo)
IDataSet getTranCtrlInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
                "FUBcTransaction.getTranCtrlInfo", getTranCtrlInfoReq, onlineCtx);

// @@ 출력 데이터 셋 처리
String outaplystartymd = getTranCtrlInfoRes.getString("outAplyStartYmd");
String outaplyendymd = getTranCtrlInfoRes.getString("outAplyEndYmd");
String outtranctrlcdname = getTranCtrlInfoRes.getString("outtranCtrlCdName");
String outspvsrathordscnyn = getTranCtrlInfoRes.getString("outSpvsrAthorDscnYn");
String outspcfcbrnathexstyn = getTranCtrlInfoRes.getString("outSpcfcBrnAthExstYn
String outkycyn = getTranCtrlInfoRes.getString("outKycYn");
String outpcutpsngtagetyn = getTranCtrlInfoRes.getString("outPcutPsngTagetYn");
String outkycsupplyn = getTranCtrlInfoRes.getString("outKycSupplYn");
String outcnclabilyn = getTranCtrlInfoRes.getString("outCnclAbilYn");
String outcorrcabilyn = getTranCtrlInfoRes.getString("outCorrcAbilYn");
String outnonopbrnabilyn = getTranCtrlInfoRes.getString("outNonOpbrnAbilYn");
String outlsdtabilyn = getTranCtrlInfoRes.getString("outLsdtAbilYn");
String outidyrekabilyn = getTranCtrlInfoRes.getString("outIdyrekAbilYn");
```

```
String outclsngafabilyn = getTranCtrlInfoRes.getString("outClsngAfAbilYn");
String outgnldrotrandstcd = getTranCtrlInfoRes.getString("outGnldrOTranDstcd");
String outnbngraltrabilyn = getTranCtrlInfoRes.getString("outNbngRAltrAbilYn");
String outacidntnfshabilyn = getTranCtrlInfoRes.getString("outAcidntNfshAbilYn");
String outplanstranabilyn = getTranCtrlInfoRes.getString("outPlanSTranAbilYn");
String outhqdvsnabildstcd = getTranCtrlInfoRes.getString("outHqDvsnAbilDstcd
String outbzoprshabildstcd = getTranCtrlInfoRes.getString("outBzoprShAbilDstcd");
String outbrnabildstcd = getTranCtrlInfoRes.getString("outBrnAbilDstcd");
String outsubrnabildstcd = getTranCtrlInfoRes.getString("outSubrnAbilDstcd");
String outovsesoabildstcd = getTranCtrlInfoRes.getString("outOvsesOAbilDstcd");
String outovsesbabildstcd = getTranCtrlInfoRes.getString("outOvsesBAbilDstcd");
String outovseslcabildstcd = getTranCtrlInfoRes.getString("outOvsesLcAbilDstcd");
String outsportcabildstcd = getTranCtrlInfoRes.getString("outSportCAbilDstcd");
String outpbcnterabildstcd = getTranCtrlInfoRes.getString("outPbCnterAbilDstcd");
String outpbsubrnabildstcd = getTranCtrlInfoRes.getString("outPbSubrnAbilDstcd");
String outlnecabildstcd = getTranCtrlInfoRes.getString("outLnEcAbilDstcd");
String outlnbzmcabildstcd = getTranCtrlInfoRes.getString("outLnbzMcAbilDstcd");
String outclcntrabildstcd = getTranCtrlInfoRes.getString("outClcntrAbilDstcd");
String outareaecabildstcd = getTranCtrlInfoRes.getString("outAreaEcAbilDstcd");
String outcardhdabildstcd = getTranCtrlInfoRes.getString("outCardHdAbilDstcd");
String outcardbabildstcd = getTranCtrlInfoRes.getString("outCardBAbilDstcd");
String outbzwkscabildstcd = getTranCtrlInfoRes.getString("outBzwkScAbilDstcd
String outcardsbabildstcd = getTranCtrlInfoRes.getString("outCardSbAbilDstcd");
String outscrtbrnabildstcd = getTranCtrlInfoRes.getString("outScrtBrnAbilDstcd");
String outcorpbrnabildstcd = getTranCtrlInfoRes.getString("outCorpBrnAbilDstcd");
String outcorpsabildstcd = getTranCtrlInfoRes.getString("outCorpSAbilDstcd");
String outcnglbrnabildstcd = getTranCtrlInfoRes.getString("outCnglBrnAbilDstcd");
String outcnglsabildstcd = getTranCtrlInfoRes.getString("outCnglSAbilDstcd");
String outppsncbabildstcd = getTranCtrlInfoRes.getString("outPpsnCbAbilDstcd");
String outcorpcbabildstcd = getTranCtrlInfoRes.getString("outCorpCbAbilDstcd");
String outpbcbrnabildstcd = getTranCtrlInfoRes.getString("outPbCBrnAbilDstcd");
String outfmlycoabildstcd = getTranCtrlInfoRes.getString("outFmlyCoAbilDstcd");
String outspltib1abildstcd = getTranCtrlInfoRes.getString("outSpltiB1AbilDstcd");
String outspltib2abildstcd = getTranCtrlInfoRes.getString("outSpltiB2AbilDstcd");
String outspltib3abildstcd = getTranCtrlInfoRes.getString("outSpltiB3AbilDstcd");
String outspltib4abildstcd = getTranCtrlInfoRes.getString("outSpltiB4AbilDstcd");
String outspltib5abildstcd = getTranCtrlInfoRes.getString("outSpltiB5AbilDstcd");
String outaplyplfmdstcd = getTranCtrlInfoRes.getString("outaplyPlfmDstcd");
String outrspblpsnempid = getTranCtrlInfoRes.getString("outRspblPsnEmpid");
String outrsemptelno = getTranCtrlInfoRes.getString("outRsempTelno");
String outsodtranptrndstcd = getTranCtrlInfoRes.getString("outSODTranPtrnDstcd");
String outsodetagettranyn = getTranCtrlInfoRes.getString("outSODETagetTranYn");
```

비 고	

1.73 양편넣기-일수/월수/영업일수/공휴일수 양편넣기 산출 (FUBcDate.calculateNumberOfBsdeDays())

PoC

프로그램 ID	FUBcDate. calculateNumberOfBsdeDays()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	휴일 목록	SQLMAP 명	NAME SPACE	fubcdate
	영문명	THKJIH001		SQL ID	selectHoldyCnt

프로그램 설명	시작년월일과 종료년월일에 대한 일수, 영업일수, 공휴일수를 산출한다. 일수, 영업일수, 공휴일수는 양편넣기로 산출한다.
프로그램 규칙 및 제약사항	1. 구분코드는 아래와 같이 입력받는다. 1) '1' : 일수 산출 2) '2' : 일수/영업일수/공휴일수 산출 2. 1976 년 부터 2200 년 까지의 일자만 산출이 가능하다. 3. 일수, 영업일수, 공휴일수는 양편넣기(시작년월일 종료년월일 포함)를 한다. 4. 일수산출시 윤년을 포함한 일수를 산출함으로 상품 중 목돈마련상여금저축, 목돈마련월급여저축은 비즈니스 요건에 맞추어 일수에서 윤년일수를 차감하여 사용한다.

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcDate **extends** com.kbstar.sqc.base.FunctionUnit
	
	[입력항목] PoC
	String inGroupCoCd; ● 그룹회사코드
	String inDstcd; ● 구분코드
	String inStartYmd; ● 시작년월일
	String inEndYmd; ● 종료년월일
	PoC
	
	[출력항목] PoC
	String rtnStat; // 리턴상태코드
	int outNoday; PoC // 양편일수
	int outBzoprNoday; // 양편영업일수
	int outPbhdayNoday; // 양편공휴일수

RTNSTAT 설명	00 : 정상

사용 설명(예시)	**// 공유메소드 호출 - 양편넣기-일수/월수/영업일수/공휴일수 산출 (com.kbstar.kji.enbncmn, calculateNumberOfBsdeDays)**
	
	IDataSet calculateNumberOfBsdeDaysReq = **new** DataSet();
	
	calculateNumberOfBsdeDaysReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateNumberOfBsdeDaysReq.put("inDstcd", requestData.get("inDstcd"));
	calculateNumberOfBsdeDaysReq.put("inStartYmd", requestData.get("inStartYmd");

	calculateNumberOfBsdeDaysReq.put("inEndYmd", requestData.get("inEndYmd")); // @@ 공유메소드 호출 (양편넣기-일수/월수/영업일수/공휴일수 산출) IDataSet calculateNumberOfBsdeDaysRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDate.calculateNumberOfBsdeDays", calculateNumberOfBsdeDaysReq, onlineCtx); // @@ 출력 데이터 셋 처리 String outnoday = calculateNumberOfBsdeDaysRes.getString("outNoday"); String outbzoprnoday = calculateNumberOfBsdeDaysRes.getString("outBzoprNoday"); String outpbhdaynoday = calculateNumberOfBsdeDaysRes.getString("outPbhdayNoday"); String rtnstat = calculateNumberOfBsdeDaysRes.getString("rtnStat");
비 고	

PoC

PoC

1.74

PoC

PoC

PoC

PoC

양편넣기-입력월수 차감후의 일수산출(FUBcDate.calculateSubtractionNumber0fBsdeDays())

프로그램 ID	FUBcDate. calculateSubtractionNumber0fBsdeDays()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	시작년월일과 종료년월일에 대한 입력월수가 주어질 때, 입력월수를 차감 후 양편넣기를 적용 일수를 산출한다.
프로그램 규칙 및 제약사항	1. 구분코드 처리 '1' : 입력 월수만큼 차감후 산출한다. 예) 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 0 출력일수 : 91 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 1 출력일수 : 62 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 2 출력일수 : 31 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 3 출력일수 : 0 '2' : 입력 월수에 -1 을 한 월수만큼 산출한다. 예) 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 0 출력일수 : 오류 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 1 출력일수 : 91 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 2 출력일수 : 31 시작년월일 : '20080210', 종료년월일 : '20080510', 월수 : 3 출력일수 : 31 2. *일수가 1 일 이상이고 7 일 미만일 경우, 입력 월수를 차감하여 산출된 시작일을 휴일체크한다. 산출된 시작일이 영업일이 아니면 이후 도래하는 최초의 영업일을 산출(CIIDT03)하여 입력된 종료일과 비교한 후 산출된 영업일이 종료일과 같거나 큰 경우 출력일수는 '0'으로 처리한다. 단, 산출된 영업일이 종료일보다 작은 경우에는 입력월수를 차감하여 산출된 시작일과 입력된 종료일 사이의 일수를 RETURN 한다.* 예) 시작년월일 : '20081002', 종료년월일 : '20081006', 구분코드: '1', 월수 : 0 출력일수 : 5 시작년월일 : '20081002', 종료년월일 : '20081006', 구분코드: '1', 월수 : 1 출력일수 : 0 시작년월일 : '20081003', 종료년월일 : '20081006', 구분코드: '1', 월수 : 0 출력일수 : 0 시작년월일 : '20081003', 종료년월일 : '20081007', 구분코드: '1', 월수 : 0 출력일수 : 5 3. 입력월수는 음수를 입력할 수 없다. *4. 산출 시작년월일이 입력 종료일보다 큰 경우는 에러출력하지않고 일수를 '0'으로 처리한다. 처리 상태코는 '00' 정상으로 출력한다.*

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcDate extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inDstcd;` ● 구분코드
	`String inStartYmd;` ● 시작년월일

	String inEndYmd ; int inNomn ; [출력항목] int outNoday;	● 종료년월일 ● 월수 // 양편일수
RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 양편넣기-입력월수 차감후의 일수산출
	(com.kbstar.kji.enbncmn, calculateSubtractionNumberOfBsdeDays)
	PoC
	IDataSet calculateSubtractionNumberOfBsdeDaysReq = new DataSet();
	
	calculateSubtractionNumberOfBsdeDaysReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateSubtractionNumberOfBsdeDaysReq.put("inDstcd", requestData.get("inDstcd"));
	calculateSubtractionNumberOfBsdeDaysReq.put("inStartYmd", requestData.get("inStartYmd"));
	calculateSubtractionNumberOfBsdeDaysReq.put("inEndYmd", requestData.get("inEndYmd"));
	calculateSubtractionNumberOfBsdeDaysReq.put("inNomn", requestData.get("inNomn"));
	PoC
	// @@ 공유메소드 호출 (양편넣기-입력월수 차감후의 일수산출
	PoC
	IDataSet calculateSubtractionNumberOfBsdeDaysRes =
	callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateSubtractionNumberOfBsdeDays",
	calculateSubtractionNumberOfBsdeDaysReq, onlineCtx);
	// @@ 출력 데이터 셋 처리
	String outnoday = calculateSubtractionNumberOfBsdeDaysRes.getString("outNoday");	

비 고		

1.75

PoC

PoC

PoC

양편넣기-만기일/역만기일 산출(FUBcDate.calculateBsdeExpirDd())

프로그램 ID	FUBcDate.calculateBsdeExpirDd ()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	시작년월일과 일수/월수를 입력받아서 만기일과 역만기일을 산출한다. 단 구분코드 1, 3, 5, 6 은 양편넣기를 적용 만기일과 역만기일을 산출 한다.
프로그램 규칙 및 제약사항	1. 1976 년과 2200 의 사이의 일자만 산출이 가능하다 2. 구분코드는 아래와 같다. 1) '1' : 시작년월일과 일수를 입력하여 만기일을 산출한다. 만기일이 영업일이 아닌경우라도 산출한다. 예시] 시작년월일 : 2008-01-28, 일수 : 90 => 만기일 : 2008-04-27 2) '3' : 시작년월일과 일수를 입력하여 역만기일을 산출한다. 역만기일이 영업일이 아닌경우라도 산출한다. 예시] 시작년월일 : 2008-01-28, 일수 : 90 => 역만기일 : 2007-10-30 3) '5' : 시작년월일과 영업일수를 입력하여 만기일을 산출한다. 예시] 시작년월일 : 2008-01-28, 일수 : 90 => 만기일 : 2008-06-12 4) '6' : 시작년월일과 영업일수를 입력하여 역만기일을 산출한다. 예시] 시작년월일 : 2008-06-12, 일수 : 90 => 역만기일 : 2008-01-28 3. 윤년이 포함된 경우 윤년일수를 합산하여 만기일과 역만기일을 산출하였다. 4. '5', '6'에 대한 만기일/역만기일은 영업일수만을 계산하여 산출한 일자이다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcDate extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inDstcd;` ● 구분코드
	`String inYmd;` ● 년월일
	`int inNodayNomn;` ● 일수/월수
	
	[출력항목]
	`String outYmd;` // 출력년월일

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 양편넣기-만기일/역만기일 산출 PoC
	(com.kbstar.kji.enbncmn, calculateBsdeExpirDd)
	
	IDataSet calculateBsdeExpirDdReq = new DataSet();
	
	calculateBsdeExpirDdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateBsdeExpirDdReq.put("inDstcd", requestData.get("inDstcd"));
	calculateBsdeExpirDdReq.put("inYmd", requestData.get("inYmd"));
	calculateBsdeExpirDdReq.put("inNodayNomn", requestData.get("inNodayNomn"));
	
	// @@ 공유메소드 호출 (양편넣기-만기일/역만기일 산출 calculateBsdeExpirDd)
	IDataSet calculateBsdeExpirDdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateBsdeExpirDd", calculateBsdeExpirDdReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outymd = calculateBsdeExpirDdRes.getString("outYmd"); // 출력년월일

비 고	

PoC

PoC

PoC

PoC

PoC

1.76 이미지 인식 코드(ECC) 암호화 I (FUBcEcc.getEncodeStringCode1())

프로그램 ID	FUBcEcc.getEncodeStringCode()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	입력 문서번호 데이터를 암호화한 문자를 제공한다.
프로그램 규칙 및 제약사항	* 암호화된 문자 훼손시 복원에 필요한 **ECC 코드를 제외하여 암호화**하는 모듈이며, 해당 모듈을 사용하여 암호화된 문자를 복원할 시에는 **BCEcc.getDecodingCode1()**을 호출하여 사용한다.

※ 업무팀은 본 유틸리티를 사용하지 말고 " 이미지 인식 코드(ECC) 암호화 II (FUBcEcc.getEncodeStringCode2()) " 유틸리티를 사용한다.

PoC

1. 입력가능한 문자는 아래와 같다.
   - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, -, ' '(SPACE)

2. 인식구분코드별 입력 값과 자릿수가 결정되며 반드시 입력데이터를 조립하여 호출하여야 한다.

번호	인식구분	인식구분명	입력 값
1	AA	기본인식정보	부점코드(4 자리) + 거래일시(14 자리) + 직원번호(7 자리)
2	BA	전표인식정보	부점코드(4 자리) + 거래일자(8 자리)+ 직원번호(7 자리) + 전표번호(5 자리)
3	CA	카드인식정보	카드번호(16 자리) + 고객번호(13 자리)
4	CB	가맹점인식정보	가맹점번호(15 자리) + SPACE(1) + 사업자등록번호(10 자리)
5	EA	장표인식정보	거래일시(14 자리)+직원번호(7 자리)+식별자(최대 20 자리)

PoC

※ 인식구분 AB, AC, DA,DC 삭제됨.(2010.09.30)

3. 장표서식에 출력되는 ECC 는 최대 2 개의 인식구분코드를 사용하여 ECC 출력데이터 1 에 출력되며, 장표서식별 인식구분자는 " 6. 장표서식별 인식구분 내용 " 을 참조한다.

4. ECC 출력데이터 2 의 내용은 ECC 원문에 해당하는 입력데이터를 출력하며 항목간의 구분을 BLANK 로 한다.

5. 임의장표 또는 지정되지 않은 장표서식구분은 입력인식구분에 의하여 판단하므로 1 개 또는 2 개의 인식구분코드를 입력한다.

PoC

PoC

PoC

6. 장표서식별 인식구분 내용

번호	출력양식명	인식구분 내용	식별자 종류
1	가맹금예치금예치신청서	EA	계좌번호
2	KB 가맹금예치제도이용신청서	EA	계좌번호
3	대여금고차용신청서(신규갱신)	EA	계좌번호
4	대여금고열쇠분실(손상)신고및재교부의뢰서	EA	계좌번호
5	대리인(추가해지변경)신청서	EA	계좌번호
6	대여금고개문신청서	EA	계좌번호

	7	대여금고차용약정해지서	EA	계좌번호
	8	보호예수의뢰서	EA	계좌번호
	9	예금잔액조회장(잔액통보거절신청서)	EA	계좌번호
	10	사고신고서(수표,어음)	EA	주민(사업자)번호
	11	적립식저축연단위이자지급(지급해지)신청서	EA	계좌번호
	12	제신고서	EA	계좌번호
	13	신규거래신청서	EA	계좌번호
	14	자동이체신청서	EA	계좌번호
	15	명의변경신청서	EA	계좌번호
	16	제증명의뢰서	EA	계좌번호
	17	관리점처리의뢰서	EA	계좌번호
	18	KB 골드투자통장거래신청서	EA	계좌번호
	19	잔액증명정기발급신청서	EA	계좌번호
	20	비거주자판정기준표(수신)	EA	계좌번호
	21	금융결제원 CMS 이용계약서학교 CMS	EA	계좌번호
	22	금융결제원 CMS 이용계약서	EA	계좌번호
	23	칩없는(인증서)방식 KB 모바일뱅킹서비스등록신청서	EA	주민(사업자)번호
	24	계좌개설대행신청서(제휴카드신청서겸용)	EA	계좌번호
	25	인터넷뱅킹전화승인서비스신청서	EA	주민(사업자)번호
	26	전자금융거래제한계좌등록해제신청서	EA	주민(사업자)번호
	27	전자금융추가이용신청서(KBsERP 다단계결재신청용)	EA	주민(사업자)번호
	28	전자금융거래신청서(출금계좌신규겸용)	EA	주민(사업자)번호
	29	입출금내역통지서비스신청서	EA	계좌번호
	30	기업프리미엄뱅킹서비스신청서(자금관리서비스신청서겸용)	EA	주민(사업자)번호
	31	CyberCFO 이용신청서	EA	주민(사업자)등록번호
	32	주류구매전용카드사용금액확인서	EA	주민(사업자)등록번호
	33	주류구매카드	EA	주민(사업자)등록번호
	34	KBWISE 폰뱅킹등록신청서	EA	주민(사업자)번호
	35	인터넷예부적금담보대출사전등록신청서(해지겸용)	EA	주민(사업자)번호
	36	이자통지서비스신청서	EA	계좌번호
	37	금리옵션거래약정서(금리상하한옵션연계대출용)	EA	계좌번호
	38	금융거래확인서	EA	주민(사업자)번호
	39	국내간외화자금이체신청서	EA	계좌번호
	40	국민웨스턴유니온특급송금신청서	EA	계좌번호
	41	외화송금신청서	EA	계좌번호
	42	KBWISE 해외송금신청서	EA	계좌번호
	43	월드링크수표발행신청서	EA	계좌번호
	44	KB 적립식외화정기예금자동이체등록(해지)확	EA	계좌번호

		인서		
	45	외화예금거래신청서	EA	계좌번호
	46	캥거루가족사랑외화예금자동이체등록(해지)확인서	EA	계좌번호
	47	MYQ(마이큐)카드발급신청서	EA	계좌번호
	48	회원제신고서(개인기업카드공통)	EA	카드번호
	49	직불카드가맹점가입신청서	EA	계좌번호
	50	제 3 자명의계좌등록요청서	EA	가맹점번호
	51	가맹점제신고서	EA	가맹점번호
	52	직불카드가맹점제신고서	EA	계좌번호
	53	은행신용카드가맹점제신고서	EA	가맹점번호
	54	본인확인생략(NoCVM)가맹점특약서	EA	가맹점번호
	55	후취정산특약서	EA	가맹점번호
	56	월간매입한도조정신청서	EA	가맹점번호
	57	바우처가맹점및신용판매대금자동청구(EDC)신청동의서	EA	가맹점번호
	58	매출전표(원본사본)신청서	EA	매출전표요청총괄내역참조번호
	59	불비매출전표추심의뢰서	EA	가맹점번호
	60	카드수령증	EA	카드번호
	61	(개인)연금저축금융기관별저축한도등록신청서	EA	계좌번호
	62	보험가입및신용정보제공(이용)동의서	EA	주민(사업자)번호
	63	개인연금저축(연금저축)불입초과금액정리명세서	EA	계좌번호
	64	연금(지급변경)신청서	EA	계좌번호
	65	(개인연금신탁연금신탁)계약(이전취소)신청서	EA	계좌번호
	66	(개인연금신탁연금신탁)계약이전(예정취소)통보서	EA	계좌번호
	67	(개인연금신탁연금신탁)송금내역통보서	EA	접수번호
	68	수익자변경신청서	EA	계좌번호
	69	신탁(기간종별)변경신청서	EA	계좌번호
	70	신탁양도승낙신청서	EA	계좌번호
	71	[]신탁(해지일부출금)(신청취소신청)서	EA	신탁계좌번호
	72	KB 유언신탁신청서	EA	주민사업자번호
	73	KB 유언신탁제변경신고서	EA	계좌번호
	74	퇴직신탁계약해지(이전)동의서	EA	계좌번호
	75	중간정산전원 · 전액지급지시서	EA	계좌번호
	76	퇴직신탁중간정산일부지급지시서	EA	계좌번호
	77	퇴직신탁퇴직급부금지급지시서	EA	계좌번호
	78	퇴직신탁계약내용변경신청서	EA	계좌번호
	79	퇴직연금제도실시를위한퇴직신탁계약해지신청서	EA	계좌번호
	80	퇴직신탁계약해지신청및지급지시서	EA	계좌번호
	81	퇴직신탁계약이전신청서	EA	계좌번호
	82	퇴직신탁가입자명부확인서	EA	계좌번호
	83	자사주특정금전신탁중도해지의뢰서	EA	계좌번호

	84	특정금전신탁연결계좌등록신청서	EA	계좌번호
	85	특정금전신탁중도해지의뢰서	EA	계좌번호
	86	특정금전신탁계약추가약정서	EA	계좌번호
	87	국민주택채권사고신고및취하서	EA	채권번호
	88	집합투자증권추가매입및환매예약신청서	EA	계좌번호
	89	판매회사 이동을 위한 계좌 확인서 발급 신청서	EA	계좌번호
	90	연금저축펀드전환예약계약변경신청서	EA	계좌번호
	91	집합투자증권신규가입신청서(역외펀드겸용)	EA	계좌번호
	92	KB Smart 플랜적립이체신청서	EA	계좌번호
	93	보험계약청약서(2-1)	EA	계좌번호
	94	고객정보변경신청서(방카슈랑스)	EA	계좌번호
	95	자동이체신청서(방카슈랑스)	EA	계좌번호
	96	품질보증신청서	EA	계좌번호
	97	지급신청서(방카슈랑스)	EA	계좌번호
	98	KBTEZECard 가입신청서	AA+CA	
	99	하나투어마일리지클럽회원가입신청서	AA+CA	
	100	SOILKB 카드회원가입신청서	AA+CA	
PoC	101	SK 엔크린보너스카드회원가입신청서	AA+CA	
	102	신용카드회원가입신청서(창구용)(신규추가교체)	AA+CA	
	103	KB 카드회원가입신청서(창구용)	AA+CA	
	104	KB 카드회원가입신청서(결제계좌신규겸용)	AA+CA	
	105	KB 카드회원가입신청서(신용체크겸용)	AA+CA	
	106	(모집인용)SK 스마트 KB 카드회원가입신청서	AA+CA	
	107	(모집인용)플래티늄회원가입신청서	AA+CA	
	108	(모집인용)KB 마이원회원가입신청서	AA+CA	
	109	KB 소형화물차유류세환급카드가입신청서	AA+CA	
	110	GS 칼텍스보너스(GS/POINT)카드회원가입신청서	AA+CA	
	111	해피포인트카드회원가입신청서	AA+CA	
	112	KB 고운맘카드(체크카드포함)가입신청서	AA+CA	
	113	대한항공스카이패스(아시아나클럽)멤버십가입신청서	AA+CA	
	114	SOIL 보너스카드회원가입신청서	AA+CA	
	115	d 카드회원가입신청서	AA+CA	
	116	KBCheckCardApplication	AA+CA	
	117	KBPlatinum 카드가입신청서	AA+CA	
	118	GS 칼텍스스마트세이브회원가입신청서	AA+CA	
	119	GS 칼텍스보너스카드회원가입신청서	AA+CA	
	120	KBmyOneCard 회원가입신청서	AA+CA	
	121	myOnecard 서비스멤버십가입신청서	AA+CA	
	122	SK 스마트 KB 카드회원가입신청서	AA+CA	
	123	(모집인용)SK 멤버십신청서	AA+CA	

	124	SOILLPGKB 카드회원가입신청서	AA+CA	
	125	SOIL 보너스카드회원가입신청서	AA+CA	
	126	KBCheck 카드발급신청서	AA+CA	
	127	Myone(CGV)멤버십가입신청서	AA+CA	
	128	CGV 멤버십가입신청서	AA+CA	
	129	KBCGVMANIACard 회원가입신청서	AA+CA	
	130	GS 칼텍스스마트세이브 KB 카드회원가입신청서	AA+CA	
	131	GS 칼텍스스마트세이브 KB 카드회원가입신청서	AA+CA	
	132	KBROVLCard 가입신청서	AA+CA	
	133	(모집인용)하나투어멤버십신청서	AA+CA	
	134	()대학교학생증신분증겸용카드(신규재)발급신청서	AA+CA	
	135	하나투어 KB 카드회원가입신청서	AA+CA	
	136	(모집인용)신용카드회원가입신청서	AA+CA	
	137	대한항공스카이패스멤버십가입신청서	AA+CA	
	138	KB 기업 ROVLCard 회원가입신청서패키지	AA+CA	
	139	KB 기업 TEZECard 회원가입신청서패키지	AA+CA	
	140	KBmybiz 체크카드가입신청서	AA+CA	
	141	기업카드회원가입신청서	AA+CA	
	142	KB 기업 TEZECard 회원가입신청서	AA+CA	
	143	KB 기업 ROVLCard 회원가입신청서	AA+CA	
	144	KBmybiz 카드가입신청서	AA+CA	
	145	가맹점가입신청서	AA+CB	
	146	KB 가맹점가입동의서	AA+CB	
	147	공동대표자가맹점가입동의서	AA+CB	
	148	가맹점거래승인한도조정(해제)신청서	AA+CB	
	149	신용카드가맹점특약서(할부특약)	AA+CB	
	150	전자상거래결제대행업무특약서(전자상거래오픈마켓용)	AA+CB	
	151	가맹점특약서(MT 처리가맹점용)	AA+CB	
	152	전자서명에의한신용판매특약서	AA+CB	
	153	EDC 특약가맹점확약서	AA+CB	
	154	전자상거래결제대행업무특약서	AA+CB	
	155	신용판매내역전송결제특약서	AA+CB	

156	예적금질권설정계약서(예적금용)	AA+CB	
157	신용판매내역무전표 EDI 서비스특약서	AA+CB	
158	국민은행신용판매대금자동청구(EDC)신청서	AA+CB	
159	신용카드가맹점특약서(수기매출전표(인터넷전자상거래포함)특약)	AA+CB	
160	공용전표사용특약서	AA+CB	
161	KB Wise Members 카드회원가입신청서	AA+CA	
162	KB 비씨대한민국카드회원가입신청서	AA+CA	
163	한국자원봉사센터협회 KB Remember 카드가입신청서	AA+CA	
164	KB 내고장사랑카드 회원가입신청서	AA+CA	
165	에버리치 KB 고운맘카드(체크카드포함)가입신청서	AA+CA	
166	edu&d 카드 회원가입 신청서	AA+CA	
167	MyOne 서비스 멤버십 가입 신청서	AA+CA	
168	GS & POINTKB 카드 회원가입신청서	AA+CA	
169	KB STARTLUCK 카드 회원가입신청서	AA+CA	
170	GS & POINT 회원가입신청서	AA+CA	
998	임의장표	출력 용도에 맞게 인식정보 사용 (인식구분정보 1 개 또는 2 개 사용)	
999	전표	BA-전표인식정보	

PoC

PoC

PoC

PoC

PoC

입출력 설명

PoC

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcEcc extends
                    com.kbstar.sqc.base.FunctionUnit
```

[입력항목]

```
String inRecogDstic1;
String inRecogDstic2;
String inBrncd;
String inTranYms;
String inEmpid;
String inCno;
String inBzno;
```

PoC

- ● 입력인식구분 1
- ◎ 입력인식구분 2
- ◎ 부점코드
- ◎ 거래일시
- ◎ 직원번호
- ◎ 고객번호
- ◎ 사업자등록번호

	String inAcno;
	String inSlipNo;
	String inCardNo;
	String inMbrmchNo;
	String inBndNo;
	String inVaultNo;
	String inTranDte;
	String inIdnfr;
	
	[출력항목]
	String outEccOutptData1;
	String outEccOutptData2;
		◎ 계좌번호
		◎ 전표번호
		◎ 카드번호
		◎ 가맹점번호
		◎ 채권번호
		◎ 금고번호
		◎ 거래일자
		◎ 식별자
		
		//ECC 출력데이터 1
		//ECC 출력데이터 2

RTNSTAT 설명	N/A	

사용 설명(예시)	**// 공유메소드 호출 - 이미지 인식 코드(ECC) 암호화 I (com.kbstar.kji.enbncmn, getEncodeStringCode1)**
	
	IDataSet getEncodeStringCode1Req = new DataSet();
	
	getEncodeStringCode1Req.put("inRecogDstic1", requestData.get("inRecogDstic1"));
	getEncodeStringCode1Req.put("inRecogDstic2", requestData.get("inRecogDstic2"));
	getEncodeStringCode1Req.put("inBrncd", requestData.get("inBrncd")); // 부점코드
	getEncodeStringCode1Req.put("inTranYms", requestData.get("inTranYms")); // 거래일시
	getEncodeStringCode1Req.put("inEmpid", requestData.get("inEmpid")); // 직원번호
	getEncodeStringCode1Req.put("inCno", requestData.get("inCno")); // 고객번호
	getEncodeStringCode1Req.put("inBzno", requestData.get("inBzno")); // 사업자등록번호
	getEncodeStringCode1Req.put("inAcno", requestData.get("inAcno")); // 계좌번호
	getEncodeStringCode1Req.put("inSlipNo", requestData.get("inSlipNo")); // 전표번호
	getEncodeStringCode1Req.put("inCardNo", requestData.get("inCardNo")); // 카드번호
	getEncodeStringCode1Req.put("inMbrmchNo", requestData.get("inMbrmchNo")); // 가맹점
	getEncodeStringCode1Req.put("inBndNo", requestData.get("inBndNo")); // 채권번호
	getEncodeStringCode1Req.put("inVaultNo", requestData.get("inVaultNo")); // 금고번호
	getEncodeStringCode1Req.put("inTranDte", requestData.get("inTranDte")); // 거래일자
	getEncodeStringCode1Req.put("inIdnfr", requestData.get("inIdnfr")); // 식별자
	
	// @@ 공유메소드 호출 (이미지 인식 코드(ECC) 암호화 I getEncodeStringCode1)
	IDataSet getEncodeStringCode1Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcEcc.getEncodeStringCode1", getEncodeStringCode1Req, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outeccoutptdata1 = getEncodeStringCode1Res.getString("outEccOutptData1");
	String outeccoutptdata2 = getEncodeStringCode1Res.getString("outEccOutptData2");	

비 고		

1.77 이미지 인식 코드(ECC) 암호화 II (FUBcEcc.getEncodeStringCode2())

프로그램 ID	FUBcEcc.getEncodeStringCode()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력 문서번호 데이터를 암호화한 문자를 제공한다.
프로그램 규칙 및 제약사항	* 암호화된 문자 훼손시 복원에 필요한 ECC 코드를 포함하여 암호화하는 모듈이며, 해당 모듈을 사용하여 암호화된 문자를 복원할 시에는 BCEcc.getDecodingCode2() 를 호출하여 사용한다.

1. 입력가능한 문자는 아래와 같다.
   - 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S, T, U, V, W, X, Y, Z, -, ‘ ‘(SPACE)

2. 인식구분코드별 입력 값과 자릿수가 결정되며 반드시 입력데이터를 조립하여 호출하여야 한다.

번호	인식구분	인식구분명	입력 값
1	AA	기본인식정보	부점코드(4 자리) + 거래일시(14 자리) + 직원번호(7 자리)
2	BA	전표인식정보	부점코드(4 자리) + 거래일자(8 자리)+ 직원번호(7 자리) + 전표번호(5 자리)
3	CA	카드인식정보	카드번호(16 자리) + 고객번호(13 자리)
4	CB	가맹점인식정보	가맹점번호(15 자리) + SPACE(1) + 사업자등록번호(10 자리)
5	EA	장표인식정보	거래일시(14 자리)+직원번호(7 자리)+식별자(최대 20 자리)

※ 인식구분 AB, AC, DA,DC 삭제됨.(2010.09.30)

3. 장표서식에 출력되는 ECC 는 최대 2 개의 인식구분코드를 사용하여 ECC 출력데이터 1 에 출력되며, 장표서식별 인식구분자는 “6. 장표서식별 인식구분 내용”을 참조한다.

4. ECC 출력데이터 2 의 내용은 ECC 원문에 해당하는 입력데이터를 출력하며 항목간의 구분을 BLANK 로 한다.

5. 임의장표 또는 지정되지 않은 장표서식구분은 입력인식구분에 의하여 판단하므로 1 개 또는 2 개의 인식구분코드를 입력한다.

6. 장표서식별 인식구분 내용

번호	출력양식명	인식구분 내용	식별자 종류
1	가맹금예치금예치신청서	EA	계좌번호
2	KB 가맹금예치제도이용신청서	EA	계좌번호
3	대여금고차용신청서(신규갱신)	EA	계좌번호
4	대여금고열쇠분실(손상)신고및재교부의뢰서	EA	계좌번호
5	대리인(추가해지변경)신청서	EA	계좌번호
6	대여금고개문신청서	EA	계좌번호
7	대여금고차용약정해지서	EA	계좌번호
8	보호예수의뢰서	EA	계좌번호

	9	예금잔액조회장(잔액통보거절신청서)	EA	계좌번호
	10	사고신고서(수표,어음)	EA	주민(사업자)번호
	11	적립식저축연단위이자지급(지급해지)신청서	EA	계좌번호
	12	제신고서	EA	계좌번호
	13	신규거래신청서	EA	계좌번호
	14	자동이체신청서	EA	계좌번호
PoC	15	명의변경신청서	EA	계좌번호
	16	제증명의뢰서	EA	계좌번호
	17	관리점처리의뢰서	EA	계좌번호
	18	KB 골드투자통장거래신청서	EA	계좌번호
	19	잔액증명정기발급신청서	EA	계좌번호
	20	비거주자판정기준표(수신)	EA	계좌번호
	21	금융결제원 CMS 이용계약서학교 CMS	EA	계좌번호
PoC	22	금융결제원 CMS 이용계약서	EA	계좌번호
	23	칩없는(인증서)방식 KB 모바일뱅킹서비스등록신청서 PoC	EA	주민(사업자)번호
	24	계좌개설대행신청서(제휴카드신청서겸용)	EA	계좌번호
	25	인터넷뱅킹전화승인서비스신청서	EA	주민(사업자)번호
	26	전자금융거래제한계좌등록해제신청서	EA	주민(사업자)번호
	27	전자금융추가이용신청서(KBsERP 다단계결재신청용)	EA	주민(사업자)번호
	28	전자금융거래신청서(출금계좌신규겸용)	EA	주민(사업자)번호
	29	입출금내역통지서비스신청서	EA	계좌번호
	30	기업프리미엄뱅킹서비스신청서(자금관리서비스신청서겸용)	EA	주민(사업자)번호
	31	CyberCFO 이용신청서	EA	주민(사업자)등록번호
	32	주류구매전용카드사용금액확인서	EA	주민(사업자)등록번호
	33	주류구매카드 PoC	EA	주민(사업자)등록번호
	34	KBWISE 폰뱅킹등록신청서	EA	주민(사업자)번호
	35	인터넷예부적금담보대출사전등록신청서(해지겸용)	EA	주민(사업자)번호
	36	이자통지서비스신청서	EA	계좌번호
	37	금리옵션거래약정서(금리상하한옵션연계대출용)	EA	계좌번호
	38	금융거래확인서 PoC	EA	주민(사업자)번호
	39	국내간외화자금이체신청서	EA	계좌번호
	40	국민웨스턴유니온특급송금신청서	EA	계좌번호
	41	외화송금신청서	EA	계좌번호
	42	KBWISE 해외송금신청서 PoC	EA	계좌번호
	43	월드링크수표발행신청서	EA	계좌번호
	44	KB 적립식외화정기예금자동이체등록(해지)확인서	EA	계좌번호
	45	외화예금거래신청서	EA	계좌번호

46	캥거루가족사랑외화예금자동이체등록(해지)확인서	EA	계좌번호
47	MYQ(마이큐)카드발급신청서	EA	계좌번호
48	회원제신고서(개인기업카드공통)	EA	카드번호
49	직불카드가맹점가입신청서	EA	계좌번호
50	제 3 자명의계좌등록요청서	EA	가맹점번호
51	가맹점제신고서	EA	가맹점번호
52	직불카드가맹점제신고서	EA	계좌번호
53	은행신용카드가맹점제신고서	EA	가맹점번호
54	본인확인생략(NoCVM)가맹점특약서	EA	가맹점번호
55	후취정산특약서	EA	가맹점번호
56	월간매입한도조정신청서	EA	가맹점번호
57	바우처가맹점및신용판매대금자동청구(EDC)신청동의서	EA	가맹점번호
58	매출전표(원본사본)신청서	EA	매출전표요청총괄내역참조번호
59	불비매출전표추심의뢰서	EA	가맹점번호
60	카드수령증	EA	카드번호
61	(개인)연금저축금융기관별저축한도등록신청서	EA	계좌번호
62	보험가입및신용정보제공(이용)동의서	EA	주민(사업자)번호
63	개인연금저축(연금저축)불입초과금액정리명세서	EA	계좌번호
64	연금(지급변경)신청서	EA	계좌번호
65	(개인연금신탁연금신탁)계약(이전취소)신청서	EA	계좌번호
66	(개인연금신탁연금신탁)계약이전(예정취소)통보서	EA	계좌번호
67	(개인연금신탁연금신탁)송금내역통보서	EA	접수번호
68	수익자변경신청서	EA	계좌번호
69	신탁(기간종별)변경신청서	EA	계좌번호
70	신탁양도승낙신청서	EA	계좌번호
71	[]신탁(해지일부출금)(신청취소신청)서	EA	신탁계좌번호
72	KB 유언신탁신청서	EA	주민사업자번호
73	KB 유언신탁제변경신고서	EA	계좌번호
74	퇴직신탁계약해지(이전)동의서	EA	계좌번호
75	중간정산전원 · 전액지급지시서	EA	계좌번호
76	퇴직신탁중간정산일부지급지시서	EA	계좌번호
77	퇴직신탁퇴직급부금지급지시서	EA	계좌번호
78	퇴직신탁계약내용변경신청서	EA	계좌번호
79	퇴직연금제도실시를위한퇴직신탁계약해지신청서	EA	계좌번호
80	퇴직신탁계약해지신청및지급지시서	EA	계좌번호
81	퇴직신탁계약이전신청서	EA	계좌번호
82	퇴직신탁가입자명부확인서	EA	계좌번호
83	자사주특정금전신탁중도해지의뢰서	EA	계좌번호
84	특정금전신탁연결계좌등록신청서	EA	계좌번호
85	특정금전신탁중도해지의뢰서	EA	계좌번호

86	특정금전신탁계약추가약정서	EA	계좌번호
87	국민주택채권사고신고및취하서	EA	채권번호
88	집합투자증권추가매입및환매예약신청서	EA	계좌번호
89	판매회사 이동을 위한 계좌 확인서 발급 신청서	EA	계좌번호
90	연금저축펀드전환예약계약변경신청서	EA	계좌번호
91	집합투자증권신규가입신청서(역외펀드겸용)	EA	계좌번호
92	KB Smart 플랜적립이체신청서	EA	계좌번호
93	보험계약청약서(2-1)	EA	계좌번호
94	고객정보변경신청서(방카슈랑스)	EA	계좌번호
95	자동이체신청서(방카슈랑스)	EA	계좌번호
96	품질보증신청서	EA	계좌번호
97	지급신청서(방카슈랑스)	EA	계좌번호
98	KBTEZECard 가입신청서	AA+CA	
99	하나투어마일리지클럽회원가입신청서	AA+CA	
100	SOILKB 카드회원가입신청서	AA+CA	
101	SK 엔크린보너스카드회원가입신청서	AA+CA	
102	신용카드회원가입신청서(창구용)(신규추가교체)	AA+CA	
103	KB 카드회원가입신청서(창구용)	AA+CA	
104	KB 카드회원가입신청서(결제계좌신규겸용)	AA+CA	
105	KB 카드회원가입신청서(신용체크겸용)	AA+CA	
106	(모집인용)SK 스마트 KB 카드회원가입신청서	AA+CA	
107	(모집인용)플래티늄회원가입신청서	AA+CA	
108	(모집인용)KB 마이원회원가입신청서	AA+CA	
109	KB 소형화물차유류세환급카드가입신청서	AA+CA	
110	GS 칼텍스보너스(GS/POINT)카드회원가입신청서	AA+CA	
111	해피포인트카드회원가입신청서	AA+CA	
112	KB 고운맘카드(체크카드포함)가입신청서	AA+CA	
113	대한항공스카이패스(아시아나클럽)멤버십가입신청서	AA+CA	
114	SOIL 보너스카드회원가입신청서	AA+CA	
115	d 카드회원가입신청서	AA+CA	
116	KBCheckCardApplication	AA+CA	
117	KBPlatinum 카드가입신청서	AA+CA	
118	GS 칼텍스스마트세이브회원가입신청서	AA+CA	
119	GS 칼텍스보너스카드회원가입신청서	AA+CA	
120	KBmyOneCard 회원가입신청서	AA+CA	
121	myOnecard 서비스멤버십가입신청서	AA+CA	
122	SK 스마트 KB 카드회원가입신청서	AA+CA	
123	(모집인용)SK 멤버십신청서	AA+CA	
124	SOILLPGKB 카드회원가입신청서	AA+CA	

	125	SOIL 보너스카드회원가입신청서	AA+CA	
	126	KBCheck 카드발급신청서	AA+CA	
	127	Myone(CGV)멤버십가입신청서	AA+CA	
	128	CGV 멤버십가입신청서	AA+CA	
	129	KBCGVMANIACard 회원가입신청서	AA+CA	
	130	GS 칼텍스스마트세이브 KB 카드회원가입신청서	AA+CA	
	131	GS 칼텍스스마트세이브 KB 카드회원가입신청서	AA+CA	
	132	KBROVLCard 가입신청서	AA+CA	
	133	(모집인용)하나투어멤버십신청서	AA+CA	
	134	()대학교학생증신분증겸용카드(신규재)발급신청서	AA+CA	
	135	하나투어 KB 카드회원가입신청서	AA+CA	
	136	(모집인용)신용카드회원가입신청서	AA+CA	
	137	대한항공스카이패스멤버십가입신청서	AA+CA	
	138	KB 기업 ROVLCard 회원가입신청서패키지	AA+CA	
	139	KB 기업 TEZECard 회원가입신청서패키지	AA+CA	
	140	KBmybiz 체크카드가입신청서	AA+CA	
	141	기업카드회원가입신청서	AA+CA	
	142	KB 기업 TEZECard 회원가입신청서	AA+CA	
	143	KB 기업 ROVLCard 회원가입신청서	AA+CA	
	144	KBmybiz 카드가입신청서	AA+CA	
	145	가맹점가입신청서	AA+CB	
	146	KB 가맹점가입동의서	AA+CB	
	147	공동대표자가맹점가입동의서	AA+CB	
	148	가맹점거래승인한도조정(해제)신청서	AA+CB	
	149	신용카드가맹점특약서(할부특약)	AA+CB	
	150	전자상거래결제대행업무특약서(전자상거래오픈마켓용)	AA+CB	
	151	가맹점특약서(MT 처리가맹점용)	AA+CB	
	152	전자서명에의한신용판매특약서	AA+CB	
	153	EDC 특약가맹점확약서	AA+CB	
	154	전자상거래결제대행업무특약서	AA+CB	
	155	신용판매내역전송결제특약서	AA+CB	
	156	예적금질권설정계약서(예적금용)	AA+CB	

157	신용판매내역무전표 EDI 서비스특약서	AA+CB	
158	국민은행신용판매대금자동청구(EDC)신청서	AA+CB	
159	신용카드가맹점특약서(수기매출전표(인터넷전자상거래포함)특약)	AA+CB	
160	공용전표사용특약서	AA+CB	
161	KB Wise Members 카드회원가입신청서	AA+CA	
162	KB 비씨대한민국카드회원가입신청서	AA+CA	
163	한국자원봉사센터협회 KB Remember 카드가입신청서	AA+CA	
164	KB 내고장사랑카드 회원가입신청서	AA+CA	
165	에버리치 KB 고운맘카드(체크카드포함)가입신청서	AA+CA	
166	edu&d 카드 회원가입 신청서	AA+CA	
167	MyOne 서비스 멤버십 가입 신청서	AA+CA	
168	GS & POINTKB 카드 회원가입신청서	AA+CA	
169	KB STARTLUCK 카드 회원가입신청서	AA+CA	
170	GS & POINT 회원가입신청서	AA+CA	
998	임의장표	출력 용도에 맞게 인식정보 사용 (인식구분정보 1 개 또는 2 개 사용)	
999	전표	BA-전표인식정보	

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcEcc extends
                    com.kbstar.sqc.base.FunctionUnit
```

[입력항목]

```
String inRecogDstic1;
String inRecogDstic2;
String inBrncd;
String inTranYms;
String inEmpid;
String inCno;
String inBzno;
String inAcno;
String inSlipNo;
```

- ● 입력인식구분 1
- ◎ 입력인식구분 2
- ◎ 부점코드
- ◎ 거래일시
- ◎ 직원번호
- ◎ 고객번호
- ◎ 사업자등록번호
- ◎ 계좌번호
- ◎ 전표번호

	String inCardNo;
	String inMbrmchNo;
	String inBndNo;
	String inVaultNo;
	String inTranDte;
	String inIdnfr;
	
	[출력항목]
	String outEccOutptData1;
	String outEccOutptData2;
		◎ 카드번호
		◎ 가맹점번호
		◎ 채권번호
		◎ 금고번호
		◎ 거래일자
		◎ 식별자
		
		//ECC 출력데이터 1
		//ECC 출력데이터 2

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 이미지 인식 코드(ECC) 암호화 II
	(com.kbstar.kji.enbncmn, getEncodeStringCode2)
	
	IDataSet getEncodeStringCode2Req = new DataSet();
	
	getEncodeStringCode2Req.put("inRecogDstic1", requestData.get("inRecogDstic1"));
	getEncodeStringCode2Req.put("inRecogDstic2", requestData.get("inRecogDstic2"));
	getEncodeStringCode2Req.put("inBrncd", requestData.get("inBrncd")); // 부점코드
	getEncodeStringCode2Req.put("inTranYms", requestData.get("inTranYms")); // 거래일시
	getEncodeStringCode2Req.put("inEmpid", requestData.get("inEmpid")); // 직원번호
	getEncodeStringCode2Req.put("inCno", requestData.get("inCno")); // 고객번호
	getEncodeStringCode2Req.put("inBzno", requestData.get("inBzno")); // 사업자등록번호
	getEncodeStringCode2Req.put("inAcno", requestData.get("inAcno")); // 계좌번호
	getEncodeStringCode2Req.put("inSlipNo", requestData.get("inSlipNo")); // 전표번호
	getEncodeStringCode2Req.put("inCardNo", requestData.get("inCardNo")); // 카드번호
	getEncodeStringCode2Req.put("inMbrmchNo", requestData.get("inMbrmchNo")); // 가맹점
	getEncodeStringCode2Req.put("inBndNo", requestData.get("inBndNo")); // 채권번호
	getEncodeStringCode2Req.put("inVaultNo", requestData.get("inVaultNo")); // 금고번호
	getEncodeStringCode2Req.put("inTranDte", requestData.get("inTranDte")); // 거래일자
	getEncodeStringCode2Req.put("inIdnfr", requestData.get("inIdnfr")); // 식별자
	
	// @@ 공유메소드 호출 (이미지 인식 코드(ECC) 암호화 II getEncodeStringCode2)
	IDataSet getEncodeStringCode2Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEcc.getEncodeStringCode2", getEncodeStringCode2Req, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outeccoutptdata1 = getEncodeStringCode2Res.getString("outEccOutptData1");
	String outeccoutptdata2 = getEncodeStringCode2Res.getString("outEccOutptData2");	

비 고		

PoC

1.78 이미지 인식 코드(ECC) 복호화 I (FUBcEcc.getDecodeStringCode1())

PoC

프로그램 ID	FUBcEcc.getDecodeStringCode()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	암호화되어 입력된 문서번호 데이터를 복호화한다.
프로그램 규칙 및 제약사항	* BCEcc.getEncodingCode1() 를 통하여 암호화된 문자를 복원할 시에 사용하는 모듈이며 암호화된 문자 훼손시 복원에 필요한 ECC 코드가 포함되지 않은 문자를 복원한다. 1. 입력가능한 암호화된 문자는 아래와 같다. - 0, 4, 5, A, E, G, H, J, L, M, P, T, V, W, X, Z

PoC

2. 입력된 1~2 자리 문자열은 인식구분코드로 인식되며, 아래와 같다.

번호	인식구분	인식구분명	입력 값
1	AA	기본인식정보	부점코드(4 자리) + 거래일시(14 자리) + 직원번호(7 자리)
2	AB	고객인식정보	부점코드(4 자리) + 거래일시(14 자리) + 직원번호(7 자리) + 고객번호(13 자리)
3	AC	계좌인식정보	계좌번호(15 자리) + 고객번호(13 자리)
4	BA	전표인식정보	부점코드(4 자리) + 직원번호(7 자리) + 전표번호(5 자리)
5	CA	카드인식정보	카드번호(16 자리) + 고객번호(13 자리)
6	CB	가맹점인식정보	가맹점번호(15 자리) + SPACE(1) + 사업자등록번호(10 자리)
7	DA	채권인식정보	채권번호(8 자리) + 고객번호(13 자리)
8	DC	대여금고인식정보	금고번호(5 자리) + 고객번호(13 자리)

PoC

3. 장표서식에 출력된 ECC 를 라인단위로 입력하면 복호화하여 최대 2 라인에 출력한다.

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcEcc extends
                com.kbstar.sqc.base.FunctionUnit
```

[입력항목]

```
String inEccInptData1;
String inEccInptData2;
```

PoC

● ECC 입력데이터 1
◎ ECC 입력데이터 2

[출력항목] PoC

```
String outBrncd;
String outTranYms;
String outEmpid;
String outCno;
String outBzno;
```

◎ 부점코드
◎ 거래일시
◎ 직원번호
◎ 고객번호
◎ 사업자등록번호

	String outAcno; String outSlipNo; String outCardNo; String outMbrmchNo; String outBndNo; String outVaultNo; String outTranDte;	◎ 계좌번호 ◎ 전표번호 ◎ 카드번호 ◎ 가맹점번호 ◎ 채권번호 ◎ 금고번호 ◎ 거래일자
RTNSTAT 설명	N/A	

사용 설명(예시)	**// 공유메소드 호출 - 이미지 인식 코드(ECC) 복호화 I (com.kbstar.kji.enbncmn, getDecodeStringCode1)**
	
	IDataSet getDecodeStringCode1Req = new DataSet();
	
	getDecodeStringCode1Req.put("inEccInptData1", requestData.get("inEccInptData1"));
	getDecodeStringCode1Req.put("inEccInptData2", requestData.get("inEccInptData2"));
	
	// @@ 공유메소드 호출 (이미지 인식 코드(ECC) 복호화 I getDecodeStringCode1)
	IDataSet getDecodeStringCode1Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcEcc.getDecodeStringCode1", getDecodeStringCode1Req, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outbrncd = getDecodeStringCode1Res.getString("outBrncd"); // 부점코드
	String outtranyms = getDecodeStringCode1Res.getString("outTranYms"); // 거래일시
	String outempid = getDecodeStringCode1Res.getString("outEmpid"); // 직원번호
	String outcno = getDecodeStringCode1Res.getString("outCno"); // 고객번호
	String outbzno = getDecodeStringCode1Res.getString("outBzno"); // 사업자등록번호
	String outacno = getDecodeStringCode1Res.getString("outAcno"); // 계좌번호
	String outslipno = getDecodeStringCode1Res.getString("outSlipNo"); // 전표번호
	String outcardno = getDecodeStringCode1Res.getString("outCardNo"); // 카드번호
	String outmbrmchno = getDecodeStringCode1Res.getString("outMbrmchNo"); // 가맹점번호
	String outbndno = getDecodeStringCode1Res.getString("outBndNo"); // 채권번호
	String outvaultno = getDecodeStringCode1Res.getString("outVaultNo"); // 금고번호
	String outteandte = getDecodeStringCode1Res.getString("outTeanDte"); // 거래일자	

비 고		

1.79 이미지 인식 코드(ECC) 복호화 II (FUBcEcc.getDecodeStringCode2())

프로그램 ID	FUBcEcc.getDecodeStringCode()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	암호화되어 입력된 문서번호 데이터를 복호화한다.
프로그램 규칙 및 제약사항	* BCEcc.getEncodingCode2() 를 통하여 암호화된 문자를 복원할 시에 사용하는 모듈이며 암호화된 문자 훼손시 복원에 필요한 ECC 코드가 포함된 문자를 복원한다. 1. 입력가능한 암호화된 문자는 아래와 같다. - 0, 4, 5, A, E, G, H, J, L, M, P, T, V, W, X, Z 2. 입력된 1~2 자리 문자열은 인식구분코드로 인식되며, 아래와 같다. 3. 장표서식에 출력된 ECC 를 라인단위로 입력하면 복호화하여 최대 2 라인에 출력한다.

번호	인식구분	인식구분명	입력 값
1	AA	기본인식정보	부점코드(4 자리) + 거래일시(14 자리) + 직원번호(7 자리)
2	AB	고객인식정보	부점코드(4 자리) + 거래일시(14 자리) + 직원번호(7 자리) + 고객번호(13 자리)
3	AC	계좌인식정보	계좌번호(15 자리) + 고객번호(13 자리)
4	BA	전표인식정보	부점코드(4 자리) + 직원번호(7 자리) + 전표번호(5 자리)
5	CA	카드인식정보	카드번호(16 자리) + 고객번호(13 자리)
6	CB	가맹점인식정보	가맹점번호(15 자리) + SPACE(1) + 사업자등록번호(10 자리)
7	DA	채권인식정보	채권번호(8 자리) + 고객번호(13 자리)
8	DC	대여금고인식정보	금고번호(5 자리) + 고객번호(13 자리)

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcEcc extends
                    com.kbstar.sqc.base.FunctionUnit

[입력항목]
    String inEccInptData1;
    String inEccInptData2;

[출력항목]
    String outBrncd;
    String outTranYms;
    String outEmpid;
    String outCno;
    String outBzno;
```

● ECC 입력데이터 1
◎ ECC 입력데이터 2

◎ 부점코드
◎ 거래일시
◎ 직원번호
◎ 고객번호
◎ 사업자등록번호

This document is strictly limited to the 2025 C2J conversation PoC

	String outAcno; ◎ 계좌번호 String outSlipNo; ◎ 전표번호 String outCardNo; ◎ 카드번호 String outMbrmchNo; ◎ 가맹점번호 String outBndNo; ◎ 채권번호 String outVaultNo; ◎ 금고번호 String outTranDte; ◎ 거래일자
RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 이미지 인식 코드(ECC) 복호화 II (com.kbstar.kji.enbncmn, getDecodeStringCode2)
	IDataSet getDecodeStringCode2Req = new DataSet();
	getDecodeStringCode2Req.put("inEccInptData1", requestData.get("inEccInptData1"));
	getDecodeStringCode2Req.put("inEccInptData2", requestData.get("inEccInptData2"));
	// @@ 공유메소드 호출 (이미지 인식 코드(ECC) 복호화 II getDecodeStringCode2)
	IDataSet getDecodeStringCode2Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcEcc.getDecodeStringCode2", getDecodeStringCode2Req, onlineCtx);
	// @@ 출력 데이터 셋 처리
	String outbrncd = getDecodeStringCode2Res.getString("outBrncd"); // 부점코드
	String outtranyms = getDecodeStringCode2Res.getString("outTranYms"); // 거래일시
	String outempid = getDecodeStringCode2Res.getString("outEmpid"); // 직원번호
	String outcno = getDecodeStringCode2Res.getString("outCno"); // 고객번호
	String outbzno = getDecodeStringCode2Res.getString("outBzno"); // 사업자등록번호
	String outacno = getDecodeStringCode2Res.getString("outAcno"); // 계좌번호
	String outslipno = getDecodeStringCode2Res.getString("outSlipNo"); // 전표번호
	String outcardno = getDecodeStringCode2Res.getString("outCardNo"); // 카드번호
	String outmbrmchno = getDecodeStringCode2Res.getString("outMbrmchNo"); // 가맹점번호
	String outbndno = getDecodeStringCode2Res.getString("outBndNo"); // 채권번호
	String outvaultno = getDecodeStringCode2Res.getString("outVaultNo"); // 금고번호

비 고	

1.80

책임자승인사유 판단(FUBcAuthorization.prcssSpvsrAthorPtrn())

프로그램 ID	FUBcAuthorization. prcssSpvsrAthorPtrn()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	에러메시지목록 책임자승인규칙기본	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIER01 THKJITR04		SQL ID	N/A

프로그램 설명	모든 책임자승인 처리를 공통 유형화하여 승인처리하는 기능으로 기 등록된 책임자승인 규칙목록의 테이블 데이터를 읽어 승인 전 거래시 책임자승인 조건을 판단하고 해당 책임자승인사유코드를 결정하여 Common Area 에 등록하며, 승인 후 거래시는 결정된 책임자승인사유코드와 Common Area 의 책임자승인사유코드를 확인하여 승인요청 여부를 결정한다.
프로그램 규칙 및 제약사항	※ N-KESA 서버 M/F 과 연계되는 책임자승인 요건없음 - N-KESA 단독 책임자승인은 가능하며, M/F 과 연계하여 책임자승인 필요 시 연계시스템개발팀(N-KESA) 전행공통 협의 필요함 - U-KESA 와 같은 책임자승인관련 후처리 구현필요 - 확인 : N-KESA 이상혁, 전행공통 이향림, 20210706 N-KESA 이상혁,이은권(재 확인_):2022.01.28(단독책임자승인 가능) 1. 금액관련 승인규칙과 목록형 규칙, 범위형 규칙을 입력파라메타로 전달받아 승인완료구분코드, 책임자승인구분코드, 책임자승인사유건수, 책임자승인사유코드를 Common Area 에 발행한다. 단, 승인완료구분코드는 초기값이 '0'이며 승인요청시 '1'이 되고 계속적으로 승인 요청할 경우에는 무시한다. 그리고 책임자의 승인이 완료되면 단말에서 '2'로 설정한다. 2. 승인완료구분코드는 다음과 같다. 1) '0' : 초기값 2) '1' : 승인요청 3) '2' : 승인완료 4) '3' : 승인거절 5) '4' : 승인취소 6) '5' : 사전승인 7) '6' : 결재 ※ 최종책임자승인구분코드가 'A'(단순결재) 또는 'B'(복수결재)의 경우, 승인완료구분코드는 '6'(결재)로 그외의 경우는 '2'(승인완료)로 Common 영역에 설정한다. 3. 책임자승인구분코드 값은 다음과 같다. 1) '1' : 단수승인 2) '2' : 단수승인 3) '3' : 단수승인 4) '4' : 단수승인 5) '5' : 복수승인 6) 'A' : 단수결재 7) 'B' : 단수결재 ※ 승인 : 책임자승인 팝업창 O, E 전표,E 장표 결재시스템 표시 O 결재 : 책임자승인 팝업창 X, E 전표,E 장표 결재시스템 표시 O ※ 20171023 팀장전결안내 시행 전 내용 1) '1' : 단수승인 2) '2' : 복수승인(책임자승인 + 책임자결재) 3) '3' : 복수승인(책임자승인 + 지점장결재) 4) '4' : 복수승인(책임자승인 + 내부통제결재) 5) '5' : 복수승인(책임자승인 + 책임자승인) 6) 'A' : 단수결재 7) 'B' : 복수결재

> 20181023 시행 관련 프로젝트
- P20171830909 전결권 안내 시스템 및 전자결재 시스템
- 전행공통 전표 및 임의장표 "팀원전결" 인자
  B88A5007 팀장전결 신규
- 결재알리미 팝업,결재대상 LIST 조회 없어짐
  커뮤니케이션 : 책임자승인 메세지 삭제 요청(스마트플랫폼부/정보개발부)

PoC

4. 멀티 책임자승인사유발생의 경우 책임자승인사유코드의 가장 큰 값의 역순으로 Common Area의 책임자승인구분코드에 반영한다.

PoC

※ 책임자승인구분코드 (단순/복수결재) 경합의 경우

경 합		최종책임자승인구분코드	경 합		최종책임자승인구분코드
1 + A	=	1	1 + B	=	1
2 + A	=	2	2 + B	=	2
3 + A	=	3	3 + B	=	3
4 + A	=	4	4 + B	=	4
A + A	=	A	A + B	=	B
B + A	=	B	B + B	=	B

PoC

PoC

5. Common Area 의 책임자승인사유코드의 건수를 10 개까지 관리하며 10 개가 초과되는 경우 우선순위가 높은 책임자승인코드인지 비교하여 순서조정을 한다. (에러를 발행하지 않으며, Common Area 에 중복되는 책임자승인사유코드가 있는지 확인하여 중복된 사유코드가 있으면 무시한다.)

6. 책임자승인처리시 단말에 고객정보를 내려주기 위해 승인요청추가메시지내용(BOCOM-ATHOR-DS-MSG-CTNT)에 고객고유번호(20BYTE)+고객명(52BYTE)를 SET 한다. 단일거래와 기동거래는 모두 고객정보를 SET 하고, 수동거래는 고객정보출력여부(XCJISUP1-I-CUST-INFO-OUTPT-YN) 입력값이 여('1')인 경우만 고객정보를 SET 하여 단말로 내려준다.

7. Return Code '05' 처리중단은 책임자승인판단 이후 추가판단이 필요 없는 경우 프로그램 로직을 끝까지 수행하지 않고 중단시켜 책임자승인처리를 하고자 하는 경우 사용되며, 거래 제어정보에 책임자중단여부에 값이 '1' Set 된 경우만 Return Code '05' 를 줄 수 있다.

8. 입력파라메터 설명

입력항목		설명	비고
금액관련승인규칙	통화코드	- 입력금액이 원화인 경우 KRW - 외화인 경우 해당통화코드 USD, JPY 등..	
	금액	- 해당금액	
	금액종류구분	- 1.거래금액, 2.누적금액, 3.원금 4.이자 5. 원금+이자 6. 잔액 7. 타점권	
목록형배열	목록형결정조건명	- 목록형 항목명을 직접 입력 (절대 띄어쓰기 금지) 예) 무인감여부, 질권설정계좌여부	
	목록형결정조건값	- 목록형 항목명의 조건값을 직접 입력 예) Y, N	
범위형배열	범위형결정조건명	- 범위형 항목명을 직접 입력 목록형 항목명을 직접 입력 (절대 띄어쓰기 금지) 예) 나이, 수수료할인율	
	범위형결정조건값	- 범위형 항목명의 조건값(숫자만 입력가능)을 직접 입력 예)30, 0.5	
예외목록형	예외목록형조건명	- 목록형결정조건명과 일치된 조건명이 해당	
	예외목록형조건값	- 목록형결정조건값의 개수가 많아 외 값으로 판단하는 것이 효율적인 경우 사용	

PoC

			예) 무역사유코드 011,012,013,019,021,022, 029 이외 코드가 1 만불초과인 경우 책임자승인처리	

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcAuthorization extends  PoC
                        com.kbstar.sqc.base.FunctionUnit

[입력항목]
     String inGroupCoCd;                          ● 그룹회사코드
     String inAthorRuleCncycd;                    ◎ 승인규칙통화코드
     BigDecimal inAthorRuleAmt;                   ◎ 승인규칙금액
     String inAthorRuleAmtKnd;                    ◎ 승인규칙금액종류
     List inCatlguTypList;                        ◎ 목록형 리스트
     List inRangTypList;                          ◎ 범위형 리스트
     String inEceptCtCndnName;                    ◎ 예외목록형조건명
     String inEceptCtCndnVal;                     ◎ 예외목록형조건값
     String inSpare;

[출력항목]                                        // 리턴상태코드
     String rtnStat;

  < inCatlguTypList 항목 >                        // 목록형조건명
     String inCatlguTdCndnName;                   // 목록형조건값
     String inCatlguTdCndnVal;
                                  PoC
  < inRangTypList 항목 >                          // 범위형조건명
     String inRangTdCndnName;                     // 범위형조건값
     BigDecimal inRangTdCndnVal;
```

RTNSTAT 설명

00 : 정상(책임자승인 요건이 없거나 추가판단이 필요한 경우)
05 : 처리중단(책임자승인 요건이 있고 추가판단이 필요없는 경우)

사용 설명(예시)

```
// 공유메소드 호출 - 책임자승인사유 판단 (com.kbstar.kji.enbncmn, prcssSpvsrAthorPtrn)

  IDataSet prcssSpvsrAthorPtrnReq = new DataSet();

  prcssSpvsrAthorPtrnReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
  prcssSpvsrAthorPtrnReq.put("inAthorRuleCncycd", requestData.get("inAthorRuleCncycd"));
  prcssSpvsrAthorPtrnReq.put("inAthorRuleAmt", requestData.get("inAthorRuleAmt"));
  prcssSpvsrAthorPtrnReq.put("inAthorRuleAmtKnd", requestData.get("inAthorRuleAmtKnd"));
  prcssSpvsrAthorPtrnReq.put("inCatlguTypList",
                                    requestData.getRecordSet("inCatlguTypList"));
  prcssSpvsrAthorPtrnReq.put("inRangTypList",
                                    requestData.getRecordSet("inRangTypList"));
  prcssSpvsrAthorPtrnReq.put("inEceptCtCndnName", requestData.get("inEceptCtCndnName"));
  prcssSpvsrAthorPtrnReq.put("inEceptCtCndnVal", requestData.get("inEceptCtCndnVal"))
  prcssSpvsrAthorPtrnReq.put("inSpare", requestData.get("inSpare"));

// @@ 공유메소드 호출 (책임자승인사유 판단 prcssSpvsrAthorPtrn)
  IDataSet prcssSpvsrAthorPtrnRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
        "FUBcAuthorization.prcssSpvsrAthorPtrn", prcssSpvsrAthorPtrnReq, onlineCtx);

// @@ 출력 데이터 셋 처리
  String rtnstat = prcssSpvsrAthorPtrnRes.getString("rtnStat");
```

비 고	

1.81

EBCDIC 한글코드(1364) 검증(N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력데이터가 Mainframe 에서 지원하는 EBCDIC 한글 1364 코드인지를 검증한다.
프로그램 규칙 및 제약사항	1. Mixed 모드의 한글데이터의 입력이 가능하다. 2. 입력데이터 검증 사항 가) 입력된 데이터 중 한글코드의 SOSI 가 있는지 체크한다. SOSI 가 붙어있는 경우 SOSI 내부의 한글데이터 Bytes 가 짝수인지를 검증한다. 예시] 입력데이터 : 나) 입력된 데이터 중 한글코드에 SO 붙었으면 SI 가 붙어 있는지를 체크한다. 예시] 입력데이터 : SO 한국 다) 입력된 데이터 중 한글코드에 SI 붙었으면 SO 가 붙어 있는지를 체크한다. 예시] 입력데이터 : 한국 SI ** 가),나),다)번의 예시인 경우 오류 처리한다. 3. 입력데이터를 ASCII 값으로 변환하여 정상으로 변환이 되는 경우에 정상 코드값으로 판단한다.
비 고	

1.82

분리된 데이터(TEXT) 결합(N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	주소와 같이 하나의 데이터가 주소 1, 주소 2 등으로 분리되어 저장된 경우에 하나의 데이터로 결합하여 출력하기 위해서 사용된다.
프로그램 규칙 및 제약사항	1. 총 4 개의 입력데이터를 하나의 데이터로 결합해 준다. PoC 2. 하나의 입력데이터는 1,000 BYTES 를 넘지 못하며, MIXED 모드의 한글데이터 입력이 가능하다. PoC 3. 입력데이터의 정상여부는 확인하지 않는다. 4. 입력데이터 1, 2 는 필수이며, 첫번째 데이터의 끝에 SPACE 1BYTE 를 추가하여 두번째 데이터를 붙인다. PoC
비 고	

1.83 대외기관에러메시지코드별 표준에러/조치메시지코드 조회 (FUBcMessage.getTermlEcptnlChnlMsgCd())

프로그램 ID	FUBcMessage.getTermlEcptnlChnlMsgCd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(X)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	단말외채널에러메시지목록	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIER03		SQL ID	selectTermlEcptnlChnlMsgCd

프로그램 설명	단말외 채널의 에러메시지를 조회한다.
프로그램 규칙 및 제약사항	1. 보안팀에서 수신한 지주직원정보를 기반으로 한다. 2. 직원의 부서이동 이력정보는 조회할 수 없다. 3. 직원번호를 반드시 입력하여야 한다. 4. 퇴직한 지주직원정보를 포함한다.. 현재 존재하는 직원정보를 조회하고자 할 경우 퇴직년월일로 업무가 판단한다.

DTO 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcMessage extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	String inGroupCoCd;
	String inLangDstcd;
	String inMsgChnlDstcd;
	String inChnlErrMsgCd;
	
	[출력항목]

- 그룹회사코드
- 언어구분
- 메시지채널구분
- 채널에러메시지코드

PoC

PoC

	String outLangDstcd;
	String outMsgChnlDstcd;
	String outChnlErrMsgCd;
	String outRegiYmd;
	String outErrStusDstcd;
	String outBascErrMsgCd;
	String outBascTreatMsgCd;
	String outRspblUapplCd;
	String outOutptCtnt;
		// 언어구분
		// 메시지채널구분
		// 채널에러메시지코드
		// 등록년월일
		// 에러상태구분
		// 기본에러메시지코드
		// 기본조치메시지코드
		// 담당어플리케이션코드
		// 출력내용

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 – 단말외채널에러메시지목록 조회 (com.kbstar.kji.enbncmn, getTermlEcptnlChnlMsgCd)
	
	IDataSet getTermlEcptnlChnlMsgCdReq = new DataSet();
	
	getTermlEcptnlChnlMsgCdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getTermlEcptnlChnlMsgCdReq.put("inLangDstcd", requestData.get("inLangDstcd"));
	getTermlEcptnlChnlMsgCdReq.put("inMsgChnlDstcd", requestData.get("inMsgChnlDstcd"));
	getTermlEcptnlChnlMsgCdReq.put("inChnlErrMsgCd", requestData.get("inLangDstcd"));
	
	// @@ 공유메소드 호출 (지주직원기본 조회 getGroupEmpInfo)
	IDataSet getGroupEmpInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcEmployee.getGroupEmpInfo", getGroupEmpInfoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outEmpid = calculateExpirDdRes.getString("outEmpid"); // 직원번호
	String outEmpzFname = calculateExpirDdRes.getString("outEmpzFname"); // 직원성명
	String outAfltIdnfr = calculateExpirDdRes.getString("outAfltIdnfr"); // 계열사식별자
	String outAfltName = calculateExpirDdRes.getString("outAfltName"); // 계열사명
	String outBelngIdnfr = calculateExpirDdRes.getString("outBelngIdnfr"); // 소속식별자
	String outBelngName = calculateExpirDdRes.getString("outBelngName"); // 소속명
	String outJobclIdnfr = calculateExpirDdRes.getString("outJobclIdnfr"); // 직급식별자
	String outJobclCtnt = calculateExpirDdRes.getString("outJobclCtnt"); // 직급내용
	String outJobtlIdnfr = calculateExpirDdRes.getString("outJobtlIdnfr"); // 직위식별자
	String outJobtlCtnt = calculateExpirDdRes.getString("outJobtlCtnt"); // 직위내용
	String outRtireYmd = calculateExpirDdRes.getString("outRtireYmd"); // 퇴직년월일	

비 고		

PoC

1.84 데이터 공백(Space) 제거(N/A)

PoC

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	하나의 데이터중 SPACE 를 제거하기 위하여 사용되는 기능으로 데이터의 비교에 활용된다.
프로그램 규칙 및 제약사항	1. 입력데이터는 1,000 BYTES 를 넘지 못하며, MIXED 모드(전자/반자 포함)의 문자도 사용가능하다. 2. 입력데이터의 코드값 정상여부는 확인하지 않는다. 3. 전자문자는 2BYTES SPACE , 반자문자는 1BYTE SPACE 에 해당되는 데이터만 제거한다.
비 고	

PoC

1.85 전표번호 채번-유/무전표 자동판단(N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	단말 사용자별(자동화기기 포함) 전표일련번호를 유/무전표 자동판단 기준표에 의하여 TABLE 을 통해 채번한다.
프로그램 규칙 및 제약사항	1. 폐쇄부점에서는 채번 할수 없다. 활동점인 경우만 채번이 가능하다. 2. 부점코드와 사용자번호에 대해 전표구분별 Common Area 거래정보의 마감후여부와 전일자거래여부 단위로 채번한다. 3. Common Area 거래정보별 채번 기준은 다음과 같다. 가) 마감후 거래인 경우 : Common Area 일자정보의 익영업년월일 기준으로 채번한다. 나) 전일자 거래인 경우 : Common Area 일자정보의 전영업년월일 기준으로 채번한다. 다) 당일자 거래인 경우 : Common Area 일자정보의 회계기준년월일 기준으로 채번한다. 4. 전표번호 채번 TABLE 에 부점 + 사용자번호 + 일계영업일(KEY)이 존재하지 않은 경우(예 : 정기인사이동) 자동으로 KEY 가 생성 된다. 5. 거래별 유/무전표 자동판단 규칙에 의해 전표구분코드(JICOM-SLIP-DSTCD)와 전표번호(JICOM-SLIP-NO)를 Common Area JICOM 영역에 Set 한다. 6. 전표구분코드 내용은 다음과 같다. 가) '1' : 유전표 채번 - 유전표일련번호 + 별단예금매수로 채번한다. 나) '2' : 유전표 무인자 채번 - 일련번호 1 씩 증가한다 다) '3' : 무전표 채번 - 일련번호 1 씩 증가한다 7. 한 개의 거래코드(BMS 거래파라메터)에서 전표채번을 1 번이상 호출하면 오류처리한다. 8. 연동거래의 전표번호, 유/무 전표구분, 책임자승인사유 판단기준은 기동이 기준이되어 처리한다. (단, 수동이 먼저 호출되어 전표채번을 한 경우 무조건 유전표 처리한다.)

PoC

9. 입력파라메터 설명은 다음과 같다.

입력항목		설명	비고
금액관련승인규칙	통화코드	- 입력금액이 원화인 경우 KRW - 외화인 경우 해당통화코드 USD, JPY 등..	금액조건 이하
	금액	- 해당금액	
	금액종류구분	- 1.거래금액, 2.누적금액, 3.원금 4.이자 5. 원금+이자 6. 잔액 7. 타점권	
목록형배열	목록형결정조건명	- 목록형 항목명을 직접 입력 (절대 띄어쓰기 금지) 예) 무인감여부, 질권설정계좌여부	최대 20 개까지 입력 가능
	목록형결정조건값	- 목록형 항목명의 조건값을 직접 입력 예) Y, N	

비 고	

1.86

PoC
PoC
PoC
PoC
PoC
PoC

KSC5601 한글코드 검증(N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	입력데이터가 Mainframe 에서 지원하는 KSC5601 한글코드인지를 검증한다.
프로그램 규칙 및 제약사항	1. 입력데이터 검증 사항 가) 입력된 데이터 중 한글코드의 SOSI 가 있는지 체크한다. SOSI 가 붙어있는 경우 SOSI 내부의 한글데이터 Bytes 가 짝수인지를 검증한다. 예시] 입력데이터 : 나) 입력된 데이터 중 한글코드에 SO 붙었으면 SI 가 붙어 있는지를 체크한다. 예시] 입력데이터 : SO 한국 다) 입력된 데이터 중 한글코드에 SI 붙었으면 SO 가 붙어 있는지를 체크한다. 예시] 입력데이터 : 한국 SI ** 가),나),다)번의 예시인 경우 오류 처리한다.
비 고	

PoC

PoC

PoC

PoC

1.87 월말 년월일 산출 (FUBcDate.calculateEnmnYmd())

프로그램 ID	FUBcDate.calculateEnmnYmd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력년월일에 대해 월말년월일을 제공한다
프로그램 규칙 및 제약사항	1. 입력년월일은 아래의 조건을 만족하야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 3) 입력일이 입력월의 말일자보다 크지않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 일로 한다. 예시] 입력년월일 2008 년 4 월 15 일

월	화	수	목	금	토	일
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

가) 입력년월일 : 20080415
=> 출력월말년월일: 2008-04-30, 리턴상태코드: 00

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDate extends
                    com.kbstar.sqc.base.FunctionUnit

[입력항목]
  String inGroupCoCd;          ● 그룹회사코드
  String inYmd;                ● 년월일

[출력항목]
  String outEnmnYmd;           // 월말년월일
```

RTNSTAT 설명	N/A

사용 설명(예시)

```
// 공유메소드 호출 - 월말 년월일 산출 (com.kbstar.kji.enbncmn, calculateEnmnYmd)

IDataSet calculateEnmnYmdReq = new DataSet();

calculateEnmnYmdReq.put("inGroupCoCd", requestData.get("inGroupCoCd ")); // 그룹회사코드
calculateEnmnYmdReq.put("inYmd", requestData.get("inYmd")); // 년월일
```

	// @@ 공유메소드 호출 (월말 년월일 산출 calculateEnmnYmd) IDataSet calculateEnmnYmdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDate.calculateEnmnYmd", calculateEnmnYmdReq, onlineCtx); // @@ 출력 데이터 셋 처리 String outenmnymd = calculateEnmnYmdRes.getString("outEnmnYmd");
비 고	

PoC

PoC

1.88

PoC

PoC

PoC

PoC

This document is strictly limited to the 2025 C2J conversation PoC

PoC

공통영역 기준일자 SET(N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	단순배치, F/W 배치에서 공통의 일자영역에 해당하는 전전영업년월일, 전영업년월일, 금영업(회계기준)년월일, 익영업년월일, 익익영업년월일, 음력거래기준년월일, 영업일구분코드, 요일구분코드의 값을 입력된 일자를 기준으로 SET 한다.
프로그램 규칙 및 제약사항	1. 단순배치, F/W 배치만 해당되며 공통영역을 사용할 수 없는 O/S 배치는 호출할 수 없다. 2. S1000-INITIALIZE-RTN PARAGRAPH 에서 호출한다. 3. F/W 배치인 경우는 배치초기화 유틸리티(ZUGBTIN)를 호출하기 전에 본 프로그램을 호출해야 한다.
비 고	

PoC
PoC
PoC
PoC
PoC

1.89 영업일 산출 II (익영업일/익익영업일/전영업일/전전영업일) (FUBcDate.calculateBzoprDd2())

프로그램 ID	FUBcDate.calculateBzoprDd2()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	공휴일목록	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIH001		SQL ID	N/A

프로그램 설명	입력일자에 대해 익영업년월일, 익익영업년월일, 전영업년월일, 전전영업년월일 제공 한다
프로그램 규칙 및 제약사항	1. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력년은 1976 ~ 2200 년 사이 값이어야 한다. 2) 입력월은 01 ~ 12 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 일로 한다. 2. 입력된 구분값에 따른 익영업년월일, 익익영업년월일, 전영업년월일, 전전영업년월일의 산출 - 휴일을 체크하여 평일 및 근무토요일인 경우의 일자를 구한다 1) 구분 1: 전영업일 제공 2) 구분 2: 익영업일 제공 3) 구분 3: 전영업일/ 익영업일 제공 4) 구분 4: 전영업일/ 전전영업일 제공 5) 구분 5: 익영업일/ 익익영업일 제공

예시] 입력일 2008 년 4 월 5 일

월	화	수	목	금	토	일
3 월 31 일	1	2	3 전전영업년월일	4 전영업년월일	5 입력년월일	6
7 익영업년월일	8 익익영업년월일	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

=> 출력결과:

구분 1 : XCJIDT16-O-BF-BZOPR-YMD = 20080404

구분 2 : XCJIDT16-O-NXBIZ-YMD = 20080407

구분 3 : XCJIDT16-O-BF-BZOPR-YMD = 20080404
XCJIDT16-O-NXBIZ-YMD = 20080407

구분 4 : XCJIDT16-O-BF-BZOPR-YMD = 20080404
XCJIDT16-O-BF-BF-BZOPR-YMD = 20080403

구분 5 : XCJIDT16-O-NXBIZ-YMD = 20080407
XCJIDT16-O-TDAT-YMD = 20080408

PoC

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	public class FUBcDate extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd;
	String inYmd;
	
	[출력항목]
	String outBfBfBzoprYmd
	String outBfBzoprYmd;;
	String outNxbizYmd;
	String outTdatYmd;
	String out3bzoprYmd;
	String out4bzoprYmd;
	String out5bzoprYmd;
		● 그룹회사코드
		● 년월일
		
		// 전전영업년월일
		// 전영업년월일
		// 익영업년월일
		// 익익영업년월일
		// 3 영업년월일
		// 4 영업년월일
		// 5 영업년월일

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 영업일산출 II (익/익익/전/전전영업일)
	(com.kbstar.kji.enbncmn, calculateBzoprDd2)
	
	IDataSet calculateBzoprDd2Req = new DataSet();
	
	calculateBzoprDd2Req.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateBzoprDd2Req.put("inYmd", requestData.get("inYmd"));
	calculateBzoprDd2Req.put("inDstcd", requestData.get("inDstcd"));
	
	// @@ 공유메소드 호출 (영업일산출 II (익/익익/전/전전영업일) calculateBzoprDd2)
	IDataSet calculateBzoprDd2Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateBzoprDd2", calculateBzoprDd2Req, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outbfbfbzoprymd = calculateBzoprDd2Res.getString("outBfBfBzoprYmd");
	String outbfbzoprymd = calculateBzoprDd2Res.getString("outBfBzoprYmd");
	String outnxbizymd = calculateBzoprDd2Res.getString("outNxbizYmd");
	String outtdatymd = calculateBzoprDd2Res.getString("outTdatYmd");	

비 고		

This document is strictly limited to the 2025 C2J conversation PoC

1.90 일자 산출 II(익일/익익일/전일/전전일) (FUBcDate.calculateDate2())

프로그램 ID	FUBcDate.calculateDate2()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명

입력일자에 대해 전전년월일, 전년월일 ,익년월일, 익익년월일을 제공한다

프로그램 규칙 및 제약사항

1. 입력년월일은 아래의 조건을 만족해야 한다.
   1) 입력월은 01 ~ 12 사이 값이어야 한다.
   2) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다.
      - 입력년의 윤년여부를 판단하여 윤년인 경우에 2 월의 마지막일자는 29 일로 한다.

2. 입력된 구분값에 따라 전일, 익일, 전일/익일, 전일/전전일, 익일/익익일 값을 제공한다.

   1) 구분 1: 전일 제공
   2) 구분 2: 익일 제공
   3) 구분 3: 전일/ 익일 제공
   4) 구분 4: 전일/ 전전일 제공
   5) 구분 5: 익일/ 익익일 제공

   예시] 입력일 2008 년 4 월 5 일

월	화	수	목	금	토	일
	1	2	3 전전일	4 전일	5 입력일	6 익일
7 익익일	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30				

가) 구분값 1, 입력년월일 : 2008-04-05
=> 출력전일: 2008-04-04
나) 구분값 2, 입력년월일 : 2008-04-05
=> 출력익일: 2008-04-06
다) 구분값 3, 입력년월일 : 2008-04-05
=> 출력전일: 2008-04-04, 출력익일: 2008-04-06
라) 구분값 4, 입력년월일 : 2008-04-05
=> 출력전일: 2008-04-04, 출력전전일: 2008-04-03
마) 구분값 5, 입력년월일 : 2008-04-05
=> 출력익일: 2008-04-06, 출력익익일: 2008-04-07

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcDate extends
                com.kbstar.sqc.base.FunctionUnit

[입력항목]
    String inYmd;                    ● 년월일
    String inDstcd;                  ● 구분코드

[출력항목]
    String outNextYmd;               // 익년월일
    String outNextNextYmd;           // 익익년월일
    String outBfYmd;                 // 전년월일
    String outBfBfYmd;               // 전전년월일
```

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 일자산출 II (익일/익익일/전일/전전일) (com.kbstar.kji.enbncmn, calculateDate2)
	IDataSet calculateDate2Req = new DataSet();
	
	calculateDate2Req.put("inYmd", requestData.get("inYmd")); // 년월일
	calculateDate2Req.put("inDstcd", requestData.get("inDstcd")); // 구분코드
	
	// @@ 공유메소드 호출 (일자산출 II (익일/익익일/전일/전전일) calculateDate2)
	IDataSet calculateDate2Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcDate.calculateDate2", calculateDate2Req, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outnextymd = calculateDate2Res.getString("outNextYmd");
	String outnextnextymd = calculateDate2Res.getString("outNextNextYmd");
	String outbfymd = calculateDate2Res.getString("outBfYmd");
	String outbfbfymd = calculateDate2Res.getString("outBfBfYmd");

비 고	

1.91

PoC

거래소휴장일 만기일/역만기일 산출(FUBcDate.calculateExchHoldyExpirDd())

프로그램 ID	FUBcDate.calculateExchHoldyExpirDd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	국내휴일목록 거래소휴일목록	SQLMAP 명	NAME SPACE	fubcdate
	영문명	THKJIH001 THKJIH003		SQL ID	selectExchHoldyCnt

프로그램 설명	시작년월일과 일수/월수를 입력받아서 만기일과 역만기일을 산출한다. 단, 한편넣기(시작일불산입,종료일산입)를 적용한 만기일과 역만기일을 산출 한다.
프로그램 규칙 및 제약사항	1. 1976 년과 2200 의 사이의 일자만 산출이 가능하다 2. 구분코드는 아래와 같다. 1) '1' : 시작년월일과 영업일수를 입력하여 만기일을 산출한다. 예시] 시작년월일 : 2008-01-28, 일수 : 90 => 만기일 : 2008-06-12 2) '2' : 시작년월일과 영업일수를 입력하여 역만기일을 산출한다. 예시] 시작년월일 : 2008-06-12, 일수 : 90 => 역만기일 : 2008-01-28 3. 윤년이 포함된 경우 윤년일수를 합산하여 만기일과 역만기일을 산출하였다. 4. 만기일/ 역만기일은 증권거래소 휴일을 포함한 휴일을 제외하여 영업일수만을 계산하여 산출한 일자이다.
입출력 설명	**package** com.kbstar.kji.enbncmn.biz; **public class** FUBcDate **extends** com.kbstar.sqc.base.FunctionUnit **[입력항목]** String inGroupCoCd; ● 그룹회사코드 String inDstcd; ● 구분코드 String inYmd; ● 년월일 int inNodayNomn; ● 일수/월수 **[출력항목]** String outYmd; // 출력년월일
RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - 거래소휴장일 만기일/역만기일 산출 (com.kbstar.kji.enbncmn, calculateExchHoldyExpirDd)**
	
	IDataSet calculateExchHoldyExpirDdReq = **new** DataSet();
	
	calculateExchHoldyExpirDdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	calculateExchHoldyExpirDdReq.put("inDstcd", requestData.get("inDstcd"));
	calculateExchHoldyExpirDdReq.put("inYmd", requestData.get("inYmd"));
	calculateExchHoldyExpirDdReq.put("inNodayNomn", requestData.get("inNodayNomn"));
	
	// @@ 공유메소드 호출 (거래소휴장일 만기일/역만기일 산출 calculateExchHoldyExpirDd)
	IDataSet calculateExchHoldyExpirDdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDate.calculateExchHoldyExpirDd", calculateExchHoldyExpirDdReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outymd = calculateExchHoldyExpirDdRes.getString("outYmd");

비 고	

PoC

1.92

PoC

PoC

PoC

PoC

PoC

거래소휴장일 양편넣기-만기일/역만기일 산출 (FUBcDate.calculateBsdeExchHoldyExpirDd())

프로그램 ID	FUBcDate.calculateBsdeExchHoldyExpirDd()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	국내휴일목록 거래소휴일목록	SQLMAP 명	NAME SPACE	fubcdate
	영문명	THKJIH001 THKJIH003		SQL ID	selectExchHoldyCnt

프로그램 설명	시작년월일과 일수/월수를 입력받아서 만기일과 역만기일을 산출한다.
프로그램 규칙 및 제약사항	1. 1976 년과 2200 의 사이의 일자만 산출이 가능하다 2. 구분코드는 아래와 같다. 1) '1' : 시작년월일과 영업일수를 입력하여 만기일을 산출한다. 예시] 시작년월일 : 2008-01-28, 일수 : 90 => 만기일 : 2008-06-11 2) '2' : 시작년월일과 영업일수를 입력하여 역만기일을 산출한다. 예시] 시작년월일 : 2008-06-12, 일수 : 90 => 역만기일 : 2008-01-29 3. 윤년이 포함된 경우 윤년일수를 합산하여 만기일과 역만기일을 산출하였다. 4. 만기일/ 역만기일은 증권거래소 휴일을 포함한 휴일을 제외하여 영업일수만을 계산하여 산출한 일자이다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcDate extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inDstcd;` ● 구분코드
	`String inYmd;` ● 년월일
	`int inNodayNomn;` ● 일수/월수
	[출력항목]
	`String outBsdeYmd;` // 양편년월일

RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 - 거래소휴장일 양편넣기-만기일/역만기일 산출 (com.kbstar.kji.enbncmn, calculateBsdeExchHoldyExpirDd)**
	
	`IDataSet calculateBsdeExchHoldyExpirDdReq = new DataSet();`
	
	`calculateBsdeExchHoldyExpirDdReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));`
	`calculateBsdeExchHoldyExpirDdReq.put("inDstcd", requestData.get("inDstcd"));`
	`calculateBsdeExchHoldyExpirDdReq.put("inYmd", requestData.get("inYmd"));`
	`calculateBsdeExchHoldyExpirDdReq.put("inNodayNomn", requestData.get("inNodayNomn"));`
	
	// @@ 공유메소드 호출 (거래소휴장일 양편넣기-만기일/역만기일 산출)
	`IDataSet calculateBsdeExchHoldyExpirDdRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDate.calculateBsdeExchHoldyExpirDd", calculateBsdeExchHoldyExpirDdReq, onlineCtx);`

	// @@ 출력 데이터 셋 처리 String outbsdeymd = calculateBsdeExchHoldyExpirDdRes.getString("outBsdeYmd");
비 고	

PoC

1.93

PoC

PoC

PoC

PoC

PoC

일자 변환 2(요청 차수의 요일에 해당하는 일자 반환)(FUBcDateChange. changeDteChng2())

프로그램 ID	FUBcDateChange. changeDteChng2() PoC		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	PoC SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력년월과 요청차수와 요일을 입력받아 일자(년월일)를 반환한다.
프로그램 규칙 및 제약사항 PoC	1. 입력년월에서 요청차수에 해당하는 요일의 일자(년월일)을 반환한다. 예시] 입력년월:200908, 요청차수:2, 요일구분:5 출력년월일:20090815 -> 둘째주 금요일이 아닌 해당월의 두번째 금요일을 출력함 2. 요일구분 값 1) '1' - 월요일 2) '2' - 화요일 3) '3' - 수요일 4) '4' - 목요일 5) '5' - 금요일 6) '6' - 토요일 7) '7' - 일요일 3. 요청차수는 5 번째 요청차수까지 요청할 수 있으며, 년월의 5 번째 요청차수 요일의 일자가 없으면 리턴상태코드에 '07'을 전달한다. PoC

입출력 설명	PoC
	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcDateChange extends com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inYm; ● 년월
	String inDmndDgre; ● 요청차수
	String inDwkDstic; ● 요일구분 PoC
	
	[출력항목]
	String rtnStat; // 리턴상태코드
	String outYmd ; // 년월일

RTNSTAT 설명	00 : 정상, 07 : 입력년월의 5 번째 요청차수 요일의 일자가 없는경우

사용 설명(예시)	// 공유메소드 호출 - 요청 차수의 요일에 해당하는 일자변환 (com.kbstar.kji.enbncmn, changeDteChng2)
	
	IDataSet changeDteChng2Req = new DataSet();
	
	changeDteChng2Req.put("inYm", requestData.get("inYm"));
	changeDteChng2Req.put("inDmndDgre", requestData.get("inDmndDgre"));
	changeDteChng2Req.put("inDwkDstic", requestData.get("inDwkDstic"));
	
	PoC

This document is strictly limited to the 2025 C2J conversation PoC

PoC

	// @@ 공유메소드 호출 (요청 차수의 요일에 해당하는 일자변환 changeDteChng2) IDataSet changeDteChng2Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcDateChange.changeDteChng2", changeDteChng2Req, onlineCtx); // @@ 출력 데이터 셋 처리 String outymd = changeDteChng2Res.getString("outYmd"); String rtnstat = changeDteChng2Res.getString("rtnStat");
비 고	

PoC

1.94

PoC

PoC

PoC

PoC

거래소휴장일 체크(FUBcDateCheck.checkExchHoldyChk())

프로그램 ID	FUBcDateCheck.checkExchHoldyChk()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	거래소휴일목록	SQLMAP 명	NAME SPACE	fubcdatecheck
	영문명	THKJIH003		SQL ID	selectExchHoldy

PoC

프로그램 설명	입력일자에 대해 거래소휴일구분을 제공 한다. (리턴상태코드 00 : 비휴장일, 07 : 휴장일)
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 입력된 일자의 휴장일을 체크 한다. 2. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2월의 29일보다 크지 않아야 한다. - 입력년의 윤년여부를 판단하여 윤년이 아닌경우 2월 28일보다 크지 않아야 한다 3. 휴장일구분코드는 아래와 같다. 1) '0' : 거래소비휴장일(영업일과는 다름) 2) '1' : 거래소휴장일 4. 휴장일 체크 1) 거래소휴일목록 테이블을 입력된 년월일로 조회한다. 거래소휴일목록 테이블에는 '1'(휴장일)만 존재한다. 2) 1)의 요건이 아닌 경우는 '0'(비휴장일)으로 출력한다. 5. 리턴상태코드 '00'(정상)은 휴장일구분코드가 '0'(비휴장일)인 경우이고, '07'(휴장일)은 휴장일구분코드가 '1' 인 경우이다. 참조) 거래소 휴장일 관련 규정(한국증권거래소 유가증권 시장 업무규정) 제5조(휴장) 거래소는 다음 각호의 하나에 해당하는 날에는 매매거래를 하지 아니한다 1. 관공서의 공휴일에 관한 규정에 의한공휴일 2. 근로자의날 규정에 관에 의한 근로자의날 3. 토요일 4. 12월 31일(고휴일 또는 일요잉인경우에는 직전의 매매거래일로 한다) 5. 그밖의 경제사정의 급변 또는 급변이 예상 되거나 거래소가 시장관리상 필요하다고 인정하는날 용어설명 : 휴장일 - 거래소 휴일목록테이블에 등록된 특정일자(영업일이 휴장일이 될 수 없음) 비휴장일 - 거래소 휴일목록테이블에 등록 되지 않은 일자 거래소 휴장일은 매년말(12월초) 제휴 상품부에서 문서로 받아처리 한다
비 고	

PoC

1.95

PoC

만기일 산출 II (N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	시작년월일, 종료년월일,월수를 입력 받아 월수에 해당하는 해당일 및 구간별 일수를 산출한다.(한편넣기(시작일불산입,종료일산입)를 적용하여 산출 한다.)
프로그램 규칙 및 제약사항	1. 시작년월일은 1601 년 01 월 01 일 보다 같거나 큰 값 이어야한다 2. 종료년월일은 9900 년 12 월 31 일 보다 같거나 작은 값 이어야한다 2. 양일자간의 차이가 50 년을 넘지 않아야 한다 3. 1 개월이 넘지 않으면 리턴상태코드는 정상으로 출력되며 출력건수, 년월일,일수는 모두 ZERO 로 SETTING 된다. 4. 구분코드는 아래와 같다. 1) '1' : 시작년월일,종료년월일, 월수를 입력받아월수에 해당하는 해당일 및 구간별 일수를 산출한다. 해당일은 영업일과 관계없이 산출한다. 5. 윤년이 포함된 경우 윤년일수를 합산하여 산출한다. 6 출력건수는 600 건(50 년) 까지 가능하다.
비 고	

1.96

PoC
PoC
PoC
PoC
PoC

전행 인스턴스코드 존재여부 조회(FUBcCode.getEnbnIncdExstYn())

프로그램 ID	FUBcCode.getEnbnIncdExstYn()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	업무 인스턴스 코드	SQLMAP 명	NAME SPACE	N/A
	영문명	THKJIUI02		SQL ID	N/A

프로그램 설명	인스턴스식별자, 인스턴스코드,년월일 을 복수개로 받아 존재여부를 확인한다.
프로그램 규칙 및 제약사항	1. 입력조회 건수는 입력항목의 인스턴스 개수와 같다. 2. 일자에 대한 처리 1) 기준년월일 입력 : 입력된 일자 당시의 인스턴스코드 조회 2) 기준년월일 미입력 : 현재 시점의 인스턴스코드 조회 - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준일 3. 입력항목의 인스턴스식별자, 인스턴스코드에 대한 개별 조회를 하여 찾지 못하면 (SQLCODE=100: 데이터없음) 출력항목별로 존재여부에 '1'(존재), '0'(미존재)을 전달한다. 4. 한 건이라도 미존재건수가 있으면 리턴상태코드는 '08'(미존재건수 있음)을 전달하고 해당되는 인스턴스식별자와.인스턴스코드를 입력 그대로 출력 해준다 5. CJIUI03 모듈은 인스턴스 내용은 출력하지 않는다(성능 문제 해결)

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcCode extends com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	int inInquryNoitm; ● 조회건수
	List LIST; ● 입력 list
	
	[출력항목]
	String rtnStat; // 리턴상태코드
	int outInquryNoitm; // 조회건수
	List LIST; // 출력 List
	
	< 입력 LIST 항목 >
	String inInstncIdnfr; // 인스턴스식별자
	String inInstncCd; // 인스턴스코드
	String inYmd; // 년월일
	
	< 출력 LIST 항목 >
	String outInstncIdnfr; // 인스턴스식별자
	String outInstncCd; // 인스턴스코드
	String outExstYn; // 존재여부

RTNSTAT 설명	00 : 정상, 08 : 미존재 건수 있음

사용 설명(예시)	// 공유메소드 호출 - 전행 인스턴스코드 존재여부 조회
	(com.kbstar.kji.enbncmn, getEnbnIncdExstYn)
	
	IDataSet getEnbnIncdExstYnReq = new DataSet();
	
	getEnbnIncdExstYnReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEnbnIncdExstYnReq.put("inInquryNoitm", requestData.get("inInquryNoitm"));
	getEnbnIncdExstYnReq.put("LIST", requestData.getRecordSet("LIST"));
	
	// @@ 공유메소드 호출 (전행 인스턴스코드 존재여부 조회 getEnbnIncdExstYn)
	IDataSet getEnbnIncdExstYnRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getEnbnIncdExstYn", getEnbnIncdExstYnReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getEnbnIncdExstYnRes.getString("rtnStat");
	String outinquryNoitm = getEnbnIncdExstYnRes.getString("outInquryNoitm");
	IRecordSet getEnbnIncdExstYnResRsList = getEnbnIncdExstYnRes.getRecordSet("LIST");

비 고	

1.97 전행 인스턴스코드 내용 조회 II(입력일자 없음) (FUBcCode.getEnbnIncdContent2())

프로그램 ID	FUBcCode.getEnbnIncdContent2()		언어 구분	JAVA(○)	
DTO 명	EnbnIncdCtnt2BDTO EnbnIncdCtnt2ListBDTO		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	업무 인스턴스 코드	SQLMAP 명	NAME SPACE	bccode
	영문명	THKJIUI02		SQL ID	getEnbnIncdContent2

프로그램 설명	인스턴스식별자, 인스턴스코드에 맞는 업무 인스턴스 내용을 조회한다.(입력일자 없음)
프로그램 규칙 및 제약사항	1. 구분코드 체크 1) '1' : 업무인스턴스코드 단건 조회 2) '2' : 업무인스턴스코드 다건 조회 2. 구분코드별 처리 1) '1' : 인스턴스식별자, 인스턴스코드를 입력 받아 단건을 조회한다. 2) '2' : 인스턴스식별자를 입력받아 식별자에 해당하는 인스턴스코드를 전체를 조회한다. 3. 인스턴스 변경 이력이 있을 경우 최종 변경분에 대하여 조회하여 출력한다. 4. 인스턴스를 조회하는 현재 시점에 인스턴스코드가 이미 삭제되어 사용 불가능한 경우, 사용가능여부 항목을 '0'으로 SET 한다. - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준일 5. 리턴상태코드 1) '00' : 조회 정상 2) '02' : 조회 자료 NOT FOUND 3) '08' : 사용불가건수있음(사용가능여부 '0')

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcCode **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd ; ● 그룹회사코드
	String inDstcd ; ● 구분코드
	String inInstncIdnfr ; ● 업무인스턴스 식별자
	String inInstncCd ; ◎ 인스턴스 코드
	
	[출력항목]
	String rtnStat; // 리턴상태코드
	int outInquryNoitm ; // 조회건수
	List LIST; // 전행인스턴스 List
	
	< LIST 항목 >
	
	String outInstncCd; // 인스턴스코드
	// 유효시작년월일

	String outValdStartYmd;
	String outValdEndYmd;
	String outUseAbilYn;
	String outInstncCtnt;
	PoC
	PoC
	PoC
		// 유효종료년월일
		// 사용가능여부
		// 인스턴스내용

RTNSTAT 설명	00 : 정상, 02 : Data NotFound, 08 : 사용불가건수있음(사용가능여부 '0')	

사용 설명(예시)	**// 공유메소드 호출 - 전행 인스턴스코드 내용 조회 II(입력일자 없음)**
	(com.kbstar.kji.enbncmn, getEnbnIncdContent2)
	PoC
	IDataSet getEnbnIncdContent2Req = new DataSet();
	
	getEnbnIncdContent2Req.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEnbnIncdContent2Req.put("inDstcd", requestData.get("inDstcd"));
	getEnbnIncdContent2Req.put("inInstncIdnfr", requestData.get("inInstncIdnfr
	getEnbnIncdContent2Req.put("inInstncCd", requestData.get("inInstncCd"));
	
	// @@ 공유메소드 호출 (전행 인스턴스코드 내용 조회 II(입력일자 없음)
	IDataSet getEnbnIncdContent2Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getEnbnIncdContent2", getEnbnIncdContent2Req, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getEnbnIncdContent2Res.getString("rtnStat");
	String outinqurynoitm = getEnbnIncdContent2Res.getString("outInquryNoitm");
	PoC IRecordSet getEnbnIncdContent2ResRsList = getEnbnIncdContent2Res.getRecordSet("LIST");	

비 고		

1.98

PoC

PoC

VIP 라운지 여부 조회 (N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	VIP 라운지 여부를 조회한다 PoC
프로그램 규칙 및 제약사항	1. 구분코드 PoC 1) '1': 공통영역을 참조하여 조회 한다 2. 구분코드별 처리 1) '1': 입력 없이 공통영역을 참조하여 VIP 라운지여부를 조회한다 3. 구분코드 '1' 인경우 거래전처리 및 업무 전처리가 끝나고 호출 되어야 한다 공통역역을(BICOM)참조 하기 때문에 그 이전에 호출 되면 부정확 한 값이 출력 될 수 있다 4. SOD 사용자유형구분코드(BICOM-SOD-USER-PTRN-DSTCD)의 VIP 라운지와 관련된 사용자유형에 해당될 때 VIP 라운지 여부를 SET 한다. 1) 10: SOD 분리상판팀장 2) 12: SOD 분리기업금융팀장 3) 14: SOD 분리 VM 4) 26: JR. VM 5. 리턴상태코드 1) '00' : 조회 정상 2) '09' :일반적인인 오류 인 경우 '09(오류)' PoC 를 전달한다.
비 고	PoC

1.99

PoC

직원 존재여부 다건 조회(FUBcEmployee.getEmpExstYn())

프로그램 ID	FUBcEmployee.getEmpExstYn()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	직원기본	SQLMAP 명	NAME SPACE	fuBCEmployee
	영문명	THKJIHR01		SQL ID	selectEmpExstYn

PoC

프로그램 설명	직원번호 복수개를 입력 받아, 직원기본 테이블에 존재여부를 확인 해 준다 PoC
프로그램 규칙 및 제약사항	1. 구분코드 '1' : 직원 존재여부 다건조회(퇴직직원포함) 2. 입력항목의 직원을 찾지 못하면 (SQLCODE=100: 데이터없음) 출력항목별로 존재여부에 '0'(미존재), 찾으면 '1'(존재)을 출력한다 3. 한 건이라도 미 존재건수가 있으면 리턴상태코드는 '08'(미존재건수 있음)을 전달하고 해당되는 직원번호는 입력 그대로 출력 해 준다 PoC 5. 직원정보는 출력하지 않는다(성능 문제)

입출력 설명	**package** com.kbstar.kji.enbncmn.biz; PoC
	
	public class FUBcEmployee **extends** com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd ; ● 그룹회사코드
	List LIST; ● 입력 List
	
	[출력항목] PoC
	String rtnStat; // 리턴상태코드
	List LIST; // 출력 List
	
	< 입력 LIST 항목 >
	String inEmpId; // 직원번호
	
	< 출력 LIST 항목 >
	String outEmpid; // 직원번호
	String outExstYn; // 존재유무

RTNSTAT 설명	00 : 정상, 08: 미존재 직원번호 건수가 있는경우

사용 설명(예시)	**// 공유메소드 호출 - KB직원존재여부 다건조회 (com.kbstar.kji.enbncmn, getEmpExstYn)**
	
	IDataSet getEmpExstYnReq = **new** DataSet();
	
	getEmpExstYnReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEmpExstYnReq.put("inDstcd", requestData.get("inDstcd"));
	getEmpExstYnReq.put("InInquryNoitm", requestData.get("InInquryNoitm"));
	getEmpExstYnReq.put("LIST", requestData.getRecordSet("LIST"));
	
	// @@ 공유메소드 호출 (KB직원존재여부 다건조회 getEmpExstYn)
	IDataSet getEmpExstYnRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", PoC

	"FUBcEmployee.getEmpExstYn", getEmpExstYnReq, onlineCtx); // @@ 출력 데이터 셋 처리 String rtnstat = getEmpExstYnRes.getString("rtnStat"); String outlistcnt = getEmpExstYnRes.getString("outListCnt"); IRecordSet getEmpExstYnResRsList = getEmpExstYnRes.getRecordSet("LIST");
비 고	

PoC

1.100

PoC

PoC

PoC

PoC

PoC

직원별 담당어플리케이션코드 및 처리권한 여부 확인 (FUBcEmployee.getEmpPerUapplCdPrcssAthYn())

PoC

프로그램 ID	FUBcEmployee. getEmpPerUapplCdPrcssAthYn()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	IT직원기본	SQLMAP 명	NAME SPACE	fuBCEmployee
	영문명	THKJIHR03		SQL ID	

PoC

프로그램 설명	IT 직원정보에 속한 직원중에 직원번호로 현재 등록된 담당 어플리케이션 코드를 조회하고 입력된 어플리케이션에 의한 처리권한을 확인하는 프로그램이다.
프로그램 규칙 및 제약사항	1. 작업구분코드는 아래와 같다. 1) '1' : 요청된 직원번호의 소속어플리케이션코드를 조회한다. 2) '2' : 요청된 직원번호의 소속어플리케이션코드를 조회하여 리턴하고, 입력된 어플리케이션코드에 대한 업무권한이 있는지 여부를 결과값으로 반환한다. (입력된 어플리케이션코드가 결과값으로 SET 된 어플리케이션코드들 중에 존재하면 업무 권한이 있는 것으로 판단함) 2. 필수 입력항목은 다음과 같다. 1) 반드시 직원번호는 필수입력 하여야 한다. 2) 작업구분코드가 '2'인 경우는 직원번호와 어플리케이션코드를 필수입력 하여야 한다. 3. 년월일이 입력되지 않은 경우에는 거래기준일을 입력한것으로 간주한다. 4. IT 직원번호 별 소속어플리케이션코드는 한 번에 100 개까지 가져올 수 있다. 5. 입력항목 작업구분코드가 '2'인 경우는 출력항목 처리여부결과 항목에 '0'-처리가능 '1'-처리불능 이 필수값으로 SET 되어 결과값으로 반환된다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcEmployee extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` // 그룹회사코드
	`String inDstcd;` // 입력구분코드
	`String inEmpid;` // 입력직원번호
	`String inApplCd;` // 어플리케이션코드
	`String inYmd;` // 입력년월일
	
	[출력항목]
	`List LIST;` // 출력 List
	`String outResult;` // 처리여부결과
	
	< LIST 항목 >
	`String outApplCd;` // 어플리케이션코드
	`String outEmpHanglFname;` // 직원한글성명

RTNSTAT 설명	N/A
사용 설명(예시)	// 공유메소드 호출 - 직원별 담당어플리케이션코드 및 처리권한 여부 확인 (com.kbstar.kji.enbncmn, getEmpPerUapplCdPrcssAthYn)

PoC

```
IDataSet getEmpPerUapplCdPrcssAthYnReq = new DataSet();

getEmpPerUapplCdPrcssAthYnReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
getEmpPerUapplCdPrcssAthYnReq.put("inDstcd", requestData.get("inDstcd"));
getEmpPerUapplCdPrcssAthYnReq.put("inEmpid", requestData.get("inEmpid"));
getEmpPerUapplCdPrcssAthYnReq.put("inYmd", requestData.get("inYmd"));
getEmpPerUapplCdPrcssAthYnReq.put("inApplCd", requestData.get("inApplCd"));

//@@ 공유메소드 호출 (직원별 담당어플리케이션코드 및 처리권한 여부 확인
IDataSet getEmpPerUapplCdPrcssAthYnRes = callSharedMethodByDirect(
        "com.kbstar.kji.enbncmn", "FUBcEmployee.getEmpPerUapplCdPrcssAthYn",
                                    getEmpPerUapplCdPrcssAthYnReq, onlineCtx);

// @@ 출력 데이터 셋 처리
String outresult = getEmpPerUapplCdPrcssAthYnRes.getString("outResult");
String outlistcnt = getEmpPerUapplCdPrcssAthYnRes.getString("outListCnt");
IRecordSet getEmpPerUapplCdPrcssAthYnResRsList =
                                getEmpPerUapplCdPrcssAthYnRes.getRecordSet("LIST");
```

비 고	

PoC PoC PoC PoC

1.101

PoC PoC

전행 인스턴스코드 내용 조회Ⅲ (복수건조회-입력일자 없음)

(FUBcCode.getEnbnIncdContent3())

프로그램 ID	FUBcCode.getEnbnIncdContent3( )		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	업무 인스턴스 코드	SQLMAP 명	NAME SPACE	fubccode
	영문명	THKJIUI02		SQL ID	selectEnbnIncdContent3

프로그램 설명	인스턴스식별자, 인스턴스코드에 맞는 업무 인스턴스 내용을 조회한다.(입력일자 없음)
프로그램 규칙 및 제약사항	1. 인스턴스식별자, 인스턴스코드를 입력 받아 조회요청 건수만큼 조회할 수 있다. 2. 인스턴스 변경 이력이 있을 경우 최종 변경분을 출력한다. 3. 인스턴스를 조회하는 현재 시점에 인스턴스코드가 이미 삭제되어 사용 불가능한 경우, 사용가능여부 항목을 '0'으로 SET 한다. - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준일 4. 입력항목의 인스턴스식별자, 인스턴스코드에 대한 개별 조회를 하여 찾지 못하면 (SQLCODE=100: 데이터없음) 출력항목별로 사용가능여부에 '2'(불가)를 출력한다. 5. 한 건이라도 불가능건수가 있으면 리턴상태코드는 '08'(사용불가건수 존재)를 전달하고 해당되는 인스턴스식별자와.인스턴스코드를 입력 그대로 출력 해준다

입출력 설명

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcCode extends
                    com.kbstar.sqc.base.FunctionUnit

[입력항목]
  String inGroupCoCd      ;          ● 그룹회사코드
  int inInquryDmnoNoitm ;            ● 조회요청건수
  List inInstncIdnfr  ;              ● 업무인스턴스 식별자
  List inInstncCd     ;              ● 인스턴스 코드

[출력항목]
  String rtnStat;                    // 리턴상태코드
  int outInquryRsultNoitm    ;       // 조회결과건수
  List LIST;                         // 전행인스턴스List

  < LIST 항목 >
  String outInstncIdnfr;             // 인스턴스식별자
  String outInstncCd;                // 인스턴스코드
  String outValdStartYmd;            // 유효시작년월일
  String outValdEndYmd;              // 유효종료년월일
  String outUseAbilYn;               // 사용가능여부
```

	String outInstncCtnt;	// 인스턴스내용
RTNSTAT 설명	00 : 정상, 08 : 사용불가능여부에 '0'이 1건이라도 있는경우 or 미존재 인스턴스식별자의 경우	

사용 설명(예시)	// 공유메소드 호출 - 전행 인스턴스코드 내용 조회 III(복수건조회-입력일자 없음)
	(com.kbstar.kji.enbncmn, getEnbnIncdContent3)
	
	IDataSet getEnbnIncdContent3Req = new DataSet();
	
	getEnbnIncdContent3Req.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEnbnIncdContent3Req.put("inInquryDmnoNoitm", requestData.get("inInquryDmnoNoitm"));
	getEnbnIncdContent3Req.put("LIST1", requestData.getRecordSet("LIST1"));
	getEnbnIncdContent3Req.put("LIST2", requestData.getRecordSet("LIST2"));
	
	// @@ 공유메소드 호출 (전행 인스턴스코드 내용 조회 III(복수건조회-입력일자 없음)
	IDataSet getEnbnIncdContent3Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getEnbnIncdContent3", getEnbnIncdContent3Req, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getEnbnIncdContent3Res.getString("rtnStat");
	String outinquryrsultnoitm = getEnbnIncdContent3Res.getString("outInquryRsultNoitm");
	IRecordSet getEnbnIncdContent3ResRsList = getEnbnIncdContent3Res.getRecordSet("LIST");	

비 고		

PoC

1.102

PoC

PoC

PoC

PoC

휴일 Check - 휴일테이블 사용안함 (N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력일자에대한 휴일구분을제공 (공통업무팀 내부용으로 업무팀에서 사용 금지)
프로그램 규칙 및 제약사항	1. 성능관련 만들어진 휴일체크 모듈 - (U-Kesa(서버) 는 해당없음) 2. 휴일테이블(THKJIH001)을 사용하지 않고 copybook(YWJIH001)에 휴일 정보를 만들어 사용 3. 휴일 변경시 아래 순서에 의해 진행 - 인사부에 시행문 발송 요청 - 적용일정 확정하여 전산정보그룹 전체 메시지 공지(z-Kesa,u-Kesa) - 프로젝트 등록(자체개선) - 개발(ZADA) 테이블(THKJIH001)에 휴일 변경 추가(개발 CRUD 화면) - DATASET 확인 (없으면생성) : KJI.DT.TKJIH001.COPYBOOK.BACK (Record length-80,CYLS-10) KJI.DT.TKJIH001.COPYBOOK.BACK01 (Record length-80,CYLS-10) - KJI.DBATCH.JCL(BJIHOCP) 실행 : THKJIH001 테이블을 READ 하여 COPYBOOK(YWJIH001)생성 - CJIDT23 COMP & LINK - 테스트 진행 - 운영 CRUD 화면을 이용 운영(ZAPA) 휴일 변경 (THKJIH001) - CJIDT23, COPYBOOK(YWJIH001) CM 적용 - u-Kesa(서버)는 테이블(THKJIH001)변경 만 있음(배포로 적용) * 타팀사용불가 사유 : 기존 휴일체크 (CJIDT02)내부에서 CJIDT23 을 호출하여 휴일 정보 활용
비 고	

PoC PoC PoC PoC PoC

1.103

PoC PoC PoC

PoC

부점한글명 조회(FUBcBranch.getBrnHanglName())

프로그램 ID	FUBcBranch.getBrnHanglName()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR01		SQL ID	selectBrnHanglName selectClsrBrnHanglName

프로그램 설명	조회 요청일자 시점의 부점기본 정보를 조회한다
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 부점코드는 숫자로 입력되어야 한다. - 부점코드를 '0000' 입력하면 오류를 발생한다. 2. 일자에 대한 부점 기본 정보 조회 1) 년월일 입력 : 입력된 일자 당시의 부점 정보 2) 년월일 미입력 : 현재시점의 부점정보 조회 - 현재시점 : Common Area 에 포함되어있는 KB 기준정보의 거래처리기준년월일 3) 적용시작일자 및 적용종료일자 기준으로 조회 된다 (단 개점년월일이 적용시작일자보다 클 수 있고, 폐쇄년월일이 적용종료일자보다 +1 이므로 반드시 부점기본 정보에 포함되어 있는 개점년월일과 폐쇄년월일을 확인 해야 한다.) 3. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일은 01 ~ 31 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다. 4. 입력된 정보를 기준으로 부점기본 테이블을 조회하여 찾지 못하면 폐쇄부점인지 확인한다. 5. 폐쇄부점일경우 리턴상태코드를 '07' 로 한다. - 폐쇄부점의 최종 통합부점을 조회하려면 부점 폐쇄에 의한 통합점 조회 (FUBcBranch.getIntegrationBranch())를 한다. - 폐쇄부점도 부점기본 정보와 동일하게 출력한다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcBranch extends`
	`com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd ;` ● 그룹회사코드
	`String inBrncd ;` ● 부점코드
	`String inYmd ;` ◎ 년월일
	
	[출력항목]
	`String rtnStat;` // 리턴상태코드
	`String outBrnHanglName;` // 부점한글명

RTNSTAT 설명	00 : 정상, 07 : 입력:부점이 폐쇄점인 경우
사용 설명 (예시)	**// 공유메소드 호출 - 부점한글명 조회 (com.kbstar.kji.enbncmn, getBrnHanglName)** `IDataSet getBrnHanglNameReq = new DataSet();`

PoC

	getBrnHanglNameReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getBrnHanglNameReq.put("inBrncd", requestData.get("inBrncd"));
	getBrnHanglNameReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (부점한글명 조회 getBrnHanglName)
	IDataSet getBrnHanglNameRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getBrnHanglName", getBrnHanglNameReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getBrnHanglNameRes.getString("rtnStat");
	String outbrnhanglname = getBrnHanglNameRes.getString("outBrnHanglName");

비 고	

PoC

PoC

1.104

PoC

PoC

PoC

전행 인스턴스코드 내용 조회 IV(인스턴스내용출력안함) (FUBcCode.getEnbnIncdContent4())

PoC

프로그램 ID	FUBcCode.getEnbnIncdContent4()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	업무 인스턴스 코드	SQLMAP 명	NAME SPACE	fubccode
	영문명	THKJIUI02		SQL ID	selectEnbnIncdContent4 selectMultiEnbnIncdContent4

PoC

프로그램 설명	인스턴스식별자, 인스턴스코드에 맞는 업무 인스턴스 내용을 조회한다.(입력일자 없음)
프로그램 규칙 및 제약사항	1. 인스턴스식별자, 인스턴스코드를 입력 받아 조회요청 건수만큼 조회할 수 있다. 2. 인스턴스 변경 이력이 있을 경우 최종 변경분을 출력한다. 3. 인스턴스를 조회하는 현재 시점에 인스턴스코드가 이미 삭제되어 사용 불가능한 경우, 사용가능여부 항목을 '0'으로 SET 한다. - 현재시점 : Common Area 에 포함되어 있는 KB 기준정보의 거래처리기준일 4. 입력항목의 인스턴스식별자, 인스턴스코드에 대한 개별 조회를 하여 찾지 못하면 (SQLCODE=100: 데이터없음) 출력항목별로 사용가능여부에 '2'(불가)를 출력한다. 5. 한 건이라도 불가능건수가 있으면 리턴상태코드는 '08'(사용불가건수 존재)를 전달하고 해당되는 인스턴스식별자와.인스턴스코드를 입력 그대로 출력 해준다

PoC

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcCode extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd ;
	int inInquryDmnoNoitm ;
	List LIST1 ;
	List LIST2 ;
	
	[출력항목]
	String rtnStat;
	int outInquryRsultNoitm ;
	List LIST;
	
	< LIST1 항목 >
	String inInstncIdnfr;
	
	< LIST2 항목 >
	String inInstncCd;
	
	< 출력 LIST 항목 >
		PoC
		
		● 그룹회사코드
		● 조회요청건수
		● 입력 List1
		● 입력 List2
		
		// 리턴상태코드
		// 조회결과건수
		// 전행인스턴스 List
		
		// 업무인스턴스 식별자
		
		// 인스턴스코드

PoC

PoC

This document is strictly limited to the 2025 C2J conversation PoC

PoC

	String outInstncIdnfr;
	String outInstncCd;
	String outValdStartYmd;
	String outValdEndYmd;
	String outUseAbilYn;
		// 인스턴스식별자
		// 인스턴스코드
		// 유효시작년월일
		// 유효종료년월일
		// 사용가능여부

RTNSTAT 설명	00 : 정상, 08 : 사용불가능여부에 '0'이 1 건이라도 있는경우 or 미존재 인스턴스식별자의 경우	

사용 설명(예시)	// 공유메소드 호출 - 전행 인스턴스코드 내용 조회 IV(인스턴스내용 출력안함)
	(com.kbstar.kji.enbncmn, getEnbnIncdContent4)
	
	IDataSet getEnbnIncdContent4Req = new DataSet();
	
	getEnbnIncdContent4Req.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEnbnIncdContent4Req.put("inInquryDmnoNoitm", requestData.get("inInquryDmnoNoitm"));
	getEnbnIncdContent4Req.put("LIST1", requestData.getRecordSet("LIST1"));
	getEnbnIncdContent4Req.put("LIST2", requestData.getRecordSet("LIST2"));
	
	// @@ 공유메소드 호출 (전행 인스턴스코드 내용 조회 IV(인스턴스내용 출력안함)
	IDataSet getEnbnIncdContent4Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcCode.getEnbnIncdContent4", getEnbnIncdContent4Req, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getEnbnIncdContent4Res.getString("rtnStat");
	String outinqurynoitm = getEnbnIncdContent4Res.getString("outInquryNoitm");
	IRecordSet getEnbnIncdContent4ResRsList = getEnbnIncdContent4Res.getRecordSet("LIST");	

비 고		

PoC

PoC

1.105 신분증이미지 Key 조립(N/A)

프로그램 ID	N/A		언어 구분	N/A
			계열사구분	N/A
COPY BOOK	N/A		사용 구분	N/A
			공통 유형	N/A
사용 TABLE 명	한글명	N/A	TABLE COPY BOOK	N/A
	영문명	N/A		

프로그램 설명	신분증 이미지 키 조립을 내부표준전문의 매체부를 읽어 처리한다.
프로그램 규칙 및 제약사항	1. 매체부 세부항목 의 입력 값을 INPUT 으로 하여 신분증 키 조립 [축약값 구성] PPR 서버거래시각(6) + 대상구분(1) + 대상일련번호(1)⇒ 8 자리 [조합된 Key 구성] 거래코드(10) + PPR 서버거래일시(14) + 영업점코드(4) + 영업점직원코드 (7) + 대상구분 (1) + 대상일련번호(1) ⇒ 37 자리 2. 매체부의 신분증스캔구분이 ‘SCAN’으로 입력된 경우만 신분증 이미지 Key 변환 조립처리를 한다. 3. 매체부의 신분증스캔구분 ‘SCAN’이 아닌 경우 정상 종료처리 한다. 4. 조립 키 건수가 ‘00’이라도 리턴상태는 ‘00’ 정상으로 출력 해 준다. 5. 내부표준전문의 매체부 영역 설명 **[실명증표 이미지 Key 정보]** 201 209 217 249 257 이미지 Key1(8) | 이미지 Key2(8) | …… … | 이미지 Key7(8) | 이미지 Key8(8) ※ 1. **축약이미지 Key (8)** : PPR 서버거래시각(6)+대상구분(1)+대상일련번호(1) 2. **실명증표 이미지 Key (37)** : 거래코드(10)+PPR 서버거래일시(14)+영업점코드(4)+영업점직원코드(7)+대상구분(1)+대상일련번호(1) [실명증표 스캔 여부] **스캔구분(5)** ※ SCAN1 = 일반스캔 SCAN2 = 추가스캔 SCAN3 = 재사용 [실명증표 스캔필 인자정보] 270 272 282 295 297 **신분증구분(2) | 성명(10) | 고객번호(13) | 스캔건수(2) | 진위확인필(1)** ※ 신분증구분 정의
비 고	

인스턴스	장표(5 자리)	전표(3 자리)
01=주민등록증	주민등록증	주민증
02=운전면허증	운전면허증	운전증
03=외국인등록증	외국인증	외국인
04=외국국적동포거소신고증	외국국적증	거소외
05=재외국민국내거소신고증	재외국민증	거소재
06=공무원증	공무원증	공무원
07=노인복지카드	노인복지증	복지노
08=장애인복지카드	장애인증	복지장
09=기타	기타	기타

1.106 실명증표 이미지키 저장 및 조회(N/A)

프로그램 ID	N/A		언어 구분	N/A
			계열사구분	N/A
COPY BOOK	N/A		사용 구분	N/A
			공통 유형	N/A
사용 TABLE 명	한글명	N/A	TABLE COPY BOOK	N/A
	영문명	N/A		

프로그램 설명	실명증표 이미지 Key 내역을 저장 및 조회 한다.
프로그램 규칙 및 제약사항	1. 실명증표 스캐닝 시 발생된 실명증표 이미지 Key 정보는 상품처리계(Mainframe)에 집중하여 저장관리 한다. 2. 실명증표 스캐닝시 발생되는 이미지 Key 정보(최대 8 개)는 내부표준전문의 매체부(300byte 중 201byte 부터 100byte)에 값을 읽어 실명증표축약이미지 Key(8bytes)를 실명증표이미지 Key(37bytes)로 변환하여 사용한다. 3. 이미지 추가연결(이미지저장구분 “3. 추가연결”)은 실명증표 스캔을 바로처리하지 않고 나중에 기 등록된 이미지 Key 정보에 추가연결하는데 사용한다. 1) 이미지 Key 영역은 최대 8 개까지 저장되며 순차적으로 빈 Key 영역에 저장된다. 2) 이미지 Key 정보 저장 당일에 한하여 추가연결등록 거래를 할 수 있으며 일자가 경과한 경우 등록할 수 없다. 4. 단말거래와 관련된 매체에 한하여 이미지 Key 내역을 관리하며 그외 매체는 Skip 처리 한다.(채널(BICOM-CHNL-DSTCD) ‘01’ 단말, ‘06’ 대외 만 허용) 이때 리턴코드는 ‘00’ 정상 으로 처리한다. 5. 연동거래의 수동일 경우 이미지저장구분이 복원(4)이 아닐 경우 Skip 처리하며, 리턴코드는 ‘00’ 정상 으로 처리한다. 6. 대외거래 또는 여신신규거래인 경우 기 등록된 이미지 Key 복원용 정보를 읽어 새로운 이미지 Key 내역을 저장(이미지저장구분 “4. 복원정보”)하며 “실명증표 이미지키 복원 저장(CJIIM03)” 유틸리티를 이용하여 이미지 Key 복원용 정보를 저장한다. 1) 실명증표 이미지 Key 정보를 이미지식별번호에 연결하여 저장하여야 되나 이미지식별번호가 확정되지 않아 임시로 복원용 정보에 저장하였다가 최종확정시 복원정보를 읽어 이미지 Key 내역을 저장 관리한다. 2) 복원식별번호로 기 저장된 이미지 Key 정보를 읽어 실명증표이미지 Key 를 조립하고 이미지 Key 내역을 등록 처리한다. 3) 이미지 Key 복원용 정보의 조회대상제외여부(RJIGB01_INQURY-T-ECLUD-YN)에 ‘1’을 자동으로 Set 처리한다. 4) 이미지식별번호 : 최종이미지키 저장을 위한 번호(고객번호, 계좌번호 등)를 SET 한다. 5) 복원식별번호 : 대외기관 송신전문의 전문번호 또는 여신승인신청번호를 SET 한다. 7. 본 유틸리티가 정상처리되어 수행되면 JICOM 영역에 이미지키저장호출여부(JICOM-IMG-KS-CALLG-YN)에 ‘1’을 Set 한다. 8. 이미지 Key 내역정보를 PPR 서버에 전달하기 위하여 후처리 로그조립을 한다. 9. 실명증표 이미지 Key 내역 조회의 역순조회는 항상 100 건까지만 조회 된다. 10. 구분코드 아래와 같다. 1) ‘1’ : 실명증표 이미지 Key 내역을 저장한다. 2) ‘2’ : 실명증표 이미지 Key 내역을 조회한다. 11. 구분코드 “1. 실명증표 이미지 Key 내역을 저장” 관련 유의사항 1) 거래일시가 입력되면 입력된 거래일시를 사용하고 입력 안하면 시스템 거래일시를 사용하여 이미지 Key 내역정보를 저장한다.

2) 연동거래인 경우 거래코드/거래명은 기동거래코드(연동대표코드)를 사용하여 저장한다.
3) 실명증표이미지 Key1~8 입력되면 입력된 값을 사용하고 입력 안하면 매체부의 실명증표 축약이미지 Key1~8 을 변환하여 이미지 Key 내역정보를 저장한다. 이때 기존에 등록된 이미지 Key 정보는 모두 없어지고 입력한 이미지 Key 정보로 채워진다.
4) 중단거래일 경우 Skip 처리하며, 리턴코드는 '00' 정상 으로 처리한다.

12. 이미지식별자구분코드는 아래와 같다.
1) '00' : 해당없음
2) '01' : 고객번호
3) '02' : 계좌번호
4) '03' : 카드번호
5) '04' : 관리번호
6) '05' : 여신승인신청번호
7) '06' : 가맹점번호
8) '07' : 전문번호
9) '99' : 기타

13. 이미지식별번호는 이미지식별자구분코드에 해당하는 값을 SET 한다.

14. 이미지저장구분코드는 아래와 같다.
1) '0' : 해당없음
2) '1' : 신규
3) '2' : 갱신
4) '3' : 추가연결
5) '4' : 복원정보

15. 이미지거래구분코드는 아래와 같다.
1) '01' : 신규
2) '02' : 등록/해제/변경
3) '03' : 제신고
4) '04' : 입/출금
5) '05' : 심사승인
6) '06' : 취소
7) '07' : 정정취소
8) '08' : 정정
9) '09' : 조회
10) '10' : 해약/해지
11) '11' : 전입/전출
12) '12' : 제증명
13) '13' : 재발행
14) '14' : 평가
15) '15' : 상담
16) '16' : 신청/접수
17) '99' : 기타

16. 역순조회구분코드는 아래와 같다.
1) ' ' : 해당없음
2) '0' : 해당없음
3) '1' : 역순조회

17. 입력파라메터 항목명 설명

구분	항목명	필수/임의
이미지저장	그룹회사코드	필수
	부점코드	필수
	이미지식별자구분코드	필수
	이미지식별번호	필수
	이미지저장구분코드	필수
	이미지거래구분코드	필수
	복원식별번호	임의
	거래일시	임의
	실명증표이미지 Key1	임의
	실명증표이미지 Key2	임의
	실명증표이미지 Key3	임의
	실명증표이미지 Key4	임의
	실명증표이미지 Key5	임의
	실명증표이미지 Key6	임의
	실명증표이미지 Key7	임의
	실명증표이미지 Key8	임의

PoC

이미지조회	그룹회사코드	필수
	부점코드	임의
	이미지식별자구분코드	필수/임의
	이미지식별번호	필수/임의
	사용자번호	임의
	Next 시작일시	임의
	역순조회구분코드	임의

※ 일반조회시 이미지식별자구분코드와 이미지식별번호는 반드시 필수입력하여야 DB FULL SCAN 으로 TIME OVER 가 발생하지 않음.

18. 출력파라메터 항목명 설명

출력구분	설명		필수/임의
이미지저장	- 리턴코드 출력 : '00' 정상, '07' 기등록, '09' 오류		필수
이미지조회	이미지키내역건수		필수
	Next 시작일시		임의
	100 개	부점코드	필수
		이미지식별번호	
		거래일시	
		이미지식별자구분코드	
		이미지거래구분코드	
		복원식별번호	
		조회대상제외여부	
		거래코드	
		거래명	
		스캔여부	
		신규여부	
		갱신여부	
		추가연결여부	
		이미지 Key 개수	
		이미지 Key1	
		이미지 Key2	
		이미지 Key3	
		이미지 Key4	
		이미지 Key5	
		이미지 Key6	
		이미지 Key7	
		이미지 Key8	
		GUID 번호	
		시스템최종처리일시	
		시스템최종사용자번호	

PoC PoC PoC PoC

비 고	

PoC

1.107 실명증표 이미지키 복원저장(N/A)

프로그램 ID	N/A		언어 구분	N/A
			계열사구분	N/A
COPY BOOK	N/A		사용 구분	N/A
			공통 유형	N/A
사용 TABLE 명	한글명	N/A	TABLE COPY BOOK	N/A
	영문명	N/A		

프로그램 설명	실명증표 이미지 Key 내역을 복원하기 위하여 저장한다.
프로그램 규칙 및 제약사항	1. 실명증표 스캐닝 시 발생된 실명증표 이미지 Key 정보는 Unique 한 이미지식별번호로 복원을 위하여 저장관리 한다. 1) 이미지식별번호(전문번호, 여신승인신청번호) + 부점코드(‘ 0000 ’) + 거래일시(‘ 99991231240000 ’) 2) 부점코드와 거래일시는 데이터를 입력하지 않는다. 2. 실명증표 스캐닝시 발생되는 이미지 Key 정보(최대 8 개)는 내부표준전문의 매체부(300byte 중 201byte 부터 100byte)에 값을 읽어 실명증표축약이미지 Key(8bytes)를 실명증표이미지 Key(37bytes)로 변환하여 사용한다. 3. 단말거래와 관련된 매체에 한하여 이미지 Key 내역을 관리하며 그외 매체는 Skip 처리 한다.(채널(BICOM-CHNL-DSTCD) ‘ 01 ’ 단말, ‘ 06 ’ 대외 만 허용) 이때 리턴코드는 ‘ 00 ’ 정상 으로 처리한다. 4. 이미지 Key 내역정보가 기 등록되어 있는 경우 기존 이미지 Key 값은 Clear 되고 최종의 이미지 Key 값만 유지된다. 1) 단말화면의 이미지스캔 버튼 클릭시 PPR 서버의 실명증표 이미지 Key 정보를 읽어 재스캔 및 저장하여 송신하면 최종 이미지 Key 정보를 만들어 저장한다. 2) 이때 이미지 Key 정보가 추가 또는 삭제 된다. 5. 이미지저장구분코드에 따라 조회대상제외여부(RJIGB01_INQURY-T-ECLUD-YN) 값 SET 가 달라진다. 1) ‘ 1 ’ 신규 : 여신승인신청번호가 해당되며 조회대상제외여부 ‘ 0 ’ 을 SET 한다. 2) ‘ 4 ’ 복원정보: 대외전문번호가 해당되며 조회대상제외여부 ‘ 1 ’ 을 SET 한다. 6. 조회대상제외여부(RJIGB01_INQURY-T-ECLUD-YN) 값이 ‘ 1 ’ 인 경우 이미지거래내역 등록 및 조회거래(KJI0696341)에서 조회되지 않는다. 7. 이미지 Key 내역정보를 PPR 서버에 전달하기 위하여 후처리 로그조립을 한다. 8. 실명증표이미지 Key1~8 입력되면 입력된 값을 사용하고 입력 안하면 매체부의 실명증표 축약이미지 Key1~8 을 변환하여 이미지 Key 내역정보를 저장한다. 이때 기존에 등록된 이미지 Key 정보는 모두 없어지고 입력한 이미지 Key 정보로 채워진다. 9. 이미지식별자구분코드는 아래와 같다. 1) ‘ 00 ’ : 해당없음 2) ‘ 01 ’ : 고객번호 3) ‘ 02 ’ : 계좌번호 4) ‘ 03 ’ : 카드번호 5) ‘ 04 ’ : 관리번호 6) ‘ 05 ’ : 여신승인신청번호 7) ‘ 06 ’ : 가맹점번호 8) ‘ 07 ’ : 전문번호 9) ‘ 99 ’ : 기타 10. 이미지식별번호는 이미지식별자구분코드에 해당하는 값을 SET 한다. 11. 이미지저장구분코드는 아래와 같다. 1) ‘ 0 ’ : 해당없음

2) ‘1’ : 신규
3) ‘4’ : 복원정보

12. 이미지거래구분코드는 아래와 같다.
1) ‘01’ : 신규
2) ‘02’ : 등록/해제/변경
3) ‘03’ : 제신고
4) ‘04’ : 입/출금
5) ‘05’ : 심사승인
6) ‘06’ : 취소
7) ‘07’ : 정정취소
8) ‘08’ : 정정
9) ‘09’ : 조회
10) ‘10’ : 해약/해지
11) ‘11’ : 전입/전출
12) ‘12’ : 제증명
13) ‘13’ : 재발행
14) ‘14’ : 평가
15) ‘15’ : 상담
16) ‘16’ : 신청/접수
17) ‘99’ : 기타

13. 입력파라메터 항목명 설명

구분	항목명	필수/임의
복원저장	그룹회사코드	필수
	이미지식별자구분코드	필수
	이미지식별번호	필수
	이미지저장구분코드	필수
	이미지거래구분코드	필수
	실명증표이미지 Key1	임의
	실명증표이미지 Key2	임의
	실명증표이미지 Key3	임의
	실명증표이미지 Key4	임의
	실명증표이미지 Key5	임의
	실명증표이미지 Key6	임의
	실명증표이미지 Key7	임의
	실명증표이미지 Key8	임의
	거래코드	임의
	취소구분코드	임의

14. 출력파라메터 항목명 설명

출력구분	설명	필수/임의
복원저장	- 리턴코드 출력 : ‘00’ 정상, ‘07’ 기등록, ‘09’ 오류	필수

비 고	

PoC

1.108

전자공용전표 판단규칙(N/A)

프로그램 ID	N/A		언어 구분	N/A
			계열사구분	N/A
COPY BOOK	N/A		사용 구분	N/A
			공통 유형	N/A
사용 TABLE 명	한글명	N/A	TABLE COPY BOOK	N/A
	영문명	N/A		

PoC

프로그램 설명	전자공용전표판단 규칙에 의하여 전자공용전표여부를 제공한다.
프로그램 규칙 및 제약사항	1. 전자공용전표 대상거래는 본 유틸리티를 반드시 수행하여야 하며 “전표번호 채번 & 유/무전표 자동판단 유틸리티(이하 CJICN05)” 수행 이전에 호출되어야 한다. 2. 전자공용전표판단 규칙을 수행하여 전자공용전표여부(JICOM-ELCTR-CNUS-SLIP-YN)에 판단된 여부값을 셋팅을 하고 이후 수행되는 CJICN05 유틸리티는 이를 참조한다. 3. 전표구분코드(1. 유전표, 3.무전표, 5.전자공용전표) 결정은 CJICN05 유틸리티에서 최종 결정된다. (상세내용은 CJICN05 유틸리티 설명서 참조) 4. 입력파라메터 설명

입력항목		설명	비고
그룹회사코드		그룹회사코드를 SET 한다.	필수
목록형 배열	목록형결정조건명	- 목록형 항목명을 입력 (띄어쓰기 금지) 예) 무인감여부, 질권설정계좌여부	최대 20 개까지 입력가능
	목록형결정조건값	- 목록형 항목명의 조건값을 입력 예) Y, N	

5. 출력파라메터 설명

출력항목	설명	비고
전자공용전표여부	전자공용전표여부 정보를 제공한다. 동 내용은 Common Area 전자공용전표여부(JICOM-ELCTR-CNUS-SLIP-YN) 항목에도 SET 된다. 0.부, 1.여	

PoC PoC

COPY BOOK 설명

```
01  XCJICN06-CA.
    03  XCJICN06-RETURN.                                          <리턴항목>
        05  XCJICN06-R-STAT                PIC  X(002).           리턴상태코드
            88  COND-XCJICN06-OK              VALUE  '00'.          정상
            88  COND-XCJICN06-ERROR           VALUE  '09'.          오류
            88  COND-XCJICN06-ABNORMAL        VALUE  '98'.          비정상
            88  COND-XCJICN06-SYSERROR        VALUE  '99'.          시스템오류
        05  XCJICN06-R-LINE                PIC  9(006).           오류위치
        05  XCJICN06-R-ERRCD               PIC  X(008).           오류코드
        05  XCJICN06-R-TREAT-CD            PIC  X(008).           조치코드
        05  XCJICN06-R-SQL-CD PIC  S9(005) LEADING  SEPARATE.     SQLCODE
    03  XCJICN06-IN.                                             <입력항목>
        05  XCJICN06-I-GROUP-CO-CD         PIC  X(003).           ●그룹회사코드
        05  XCJICN06-I-CATLGU-TYP-ARAY OCCURS  20   TIMES.        ◎목록형 배열(20 개)
            07  XCJICN06-I-CATLGU-TD-CNDN-NAME PIC  X(022).        - 목록형결정조건명
            07  XCJICN06-I-CATLGU-TD-CNDN-VAL  PIC  X(014).        - 목록형결정조건값
        05  XCJICN06-I-SPARE               PIC  X(050).           예비
    03  XCJICN06-OUT.                                            <출력항목>
        05  XCJICN06-O-ELCTR-CNUS-SLIP-YN PIC  X(001).            전자공용전표여부
        05  XCJICN06-O-SPARE               PIC  X(050).           예비
```

PoC PoC PoC

비 고	

1.109 전표구분 조회-유/무전표 자동판단(N/A)

PoC

프로그램 ID	N/A		언어 구분	N/A
			계열사구분	N/A
COPY BOOK	N/A		사용 구분	N/A
			공통 유형	N/A
사용 TABLE 명	한글명	N/A	TABLE COPY BOOK	N/A
	영문명	N/A		

PoC

프로그램 설명	단말 사용자별(자동화기기 포함) 전표구분코드를 반환한다.
프로그램 규칙 및 제약사항	1. 부점지주회사구분코드 01-은행 이 아닐경우 오류처리 한다. 2. 사용 방법은 CJICN05 와 동일하다.(사용법 CJICN05 참조) 3. 처리결과는 CJICN05 와 동일하나, 전표 채번을 수행하지 않고 전표구분만을 Set 하여 반환한다.
비 고	

1.110

PoC

PoC

PoC

PoC

This document is strictly limited to the 2025 C2J conversation PoC

PoC

카드인터페이스용 전표채번(N/A)

프로그램 ID	N/A		언어 구분	N/A
			계열사구분	N/A
COPY BOOK	N/A		사용 구분	N/A
			공통 유형	N/A
사용 TABLE 명	한글명	N/A	TABLE COPY BOOK	N/A
	영문명	N/A		

PoC

프로그램 설명	카드인터페이스를 통한 단말 사용자별(자동화기기 포함) 전표일련번호를 카드시스템으로부터 입력받은 전표구분에 의하여 TABLE 을 통해 채번한다.
프로그램 규칙 및 제약사항	1. 부점지주회사구분코드 02-카드 가 아닐경우 오류처리 한다. 2. 전표구분이 '0' 일경우 전표번호(JICOM-SLIP-NO)에 '00001', 전표구분코드(JICOM-SLIP-DSTCD)에 '3-무전표' 을 Set 한다. 3. 채번 규칙은 CJICN05 와 동일하다(CJICN05 참조)
비 고	

PoC

PoC

1.111 고객지정계좌 계좌번호 체크디짓 산출 및 검증(N/A)

프로그램 ID	N/A		언어 구분	N/A
			계열사구분	N/A
COPY BOOK	N/A		사용 구분	N/A
			공통 유형	N/A
사용 TABLE 명	한글명	N/A	TABLE COPY BOOK	N/A
	영문명	N/A		

PoC

프로그램 설명	관리번호 체크디지트 산출 및 검증 결과를 제공 한다.
프로그램 규칙 및 제약사항	1.입력 데이터 검증 1) 구분코드 가) '1' : 체크디지트 산출 나) '2' : 체크디지트 검증 2. 구분코드별 체크디지트 산출 및 검증 규칙 => 검증시 입력데이터의 체크디지트와 산출체크디지트와 불일치시 리턴상태코드를 '07'로 출력한다 1) 체크디지트 산출 및 검증 가) 산출연산합 : 계좌번호 1 번째자리 * 1 + 계좌번호 2 번째자리 * 3 + 계좌번호 3 번째자리 * 1 + 계좌번호 4 번째자리 * 7 + 계좌번호 5 번째자리 * 3 + 계좌번호 6 번째자리 * 1 + 계좌번호 7 번째자리 * 7 + 계좌번호 8 번째자리 * 3 + 계좌번호 9 번째자리 * 1 나) 계좌번호의 각 자릿수 연산시 결과값중 1 의자리만을 합계에 사용한다. 다) 체크디지트 산출 = 100 - 산출연산합 라) 검증일 경우 생된 체크디지트와 계좌번호의 10-11 번째 자리의 수를 비교하여 검증한다. ** 2012.01.31 부터 고객지정계좌 체크디짓 추가 2010.02 ~ 2012.01.12 까지 체크디짓 없이 사용 2012.01.13 ~ 2012.01.30 까지 고객지정 신규발급 중지
비 고	

PoC

PoC

1.112

PoC

PoC

직원권한 및 종료고객조회(FUBcEmployee.getEmpAthEndCust())

프로그램 ID	FUBcEmployee.getEmpAthEndCust ()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(X)	
PoC			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	거래사용자번호에 대한 권한 및 고객식별자에 대한 고객종료여부를 조회한다.
프로그램 규칙 및 제약사항	PoC 1. 조회구분 체크 1) '1' : 권한조회 + 종료고객조회 2) '2' : 권한조회 2. 조회구분별 입력항목을 아래와 같다. 1) '1' : 그룹회사코드, 조회구분, 거래사용자번호, 고객식별자 or 고객고유번호 or 계좌번호 중 1 개 항목은 필수 중복 입력 PoC 경우 우선순위는 식별자->고객고유번호->계좌번호순으로한다. ※ 계좌번호 계약원장만 해당(IHC1006 (IC 수신계약정보조회), DHC2234 (DC 여신계약기본조회 이용) 식별자로 변환하며, 비계약 원장인경우 식별자 변환불가하여 해당업무팀에서 고객식별자 및 고유번호로 입력 해야 한다. 2) '2' : 그룹회사코드, 조회구분, 거래사용자번호 PoC 3. 구분코드별 처리 1) '1' : 거래사용자 권한 및 종료고객 여부를 조회한다. 출력항목) 권한부여여부,권한부여목적,종료고객여부,종료고객년월일,당일거래가능여부, 고객식별자,2 단계권한부여여부,거래종료고객상태구분 2) '2' : 거래사용자 권한여부를 조회한다. 출력항목) 권한부여여부,2 단계권한부여여부 4. 로그처리는 각 서버에서 한다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcEmployee extends`
	`com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	String inGroupCoCd ; ● 그룹회사코드
	String inInquryDstic ; ● 조회구분
	String inTranUserNo ; ● 거래사용자번호
	String inCustIdnfr ; ◎ 고객식별자 PoC
	String inCuniqno ; ◎ 고객고유번호
	String inAcno ; ◎ 계좌번호
	
	[출력항목]
	String outAthGrntYn ; // 권한부여여부
	String outAthGrntObjctv ; // 권한부여목적
	String outEndCustYn ; // 종료고객여부
	String outEndCustYmd ;

	String outTtddTranAbilYn;
	String outCustIdnfr ;
	String outN2AthGrntYn ;
	String outCustEndStusDstcd;
		// 종료고객년월일
		// 당일거래가능여부
		// 고객식별자
		// 2 단계권한부여여부
		// 거래종료고객상태구분

RTNSTAT 설명	N/A	

사용 설명(예시)	//공유메소드 호출-직원권한 및 종료고객조회 (com.kbstar.kji.enbncmn, getEmpAthEndCust)
	
	IDataSet getEmpAthEndCustReq = new DataSet();
	
	getEmpAthEndCustReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEmpAthEndCustReq.put("inInquryDstic", requestData.get("inInquryDstic"));
	getEmpAthEndCustReq.put("inTranUserNo", requestData.get("inTranUserNo"));
	getEmpAthEndCustReq.put("inCustIdnfr", requestData.get("inCustIdnfr"));
	getEmpAthEndCustReq.put("inCuniqno", requestData.get("inCuniqno"));
	getEmpAthEndCustReq.put("inAcno", requestData.get("inAcno"));
	
	// @@ 공유메소드 호출 (직원권한 및 종료고객조회 getEmpAthEndCust)
	IDataSet getEmpAthEndCustRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEmployee.getEmpAthEndCust", getEmpAthEndCustReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outathgrntyn = getEmpAthEndCustRes.getString("outAthGrntYn");
	String outathgrntobjctv = getEmpAthEndCustRes.getString("outAthGrntObjctv");
	String outendcustyn = getEmpAthEndCustRes.getString("outEndCustYn");
	String outendcustymd = getEmpAthEndCustRes.getString("outEndCustYmd
	String outttddtranabilyn = getEmpAthEndCustRes.getString("outTtddTranAbilYn
	String outcustidnfr = getEmpAthEndCustRes.getString("outCustIdnfr");
	String outn2athgrntyn = getEmpAthEndCustRes.getString("outN2AthGrntYn");
	String outcustendstusdstcd = getEmpAthEndCustRes.getString("outCustEndStusDstcd");	

비 고		

1.113 실버바판매환율정보조회(N/A)

프로그램 ID	N/A		언어 구분	N/A
			계열사구분	N/A
COPY BOOK	N/A		사용 구분	N/A
			공통 유형	N/A
사용 TABLE 명	한글명	N/A	TABLE COPY BOOK	N/A
	영문명	N/A		

프로그램 설명	실버바판매 거래에 사용되는 실버바판매환율정보를 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 지정 상품선물구분에 대한 상품선물환율 정보를 조회한다. 상품선물구분은 필수항목으로 입력하여야 한다. 2. 대상일자는 기준환율 적용시작일을 기준으로 한다. 대상일자 미입력시 거래기준년월일을 기준으로 조회한다. 3. 입력적용회차 미입력시 최종적용회차로 조회한다. *1) MAX 회차가 0 인경우 리턴상태를 '07'로 한다.* 4. 100 건 초과하여 상품선물환율건수가 있을 경우 오류처리 한다.
비 고	

1.114

대기업금융센터관리부점리스트조회(FUBcBranch.getCnglFinacCnterMgtt())

프로그램 ID	FUBcBranch. getCnglFinacCnterMgtt ()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점운영기타	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR03		SQL ID	selectCnglFinacCnterMgtt

프로그램 설명	대기업금융센터 관리부점 목록 조회 및 결과를 제공 한다.
프로그램 규칙 및 제약사항	1. 입력 체크 1) 구분코드 체크 - '1', '2' 입력시 오류처리 2) 부점코드 체크 - 입력:부점코드는 숫자로 입력되어야 한다. - 입력:부점코드가 '0000' 입력되면 안됨 2. 구분코드는 아래와 같다. 1) '1' : 센터포함 2) '2' : 센터미포함

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcBranch extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inDstcd ;` ● 구분코드
	`String inCnglFinacCnterMgtt ;` ● 대기업센터부점코드
	
	[출력항목]
	`int outBrnNoitm;` // 부점건수
	`List LIST1;` // 출력 List1
	`List LIST2;` // 출력 List2
	
	< LIST1 항목 >
	`String outCnglCnterYn ;` // 대기업센터여부
	
	< LIST2 항목 >
	`String outBrncd ;` // 부점코드

RTNSTAT 설명	N/A
사용 설명(예시)	// 공유메소드 호출 - 대기업금융센터관리부점리스트 조회 (com.kbstar.kji.enbncmn, getCnglFinacCnterMgtt)

	IDataSet getCnglFinacCnterMgttReq = new DataSet();
	getCnglFinacCnterMgttReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getCnglFinacCnterMgttReq.put("inDstcd", requestData.get("inDstcd"));
	getCnglFinacCnterMgttReq.put("inCnglCnterBrnCd", requestData.get("inCnglCnterBrnCd"));
	
	// @@ 공유메소드 호출 (대기업금융센터관리부점리스트 조회 getCnglFinacCnterMgtt)
	IDataSet getCnglFinacCnterMgttRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getCnglFinacCnterMgtt", getCnglFinacCnterMgttReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outoutbrnnoit = getCnglFinacCnterMgttRes.getString("outOutBrnNoit");
	IRecordSet getCnglFinacCnterMgttResRsList1 =
	getCnglFinacCnterMgttRes.getRecordSet("LIST1");
	IRecordSet getCnglFinacCnterMgttResRsList2 =
	getCnglFinacCnterMgttRes.getRecordSet("LIST2");

비 고	

1.115

PoC

태블릿방문접수일정 조회 (N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	태블릿 고객상담을 위한 태블릿방문접수 가능부점목록 및 관련정보를 조회한다.
프로그램 규칙 및 제약사항	1.입력 데이터 검증 1) 그룹회사코드가 입력되지 않으면 에러처리한다. 2) 기준년월일은 아래의 조건을 만족해야 한다. 가) 숫자로 입력되어야 한다 나) 입력월은 01 ~ 12 사이 값이어야 한다. 다) 입력일이 입력월의 말일자보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2월의 마지막일자는 29일로 한다. 3) 지역시군구구분은 지역구분에 해당하는 값만 입력이 가능하다 2.업무신청구분코드는 아래와 같다. 1) '00' : 해당무 2) '10' : 예금 3) '20' : 신용/체크카드 4) '30' : 신용대출 5) '40' : 주택담보대출 6) '50' : SOHO 7) '60' : 신탁 8) '70' : 펀드 9) '80' : 외환 10) '90': 기타 3.출력 1) 조회 결과를 목록으로 출력 한다. 2) 대상건이 존재하지 않는 경우 리턴상태코드는 '00', 조회건수는 '0'으로 리턴한다.
비고	

1.116

PoC

대기업금융센터 및 관리부점조회(FUBcBranch.getCnglFinacCnterMgtbrn())

프로그램 ID	FUBcBranch.getCnglFinacCnterMgtbrn()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(X)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	거래사용자번호에 대한 대기업센터여부 및 대기업센터부점코드와 관리부점코드를 조회한다.
프로그램 규칙 및 제약사항	1. 입력항목을 아래와 같다. 1) 부점코드 또는 2) 부점코드, 직원번호 2. 입력항목에 따라서 1) 부점코드만 입력된 경우 대기업센터여부, 대기업센터부점코드 조회 2) 부점코드, 직원번호가 입력된 경우 대기업센터여부, 대기업센터부점코드, 관리부점코드 조회 3. 로그처리는 각 서버에서 한다. 4. 호출시 EAI 거래를 통해 메인프레임의 정보를 실시간제공 하므로 모든거래에서 호출하지 말고 반드시 채널이 창구(‘ 01 ’) 이고 부점구분이 대기업금융(‘ 22 ’)인 경우만 호출

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcBranch extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inBrncd ;
	String inEmpid ;
	
	[출력항목]
	String outCnglCnterYn ;
	String outCnglCnterBrncd ;
	String outMgtBrncd ;
	
	● 부점코드
	◎ 직원번호
	
	// 대기업센터여부
	// 대기업센터부점코드
	// 관리부점코드

RTNSTAT 설명	N/A

PoC
PoC
PoC
PoC
PoC

PoC

사용 설명(예시)	//공유메소드 호출-대기업금융센터 관리부점 조회 (com.kbstar.kji.enbncmn, getCnglFinacCnterMgtbrn)
	
	IDataSet getCnglFinacCnterMgtbrnReq = new DataSet();
	
	getCnglFinacCnterMgtbrnReq.put("inBrncd", requestData.get("inBrncd"));
	getCnglFinacCnterMgtbrnReq.put("inEmpid", requestData.get("inEmpid"));
	
	// @@ 공유메소드 호출 (대기업금융센터 관리부점 조회 getCnglFinacCnterMgtbrn)
	IDataSet getCnglFinacCnterMgtbrnRes = callSharedMethodByDirect(
	"com.kbstar.kji.enbncmn", "FUBcBranch.getCnglFinacCnterMgtbrn",
	getCnglFinacCnterMgtbrnReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outcnglcnteryn = getCnglFinacCnterMgtbrnRes.getString("outCnglCnterYn");
	String outcnglcnterbrncd = getCnglFinacCnterMgtbrnRes.getString("outCnglCnterBrnCd");
	String outmgtbrncd = getCnglFinacCnterMgtbrnRes.getString("outMgtBrncd");

비 고	

PoC
PoC
PoC
1.117
PoC

This document is strictly limited to the 2025 C2J conversation PoC

고객유형체크(FUBcValidation.validationCustomerPattern())

프로그램 ID	FUBcValidation. validationCustomerPattern()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	고객유형을 확인하여 결과를 제공 한다.
프로그램 규칙 및 제약사항	1.입력 데이터 검증 1) 고객고유번호 => 검증 할 고객고유번호(20) 입력 2) 고객고유번호 공백인 경우 오류(리턴상태코드 : 09) 이외 정상처리(리턴상태코드 : 00) 하면서 고객유형 출력 2. 고객고유번호 검증 기준 1) 길이 13 가) 7 번째자리 : 1~4,0,9 ← 1 - 개인(주민번호,재외국민주민번호) 나) 7 번째자리 : 5~8 ← 1 - 개인(외국인등록번호,국내거소신고증번호) 2) 길이 10 가) 4~5 번째자리 : 81~88 ← 4 - 법인사업자(법인사업자) 라) 4~5 번째자리 : 80,89 ← 5 - 임의단체(임의단체) 3) 입력 데이터는 숫자형 만 유효하며, 숫자형이 아닌 값이 들어온 경우 즉 입력한 값 그대로 참조하며 공백,특수문자,알파벳 등의 값이 중간에 있으면 “Z : 기타대체번호’로 출력한다. 4) 정상 숫자형(13 자리,10 자리)인 경우 체크디지 유틸리티(lhl) 호출결과 정상이면 “1,4,5” 정상이 아니면 “Z : 기타대체번호’ 출력 한다. 3. 고객유형 출력 1) 1 : 개인, 4 : 법인사업자, 5 : 임의단체, Z : 기타대체번호

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcValidation extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inCustUniqNo;` ● 고객고유번호
	
	[출력항목]
	`String rtnStat;` // 리턴상태코드
	`String outCustPtrn;` // 고객유형

RTNSTAT 설명	00 : 정상

사용 설명(예시)	`// 공유메소드 호출 - 고객유형체크 (com.kbstar.kji.enbncmn, validationCustomerPattern)`
	
	`IDataSet validationCustomerPatternReq = new DataSet();`
	
	`validationCustomerPatternReq.put("inCustUniqNo", requestData.get("inCustUniqNo"));`
	
	`//@@ 공유메소드 호출 (고객유형체크 validationCustomerPattern)`
	`IDataSet validationCustomerPatternRes =`
	`callSharedMethodByDirect("com.kbstar.kji.enbncmn",`

```
"FUBcValidation.validationCustomerPattern", validationCustomerPatternReq, onlineCtx);

// @@ 출력 데이터 셋 처리
 String rtnstat = validationCustomerPatternRes.getString("rtnStat");
 String outcustptrn = validationCustomerPatternRes.getString("outCustPtrn");
```

PoC

1.118

PoC
PoC
PoC
PoC
PoC

직원 KB-PIN 정보 조회(FUBcEmployee.getEmpKBPinInfo())

프로그램 ID	FUBcEmployee.getEmpKBPinInfo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(X)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	직원KB-PIN정보	SQLMAP 명	NAME SPACE	fuBcemployee
	영문명	THKJIHR12		SQL ID	selectEmpKBPinInfo1 selectEmpKBPinInfo2

PoC

프로그램 설명	직원번호, 고객식별자, 고객관리번호, 직원주민번호로 직원 KB-PIN 정보를 조회한다.
프로그램 규칙 및 제약사항	1. 입력항목은 아래와 같다. 1) 직원번호 2) 고객식별자, 고객관리번호 항목은 선택 3) 직원주민번호 3 개 항목 중에 1 개 정보만 입력 2. 입력항목별 처리 1) 직원번호 or 고객식별자(고객관리번호는 선택)가 입력된 경우, 입력된 항목을 KEY 로 직원 KB-PIN 정보를 조회한 후, KB-PIN 조회 및 생성 EAI 거래 호출 출력항목) - 정상 직원의 경우 : 직원번호,고객식별자,고객관리번호,직원주민번호 - 미존재 직원의 경우 : 직원번호,고객식별자,고객관리번호, 직원주민번호(공백) 2) 직원주민번호가 입력된 경우, KB-PIN 조회 및 생성 EAI 거래 호출한 후, 정상 직원의 경우, 고객식별자를 KEY 로 직원 KB-PIN 정보를 조회 출력항목) - 정상 직원의 경우 : 직원번호,고객식별자,고객관리번호,직원주민번호 - 미존재 직원의 경우, Exception 처리 3. 로그처리는 각 서버에서 한다. 4. 직원번호로 조회시 동일한 직원고객식별자 및 직원주민번호 출력 주의) 직원고객식별자 및 직원주민번호로 조회시 마지막 정보출력 (퇴사 후 재 입행한 경우)

PoC PoC PoC

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcEmployee extends`
	`com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	String inGroupCoCd ;
	String inEmpid ;
	String inCustIdnfr ;
	String inCustMgtNo ;
	String inEmpSsno ;
	[출력항목]
	String outEmpid ;
		◎ 그룹회사코드
		◎ 직원번호
		◎ 고객식별자
		◎ 고객관리번호
		◎ 직원주민번호
		
		// 직원번호

PoC

PoC

	String outCustIdnfr ;
	String outCustMgtNo ;
	String outEmpSsno ;
		// 고객식별자
		// 고객관리번호
		// 직원주민번호

RTNSTAT 설명	N/A	

사용 설명(예시)	// 공유메소드 호출 - 직원 KB-PIN 정보 조회 (com.kbstar.kji.enbncmn, getEmpKBPinInfo)
	
	IDataSet getEmpKBPinInfoReq = new DataSet();
	
	getEmpKBPinInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEmpKBPinInfoReq.put("inEmpid", requestData.get("inEmpid"));
	getEmpKBPinInfoReq.put("inCustIdnfr", requestData.get("inCustIdnfr"));
	getEmpKBPinInfoReq.put("inCustMgtNo", requestData.get("inCustMgtNo"));
	getEmpKBPinInfoReq.put("inEmpSsno", requestData.get("inEmpSsno"));
	
	// @@ 공유메소드 호출 (직원 KB-PIN 정보 조회 getEmpKBPinInfo)
	IDataSet getEmpKBPinInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEmployee.getEmpKBPinInfo", getEmpKBPinInfoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outempid = getEmpKBPinInfoRes.getString("outEmpid");
	String outcustidnfr = getEmpKBPinInfoRes.getString("outCustIdnfr");
	String outcustmgtno = getEmpKBPinInfoRes.getString("outCustMgtNo");
	String outempssno = getEmpKBPinInfoRes.getString("outEmpSsno");	

비 고		

PoC

PoC

PoC

1.119

PoC PoC

KB-PIN 조회 및 생성(FUBcEmployee.getKBPin())

프로그램 ID	FUBcEmployee.getKBPin()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(X)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	고객식별자, 고객관리번호로 고객고유번호를 조회하거나, 고객고유번호와 고객유형을 입력값으로 KB-PIN(고객식별자+고객관리번호)를 생성한다.
프로그램 규칙 및 제약사항	1. 처리구분 체크 1) '1': 조회(KB-PIN) 2) '2': 조회(고객고유번호) 3) '3': 조회 후 생성 2. 처리구분별 입력항목은 아래와 같다. 1) '1' : 고객식별자 필수 고객관리번호 항목은 선택 ※ 미존재고객의 경우 오류처리 하지 않고 그 외 DB 오류등만 오류로 처리한다. 2) '2' : 고객고유번호 필수 고객유형분류구분코드 필수 입력값) A: 법인이외의 경우, B: 법인의 경우 3) '3' : 고객고유번호 필수 고객유형분류구분코드 필수 입력값) A: 법인이외의 경우, B: 법인의 경우 신규고객유형분류구분코드 필수 입력값) 1: 개인, 3: 법인, 4: 법인사업자, 5: 임의단체, 6: 해외현지법인 7: 외국기업 고객고유번호구분코드 항목은 선택 3. 구분코드별 처리 1) '1', '2': KB-PIN 및 종료고객 여부를 조회한다. 출력항목) - 정상고객의 경우 : 고객식별자,고객관리번호,고객유형분류코드, 고객종료상태구분,고객고유번호 - 2단계고객의 경우 : 고객식별자,고객종료상태구분 - 미존재고객의 경우 : rtnStat = '02'(NOT FOUND) 리턴 2) '3': KB-PIN을 생성한다. 출력항목) 고객식별자,고객관리번호,고객유형분류코드,고객종료상태구분,고객고유번호 4. 로그처리는 각 서버에서 한다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcEmployee extends`
	`com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inPrcssDstic;`
	`String inCustIdnfr;`
	`String inCustMgtNo;`
	`String inCustNm ;`
	`String inCuniqno ;`
	`String inCustPzClsfiDstcd ;`
		● 처리구분
		◎ 고객식별자
		◎ 고객관리번호
		◎ 고객명
		◎ 고객고유번호
		◎ 고객유형분류구분코드

	String inCuniqnoDstcd ;	◎ 고객고유번호구분코드
	[출력항목]	
	String rtnStat;	// 리턴상태코드
	String outCustIdnfr ;	// 고객식별자
	String outCustMgtNo ;	// 고객관리번호
	String outCustPzClsfiDstcd;	// 고객유형분류구분코드
	String outCustEndStusDstcd;	// 고객종료상태구분
	String outCuniqno ;	// 고객고유번호
RTNSTAT 설명	00 : 정상, 02 : 미존재고객의 경우	
사용 설명(예시)	// 공유메소드 호출 - KB-PIN조회 및 생성 (com.kbstar.kji.enbncmn, getKBPin)	
	IDataSet getKBPinReq = new DataSet();	
	getKBPinReq.put("inPrcssDstic", requestData.get("inPrcssDstic"));	
	getKBPinReq.put("inCustIdnfr", requestData.get("inCustIdnfr"));	
	getKBPinReq.put("inCustMgtNo", requestData.get("inCustMgtNo"));	
	getKBPinReq.put("inCustNm", requestData.get("inCustNm"));	
	getKBPinReq.put("inCuniqno", requestData.get("inCuniqno"));	
	getKBPinReq.put("inCustPzClsfiDstcd", requestData.get("inCustPzClsfiDstcd"));	
	getKBPinReq.put("inCuniqnoDstcd", requestData.get("inCuniqnoDstcd"));	
	getKBPinReq.put("inNewCustPzCDstcd", requestData.get("inNewCustPzCDstcd"));	
	// @@ 공유메소드 호출 (KB-PIN조회 및 생성 getKBPin)	
	IDataSet getKBPinRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcEmployee.getKBPin", getKBPinReq, onlineCtx);	
	// @@ 출력 데이터 셋 처리	
	String outrtnStat = getKBPinRes.getString("rtnStat");	
	String outcustidnfr = getKBPinRes.getString("outCustIdnfr");	
	String outcustmgtno = getKBPinRes.getString("outCustMgtNo");	
	String outcustpzclsfidstcd = getKBPinRes.getString("outCustPzClsfiDstcd");	
	String outcustendstusdstcd = getKBPinRes.getString("outCustEndStusDstcd");	
	String outcuniqno = getKBPinRes.getString("outCuniqno");	
비 고		

PoC

1.120 기타대체번호조회 및 생성(FUBcEmployee.getEtcAltrNo())

프로그램 ID	FUBcEmployee.getEtcAltrNo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(X)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	기타대체번호 또는 고객고유번호로 기타대체번호,고객고유번호를 조회하거나, 고객고유번호와 고객유형을 입력값으로 기타대체번호를 생성한다.
프로그램 규칙 및 제약사항	1. 처리구분 체크 1) '1' : 조회(기타대체번호) 2) '2' : 조회(고객고유번호) 3) '3' : 조회 후 생성 PoC 2. 처리구분별 입력항목은 아래와 같다. 1) '1' : 기타대체번호 필수 ※ 미존재고객의 경우 오류처리 하지 않고 그 외 DB 오류등만 오류로 처리한다. 2) '2' : 고객고유번호 필수 3) '3' : 고객고유번호, 신청어플리케이션코드, 고객유형분류구분코드('Z') 필수 3. 구분코드별 처리 1) '1','2' : 기타대체번호, 고객고유번호를 조회한다. 출력항목) - 정상고객의 경우 : 기타대체번호,고객고유번호 - 미존재고객의 경우 : rtnStat = '02'(NOT FOUND) 리턴 2) '3' : 기타대체번호를 생성한다. 출력항목) 기타대체번호,고객고유번호 PoC 4. 로그처리는 각 서버에서 한다.

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	PoC
	
	public class FUBcEmployee extends
	com.kbstar.sqc.base.FunctionUnit PoC
	
	[입력항목]
	String inPrcssDstic ; ● 처리구분
	String inAltrNo; ◎ 대체번호
	String inCuniqno ; ◎ 고객고유번호
	String inCustPzClsfiDstcd ; ◎ 고객유형분류구분코드
	String inCuniqnoDstcd ; ◎ 고객고유번호구분코드
	String inAplcnUapplCd ; ◎ 신청어플리케이션코드
	
	[출력항목]
	String rtnStat // 리턴상태코드
	PoC String outAltrNo ; // 대체번호
	String outCuniqno ; // 고객고유번호

RTNSTAT 설명	00 : 정상, 02 : 미존재고객의 경우

사용 설명(예시)	// 공유메소드 호출 - 기타대체번호조회 및 생성 (com.kbstar.kji.enbncmn, getEtcAltrNo)
	
	IDataSet getEtcAltrNoReq = new DataSet();
	
	getEtcAltrNoReq.put("inPrcssDstic", requestData.get("inPrcssDstic"));
	getEtcAltrNoReq.put("inAltrNo", requestData.get("inAltrNo"));
	getEtcAltrNoReq.put("inCuniqno", requestData.get("inCuniqno"));
	getEtcAltrNoReq.put("inCustPzClsfiDstcd", requestData.get("inCustPzClsfiDstcd"));
	getEtcAltrNoReq.put("inCuniqnoDstcd", requestData.get("inCuniqnoDstcd"));
	getEtcAltrNoReq.put("inAplcnUapplCd", requestData.get("inAplcnUapplCd
	
	// @@ 공유메소드 호출 (기타대체번호조회 및 생성 getEtcAltrNo)
	IDataSet getEtcAltrNoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEmployee.getEtcAltrNo", getEtcAltrNoReq, onlineCtx);
	PoC
	PoC
	
	// @@ 출력 데이터 셋 처리
	String outrtnStat = getEtcAltrNoRes.getString("rtnStat");
	String outaltrno = getEtcAltrNoRes.getString("outAltrNo");
	String outcuniqno = getEtcAltrNoRes.getString("outCuniqno");

비 고	

PoC

1.121

PoC

PoC

PoC

내맘대로계좌등록(FUBcNumbering.manageDIYAcno())

프로그램 ID	FUBcNumbering.manageDIYAcno()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	내맘대로계좌채번	SQLMAP 명	NAME SPACE	TODO
	영문명	THKJICN15		SQL ID	TODO

프로그램 설명	내맘대로계좌의 등록,취소,해지,조회 기능을 제공 한다.
프로그램 규칙 및 제약사항	0. '0' 으로 시작하는 번호로 전체 자릿수 10~11 자리 체계 1.입력 데이터 검증 1) 구분코드 => 구분코드 입력 1-등록 - 신규 계좌 등록 2-취소 - 삭제(계좌식별자(내맘대로계좌번호) 기준 마지막 데이터 삭제) 3-해지 - 사용안함(사용여부:부) 4-해지취소 - 사용함(사용여부:여) 5-계좌식별자 조회 - 계좌식별자(내맘대로계좌번호) 기준 조회 6-통합계좌번호 조회 - 통합계좌번호 기준 조회 2) 고객식별자 검증 숫자데이터, 길이 10 또는 11 자리가 아니면 에러 처리 2. 구분코드별 필수 입력 항목 1) 등록 : 계좌식별자, 통합계좌번호 2) 취소 : 계좌식별자 3) 해지 : 계좌식별자 4) 해지 취소 : 계좌식별자 5) 계좌식별자 조회 : 계좌식별자 6) 통합계좌번호 조회 : 통합계좌번호 * 계좌식별자 = 내맘대로계좌번호 ※ 1)등록,2)취소,3)해지,4)해지취소 미제공 3. 출력 - 계좌식별자, 일련번호, 통합계좌번호, 사용여부, 등록년월일, 해지년월일 - 조회 거래 시 조회 결과가 없을 경우 응답코드 '02' 로 응답
입출력 설명	01 XCJICN15-CA. 03 XCJICN15-RETURN. 05 XCJICN15R-STAT PIC X(002). 88 COND-XCJICN15-OK VALUE '00'. 88 COND-XCJICN15-KEYDUP VALUE '01'. 88 COND-XCJICN15-NOTFOUND VALUE '02'. 88 COND-XCJICN15-SPVSTOP VALUE '05'. 88 COND-XCJICN15-ERROR VALUE '09'. 88 COND-XCJICN15-ABNORMAL VALUE '98'. 88 COND-XCJICN15-SYSERROR VALUE '99'. 05 XCJICN15-R-LINE PIC 9(006). 05 XCJICN15-R-ERRCD PIC X(008). 05 XCJICN15-R-TREAT-CD PIC X(008). 05 XCJICN15-R-SQL-CD PIC S9(005) LEADING SEPARATE. 03 XCJICN15-IN. 05 XCJICN15-I-INPT. 07 XCJICN15-I-DSTCD PIC X(004). 07 XCJICN15-I-ACN-IDNFR PIC X(020). 07 XCJICN15-I-INTGRA-ACNO PIC X(020). 07 XCJICN15-I-SPARE PIC X(050). 03 XCJICN15-OUT.	<입력항목> ● 구분코드 ● 계좌식별자 ● 통합계좌번호 <출력항목> 계좌식별자 일련번호 통합계좌번호 사용여부 등록년월일 해지년월일 관리계좌여부 관리부점코드

	05 XCJICN15-O-OUTPT. 07 XCJICN15-O-ACN-IDNFR PIC X(020). 07 XCJICN15-O-SERNO PIC X(004). 07 XCJICN15-O-INTGRA-ACNO PIC X(020). 07 XCJICN15-O-USE-YN PIC X(001). 07 XCJICN15-O-REGI-YMD PIC X(008). 07 XCJICN15-O-TERMI-YMD PIC X(008). 07 XCJICN15-O-MGT-ACN-YN PIC X(001). 07 XCJICN15-O-MGT-BRNCD PIC X(004). 07 XCJICN15-O-SPARE PIC X(045).	
RETURN CODE 설명	00 : 정상,	
사용 설명(예시)		
비 고		

PoC
PoC
PoC
PoC
PoC
PoC

1.122 부점운영정보 조회(FUBcBranch.getBranchOperationTime())

프로그램 ID	FUBcBranch.getBranchOperationTime()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점운영시간	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR13		SQL ID	selectBranchOperationTime

프로그램 설명	조회 요청일자 시점의 부점운영정보를 조회한다	
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 부점코드는 숫자로 입력되어야 한다. - 부점코드를 '0000' 입력하면 오류를 발생한다. 2. 일자에 대한 부점 기본 정보 조회 1) 년월일 입력 : 입력된 일자 당시의 부점 정보 2) 년월일 미입력 : 현재시점의 부점정보 조회 - 현재시점 : Common Area 에 포함되어있는 KB 기준정보의 거래처리기준년월일 3) 적용시작일자 및 적용종료일자 기준으로 조회 된다 (단 개점년월일이 적용시작일자보다 클 수 있고, 폐쇄년월일이 적용종료일자보다 +1 이므로 반드시 부점기본 정보에 포함되어 있는 개점년월일과 폐쇄년월일을 확인 해야 한다.) 3. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일은 01 ~ 31 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다. 4. 입력된 정보를 기준으로 부점운영시간 테이블을 조회한다. - 부점이 존재하지 않는 경우, 부점운영시간구분을 '00'(해당무)로 출력한다. 5. 부점운영구분코드 내용은 아래와 같다. 1) '00' : 해당무 2) '01' : 9 TO 7 Bank 3) '02' : AFTER BANK(TO 5 BANK) 4) '03' : AFTER BANK(TO 6 BANK) 5) '04' : AFTER BANK(TO 7 BANK) 6. 부점운영 11 식별자 1 은 은행-증권 복합점포 정보로 사용한다. 7. 부점운영 12 식별자 1 은 BIZ 집중형운영모델 정보로 사용하고, BIZ 집중형운영모델구분코드 내용은 아래와 같다. 1) '0' : 해당무 2) '1' : Retail Biz 집중형(SOHO 형) 3) '2' : Retail Biz 집중형(개인형) 4) '3' : SME Biz 집중형 5) '4' : WM Biz 집중형 6) '5' : Full 7) '6' : Full(거점)	
입출력 설명	package com.kbstar.kji.enbncmn.biz; public class FUBcBranch extends com.kbstar.sqc.base.FunctionUnit [입력항목] String inGroupCoCd ; String inBrncd; String inYmd;	● 그룹회사코드 ● 부점코드 ◎ 년월일

	[출력항목]
	String outBrncd;
	String outAplyStartYmd;
	String outAplyEndYmd;
	String outBrnOperTtmDstcd ;
	String outBrnOper11Idnfr1 ;
	String outBrnOper12Idnfr1 ;
	String outBrnOper21Idnfr2 ;
	String outBrnOper22Idnfr2 ;
	String outBrnOper41Idnfr4 ;
	String outBrnOper42Idnfr4 ;
	String outBrnOper61Idnfr6 ;
	String outBrnOper62Idnfr6 ;
	String outBrnOper81Idnfr8 ;
	String outBrnOper82Idnfr8 ;
	String outBzoprStartHMS ;
	String outBzoprEndHMS ;
		// 부점코드
		// 적용시작년월일
		// 적용종료년월일
		// 부점운영시간구분
		// 부점운영11식별자1
		// 부점운영12식별자1
		// 부점운영21식별자2
		// 부점운영22식별자2
		// 부점운영41식별자4
		// 부점운영42식별자4
		// 부점운영61식별자6
		// 부점운영62식별자6
		// 부점운영81식별자8
		// 부점운영82식별자8
		// 영업시작시각
		// 영업종료시각

RTNSTAT 설명	N/A	

사용 설명
(예시)
	// 공유메소드 호출-부점운영정보 조회(com.kbstar.kji.enbncmn,getBranchOperationTime)
	
	IDataSet getBranchOperationTimeReq = new DataSet();
	
	getBranchOperationTimeReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getBranchOperationTimeReq.put("inBrncd", requestData.get("inBrncd"));
	getBranchOperationTimeReq.put("inYmd", requestData.get("inYmd"));
	
	// @@ 공유메소드 호출 (부점운영정보 조회 getBranchOperationTime)
	IDataSet getBranchOperationTimeRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getBranchOperationTime", getBranchOperationTimeReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outbrncd = getBranchOperationTimeRes.getString("outBrncd");
	String outaplystartymd = getBranchOperationTimeRes.getString("outAplyStartYmd");
	String outaplyendymd = getBranchOperationTimeRes.getString("outAplyEndYmd");
	String outbrnoperttmdstcd = getBranchOperationTimeRes.getString("outBrnOperTtmDstcd");
	String outbrnoper11idnfr1 = getBranchOperationTimeRes.getString("outBrnOper11Idnfr1");
	String outbrnoper12idnfr1 = getBranchOperationTimeRes.getString("outBrnOper12Idnfr1");
	String outbrnoper21idnfr2 = getBranchOperationTimeRes.getString("outBrnOper21Idnfr2");
	String outbrnoper22idnfr2 = getBranchOperationTimeRes.getString("outBrnOper22Idnfr2");
	String outbrnoper41idnfr4 = getBranchOperationTimeRes.getString("outBrnOper41Idnfr4");
	String outbrnoper42idnfr4 = getBranchOperationTimeRes.getString("outBrnOper42Idnfr4");
	String outbrnoper61idnfr6 = getBranchOperationTimeRes.getString("outBrnOper61Idnfr6");
	String outbrnoper62idnfr6 = getBranchOperationTimeRes.getString("outBrnOper62Idnfr6");
	String outbrnoper81idnfr8 = getBranchOperationTimeRes.getString("outBrnOper81Idnfr8");
	String outbrnoper82idnfr8 = getBranchOperationTimeRes.getString("outBrnOper82Idnfr8");
	String outBzoprStartHMS = getBranchOperationTimeRes.getString("outBzoprStartHMS");	

	String outBzoprEndHMS = getBranchOperationTimeRes.getString("outBzoprEndHMS");
비 고	

1.123 주민번호검색(N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	N/A	
			공통 유형	N/A	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력된 데이터에서 주민번호 항목을 찾아 위치와 값을 리턴한다.
프로그램 규칙 및 제약사항	1. 입력 데이터 검증 - 검색데이터길이 는 5 자리 숫자로 입력되어야 함. - 검색데이터는 29,000 Byte 이내로 입력되어야 함. 2. 출력 데이터 1) 추출 건수 : 추출된 데이터 건수 2) 추출 데이터 : 검색된 데이터의 건수로 총 20 개까지 검색 - 추출 위치 : 전체 데이터 중 주민번호 시작 위치 - 추출 길이 : 검색된 주민번호의 길이(13 또는 14) - 추출값 : 검색된 주민번호의 실제 값(13 또는 14 byte)
비 고	

1.124

주민번호 존재여부 확인(N/A)

프로그램 ID	N/A		언어 구분	N/A	
입출력 명	N/A		사용 구분	PoC	N/A
			공통 유형		N/A
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력된 데이터에 주민번호가 존재하는지 여부를 리턴한다.
프로그램 규칙 및 제약사항	1. 입력 데이터 검증 - 처리구분 : '01-일반데이터검증' - 입력내용 : 주민번호를 확인 할 데이터 2. 출력 데이터 1) 확인결과 : 주민번호가 포함되어 있으면 '99' 아니면 '00' 리턴
비 고	

PoC

PoC

PoC

PoC

PoC

This document is strictly limited to the 2025 C2J conversation PoC

PoC

1.125 지주직원기본 조회(FUBcEmployee.getGroupEmpInfo())

프로그램 ID	FUBcEmployee.getGroupEmpInfo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	지주직원기본	SQLMAP 명	NAME SPACE	fubcemployee
	영문명	THKJIHR11		SQL ID	getGroupEmpInfo

프로그램 설명	직원번호에 대한 지주직원정보를 조회한다.
프로그램 규칙 및 제약사항	1. 보안팀에서 수신한 지주직원정보를 기반으로 한다. 2. 직원의 부서이동 이력정보는 조회할 수 없다. 3. 직원번호를 반드시 입력하여야 한다. 4. 퇴직한 지주직원정보를 포함한다.. 현재 존재하는 직원정보를 조회하고자 할 경우 퇴직년월일로 업무가 판단한다.

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcEmployee extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inEmpid; ● 직원번호
	
	[출력항목]
	String outEmpid; // 직원번호
	String outEmpzFname; // 직원성명
	String outAfltIdnfr; // 계열사식별자
	String outAfltName; // 계열사명
	String outBelngIdnfr; // 소속식별자
	String outBelngName; // 소속명
	String outJobclIdnfr; // 직급식별자
	String outJobclCtnt; // 직급내용
	String outJobtlIdnfr; // 직위식별자
	String outJobtlCtnt; // 직위내용
	String outRtireYmd; // 퇴직년월일

RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 - 지주직원기본조회 (com.kbstar.kji.enbncmn, getGroupEmpInfo)
	
	IDataSet getGroupEmpInfoReq = new DataSet();
	
	getGroupEmpInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getGroupEmpInfoReq.put("inEmpid", requestData.get("inEmpid"));
	
	// @@ 공유메소드 호출 (지주직원기본 조회 getGroupEmpInfo)
	IDataSet getGroupEmpInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEmployee.getGroupEmpInfo", getGroupEmpInfoReq, onlineCtx);

	// @@ 출력 데이터 셋 처리
	String outEmpid = calculateExpirDdRes.getString("outEmpid"); // 직원번호
	String outEmpzFname = calculateExpirDdRes.getString("outEmpzFname"); // 직원성명
	String outAfltIdnfr = calculateExpirDdRes.getString("outAfltIdnfr"); // 계열사식별자
	String outAfltName = calculateExpirDdRes.getString("outAfltName"); // 계열사명
	String outBelngIdnfr = calculateExpirDdRes.getString("outBelngIdnfr"); // 소속식별자
	String outBelngName = calculateExpirDdRes.getString("outBelngName"); // 소속명
	String outJobclIdnfr = calculateExpirDdRes.getString("outJobclIdnfr"); // 직급식별자
	String outJobclCtnt = calculateExpirDdRes.getString("outJobclCtnt"); // 직급내용
	String outJobtlIdnfr = calculateExpirDdRes.getString("outJobtlIdnfr"); // 직위식별자
	String outJobtlCtnt = calculateExpirDdRes.getString("outJobtlCtnt"); // 직위내용
	String outRtireYmd = calculateExpirDdRes.getString("outRtireYmd"); // 퇴직년월일

비 고	

PoC

PoC

PoC

PoC

PoC

PoC

1.126 부점유형기본 조회(FUBcBranch.getBrnPtrnInfo())

프로그램 ID	FUBcBranch.getBrnPtrnInfo()		언어 구분	JAVA(○)	
입출력 명	PoC requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점유형기본	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR07		SQL ID	selectBrnPtrnInfo selectListAcctgBrnInfo

프로그램 설명	인사부점코드, 회계부점코드에 대한 부점유형정보를 조회한다.
프로그램 규칙 및 제약사항	1. 보안팀에서 수신한 부점유형정보를 기반으로 한다. 2. 인사부점코드의 이력정보는 조회할 수 없다. 3. 인사부점코드 또는 회계부점코드 중 하나만 입력하여야 한다.

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	PoC
	`public class FUBcBranch extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	PoC
	`String inGroupCoCd;` ● 그룹회사코드
	`String inBrnNo;` ● 부점번호
	`String inAcctgBrncd;` ● 회계부점코드
	
	[출력항목]
	`int outArayCnt;` // 배열건수
	`List LIST;` // 배열
	PoC
	PoC
	< LIST 항목 >
	`String outGroupCoCd;` // 그룹회사코드
	`String outBrnNo;` // 부점번호
	`String outAcctgBrncd;` // 회계부점코드
	`String outDvsnNameCtnt;` // 부서명내용
	`String outDvsnEngName;` // 부서영문명
	`String outHQCdNo;` // 본부코드번호
	`String outHQName;` // 본부명
	`String outGroupCdNo;` // 그룹코드번호
	`String outGroupCtnt;` // 그룹내용
	`String outJntRgtrDstcd;` // 공동영업권구분코드
	`String outJntRgtrNameIdnfr;` // 공동영업권식별자
	`String outDrcnPntYn;` // 직할점여부
	`String outDvsnPtrnDstcd;` // 부서유형구분코드
	`String outDvsnPtrnName;` // 부서유형명
	`String outDvsnAddr;` // 부서주소
	PoC
	`String outTelnoCtnt;` // 전화번호내용
	`String outFaxCtnt;` // 팩스내용

RTNSTAT 설명	N/A

사용 설명(예시)	`// 공유메소드 호출 – 부점유형기본 조회 (com.kbstar.kji.enbncmn, getBrnPtrnInfo)`
	
	`IDataSet getBrnPtrnInfoReq = new DataSet();`
	
	`getBrnPtrnInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));`

	getBrnPtrnInfoReq.put("inBrnNo", requestData.get("inBrnNo"));
	getBrnPtrnInfoReq.put("inAcctgBrncd", requestData.get("inAcctgBrncd"));
	
	// @@ 공유메소드 호출 (부점유형기본 조회 getBrnPtrnInfo)
	IDataSet getBrnPtrnInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getBrnPtrnInfo", getBrnPtrnInfoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	int outArayCnt = getBrnPtrnInfoRes.getInt("outArayCnt");
	IRecordSet getBrnPtrnResRsList = getBrnPtrnInfoRes.getRecordSet("LIST");

비 고	

1.127 직원기타정보 조회(FUBcEmployee.getEmpEtcInfo())

프로그램 ID	FUBcEmployee.getEmpEtcInfo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	직원기타정보	SQLMAP 명	NAME SPACE	Fubcemployee
	영문명	THKJIHR05		SQL ID	selectEmpEtcInfo

프로그램 설명	직원번호에 대한 사용자역할내용 및 기타 정보를 조회한다.	
프로그램 규칙 및 제약사항	1. 보안팀에서 수신한 직원기타정보를 기반으로 한다. 2. 직원기타정보의 이력정보는 조회할 수 없다. 3. 직원번호를 반드시 입력해야 한다.	

입출력 설명	package com.kbstar.kji.enbncmn.biz;
	
	public class FUBcEmployee extends
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd;
	String inEmpid;
	
	[출력항목]
	String outEmpid ;
	String outUserRoleCtnt
	String outPafiarBrnDstcd
	String outBelngBrncd
	String outPafiarJobtlDstcd
	String outPafiarJobtlDsticHanglName
	String outCnpstnPafiarBrnDstcd1
	String outCnpstnAcctgBrncd1
	String outCnpstnPafiarJobtlDstcd1
	String outCnpstnPafiarJobtlDsticName1
	String outCnpstnStartYmd1
	String outCnpstnEndYmd1
	String outCnpstnPafiarBrnDstcd2
	String outCnpstnAcctgBrncd2
	String outCnpstnPafiarJobtlDstcd2
		● 그룹회사코드
		● 직원번호
		
		//직원번호
		//사용자역할내용
		//인사부점구분코드
		//소속부점코드
		//인사직위구분코드
		//인사직위구분한글명
		//겸직인사부점구분코드 1
		//겸직회계부점코드 1
		//겸직인사직위구분코드 1
		//겸직인사직위구분명 1
		//겸직시작년월일 1
		//겸직종료년월일 1
		//겸직인사부점구분코드 2
		//겸직회계부점코드 2
		//겸직인사직위구분코드 2

This document is strictly limited to the 2025 C2J conversation PoC

	String outCnpstnPafiarJobtlDsticName2	//겸직인사직위구분명 2
	String outCnpstnStartYmd2	//겸직시작년월일 2
	String outCnpstnEndYmd2	//겸직종료년월일 2
	String outCnpstnPafiarBrnDstcd3	//겸직인사부점구분코드 3
	String outCnpstnAcctgBrncd3	//겸직회계부점코드 3
	String outCnpstnPafiarJobtlDstcd3	//겸직인사직위구분코드 3
	String outCnpstnPafiarJobtlDsticName3	//겸직인사직위구분명 3
	String outCnpstnStartYmd3	//겸직시작년월일 3
	String outCnpstnEndYmd3	//겸직종료년월일 3
	String outCnpstnPafiarBrnDstcd4	//겸직인사부점구분코드 4
	String outCnpstnAcctgBrncd4	//겸직회계부점코드 4
	String outCnpstnPafiarJobtlDstcd4	//겸직인사직위구분코드 4
	String outCnpstnPafiarJobtlDsticName4	//겸직인사직위구분명 4
	String outCnpstnStartYmd4	//겸직시작년월일 4
	String outCnpstnEndYmd4	//겸직종료년월일 4
	String outCnpstnPafiarBrnDstcd5	//겸직인사부점구분코드 5
	String outCnpstnAcctgBrncd5	//겸직회계부점코드 5
	String outCnpstnPafiarJobtlDstcd5	//겸직인사직위구분코드 5
	String outCnpstnPafiarJobtlDsticName5	//겸직인사직위구분명 5
	String outCnpstnStartYmd5	//겸직시작년월일 5
	String outCnpstnEndYmd5	//겸직종료년월일 5
	String outCnpstnPafiarBrnDstcd6	//겸직인사부점구분코드 6
	String outCnpstnAcctgBrncd6	//겸직회계부점코드 6
	String outCnpstnPafiarJobtlDstcd6	//겸직인사직위구분코드 6
	String outCnpstnPafiarJobtlDsticName6	//겸직인사직위구분명 6
	String outCnpstnStartYmd6	//겸직시작년월일 6
	String outCnpstnEndYmd6	//겸직종료년월일 6
	String outCnpstnPafiarBrnDstcd7	//겸직인사부점구분코드 7
	String outCnpstnAcctgBrncd7	//겸직회계부점코드 7
	String outCnpstnPafiarJobtlDstcd7	//겸직인사직위구분코드 7
	String outCnpstnPafiarJobtlDsticName7	//겸직인사직위구분명 7
	String outCnpstnStartYmd7	//겸직시작년월일 7
	String outCnpstnEndYmd7	//겸직종료년월일 7
	String outCnpstnPafiarBrnDstcd8	//겸직인사부점구분코드 8
	String outCnpstnAcctgBrncd8	//겸직회계부점코드 8
	String outCnpstnPafiarJobtlDstcd8	//겸직인사직위구분코드 8
	String outCnpstnPafiarJobtlDsticName8	//겸직인사직위구분명 8
	String outCnpstnStartYmd8	//겸직시작년월일 8
	String outCnpstnEndYmd8	//겸직종료년월일 8
	String outCnpstnPafiarBrnDstcd9	//겸직인사부점구분코드 9
	String outCnpstnAcctgBrncd9	//겸직회계부점코드 9
	String outCnpstnPafiarJobtlDstcd9	//겸직인사직위구분코드 9
	String outCnpstnPafiarJobtlDsticName9	//겸직인사직위구분명 9
	String outCnpstnStartYmd9	//겸직시작년월일 9
	String outCnpstnEndYmd9	//겸직종료년월일 9
	String outCnpstnPafiarBrnDstcd10	//겸직인사부점구분코드 10
	String outCnpstnAcctgBrncd10	//겸직회계부점코드 10
	String outCnpstnPafiarJobtlDstcd10	// 겸직인사직위구분코드 10
	String outCnpstnPafiarJobtlDsticName10	//겸직인사직위구분명 10
	String outCnpstnStartYmd10	//겸직시작년월일 10
	String outCnpstnEndYmd10	//겸직종료년월일 10
	String outHiDfclDigrmPrdctSpvsrYn	//고난도상품책임자여부
	String outIntnlCtrlPsnSpvsrYn	//내부통제자책임자여부
	String outEmpEtc21Idnfr2	//직원기타 21 식별자 2
	String outEmpEtc22Idnfr2	//직원기타 22 식별자 2
	String outEmpEtc41Idnfr4	//직원기타 41 식별자 4
	String outEmpEtc42Idnfr4	//직원기타 42 식별자 4
	String outEnbnkYmd	//입행년월일
	String outEmpEtc82Idnfr8	//직원기타 82 식별자 8
RTNSTAT 설명	N/A	

사용 설명(예시)	

PoC	// 공유메소드 호출 – 직원기타정보 조회 (com.kbstar.kji.enbncmn, getEmpEtcInfo)
	
	IDataSet getEmpEtcInfoReq = new DataSet();
	
	getEmpEtcInfoReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getEmpEtcInfoReq.put("inEmpid ", requestData.get("inEmpid "));
	
	// @@ 공유메소드 호출 (직원기타정보 조회 getEmpEtcInfo)
	IDataSet getEmpEtcInfoRes = callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcEmployee.getEmpEtcInfo", getEmpEtcInfoReq, onlineCtx);

PoC	// @@ 출력 데이터 셋 처리

PoC	String outempid = getEmpEtcInfoRes.getString("outEmpid");
	String outuserrolectnt = getEmpEtcInfoRes.getString("outUserRoleCtnt");
	String outpafiarbrndstcd = getEmpEtcInfoRes.getString("outPafiarBrnDstcd");
	String outbelngbrncd = getEmpEtcInfoRes.getString("outBelngBrncd");
	String outpafiarjobtldstcd = getEmpEtcInfoRes.getString("outPafiarJobtlDstcd");
	String outpafiarjobtldstichanglname = getEmpEtcInfoRes.getString("outPafiarJobtlDsticHanglName");
	String outcnpstnpafiarbrndstcd1 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd1");
	String outcnpstnacctgbrncd1 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd1");
	String outcnpstnpafiarjobtldstcd1 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd1");
	String outcnpstnpafiarjobtldsticname1 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName1");
	String outcnpstnstartymd1 = getEmpEtcInfoRes.getString("outCnpstnStartYmd1");
	String outcnpstnendymd1 = getEmpEtcInfoRes.getString("outCnpstnEndYmd1");
	String outcnpstnpafiarbrndstcd2 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd2");
	String outcnpstnacctgbrncd2 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd2");
	String outcnpstnpafiarjobtldstcd2 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd2");
	String outcnpstnpafiarjobtldsticname2 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName2");
	String outcnpstnstartymd2 = getEmpEtcInfoRes.getString("outCnpstnStartYmd2");
	String outcnpstnendymd2 = getEmpEtcInfoRes.getString("outCnpstnEndYmd2");
	String outcnpstnpafiarbrndstcd3 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd3");
	String outcnpstnacctgbrncd3 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd3");
	String outcnpstnpafiarjobtldstcd3 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd3");
	String outcnpstnpafiarjobtldsticname3 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName3");
	String outcnpstnstartymd3 = getEmpEtcInfoRes.getString("outCnpstnStartYmd3");
	String outcnpstnendymd3 = getEmpEtcInfoRes.getString("outCnpstnEndYmd3");
	String outcnpstnpafiarbrndstcd4 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd4");
	String outcnpstnacctgbrncd4 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd4");
	String outcnpstnpafiarjobtldstcd4 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd4");
	String outcnpstnpafiarjobtldsticname4 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName4");
	String outcnpstnstartymd4 = getEmpEtcInfoRes.getString("outCnpstnStartYmd4");
	String outcnpstnendymd4 = getEmpEtcInfoRes.getString("outCnpstnEndYmd4");
	String outcnpstnpafiarbrndstcd5 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd5");
	String outcnpstnacctgbrncd5 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd5");
	String outcnpstnpafiarjobtldstcd5 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd5");
	String outcnpstnpafiarjobtldsticname5 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName5");
	String outcnpstnstartymd5 = getEmpEtcInfoRes.getString("outCnpstnStartYmd5");

PoC
PoC
PoC
PoC
PoC
PoC
PoC
PoC
	String outcnpstnendymd5 = getEmpEtcInfoRes.getString("outCnpstnEndYmd5");
	String outcnpstnpafiarbrndstcd6 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd6");
	String outcnpstnacctgbrncd6 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd6");
	String outcnpstnpafiarjobtldstcd6 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd6");
	String outcnpstnpafiarjobtldsticname6 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName6");
	String outcnpstnstartymd6 = getEmpEtcInfoRes.getString("outCnpstnStartYmd6");
	String outcnpstnendymd6 = getEmpEtcInfoRes.getString("outCnpstnEndYmd6");
	String outcnpstnpafiarbrndstcd7 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd7");
	String outcnpstnacctgbrncd7 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd7");
	String outcnpstnpafiarjobtldstcd7 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd7");
	String outcnpstnpafiarjobtldsticname7 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName7");
	String outcnpstnstartymd7 = getEmpEtcInfoRes.getString("outCnpstnStartYmd7");
	String outcnpstnendymd7 = getEmpEtcInfoRes.getString("outCnpstnEndYmd7");
	String outcnpstnpafiarbrndstcd8 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd8");
	String outcnpstnacctgbrncd8 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd8");
	String outcnpstnpafiarjobtldstcd8 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd8");
	String outcnpstnpafiarjobtldsticname8 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName8");
	String outcnpstnstartymd8 = getEmpEtcInfoRes.getString("outCnpstnStartYmd8");
	String outcnpstnendymd8 = getEmpEtcInfoRes.getString("outCnpstnEndYmd8");
	String outcnpstnpafiarbrndstcd9 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd9");
	String outcnpstnacctgbrncd9 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd9");
	String outcnpstnpafiarjobtldstcd9 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd9");
	String outcnpstnpafiarjobtldsticname9 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName9");
	String outcnpstnstartymd9 = getEmpEtcInfoRes.getString("outCnpstnStartYmd9");
	String outcnpstnendymd9 = getEmpEtcInfoRes.getString("outCnpstnEndYmd9");
	String outcnpstnpafiarbrndstcd10 = getEmpEtcInfoRes.getString("outCnpstnPafiarBrnDstcd10");
	String outcnpstnacctgbrncd10 = getEmpEtcInfoRes.getString("outCnpstnAcctgBrncd10");
	String outcnpstnpafiarjobtldstcd10 = getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDstcd10");
	String outcnpstnpafiarjobtldsticname10 =
	getEmpEtcInfoRes.getString("outCnpstnPafiarJobtlDsticName10");
	String outcnpstnstartymd10 = getEmpEtcInfoRes.getString("outCnpstnStartYmd10");
	String outcnpstnendymd10 = getEmpEtcInfoRes.getString("outCnpstnEndYmd10");
	String outhidfcldigrmprdctspvsryn = getEmpEtcInfoRes.getString("outHiDfclDigrmPrdctSpvsrYn");
	String outintnlctrlpsnspvsryn = getEmpEtcInfoRes.getString("outIntnlCtrlPsnSpvsrYn");
	String outempetc21idnfr2 = getEmpEtcInfoRes.getString("outEmpEtc21Idnfr2");
	String outempetc22idnfr2 = getEmpEtcInfoRes.getString("outEmpEtc22Idnfr2");
	String outempetc41idnfr4 = getEmpEtcInfoRes.getString("outEmpEtc41Idnfr4");
	String outempetc42idnfr4 = getEmpEtcInfoRes.getString("outEmpEtc42Idnfr4");
	String outenbnkymd = getEmpEtcInfoRes.getString("outEnbnkYmd");
	String outempetc82idnfr8 = getEmpEtcInfoRes.getString("outEmpEtc82Idnfr8");

비 고	

PoC

PoC

PoC

PoC

PoC

PoC

1.128 부점영업시간정보 조회(FUBcBranch.getBranchBizOperationTime())

프로그램 ID	FUBcBranch.getBranchBizOperationTime()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점영업시간정보	SQLMAP 명	NAME SPACE	Fubcbranch
	영문명	THKJIBR08		SQL ID	selectBrnBizOperationTime

프로그램 설명	조회 요청일자 시점의 부점영업시간 정보를 조회한다
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 부점코드는 숫자로 입력되어야 한다. - 부점코드를 '0000' 입력하면 오류를 발생한다. 2. 일자에 대한 부점 기본 정보 조회 1) 연월일 입력 : 입력된 일자 당시의 부점영업시간정보 조회 2) 연월일 미입력 : 현재 시점의 부점영업시간정보 조회 - 현재시점 : Common Area 에 포함되어있는 KB 기준정보의 거래처리기준년월일 3) 적용시작일자 및 적용종료일자 기준으로 조회 된다. 3. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일은 01 ~ 31 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다. 4. 입력된 정보를 기준으로 부점운영시간 테이블을 조회하여 찾지 못하면 부점운영시간구분코드를 '00'(해당무)으로 출력하고 리턴상태코드를 '00' 으로 한다. 5. 부점운영구분코드 내용은 아래와 같다. 1) '00' : 해당무 2) '01' : 9 TO 7 Bank 3) '02' : AFTER BANK(TO 5 BANK) 4) '03' : AFTER BANK(TO 6 BANK) 5) '04' : AFTER BANK(TO 7 BANK) 6. 데이터관리 1) 영업기획부 2) 화면 : [06-70-003] 부점영업시간정보 등록 및 조회 3) 프로젝트 : P20202066472 부점별 영업시간 등록 화면 신설 4) 담당자 : 심현 차장 (20201211 기준)

입출력 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcBranch extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	`String inGroupCoCd;` ● 그룹회사코드
	`String inBrncd;` ● 부점코드
	`String inYmd;` ◎ 기준년월일
	
	[출력항목]
	`String outBrncd;` //부점코드
	`String outAplyStartYmd;` //적용시작년월일
	`String outAplyEndYmd;` //적용종료년월일
	`String outBzoprStartHMS;` //영업시작시각
	`String outBzoprEndHMS;` //영업종료시각
	`String outBrnOperTtmDstcd;` //부점운영시간구분
	`String outBrnTtm11Idnfr1;` //부점시간 11 식별자 1
	`String outBrnTtm12Idnfr1;` //부점시간 12 식별자 1
	`String outBrnTtm21Idnfr2;` //부점시간 21 식별자 2
	//부점시간 22 식별자 2

PoC

	String outBrnTtm22Idnfr2; PoC
RTNSTAT 설명	N/A

사용 설명(예시)	// 공유메소드 호출 – 부점영업시간정보 조회 (com.kbstar.kji.enbncmn, getBranchBizOperationTime)
	
	IDataSet getBranchBizOperationTimeReq = new DataSet();
	
	getBranchBizOperationTimeReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getBranchBizOperationTimeReq.put("inBrncd", requestData.get("inBrncd"));
	
	// @@ 공유메소드 호출 (부점영업시간정보 조회 getBranchBizOperationTime)
	IDataSet getBranchBizOperationTimeRes =
	callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranch.getBranchBizOperationTime",
	getBranchBizOperationTimeReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outBrnOperTtmDstcd = getBranchBizOperationTimeRes.getString("outBrnOperTtmDstcd");
	< 중 략 >

비 고	

PoC

1.129 고유식별정보 검출(FUBcValidation.validationUniqIdnfiInfo())

PoC

프로그램 ID	FUBcValidation.validationUniqIdnfiInfo()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

프로그램 설명	입력된 데이터에서 고유식별정보 항목을 찾아 위치와 값을 리턴한다.
프로그램 규칙 및 제약사항	1. 입력 데이터 검증 1) 고유식별정보구분 - “ 01 ” - 주민번호 - “ 02 ” - 외국인등록번호 - “ 03 ” - 운전면허번호 - “ 04 ” - 여권번호 - “ 05 ” - 연속된 4 자리 숫자 - “ 06 ” - 연속된 13 자리 숫자 2) 최소 검색데이터길이 체크 - 주민번호 13, 외국인등록번호 13, 운전면허번호 12, 여권번호 9 이상으로 입력 되어야 함 2. 출력 데이터 1) 추출 건수 : 추출된 데이터 건수 2) 추출 데이터 : LIST 형태의 검색된 데이터 - 추출 위치 : 전체 데이터 중 고유식별정보 시작 위치 - 추출 길이 : 검색된 고유식별정보 길이 - 추출값 : 검색된 고유식별정보의 실제 값 3. 검색기준 1) 주민번호, 외국인등록번호 - 13~14 자리 연속으로 이어진 데이터가 주민등록번호 형식에 맞아야 함 - 앞 6 자리 숫자가 날짜형식에 맞아야 함 - 날짜가 미래 일자가 아니어야 함 - 날짜가 “ 20201005 ” 이전 날짜인 경우 체크디지트 검증로직을 수행하고 체크디지트가 맞으면 주민등록번호 인 것으로 간주, 이후 날자는 체크디짓 검증 없이 주민번호로 간주 - 검증형식 예시(실주민번호가 “ 2001011234567 ” 이라 가정 할때) “ 2001011234567 ” → 연속된 13 자리 숫자인 경우 “ 200101-1234567 “ → 6 자리숫자 + “ - “ + 7 자리숫자 “ 200101 1234567 ” → 6 자리숫자 + “ “ + 7 자리숫자 - 형식은 맞으나 법인번호 체크하여 법인번호인 경우 주민번호, 외국인등록번호로 보지 않는다. 2) 운전면허번호 - 12~19 자리 연속으로 이어진 데이터가 운전면허번호 형식인 경우 체크디지트 검증로직을 수행하고 체크디지트가 맞으면 운전면허번호 인 것으로 간주 - 검증형식예시(운전면허번호가 “ 112345678971 ” 또는 “ 서울 2345678971 ” 이라 가정 할때) “ 서울 2345678971 ” → 지역명 + 10 자리숫자 “ 서울 23-456789-71 ” → 지역명+2 자리숫자+ “ - “ +6 자리숫자+ “ - “ +2 자리숫자 “ 서울-23-456789-71 ” → 지역명+ “ - “ +2 자리숫자+ “ - “ +6 자리숫자+ “ - “ +2 자리숫자 “ 112345678971 ” → 지역코드 2 자리 + 12 자리 숫자 “ 11-23-456789-71 ” → 지역코드 2 자리+ “ - “ +2 자리숫자+ “ - “ +6 자리숫자+ “ - “ +2 자리숫자 - 중간에 공백이 있는 경우 고유식별정보로 검출하지 않음 (2024.10 요건 추가) “ 서울 23 456789 71 ” → 지역명+ “ “ +2 자리숫자+ “ “ +6 자리숫자+ “ “ +2 자리숫자 “ 서울 23-456789-71 ” → 지역명+ “ “ +2 자리숫자+ “ - “ +6 자리숫자+ “ - “ +2 자리숫자 “ 11 23 456789 71 ” → 지역코드 2 자리+ “ “ +2 자리숫자+ “ “ +6 자리숫자+ “ “ +2 자리숫자 “ 11 23-456789-71 ” → 지역코드 2 자리+ “ “ +2 자리숫자+ “ - “ +6 자리숫자+ “ - “ +2 자리숫자 - 운전면허번호 패턴 검출 후 추가 데이터가 있으면 검출 결과를 set 하지 않고 검출 종료

PoC

PoC

PoC

PoC

PoC

검출 종료 이후 공백이 있으면 다시 검출을 시작함(2024.10 요건 추가)
운전면허번호 패턴이 검출되고 데이터가 끝인경우 검출 결과를 set 하고 검출 종료함
" 서울 23-456789-71555 " → 검출되지 않음
" 111 서울 23-456789-71 " → 검출되지 않음
" 111 서울 23-456789-71 서울 2345678971 " → 공백 뒤 서울 2345678971 만 검출
" 111 서울 23-456789-71 11-23-456789-71 " → 공백 뒤 11-23-456789-71 만 검출
- 유효 지역명 및 지역코드 : 11 서울 ,12 부산 ,13 경기 ,14 강원 ,15 충북 ,16 충남 ,
17 전북 ,18 전남 ,19 경북 ,20 경남 ,21 제주 ,22 대구 ,
23 인천 ,24 광주 ,25 대전 ,26 울산 ,28 지역명 없음

3) 여권번호
- 9 자리 연속으로 이어진 데이터가 여권번호 형식인 경우 여권번호 인 것으로 간주
- 검증형식 예시
" D12345678 " → 영문 1 자리 + 숫자 8 자리
" AF1234567 " → 영문 2 자리 + 숫자 7 자리
- 여권번호 패턴 검출 후 추가 데이터가 있으면 검출 결과를 set 하지 않고 검출 종료함
검출 종료 이후 공백이 있으면 다시 검출을 시작함(2024.10 요건 추가)
" M10241006NS00178 " → 검출되지 않음
" 999M10241006 " → 검출되지 않음
" 999M10241006 D12345678 " → 공백 뒤 D12345678 만 검출
- 유효 영문자 (1 자리)
→ 'D' 'M' 'T' 'R' 'S' 'G'
- 유효 영문자 (2 자리)
→ 'AE' 'AF' 'AG' 'AN' 'AQ' 'AR' 'AU' 'AV' 'AY' 'BA'
'BB' 'BG' 'BL' 'BN' 'BO' 'BR' 'BS' 'BT' 'BU' 'CA'
'CB' 'CD' 'CG' 'CH' 'CL' 'CN' 'CO' 'CP' 'CS' 'CZ'
'DG' 'DJ' 'DK' 'DM' 'EP' 'EQ' 'ER' 'ES' 'ET' 'FJ'
'FK' 'FN' 'FR' 'FU' 'GA' 'GB' 'GD' 'GE' 'GG' 'GH'
'GJ' 'GK' 'GM' 'GN' 'GP' 'GR' 'GS' 'GU' 'GV' 'GW'
'GY' 'GZ' 'HC' 'HD' 'HG' 'HI' 'HK' 'HL' 'HO' 'HU'
'IC' 'IR' 'IS' 'IT' 'IV' 'JA' 'JB' 'JG' 'JJ' 'JN'
'JO' 'JR' 'JU' 'KA' 'KJ' 'KN' 'KO' 'KR' 'KU' 'KY'
'KZ' 'LA' 'LB' 'LP' 'LS' 'MA' 'MG' 'MN' 'MO' 'MP'
'MU' 'MZ' 'NA' 'ND' 'NG' 'NI' 'NJ' 'NP' 'NR' 'NW'
'NY' 'NZ' 'OC' 'OM' 'OS' 'PA' 'PD' 'PG' 'PH' 'PM'
'PN' 'PO' 'PU' 'QD' 'QT' 'RA' 'RF' 'RM' 'SA' 'SB'
'SC' 'SD' 'SE' 'SF' 'SG' 'SH' 'SJ' 'SK' 'SL' 'SM'
'SN' 'SO' 'SP' 'SQ' 'SR' 'SS' 'ST' 'SW' 'SY' 'SZ'
'TJ' 'TM' 'TN' 'TP' 'TR' 'TU' 'TZ' 'UA' 'UK' 'UL'
'UN' 'UR' 'US' 'UZ' 'VC' 'VE' 'VL' 'VT' 'VZ' 'YG'
'YO' 'YP' 'YS' 'ZB' 'DR' 'DQ' 'DP' 'DT' 'DS'

4) 법인번호 체크 (주민번호, 외국인등록번호 추출시 체크)
- 형식 패턴은 주민등록 , 외국인등록번호와 동일
- 정규식 : (1[1-9]¦2[0-8])(0[1-3]¦1[1-5]¦4[1-9]¦5[0-9]¦6[0-2])(1[1-5]¦2[1-2]¦3[1-9]¦4[0-9]¦5[0-5]¦61¦71¦8[1-6])\D{7}
- 정규식 내용 :

```
1-2 : (1[1-9] ¦  << 1  번째자리  1, 2  번째자리   1-9  OR
       2[0-8])   << 1  번째자리  2, 2  번째자리   0-8
3-4 : (0[1-3] ¦  << 3  번째자리  0, 4  번째자리   1-3  OR
       1[1-5] ¦  << 3  번째자리  1, 4  번째자리   1-5  OR
       4[1-9] ¦  << 3  번째자리  4, 4  번째자리   1-9  OR
       5[0-9] ¦  << 3  번째자리  5, 4  번째자리   0-9  OR
       6[0-2])   << 3  번째자리  6, 4  번째자리   0-2
5-6 : (1[1-5] ¦  << 5  번째자리  1, 6  번째자리   1-5  OR
       2[1-2] ¦  << 5  번째자리  2, 6  번째자리   1-2  OR
       3[1-9] ¦  << 5  번째자리  3, 6  번째자리   1-9  OR
       4[0-9] ¦  << 5  번째자리  4, 6  번째자리   0-9  OR
       5[0-5] ¦  << 5  번째자리  5, 6  번째자리   0-5  OR
       61 ¦      << 5  번째자리  6, 6  번째자리   1    OR
       71 ¦      << 5  번째자리  7, 6  번째자리   1    OR
       8[1-6])   << 5  번째자리  8, 6  번째자리   1-6
7-13:  \D{7}     << 7-13  번째자리   0-9
```

5) 연속된 4 자리 숫자 체크
- 4 자리 이상 연속된 숫자이면 숫자 4 자리를고유식별정보로 간주
- EX) INPUT : 123456
OUTPUT : 1234, 2345, 3456

PoC

	6) 연속된 13 자리 숫자 체크 - 13 자리 이상 연속된 숫자이면 숫자 13 자리를 고유식별정보로 간주 - EX) INPUT : 12345678901234 OUTPUT : 1234567890123, 2345678901234 4. 프로젝트 1) P20202067334 고유식별정보 신규 입력 방지 필터링을 위한 전행공통모듈 개발 2) 담당자 : 정보보호부 안대현 대리(2024.10 기준) 3) 요건 수정시 정보보호부 확인 필요

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcValidation **extends** com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inSearchDstcd; ● 검색구분
	String inSearchData; ● 검색데이타
	
	[출력항목]
	String outFndnCnt; //추출건수
	recordSet List; //[recordSet]
	-field outFndnPos; // 추출위치
	-field outFndnLen; // 추출길이
	-field outFndnVal; // 추출값

RTNSTAT 설명	N/A

사용 설명(예시)	**// 공유메소드 호출 – 고유식별정보 검출 (com.kbstar.kji.enbncmn, validationUniqIdnfiInfo)**
	
	IDataSet validationUniqIdnfiInfoReq = **new** DataSet();
	
	validationUniqIdnfiInfoReq.put("inSearchDstcd", requestData.get("inSearchDstcd"));
	validationUniqIdnfiInfoReq.put("inSearchData", requestData.get("inSearchData"));
	
	// @@ 공유메소드 호출 (고유식별정보 검출 validationUniqIdnfiInfo)
	IDataSet validationUniqIdnfiInfoRes =
	callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	" FUBcValidation.validationUniqIdnfiInfo",
	validationUniqIdnfiInfoReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String outFndnCnt = validationUniqIdnfiInfoRes.getString("outFndnCnt");
	IRecordSet validationUniqIdnfiInfoResRsList =
	validationUniqIdnfiInfoRes.getRecordSet("LIST");
	For (IRecordSet validationUniqIdnfiInfoResRsListRec : validationUniqIdnfiInfoResRsList){
	String outFndnPos = validationUniqIdnfiInfoResRsListRec.getString("outFndnPos");
	String outFndnLen = validationUniqIdnfiInfoResRsListRec.getString("outFndnLen");
	String outFndnVal = validationUniqIdnfiInfoResRsListRec.getString("outFndnVal");
	}

비 고	

PoC PoC PoC PoC PoC

PoC PoC

PoC

1.130 지역본부장,부점장,내부통제자 조회 (FUBcBranch.getBrnAreahqBrnmgrCpsn())

PoC

프로그램 ID	FUBcBranch. getBrnAreahqBrnmgrCpsn()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점기본 직원기본 직원기타정보	SQLMAP 명	NAME SPACE	Fubcbranch Fubcemployee
	영문명	THKJIBR01 THKJIHR01 THKJIHR05		SQL ID	selectBranchBrnmgrInfo selectBranchIntnlCPsnInfo selectListAreaHqBrnmgrInfo selectListIntnlCPsnInfo

PoC

프로그램 설명	입력된 부점코드에 해당하는 지역본부장, 부점장, 내부통제자 정보를 조회한다.
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 부점코드는 숫자로 입력되어야 한다. - 부점코드를 '0000'입력하면 오류 처리한다. - 부점기본(THKJIBR01) 원장에 등록되지 않은 부점은 오류 처리한다. 2. 폐쇄부점코드일 경우 리턴상태코드를 '07'로 출력한다. 3. 폐쇄부점코드이거나 카드부점코드일 경우 부점기본(THKJIBR01) 원장에 등록된 부점장직원번호, 부점장휴대폰번호, 내부통제자직원번호, 내부통제자휴대폰번호 항목만 출력한다. 4. 지역본부장 출력조건은 다음과 같다. - 입력된 부점코드가 직원기본(THKJIHR01) 원장의 사용자업무부점코드나 겸직회계부점코드와 같으면서 아래 세 조건 중 하나라도 해당되는 은행 직원 1) 부점장여부가 '1' 2) 사용자유형구분이 '01'(부점장) 3) 사용자역할내용에 '17101' 포함 - 인사직위구분코드가 'B01'(지역본부장)인 경우 5. 부점장 출력조건은 다음과 같다. - 첫번째 출력값은 부점기본(THKJIBR01) 원장에 부점장직원번호로 등록되어 있는 직원을 우선 출력 (부점장부점정보등록여부 : '1') - 그 외 부점장 대상은 입력된 부점코드가 직원기본(THKJIHR01) 원장의 사용자업무부점코드와 같으면서 아래 세 조건 중 하나라도 해당되는 은행 직원 1) 부점장여부가 '1' 2) 사용자유형구분이 '01'(부점장) 3) 사용자역할내용에 '17101' 포함 (부점장역할여부 : '1') - 겸직부점장 : 위 세 조건 중 하나라도 해당되면서 입력된 부점코드와 겸직회계부점코드가 같은 경우 (겸직부점장여부 : '1') - 출력 순서는 직원한글성명 가나다순으로 정렬 6. 내부통제자 출력조건은 다음과 같다. - 첫번째 출력값은 부점기본(THKJIBR01) 원장에 내부통제자직원번호로 등록되어 있는 직원을 우선 출력 (내부통제자부점정보등록여부 : '1') - 그 외 내부통제자 대상은 입력된 부점코드가 직원기본(THKJIHR01) 원장의 사용자업무부점코드와 같으면서 직원기타정보(THKJIHR05) 원장에 내부통제자책임자여부가 '1'인 은행

PoC

	직원 - 출력 순서는 다음과 같다. 1) 직급구분 : ‘L4’ > ‘L3’ > ‘L2’ > ‘L1’ > ‘L0’ > ‘000’ > ‘S4’ > ‘S3’ ‘S2’ > ‘S1’ > 그 외 2) 직원한글성명 가나다순

입출력 설명	**package** com.kbstar.kji.enbncmn.biz;
	
	public class FUBcBranch **extends**
	com.kbstar.sqc.base.FunctionUnit
	
	[입력항목]
	String inGroupCoCd; ● 그룹회사코드
	String inBrncd; ● 부점코드
	
	[출력항목]
	String rtnStat; //리턴상태코드
	String outAreaHqChfEmpid; //지역본부장직원번호
	String outAreaHqChfFname; //지역본부장직원명
	String outAreaHqChfCphnNo; //지역본부장휴대폰번호
	String outBrnmgrCnt; //부점장수
	recordSet outBrnmgrList; //부점장 List
	-field outBrnmgrEmpid; //부점장직원번호
	-field outBrnmgrFname; //부점장직원명
	-field outBrnmgrEmpCphnNo; //부점장휴대폰번호
	-field outBrnmgrYn; //부점장여부
	-field outBrnmgrPtrnDstcd; //부점장유형구분
	-field outBrnmgrRoleYn; //부점장역할여부
	-field outCnpstnBrnmgrYn; //겸직부점장여부
	-field outBBrnInfoRegiYn; //부점장부점정보등록여부
	String outIntnlCPsnCnt; //내부통제자수
	recordSet outIntnlCPsnList; //내부통제자 List
	-field outIntnlCPsnEmpid; //내부통제자직원번호
	-field outIntnlCPsnFname; //내부통제자직원명
	-field outIntnlCPsnCphnNo; //내부통제자휴대폰번호
	-field outIntnlCPsnJDstcd; //내부통제자직급구분
	-field outIntnlCPsnPDstcd; //내부통제자유형구분
	-field outIBrnInfoRegiYn; //내부통제자부점정보등록여부

RETURN CODE 설명	00 : 정상, 07 : 폐쇄점, 09 : ERROR

사용 설명(예시)	**// 공유메소드 호출 – 지역본부장,부점장,내부통제자 조회**
	(com.kbstar.kji.enbncmn, getBrnAreahqBrnmgrCpsn)
	
	IDataSet getBrnAreahqBrnmgrCpsnReq = **new** DataSet();
	
	getBrnAreahqBrnmgrCpsnReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getBrnAreahqBrnmgrCpsnReq.put("inBrncd", requestData.get("inBrncd"));
	
	// @@ 공유메소드 호출 (지역본부장,부점장,내부통제자 조회 getBrnAreahqBrnmgrCpsn)
	IDataSet getBrnAreahqBrnmgrCpsnRes =
	callSharedMethodByDirect("com.kbstar.kji.enbncmn",
	"FUBcBranchgetBrnAreahqBrnmgrCpsn", getBrnAreahqBrnmgrCpsnReq, onlineCtx);
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getBrnAreahqBrnmgrCpsnRes.getString("rtnStat");
	String outAreaHqChfEmpid =
	getBrnAreahqBrnmgrCpsnRes.getString("outAreaHqChfEmpid");
	String outAreaHqChfFname =

	getBrnAreahqBrnmgrCpsnRes.getString("outAreaHqChfFname");
	String outAreaHqChfCphnNo =
	getBrnAreahqBrnmgrCpsnRes.getString("outAreaHqChfCphnNo");
	String outBrnmgrCnt = getBrnAreahqBrnmgrCpsnRes.getString("outBrnmgrCnt");
	IRecordSet getBrnmgrResRsList =
	getBrnAreahqBrnmgrCpsnRes.getRecordSet("outBrnmgrList");
	String outIntnlCPsnCnt =
	getBrnAreahqBrnmgrCpsnRes.getString("outIntnlCPsnCnt");
	IRecordSet getIntnlCPsnResRsList =
	getBrnAreahqBrnmgrCpsnRes.getRecordSet("outIntnlCPsnList");

비 고	

PoC

PoC

PoC

PoC

PoC

PoC

1.131 기준환율 조회 II (FUBcRate.getBaseExchangeRate2())

프로그램 ID	FUBcRate.getBaseExchangeRate2()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	기준환율 통화목록	SQLMAP 명	NAME SPACE	fubcrate
	영문명	THKJIEX04 THKJIEX01		SQL ID	selectBaseMaxExreturnNth selectCurrencyCode5 selectBaseExchangeRate1 selectBaseExchangeRate6 selectBaseExchangeRate7 selectBaseExchangeRate8 selectBaseExchangeRate9

PoC

프로그램 설명	대고객 거래에 사용되는 기준환율정보를 조회한다.
프로그램 규칙 및 제약사항	1. 구분코드 아래와 같다. 1) '1' : 지정통화에 대한 기준환율 정보를 조회한다. - 통화코드를 필수항목으로 입력하여야 한다. - 입력된 통화코드는 통화목록(THKJIEX01)에 등록 되어 있어야 하며, 사용여부가 '1-사용', 고시통화구분이 '1-고시통화' 여야 한다. 2) '2' : 전체통화에 대한 기준환율 정보를 조회한다.(57 개통화 - 유로통화 제외) - 통화목록(THKJIEX01)의 통화코드 정보 중 사용여부가 '1-사용', 고시통화구분이 '1-고시통화', 유로통화사용여부가 '0-부'인 통화를 대상으로 한다. - 조회순서일련번호가 0 보다 큰 기준환율 정보를 대상으로 한다. 3) '3' : 전체통화에 대한 기준환율 정보를 조회한다.(57 개통화 - 유로통화 포함) - 통화목록(THKJIEX01)의 통화코드 정보 중 사용여부가 '1-사용', 고시통화구분이 '1-고시통화'인 통화를 대상으로 한다. - 조회순서일련번호가 0 보다 큰 기준환율 정보를 대상으로 한다. 2. 출력구분코드는 아래와 같다. (전체통화에 대한 기준환율 정보 조회시에 필수입력한다.) 1) '1' : 통화코드일련번호순으로 오름차순 정렬하여 조회한다. 2) '2' : 조회순서일련번호순으로 오름차순 정렬하여 조회한다. 3. 대상일자는 기준환율 적용시작일을 기준으로 한다. 대상일자 미입력시 거래기준년월일을 기준으로 조회한다. 4. 입력고시회차 미입력시 최종회차로 조회한다. 1) MAX 회차가 0 인(미고시)경우 리턴상태를 '07'로 한다. 5. 1 회차 환율,고시여부를 체크 1) 1 회차 환율, 고시여부를 체크하려면 환율회차에 1 을 입력하여 조회하고, 조회 데이터가 없으면 리턴상태코드를 '07'로 한다. 6. 환율고시전에 기준년월일에 대한 1 회차 환율데이터가 없을 경우에, 전일자로 환율을 조회하지 않고 리턴상태코드를 '07'로 한다. 업무가 판단하여 환율고시전의 환율이 필요한 경우에 전일자로 조회하여 사용한다. 7. 100 건 초과하여 환율건수가 있을 경우 오류처리 한다. 8. 2009.09.08 현재 일본,인도네시아,이태리,이탈리아,스페인,베트남 6 개통화는 100 단로 관리되며 일단위 자료가 필요한 업무는 N100 단위통화여부(XCJIEX11-0-CNCYCD)에 '1' 출력 값을 확인 하여 6 개 통화에 대해서는 일단위로 변환하여 사용 한다. 9. 정상인경우 1 회차매매기준환율과 등록시각을 항상 제공한다.
입출력 설명	package com.kbstar.kji.enbncmn.biz; public class FUBcRate extends com.kbstar.sqc.base.FunctionUnit

PoC

PoC

PoC

	[입력항목]
	String inGroupCoCd;
	String inDstcd;
	String inOutptDstcd;
	String inBaseYmd;
	String inCncycd;
	int inExrtAplyNth;
	
	[출력항목]
	String rtnStat;
	String outBaseYmd;
	int outExrtAplyNth;
	int outArayCnt;
	List outBaseExrtList;
	
	< outBaseExrtList 항목 >
	String outCncycd;
	BigDecimal outCrrtz;
	BigDecimal outDealBaseExrt;
	BigDecimal outUsdRt;
	BigDecimal outTelchkBuyExrt;
	BigDecimal outTelChkSelExrt;
	BigDecimal outCshBuyExrt;
	BigDecimal outCshSellExrt;
	BigDecimal outTcSellExret;
	String outCrncyIsocd;
	String outNatnlCd;
	String outNatnlHanglName;
	String outN100UnitCrncyYn;
	String outCncycdHanglName;
	String outCncycdEngName;
	String outRegiFnshHms;
	BigDecimal outN1tmTdBaseExrt;
		● 그룹회사코드
		● 구분코드
		◎ 출력구분코드
		◎ 기준년월일
		◎ 통화코드
		◎ 환율적용회차
		
		// 리턴상태코드
		// 기준년월일
		// 환율적용회차
		// 배열개수
		// 기준환율List
		
		// 통화코드
		// CROSSRATE 율
		// 매매기준환율
		// 대미환산율
		// 전신환매입환율
		// 전신환매도환율
		// 현찰매입환율
		// 현찰매도환율
		// TC 매도환율
		// 통화ISO코드
		// 국가코드
		// 국가한글명
		// N100단위통화여부
		// 통화코드한글명
		// 통화코드영문명
		// 등록완료시각
		// 1회차매매기준환율

RTNSTAT 설명	00 : 정상, 07 : 1 회차 환율 고시 미완료	

사용 설명(예시)	// 공유메소드 호출 - 기준환율 조회 (com.kbstar.kji.enbncmn, getBaseExchangeRate2)
	
	IDataSet getBaseExchangeRate2Req = new DataSet();
	
	GetBaseExchangeRate2Req.put("inGroupCoCd", requestData.get("inGroupCoCd"));
	getBaseExchangeRate2Req.put("inDstcd", requestData.get("inDstcd"));
	GetBaseExchangeRate2Req.put("inOutptDstcd", requestData.get("inOutptDstcd"));
	getBaseExchangeRate2Req.put("inBaseYmd", requestData.get("inBaseYmd"));
	GetBaseExchangeRate2Req.put("inCncycd", requestData.get("inCncycd"));
	GetBaseExchangeRate2Req.put("inExrtAplyNth", requestData.get("inExrtAplyNth"));
	
	// @@ 공유메소드 호출 (기준환율 조회 getBaseExchangeRate2)	

	IDataSet getBaseExchangeRate2Res = callSharedMethodByDirect("com.kbstar.kji.enbncmn", "FUBcRate.getBaseExchangeRate2", getBaseExchangeRate2Req, onlineCtx); PoC
	
	// @@ 출력 데이터 셋 처리
	String rtnstat = getBaseExchangeRate2Res.getString("rtnStat");
	String outbaseymd = getBaseExchangeRate2Res.getString("outBaseYmd");
	int outexrtaplynth = getBaseExchangeRate2Res.getInt("outExrtAplyNth");
	int outaraycnt = getBaseExchangeRate2Res.getInt("outArayCnt");
	IRecordSet getBaseExchangeRate2ResRsOutbaseexrtlist = GetBaseExchangeRate2Res.getRecordSet("outBaseExrtList");

비 고 PoC	

PoC

PoC

PoC

This document is strictly limited to the 2025 C2J conversation PoC

1.132 부점점심시간정보 조회 (FUBcBranch.getLnchTtmCntzWrkBrn())

프로그램 ID	FUBcBranch.getLnchTtmCntzWrkBrn()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	부점점심시간정보	SQLMAP 명	NAME SPACE	fubcbranch
	영문명	THKJIBR17		SQL ID	selectLnchTtmCntzWrkBrn

PoC

프로그램 설명	조회 요청일자 시점의 부점점심시간 정보를 조회한다
프로그램 규칙 및 제약사항	1. 부점코드 체크 - 부점코드는 숫자로 입력되어야 한다. - 부점코드를 '0000'입력하면 오류를 발생한다. 2. 일자에 대한 부점 기본 정보 조회 1) 연월일 입력 : 입력된 일자 당시의 부점점심시간정보 조회 2) 연월일 미입력 : 현재 시점의 부점점심시간정보 조회 - 현재시점 : Common Area 에 포함되어있는 KB 기준정보의 거래처리기준년월일 3) 적용시작일자 및 적용종료일자 기준으로 조회 된다. 3. 입력년월일은 아래의 조건을 만족해야 한다. 1) 입력월은 01 ~ 12 사이 값이어야 한다. 2) 입력일은 01 ~ 31 사이 값이어야 한다. 3) 입력일이 입력월의 말일자 보다 크지 않은 값이어야 한다. - 입력년의 윤년여부를 판단하여 윤년인 경우 2 월의 마지막일자는 29 로 셋팅한다. 4. 데이터관리 1) 영업기획부 2) 화면 : [06-70-006] 부점점심시간정보 등록 및 조회 3) 프로젝트 : P20242333109 MyStar+ 단말 내 [점심시간 집중근무 등록 및 조회] 화면 개발 4) 담당자 : 박수진 과장 (20240806 기준)

COPY BOOK 설명	`package com.kbstar.kji.enbncmn.biz;`
	
	`public class FUBcBranch extends com.kbstar.sqc.base.FunctionUnit`
	
	[입력항목]
	String inGroupCoCd;
	String inBrncd;
	String inYmd;
	
	[출력항목]
	String outBrncd;
	String outAplyStartYmd;
	String outAplyEndYmd;
	String outLnchStartHms;
	String outLnchEndHms;
	String outBrnLtcwraDstcd;
	
	<입력항목>
	● 그룹회사코드
	● 부점코드
	◎ 연월일
	
	<출력항목>
	부점코드
	적용시작년월일
	적용종료년월일
	점심시작시각
	점심종료시각
	부점지역구분코드

RTNSTAT 설명	N/A

사용설명(예시)	`// 공유메소드 호출 – 부점점심시간정보 조회 (com.kbstar.kji.enbncmn, getLnchTtmCntzWrkBrn)`
	
	`IDataSet getLnchTtmCntzWrkBrnReq = new DataSet();`
	
	`getLnchTtmCntzWrkBrnReq.put("inGroupCoCd", requestData.get("inGroupCoCd"));`
	`getLnchTtmCntzWrkBrnReq.put("inBrncd", requestData.get("inBrncd"));`

PoC
PoC
PoC
PoC

This document is strictly limited to the 2025 C2J conversation PoC

	// @@ 공유메소드 호출 (부점점심시간정보 조회 getLnchTtmCntzWrkBrn) *IDataSet getLnchTtmCntzWrkBrnRes =* *callSharedMethodByDirect("com.kbstar.kji.enbncmn",* *"FUBcBranch.getLnchTtmCntzWrkBrn",* *getLnchTtmCntzWrkBrnReq, onlineCtx);* // @@ 출력 데이터 셋 처리 String outBrnLtcwraDstcd = getBranchBizOperationTimeRes.getString("outBrnLtcwraDstcd"); < 중 략 >
비 고	

PoC
PoC
PoC
PoC
PoC
PoC

1.133 고유식별정보 검출 II (FUBcValidation.validationUniqIdnfiInfo())

프로그램 ID	FUBcValidation. validationUniqIdnfiInfo2()		언어 구분	JAVA(○)	
입출력 명	requestData/responseData		사용 구분	온라인(○), 배 치(○)	
			공통 유형	공통 Utility	
사용 TABLE 명	한글명	N/A	SQLMAP 명	NAME SPACE	N/A
	영문명	N/A		SQL ID	N/A

PoC

프로그램 설명	입력된 데이터에서 고유식별정보 항목을 찾아 위치와 값을 리턴한다. FUBcValidation.validationUniqIdnfiInfo 와 동일하나 “ 99 ”-전체 (01~04) 검색 추가
프로그램 규칙 및 제약사항	1. 입력 데이터 검증 1) 고유식별정보구분 - “ 01 ” - 주민번호 - “ 02 ” - 외국인등록번호 - “ 03 ” - 운전면허번호 - “ 04 ” - 여권번호 - “ 05 ” - 연속된 4 자리 숫자 - “ 06 ” - 연속된 13 자리 숫자 - “ 99 ” - 전체 (01~04) 검색 → “ 01 ” ~ “ 04 ” 까지 순차적으로 검색 → 검색 중 추출된 데이터가 50 개이면 추가 검색 없이 종료 2) 검색데이터길이 는 5 자리 숫자(29,000 이하)로 입력되어야 함. 3) 검색데이터는 29,000 Byte 이내로 입력되어야 함. 4) 최소 검색데이터길이 체크 - 주민번호 13, 외국인등록번호 13, 운전면허번호 12, 여권번호 9 이상으로 입력 되어야 함 2. 출력 데이터 1) 고유식별정보확인구분 : 추출 데이터 있으면 “ 01 ”, 없으면 “ 02 ” 2) 추출 건수 : 추출된 데이터 건수 3) 추출 데이터 : 검색된 데이터의 건수로 총 50 개까지 검색 - 고유식별정보구분 : 추출된 고유식별정보구분 - 추출 위치 : 전체 데이터 중 고유식별정보 시작 위치 - 추출 길이 : 검색된 고유식별정보 길이 - 추출값 : 검색된 주민번호의 실제 값 3. 검색기준 1) 주민번호, 외국인등록번호 - 13~14 자리 연속으로 이어진 데이터가 주민등록번호 형식에 맞아야 함 - 앞 6 자리 숫자가 날짜형식에 맞아야 함 - 날짜가 미래 일자가 아니어야 함 - 날짜가 “ 20201005 ” 이전 날짜인 경우 체크디지트 검증로직을 수행하고 체크디지트가 맞으면 주민등록번호 인 것으로 간주, 이후 날자는 체크디지 검증 없이 주민번호로 간주 - 검증형식 예시(실주민번호가 “ 2001o11234567 ” 이라 가정 할때) “ 2001o11234567 ” → 연속된 13 자리 숫자인 경우 “ 2001o1-1234567 “ → 6 자리숫자 + “ - “ + 7 자리숫자 “ 2001o1 1234567 ” → 6 자리숫자 + “ “ + 7 자리숫자 - 형식은 맞으나 법인번호 체크하여 법인번호인 경우 주민번호, 외국인등록번호로 보지 않는다. 2) 운전면허번호 - 12~19 자리 연속으로 이어진 데이터가 운전면허번호 형식인 경우 체크디지트 검증로직을 수행하고 체크디지트가 맞으면 운전면허번호 인 것으로 간주 - 검증형식예시(운전면허번호가 “ 112345678971 ” 또는 “ 서울 2345678971 ” 이라 가정 할때) “ 서울 2345678971 ” → 지역명 + 10 자리숫자 “ 서울 23-456789-71 ” → 지역명+2 자리숫자+ “ - “ +6 자리숫자+ “ - “ +2 자리숫자 “ 서울-23-456789-71 ” → 지역명+ “ - “ +2 자리숫자+ “ - “ +6 자리숫자+ “ - “ +2 자리숫자 “ 112345678971 ” → 지역코드 2 자리 + 12 자리 숫자 “ 11-23-456789-71 ” → 지역코드 2 자리+ “ - “ +2 자리숫자+ “ - “ +6 자리숫자+ “ - “ +2 자리숫자 - 중간에 공백이 있는 경우 고유식별정보로 검출하지 않음 (2024.10 요건 추가)

PoC PoC PoC PoC PoC

This document is strictly limited to the 2025 C2J conversation PoC

“ 서울 23 456789 71 ” → 지역명+“ “ +2 자리숫자+“ “ +6 자리숫자+“ “ +2 자리숫자
“ 서울 23-456789-71 ” → 지역명+“ “ +2 자리숫자+“ - “ +6 자리숫자+“ - “ +2 자리숫자
“ 11 23 456789 71 ” → 지역코드 2 자리+“ “ +2 자리숫자+“ “ +6 자리숫자+“ “ +2 자리숫자
“ 11 23-456789-71 ” → 지역코드 2 자리+“ “ +2 자리숫자+“ - “ +6 자리숫자+“ - “ +2 자리숫자

- 운전면허번호 패턴 검출 후 추가 데이터가 있으면 검출 결과를 set 하지 않고 검출 종료
  검출 종료 이후 공백이 있으면 다시 검출을 시작함(2024.10 요건 추가)
  운전면허번호 패턴이 검출되고 데이터가 끝인경우 검출 결과를 set 하고 검출 종료함
  “ 서울 23-456789-71555 ” → 검출되지 않음
  “ 111 서울 23-456789-71 ” → 검출되지 않음
  “ 111 서울 23-456789-71 서울 2345678971 ” → 공백 뒤 서울 2345678971 만 검출
  “ 111 서울 23-456789-71 11-23-456789-71 ” → 공백 뒤 11-23-456789-71 만 검출
- 유효 지역명 및 지역코드 : 11 서울 ,12 부산 ,13 경기 ,14 강원 ,15 충북 ,16 충남 ,
  17 전북 ,18 전남 ,19 경북 ,20 경남 ,21 제주 ,22 대구 ,
  23 인천 ,24 광주 ,25 대전 ,26 울산 ,28 지역명 없음

3) 여권번호
- 9 자리 연속으로 이어진 데이터가 여권번호 형식인 경우 여권번호 인 것으로 간주
- 검증형식 예시
  “ D12345678 ” → 영문 1 자리 + 숫자 8 자리
  “ AF1234567 ” → 영문 2 자리 + 숫자 7 자리
- 여권번호 패턴 검출 후 추가 데이터가 있으면 검출 결과를 set 하지 않고 검출 종료함
  검출 종료 이후 공백이 있으면 다시 검출을 시작함(2024.10 요건 추가)
  “ M10241006NS00178 ” → 검출되지 않음
  “ 999M10241006 ” → 검출되지 않음
  “ 999M10241006 D12345678 ” → 공백 뒤 D12345678 만 검출
- 유효 영문자 (1 자리)
  → 'D' 'M' 'T' 'R' 'S' 'G'
- 유효 영문자 (2 자리)

```
→ 'AE' 'AF' 'AG' 'AN' 'AQ' 'AR' 'AU' 'AV' 'AY' 'BA'
  'BB' 'BG' 'BL' 'BN' 'BO' 'BR' 'BS' 'BT' 'BU' 'CA'
  'CB' 'CD' 'CG' 'CH' 'CL' 'CN' 'CO' 'CP' 'CS' 'CZ'
  'DG' 'DJ' 'DK' 'DM' 'EP' 'EQ' 'ER' 'ES' 'ET' 'FJ'
  'FK' 'FN' 'FR' 'FU' 'GA' 'GB' 'GD' 'GE' 'GG' 'GH'
  'GJ' 'GK' 'GM' 'GN' 'GP' 'GR' 'GS' 'GU' 'GV' 'GW'
  'GY' 'GZ' 'HC' 'HD' 'HG' 'HI' 'HK' 'HL' 'HO' 'HU'
  'IC' 'IR' 'IS' 'IT' 'IV' 'JA' 'JB' 'JG' 'JJ' 'JN'
  'JO' 'JR' 'JU' 'KA' 'KJ' 'KN' 'KO' 'KR' 'KU' 'KY'
  'KZ' 'LA' 'LB' 'LP' 'LS' 'MA' 'MG' 'MN' 'MO' 'MP'
  'MU' 'MZ' 'NA' 'ND' 'NG' 'NI' 'NJ' 'NP' 'NR' 'NW'
  'NY' 'NZ' 'OC' 'OM' 'OS' 'PA' 'PD' 'PG' 'PH' 'PM'
  'PN' 'PO' 'PU' 'QD' 'QT' 'RA' 'RF' 'RM' 'SA' 'SB'
  'SC' 'SD' 'SE' 'SF' 'SG' 'SH' 'SJ' 'SK' 'SL' 'SM'
  'SN' 'SO' 'SP' 'SQ' 'SR' 'SS' 'ST' 'SW' 'SY' 'SZ'
  'TJ' 'TM' 'TN' 'TP' 'TR' 'TU' 'TZ' 'UA' 'UK' 'UL'
  'UN' 'UR' 'US' 'UZ' 'VC' 'VE' 'VL' 'VT' 'VZ' 'YG'
  'YO' 'YP' 'YS' 'ZB' 'DR' 'DQ' 'DP' 'DT' 'DS'
```

4) 법인번호 체크 (주민번호, 외국인등록번호 추출시 체크)
- 형식 패턴은 주민등록 , 외국인등록번호와 동일
- 정규식 : (1[1-9]¦2[0-8])(0[1-3]¦1[1-5]¦4[1-9]¦5[0-9]¦6[0-2])(1[1-5]¦2[1-2]¦3[1-9]¦4[0-9]¦5[0-5]¦61¦71¦8[1-6])\D{7}
- 정규식 내용 :

```
1-2 : (1[1-9] ¦  << 1 번째자리 1, 2 번째자리  1-9  OR
       2[0-8])   << 1 번째자리 2, 2 번째자리  0-8
3-4 : (0[1-3] ¦  << 3 번째자리 0, 4 번째자리  1-3  OR
       1[1-5] ¦  << 3 번째자리 1, 4 번째자리  1-5  OR
       4[1-9] ¦  << 3 번째자리 4, 4 번째자리  1-9  OR
       5[0-9] ¦  << 3 번째자리 5, 4 번째자리  0-9  OR
       6[0-2])   << 3 번째자리 6, 4 번째자리  0-2
5-6 : (1[1-5] ¦  << 5 번째자리 1, 6 번째자리  1-5  OR
       2[1-2] ¦  << 5 번째자리 2, 6 번째자리  1-2  OR
       3[1-9] ¦  << 5 번째자리 3, 6 번째자리  1-9  OR
       4[0-9] ¦  << 5 번째자리 4, 6 번째자리  0-9  OR
       5[0-5] ¦  << 5 번째자리 5, 6 번째자리  0-5  OR
       61 ¦      << 5 번째자리 6, 6 번째자리  1    OR
       71 ¦      << 5 번째자리 7, 6 번째자리  1    OR
       8[1-6])   << 5 번째자리 8, 6 번째자리  1-6
7-13:  \D{7}     << 7-13 번째자리  0-9
```

5) 연속된 4 자리 숫자 체크
- 4 자리 이상 연속된 숫자이면 숫자 4 자리를고유식별정보로 간주
- EX) INPUT : 123456
  OUTPUT : 1234, 2345, 3456

6) 연속된 13 자리 숫자 체크
- 13 자리 이상 연속된 숫자이면 숫자 13 자리를 고유식별정보로 간주
- EX) INPUT : 12345678901234
  OUTPUT : 1234567890123, 2345678901234

4. 프로젝트
1) P20242334884 - DB 고유식별정보 암호화 실태 점검 협의체
2) 담당자 : 정보보호부 안대현대리(2024.10 기준)
3) 요건 수정시 정보보호부 확인 필요

COPY BOOK 설명

PoC

```
package com.kbstar.kji.enbncmn.biz;

public class FUBcValidation extends
                com.kbstar.sqc.base.FunctionUnit

[입력항목]
  String inSearchDstcd;
  String inSearchData;

[출력항목]
  String outFndnCnt;
  recordSet List;
    -field outUniqIdInfoDst
    -field outFndnPos;
    -field outFndnLen;
    -field outFndnVal;
```

PoC

PoC

PoC

- ● 검색구분
- ● 검색데이타

//추출건수
//[recordSet]
//고유식별정보구분
// 추출위치
// 추출길이
// 추출값

RTNSTAT 설명

N/A

사용설명(예시)

PoC

```
// 공유메소드 호출 - 고유식별정보 검출2 (com.kbstar.kji.enbncmn,
                                        validationUniqIdnfiInfo2)

  IDataSet validationUniqIdnfiInfo2Req = new DataSet();

  validationUniqIdnfiInfo2Req.put("inSearchDstcd", requestData.get("inSearchDstcd"));
  validationUniqIdnfiInfo2Req.put("inSearchData", requestData.get("inSearchData"));

// @@ 공유메소드 호출 (고유식별정보 검출 validationUniqIdnfiInfo2)
IDataSet validationUniqIdnfiInfo2Res =
      callSharedMethodByDirect("com.kbstar.kji.enbncmn",
                               " FUBcValidation.validationUniqIdnfiInfo2",
                               validationUniqIdnfiInfo2Req, onlineCtx);

// @@ 출력 데이터 셋 처리
String outFndnCnt = validationUniqIdnfiInfo2Res.getString("outFndnCnt");
IRecordSet validationUniqIdnfiInfo2ResRsList =
                        validationUniqIdnfiInfo2Res.getRecordSet("LIST");
For (IRecord validationUniqIdnfiInfo2ResRsListRec : validationUniqIdnfiInfo2ResRsList){
  String outUniqIdInfoDst =
                validationUniqIdnfiInfo2ResRsListRec.getString("outUniqIdInfoDst");
  String outFndnPos = validationUniqIdnfiInfo2ResRsListRec.getString("outFndnPos");
  String outFndnLen = validationUniqIdnfiInfo2ResRsListRec.getString("outFndnLen");
```

This document is strictly limited to the 2025 C2J conversation PoC

PoC	String outFndnVal = validationUniqIdnfiInfoResRsListRec.getString("outFndnVal"); } PoC
비 고 PoC	PoC